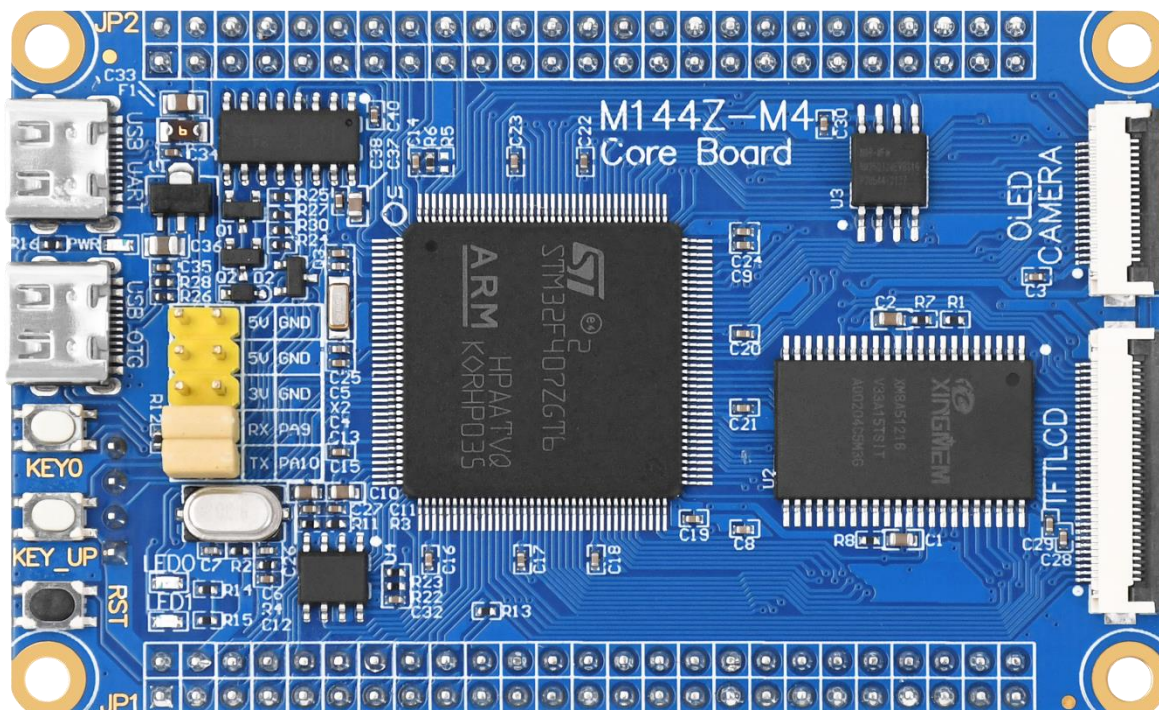


M144Z-M4 最小系统板使用指南

——STM32F407 版 V1.0

—M144Z-M4 最小系统板教程

注：本教程仅适用于 STM32F407 版



型号	版本	说明
M144Z-M3	STM32F103 版	主控芯片使用：STM32F103ZET6
M144Z-M3	APM32E103 版	主控芯片使用：APM32E103ZET6
M144Z-M4	STM32F407 版	主控芯片使用：STM32F407ZGT6
M144Z-M4	APM32F407 版	主控芯片使用：APM32F407ZGT6

修订历史:

版本	日期	修改内容
V1.0	2023/06/07	第一次发布
V1.1	2024/01/08	添加串口烧录介绍
V1.2	2024/09/14	修正引脚描述错误
V1.3	2025/02/12	LCD 驱动对 ST7796 进行了兼容，触摸驱动也对屏幕 ID 为 0x7796 带 GT 型号的电容触摸屏进行兼容。



正点原子公司名称 : 广州市星翼电子科技有限公司
原子哥在线教学平台 : www.yuanzige.com
开源电子网 / 论坛 : www.openedv.com
正点原子官方网站 : www.alientek.com
正点原子淘宝店铺 : <https://openedv.taobao.com>
正点原子 B 站视频 : <https://space.bilibili.com/394620890>

电话: 020-38271790 传真: 020-36773971

请下载原子哥 APP, 数千讲视频免费学习, 更快更流畅。
请关注正点原子公众号, 资料发布更新我们会通知。



扫码下载“原子哥”APP



扫码关注正点原子公众号

内容简介.....	1
第一篇 基础篇.....	2
第一章 本书学习方法.....	3
1.1 本书学习顺序.....	3
1.2 本书参考资料.....	3
1.3 本书编写规范.....	3
1.4 本书代码规范.....	4
1.5 例程资源说明.....	4
1.6 学习资料查找.....	5
第二章 STM32 简介	8
2.1 初识 STM32	8
2.2 STM32F407 资源简介	8
第三章 开发环境搭建.....	10
3.1 常用开发工具简介	10
3.2 MDK 安装	10
3.3 仿真器驱动安装.....	13
3.4 USB 虚拟串口驱动安装	13
第四章 STM32 初体验	15
4.1 使用 MDK 编译实验例程	15
4.2 使用串口烧录程序.....	17
4.3 使用 DAP 烧录及调试程序.....	20
4.3.1 使用 DAP 烧录程序.....	20
4.3.2 使用 DAP 调试程序.....	22
4.4 MDK 使用技巧	27
4.4.1 自定义编辑器.....	27
4.4.2 动态语法检测&代码自动补全.....	29
4.4.3 代码编辑技巧.....	30
4.4.4 其他小技巧.....	32
第五章 STM32 基础知识入门	33
5.1 C 语言基础知识复习	33
5.1.1 位操作.....	33
5.1.2 define 宏定义.....	34
5.1.3 ifdef 条件编译	34
5.1.4 extern 外部申明.....	34
5.1.5 typedef 类型别名.....	34
5.1.6 struct 结构体.....	34
5.1.7 指针.....	35
5.2 寄存器基础知识.....	35
5.3 STM32F407 系统架构	36
5.3.1 Cortex-M4 内核&芯片	36
5.3.2 STM32F407 系统框图	36
5.3.3 地址映射.....	37
第六章 认识 HAL 库	38

6.1 STM32CubeF4 简介	38
6.2 Drivers 文件夹介绍	38
6.3 ST HAL 库介绍	39
第七章 HAL 库版本 MDK 工程创建	40
7.1 HAL 库版本 MDK 工程创建	40
7.1.1 构建工程文件夹	40
7.1.2 添加文件到工程目录	41
7.1.3 创建 MDK 工程	42
7.1.4 配置 MDK 工程	44
7.2 下载验证	48
第八章 STM32F407 时钟系统介绍	50
8.1 认识时钟树	50
8.1.1 时钟源	52
8.1.2 锁相环	52
8.1.3 系统时钟	55
8.2 配置系统主频	55
8.3 开启和关闭外设时钟	56
第九章 SYSTEM 文件夹介绍	58
9.1 sys 文件夹代码介绍	58
9.2 delay 文件夹代码介绍	58
9.3 usart 文件夹代码介绍	58
9.3.1 printf 函数支持	59
第二篇 入门篇	61
第十章 跑马灯实验	62
10.1 硬件设计	62
10.1.1 例程功能	62
10.1.2 硬件资源	62
10.1.3 原理图	62
10.2 程序设计	62
10.2.1 HAL 库的 GPIO 驱动	62
10.2.2 LED 驱动	64
10.2.3 实验应用代码	66
10.3 下载验证	66
第十一章 按键输入实验	67
11.1 硬件设计	67
11.1.1 例程功能	67
11.1.2 硬件资源	67
11.1.3 原理图	67
11.2 程序设计	67
11.2.1 HAL 库的 GPIO 驱动	67
11.2.2 按键驱动	68
11.2.3 实验应用代码	70
11.3 下载验证	71
第十二章 外部中断实验	72

12.1 硬件设计.....	72
12.1.1 例程功能.....	72
12.1.2 硬件资源.....	72
12.1.3 原理图.....	72
12.2 程序设计.....	72
12.2.1 HAL 库的 GPIO 驱动	72
12.2.2 外部中断驱动.....	73
12.2.3 实验应用代码.....	75
12.3 下载验证.....	75
第十三章 串口通信实验.....	76
13.1 硬件设计.....	76
13.1.1 例程功能.....	76
13.1.2 硬件资源.....	76
13.1.3 原理图.....	76
13.2 程序设计.....	77
13.2.1 HAL 库的 GPIO 驱动	77
13.2.2 HAL 库的 UART 驱动.....	77
13.2.3 串口通讯驱动.....	78
13.2.4 实验应用代码.....	80
13.3 下载验证.....	81
第十四章 独立看门狗实验.....	82
14.1 硬件设计.....	82
14.1.1 例程功能.....	82
14.1.2 硬件资源.....	82
14.1.3 原理图.....	82
14.2 程序设计.....	82
14.2.1 HAL 库的 IWDG 驱动.....	82
14.2.2 看门狗驱动.....	83
14.2.3 实验应用代码.....	84
14.3 下载验证.....	84
第十五章 窗口看门狗实验.....	85
15.1 硬件设计.....	85
15.1.1 例程功能.....	85
15.1.2 硬件资源.....	85
15.1.3 原理图.....	85
15.2 程序设计.....	85
15.2.1 HAL 库的 WWDG 驱动	85
15.2.2 看门狗驱动.....	86
15.2.3 实验应用代码.....	87
15.3 下载验证.....	87
第十六章 基本定时器中断实验.....	88
16.1 硬件设计.....	88
16.1.1 例程功能.....	88
16.1.2 硬件资源.....	88
16.1.3 原理图.....	88
16.2 程序设计.....	88

16.2.1 HAL 库的 TIM 驱动	88
16.2.2 基本定时器驱动.....	89
16.2.3 实验应用代码.....	90
16.3 下载验证.....	90
第十七章 通用定时器中断实验.....	91
17.1 硬件设计.....	91
17.1.1 例程功能.....	91
17.1.2 硬件资源.....	91
17.1.3 原理图.....	91
17.2 程序设计.....	91
17.2.1 HAL 库的 TIM 驱动	91
17.2.2 通用定时器驱动.....	91
17.2.3 实验应用代码.....	91
17.3 下载验证.....	92
第十八章 通用定时器 PWM 输出实验	93
18.1 硬件设计.....	93
18.1.1 例程功能.....	93
18.1.2 硬件资源.....	93
18.1.3 原理图.....	93
18.2 程序设计.....	93
18.2.1 HAL 库的 TIM 驱动	93
18.2.2 通用定时器驱动.....	95
18.2.3 实验应用代码.....	96
18.3 下载验证.....	96
第十九章 通用定时器输入捕获实验.....	97
19.1 硬件设计.....	97
19.1.1 例程功能.....	97
19.1.2 硬件资源.....	97
19.1.3 原理图.....	97
19.2 程序设计.....	97
19.2.1 HAL 库的 TIM 驱动	97
19.2.2 通用定时器驱动.....	99
19.2.3 实验应用代码.....	100
19.3 下载验证.....	101
第二十章 通用定时器脉冲计数实验.....	102
20.1 硬件设计.....	102
20.1.1 例程功能.....	102
20.1.2 硬件资源.....	102
20.1.3 原理图.....	102
20.2 程序设计.....	102
20.2.1 HAL 库的 TIM 驱动	102
20.2.2 通用定时器驱动.....	103
20.2.3 实验应用代码.....	105
20.3 下载验证.....	106
第二十一章 高级定时器输出指定个数 PWM 实验	107

21.1 硬件设计.....	107
21.1.1 例程功能.....	107
21.1.2 硬件资源.....	107
21.1.3 原理图.....	107
21.2 程序设计.....	107
21.2.1 HAL 库的 TIM 驱动	107
21.2.2 高级定时器驱动.....	107
21.2.3 实验应用代码.....	110
21.3 下载验证.....	110
第二十二章 高级定时器输出比较模式实验.....	111
22.1 硬件设计.....	111
22.1.1 例程功能.....	111
22.1.2 硬件资源.....	111
22.1.3 原理图.....	111
22.2 程序设计.....	111
22.2.1 HAL 库的 TIM 驱动	111
22.2.2 高级定时器驱动.....	113
22.2.3 实验应用代码.....	114
22.3 下载验证.....	115
第二十三章 高级定时器互补输出带死区控制实验.....	116
23.1 硬件设计.....	116
23.1.1 例程功能.....	116
23.1.2 硬件资源.....	116
23.1.3 原理图.....	116
23.2 程序设计.....	116
23.2.1 HAL 库的 TIM 驱动	116
23.2.2 高级定时器驱动.....	118
23.2.3 实验应用代码.....	120
23.3 下载验证.....	120
第二十四章 高级定时器 PWM 输入模式实验.....	121
24.1 硬件设计.....	121
24.1.1 例程功能.....	121
24.1.2 硬件资源.....	121
24.1.3 原理图.....	121
24.2 程序设计.....	121
24.2.1 HAL 库的 TIM 驱动	121
24.2.2 高级定时器驱动.....	122
24.2.3 实验应用代码.....	124
24.3 下载验证.....	124
第二十五章 OLED 实验.....	125
25.1 硬件设计.....	125
25.1.1 例程功能.....	125
25.1.2 硬件资源.....	125
25.1.3 原理图.....	125
25.2 程序设计.....	126
25.2.1 HAL 库的 GPIO 驱动	126

25.2.2 OLED 驱动.....	126
25.2.3 实验应用代码.....	129
25.3 下载验证.....	130
第二十六章 TFTLCD（MCU 屏）实验.....	131
26.1 硬件设计.....	131
26.1.1 例程功能.....	131
26.1.2 硬件资源.....	131
26.1.3 原理图.....	131
26.2 程序设计.....	132
26.2.1 HAL 库的 FSMC 驱动.....	132
26.2.2 TFTLCD 驱动.....	133
26.2.3 实验应用代码.....	137
26.3 下载验证.....	139
第二十七章 USART 调试实验	140
27.1 硬件设计.....	140
27.1.1 例程功能.....	140
27.1.2 硬件资源.....	140
27.1.3 原理图.....	140
27.2 程序设计.....	140
27.2.1 USART 组件	140
27.2.2 实验应用代码.....	142
27.3 下载验证.....	143
第二十八章 RTC 实验	144
28.1 硬件设计.....	144
28.1.1 例程功能.....	144
28.1.2 硬件资源.....	144
28.1.3 原理图.....	144
28.2 程序设计.....	144
28.2.1 HAL 库的 PWR 驱动.....	144
28.2.2 HAL 库的 RTC 驱动	145
28.2.4 RTC 驱动	151
28.2.5 实验应用代码.....	154
28.3 下载验证.....	155
第二十九章 硬件随机数实验.....	156
29.1 硬件设计.....	156
29.1.1 例程功能.....	156
29.1.2 硬件资源.....	156
29.1.3 原理图.....	156
29.2 程序设计.....	156
29.2.1 HAL 库的 RNG 驱动.....	156
29.2.2 RNG 驱动	157
29.2.3 实验应用代码.....	158
29.3 下载验证.....	159
第三十章 PVD 电压监控实验.....	160
30.1 硬件设计.....	160

30.1.1 例程功能.....	160
30.1.2 硬件资源.....	160
30.1.3 原理图.....	160
30.2 程序设计.....	160
30.2.1 HAL 库的 PWR 驱动.....	160
30.2.2 PWR 驱动.....	161
30.2.3 实验应用代码.....	162
30.3 下载验证.....	163
第三十一章 睡眠模式实验.....	164
31.1 硬件设计.....	164
31.1.1 例程功能.....	164
31.1.2 硬件资源.....	164
31.1.3 原理图.....	164
31.2 程序设计.....	164
31.2.1 HAL 库的 PWR 驱动.....	164
31.2.2 PWR 驱动.....	165
31.2.3 实验应用代码.....	166
31.3 下载验证.....	167
第三十二章 停止模式实验.....	168
32.1 硬件设计.....	168
32.1.1 例程功能.....	168
32.1.2 硬件资源.....	168
32.1.3 原理图.....	168
32.2 程序设计.....	168
32.2.1 HAL 库的 PWR 驱动.....	168
32.2.2 PWR 驱动.....	169
32.2.3 实验应用代码.....	169
32.3 下载验证.....	170
第三十三章 待机模式实验.....	171
33.1 硬件设计.....	171
33.1.1 例程功能.....	171
33.1.2 硬件资源.....	171
33.1.3 原理图.....	171
33.2 程序设计.....	171
33.2.1 HAL 库的 PWR 驱动.....	171
33.2.2 PWR 驱动.....	172
33.2.3 实验应用代码.....	172
33.3 下载验证.....	173
第三十四章 DMA 实验	174
34.1 硬件设计.....	174
34.1.1 例程功能.....	174
34.1.2 硬件资源.....	174
34.1.3 原理图.....	174
34.2 程序设计.....	174
34.2.1 HAL 库的 DMA 驱动	174
34.2.2 DMA 驱动	176

34.2.3 实验应用代码.....	176
34.3 下载验证.....	178
第三十五章 单通道 ADC 采集实验	179
35.1 硬件设计.....	179
35.1.1 例程功能.....	179
35.1.2 硬件资源.....	179
35.1.3 原理图.....	179
35.2 程序设计.....	179
35.2.1 HAL 库的 ADC 驱动.....	179
35.2.2 ADC 驱动	182
35.2.3 实验应用代码.....	184
35.3 下载验证.....	184
第三十六章 单通道 ADC 采集（DMA 读取）实验	185
36.1 硬件设计.....	185
36.1.1 例程功能.....	185
36.1.2 硬件资源.....	185
36.1.3 原理图.....	185
36.2 程序设计.....	185
36.2.1 HAL 库的 ADC 驱动.....	185
36.2.2 ADC 驱动	186
36.2.3 实验应用代码.....	188
36.3 下载验证.....	189
第三十七章 多通道 ADC 采集（DMA 读取）实验	190
37.1 硬件设计.....	190
37.1.1 例程功能.....	190
37.1.2 硬件资源.....	190
37.1.3 原理图.....	190
37.2 程序设计.....	190
37.2.1 HAL 库的 ADC 驱动.....	190
37.2.2 ADC 驱动	190
37.2.3 实验应用代码.....	192
37.3 下载验证.....	193
第三十八章 内部温度传感器实验.....	195
38.1 硬件设计.....	195
38.1.1 例程功能.....	195
38.1.2 硬件资源.....	195
38.1.3 原理图.....	195
38.2 程序设计.....	195
38.2.1 HAL 库的 ADC 驱动.....	195
38.2.2 ADC 驱动	195
38.2.3 实验应用代码.....	196
38.3 下载验证.....	197
第三十九章 DAC 输出实验	198
39.1 硬件设计.....	198
39.1.1 例程功能.....	198

39.1.2 硬件资源.....	198
39.1.3 原理图.....	198
39.2 程序设计.....	198
39.2.1 HAL 库的 DAC 驱动.....	198
39.2.2 DAC 驱动.....	200
39.2.3 实验应用代码.....	202
39.3 下载验证.....	203
第四十章 DAC 输出三角波实验	204
40.1 硬件设计.....	204
40.1.1 例程功能.....	204
40.1.2 硬件资源.....	204
40.1.3 原理图.....	204
40.2 程序设计.....	204
40.2.1 HAL 库的 DAC 驱动.....	204
40.2.2 DAC 驱动.....	204
40.2.3 实验应用代码.....	205
40.3 下载验证.....	206
第四十一章 DAC 输出正弦波实验	207
41.1 硬件设计.....	207
41.1.1 例程功能.....	207
41.1.2 硬件资源.....	207
41.1.3 原理图.....	207
41.2 程序设计.....	207
41.2.1 HAL 库的 TIM 驱动	207
41.2.2 HAL 库的 DAC 驱动.....	208
41.2.3 DAC 驱动	209
41.2.4 实验应用代码.....	211
41.3 下载验证.....	212
第四十二章 IIC 实验	213
42.1 硬件设计.....	213
42.1.1 例程功能.....	213
42.1.2 硬件资源.....	213
42.1.3 原理图.....	213
42.2 程序设计.....	214
42.2.1 HAL 库的 GPIO 驱动	214
42.2.2 IIC 驱动	214
42.2.3 EEPROM 驱动	215
42.2.4 实验应用代码.....	215
42.3 下载验证.....	216
第四十三章 SPI 实验.....	217
43.1 硬件设计.....	217
43.1.1 例程功能.....	217
43.1.2 硬件资源.....	217
43.1.3 原理图.....	217
43.2 程序设计.....	217
43.2.1 HAL 库的 SPI 驱动.....	217

43.2.2 SPI 驱动.....	219
43.2.3 NOR Flash 驱动.....	221
43.2.4 实验应用代码.....	222
43.3 下载验证.....	223
第四十四章 触摸屏实验.....	224
44.1 硬件设计.....	224
44.1.1 例程功能.....	224
44.1.2 硬件资源.....	224
44.1.3 原理图.....	224
44.2 程序设计.....	224
44.2.1 触摸屏驱动.....	224
44.2.2 实验应用代码.....	225
44.3 下载验证.....	228
第四十五章 FLASH 模拟 EEPROM 实验.....	229
45.1 硬件设计.....	229
45.1.1 例程功能.....	229
45.1.2 硬件资源.....	229
45.1.3 原理图.....	229
45.2 程序设计.....	229
45.2.1 HAL 库的 Flash 驱动.....	229
45.2.2 Flash 驱动.....	232
45.2.3 实验应用代码.....	234
45.3 下载验证.....	235
第四十六章 摄像头实验.....	236
46.1 硬件设计.....	236
46.1.1 例程功能.....	236
46.1.2 硬件资源.....	236
46.1.3 原理图.....	236
46.2 程序设计.....	237
46.2.1 HAL 库的 DCMI 驱动.....	237
46.2.2 DCMI 驱动代码.....	238
46.2.3 SCCB 驱动.....	241
46.2.4 OV2640 驱动.....	241
46.2.5 实验应用代码.....	241
46.3 下载验证.....	242
第四十七章 外部 SRAM 实验.....	244
47.1 硬件设计.....	244
47.1.1 例程功能.....	244
47.1.2 硬件资源.....	244
47.1.3 原理图.....	244
47.2 程序设计.....	245
47.2.1 HAL 库的 FSMC 驱动.....	245
47.2.2 SRAM 驱动代码.....	245
47.2.5 实验应用代码.....	247
47.3 下载验证.....	249
第三篇 提高篇.....	250

第四十八章 内存管理实验.....	251
48.1 硬件设计.....	251
48.1.1 例程功能.....	251
48.1.2 硬件资源.....	251
48.1.3 原理图.....	251
48.2 程序设计.....	251
48.2.1 内存管理库的使用.....	251
48.2.2 实验应用代码.....	252
48.3 下载验证.....	254
第四十九章 SD 卡实验.....	255
49.1 硬件设计.....	255
49.1.1 例程功能.....	255
49.1.2 硬件资源.....	255
49.1.3 原理图.....	255
49.2 程序设计.....	256
49.2.1 SD 卡驱动.....	256
49.2.2 实验应用代码.....	256
49.3 下载验证.....	258
第五十章 FATFS 实验	259
50.1 硬件设计.....	259
50.1.1 例程功能.....	259
50.1.2 硬件资源.....	259
50.1.3 原理图.....	259
50.2 程序设计.....	259
50.2.1 FATFS 的使用	259
50.2.2 实验应用代码.....	260
50.3 下载验证.....	262
第五十章 汉字显示实验.....	263
51.1 硬件设计.....	263
51.1.1 例程功能.....	263
51.1.2 硬件资源.....	263
51.1.3 原理图.....	263
51.2 程序设计.....	263
51.2.1 字库管理库的使用.....	263
51.2.2 实验应用代码.....	264
51.3 下载验证.....	266
第五十二章 图片显示实验.....	267
52.1 硬件设计.....	267
52.1.1 例程功能.....	267
52.1.2 硬件资源.....	267
52.1.3 原理图.....	267
52.2 程序设计.....	267
52.2.1 图片解码库的使用.....	267
52.2.2 实验应用代码.....	268
52.3 下载验证.....	271

第五十三章 照相机实验.....	272
53.1 硬件设计.....	272
53.1.1 例程功能.....	272
53.1.2 硬件资源.....	272
53.1.3 原理图.....	272
53.2 程序设计.....	273
53.2.1 实验应用代码.....	273
53.3 下载验证.....	276
第五十四章 视频播放器实验.....	277
54.1 硬件设计.....	277
54.1.1 例程功能.....	277
54.1.2 硬件资源.....	277
54.1.3 原理图.....	277
54.2 程序设计.....	277
54.2.1 libjpeg 库的使用.....	277
54.2.2 AVI 文件解析	278
54.2.3 实验应用代码.....	278
54.3 下载验证.....	281
第五十五章 FPU 测试（Julia 分形）实验.....	282
55.1 硬件设计.....	282
55.1.1 例程功能.....	282
55.1.2 硬件资源.....	282
55.1.3 原理图.....	282
55.2 程序设计.....	282
55.2.1 实验应用代码.....	282
55.3 下载验证.....	284
第五十六章 DSP BasicMath 实验	285
56.1 硬件设计.....	285
56.1.1 例程功能.....	285
56.1.2 硬件资源.....	285
56.1.3 原理图.....	285
56.2 程序设计.....	285
56.2.1 DPS 库的使用	285
56.2.2 实验应用代码.....	285
56.3 下载验证.....	287
第五十七章 DSP FFT 实验.....	288
57.1 硬件设计.....	288
57.1.1 例程功能.....	288
57.1.2 硬件资源.....	288
57.1.3 原理图.....	288
57.2 程序设计.....	288
57.2.1 实验应用代码.....	288
57.3 下载验证.....	289
第五十八章 手写识别实验.....	291
58.1 硬件设计.....	291

58.1.1 例程功能.....	291
58.1.2 硬件资源.....	291
58.1.3 原理图.....	291
58.2 程序设计.....	291
58.2.1 手写识别库的使用.....	291
58.2.2 实验应用代码.....	293
58.3 下载验证.....	294
第五十九章 T9 拼音输入法实验	295
59.1 硬件设计.....	295
59.1.1 例程功能.....	295
59.1.2 硬件资源.....	295
59.1.3 原理图.....	295
59.2 程序设计.....	295
59.2.1 T9 拼音输入法库的使用	295
59.2.2 实验应用代码.....	296
59.3 下载验证.....	298
第六十章 串口 IAP 实验	299
60.1 硬件设计.....	299
60.1.1 例程功能.....	299
60.1.2 硬件资源.....	299
60.1.3 原理图.....	299
60.2 程序设计.....	299
60.2.1 IAP 的实现	299
60.2.2 APP 工程修改.....	300
60.2.3 实验应用代码.....	301
60.3 下载验证.....	302
第六十一章 USB 读卡器 (Slave) 实验	303
61.1 硬件设计.....	303
61.1.1 例程功能.....	303
61.1.2 硬件资源.....	303
61.1.3 原理图.....	303
61.2 程序设计.....	304
61.2.1 ST 的 USB 设备驱动库	304
61.2.2 实验应用代码.....	304
61.3 下载验证.....	306
第六十二章 USB 虚拟串口 (Slave) 实验	307
62.1 硬件设计.....	307
62.1.1 例程功能.....	307
62.1.2 硬件资源.....	307
62.1.3 原理图.....	307
62.2 程序设计.....	307
62.2.1 ST 的 USB 设备驱动库	307
62.2.2 实验应用代码.....	307
62.3 下载验证.....	309
第六十三章 FreeRTOS 移植实验.....	310

63.1 硬件设计.....	310
63.1.1 例程功能.....	310
63.1.2 硬件资源.....	310
63.1.3 原理图.....	310
63.2 程序设计.....	310
63.2.1 FreeRTOS 的移植.....	310
63.2.2 实验应用代码.....	310
63.3 下载验证.....	312

内容简介

本书将由浅入深，带领大家学习 STM32F407 的各个功能，为您开启 STM32F407 的学习之旅。

本书共分为三篇：

- 1, 基础篇，主要介绍 STM32F407 的基础入门知识，包括介绍 STM32、搭建开发环境、创建模板工程、介绍 STM32F407 的时钟系统以及 SYSTEM 文件夹等基础知识，必须好好学习并掌握；
- 2, 入门篇，主要介绍 STM32F407 中常用片上外设的使用，包括 GPIO、TIM、ADC、DAC、DMA、FSMC 等，必须好好学习并掌握；
- 3, 提高篇，主要介绍 STM32F407 中比较复杂的片上外设以及一些高级例程，包括 SDIO、USB、内存管理、文件系统、图片解码、RTOS 等，这些片上外设和例程的难度相对较高，应沉下心来，戒骄戒躁、循序渐进地学习，对于以上难度较高的软件库，仅需学习如何使用这些软件库，并不需要深入地了解软件库的实现原理和运行机制，例如文件系统、图片解码、USB 协议栈、RTOS 等。

本书为正点原子 M144Z-M4 最小系统板 STM32F407 版的配套教程，在板卡配套的资料包中提供了大量与本板卡相关的资料，其中就包括硬件原理图以及本书中所有实验例程的源代码，这些代码都作了详细的注释，并且都经过了测试，不会出现任何错误或警告，且每一个实验例程都提供了实现编译好的 Hex 文件，仅需将其通过仿真器烧录到板卡即可观察到实验现象，体验实验过程。

本书没有详细介绍 M144Z-M4 最小系统板 STM32F407 版的硬件资源，而是单独提供了一份相关的文档，请参考：《M144Z-M4 最小系统板硬件参考手册——STM32F407 版.pdf》。

本书不仅非常适合广大学生和电子爱好者学习 STM32F407，其大量的实验例程和详细的解说，也是公司产品开发的不二参考。

第一篇 基础篇

万事开头难，如果打好了基础，后面的学习将会事半功倍。本篇将详细介绍 STM32F407 学习的基础入门知识，包括介绍 STM32、搭建开发环境、创建模板工程、介绍 STM32F407 的时钟系统以及 SYSTEM 文件夹等部分。掌握本篇内容对后续的学习将会有很大的帮助，极大地提高学习效率。

对于初学者，应好好学习并理解本篇提及的知识点，手脑并用，不要漏过任何内容，一遍学不会就多学几遍，掌握好本篇涉及到的知识点。

对于有已经单片机开发基础的读者，则可以挑选学习本篇内容中。

本篇分为如下几个章节：

- 1, 本书学习方法
- 2, STM32 简介
- 3, 开发环境搭建
- 4, STM32 初体验
- 5, STM32 基础知识入门
- 6, 认识 HAL 库
- 7, HAL 库版本 MDK 工程创建
- 8, STM32F407 时钟系统介绍
- 9, SYSTEM 文件夹介绍

第一章 本书学习方法

为了让读者能够更好地学习和使用本书，本章将介绍本书的学习方法。

本章分为如下几个小节：

- 1.1 本书学习顺序
- 1.2 本书参考资料
- 1.3 本书编写规范
- 1.4 本书代码规范
- 1.5 例程资源说明
- 1.6 学习资料查找

1.1 本书学习顺序

为了让读者更好地学习和使用本书，我们做了以下几点考虑：

- 1，坚持循序渐进的思路讲解，从基础到入门，从简单到复杂；
- 2，将知识进行分类介绍，简化学习过程；
- 3，将板卡硬件资源介绍独立成一个文档（《M144Z-M4 最小系统板硬件参考手册——STM32F407 版.pdf》）。

因此，读者在学习本书的时候，我们建议：先通读一遍《M144Z-M4 最小系统板硬件参考手册——STM32F407 版.pdf》，对板卡的硬件资源有个大概的了解，然后从本书的基础篇开始，再到入门篇，最后是提高篇，循序渐进，逐一攻克。

对于初学者，更是要按照以上建议的学习路线进行学习，不要跳跃式学习，因为本书中的知识是环环相扣的，如果没有掌握前面的知识，就去学习后面的知识，就会学的非常吃力。

对于已经有了一定单片机基础的读者，就可以跳跃式地学习，学习效率，当然了，若是遇到不懂的知识点，也得查阅前面的知识点进行巩固。

1.2 本书参考资料

本书主要参考的资料有以下两份文档：

《RM0090: STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 advanced Arm®-based 32-bit MCUs》

《Arm® Cortex®-M4 Processor Technical Reference Manual》

前者是 ST 官方针对 STM32F407 系列 MCU 提供的参考手册，该参考手册向程序开发人员提供了如何使用 MCU 系统架构、存储器和外设所涉及的全部信息。

后者是 ARM 针对 ARM Cortex-M4 内核提供的技术参考手册，该技术参考手册包含了对 ARM Cortex-M4 内核和其使用的指令集、寄存器、内存映射、浮点和跟踪调试等的支持文档。

以上提及的两份文档也是读者在学习本书的过程中必不可少的参考资料，读者可以在 A 盘→8，STM32 参考资料中找到这两份文档。

1.3 本书编写规范

本书通过数十个实验例程为读者详细介绍了 STM32 几乎所有的功能和外设，按照难易程度以及知识结构，本书分为三大篇章：基础篇、入门篇和提高篇。

基础篇，共十章，主要是一些基础知识的介绍、包括开发环境搭建、HAL 库介绍、创建 MDK 工程、时钟系统介绍、SYSTEM 文件夹介绍等，这些章节在结构上没有共性，但相互有关联，即：必须先学习前面的知识，才能更好地学习后面的知识。

入门篇和提高篇，共 53 章，介绍了 STM32F407 的绝大部分外设及其驱动代码，并且还介绍了一些非常实用的程序代码（纯软件例程），如：内存管理、文件系统、T9 拼音输入法、手写识别、图片解码、IAP 等。这部分内容占了本书的绝大部分篇幅，并且这些章节在结构上比较有共

性，一般分为三个部分，如下：

- 1, 硬件设计
- 2, 程序设计
- 3, 下载验证

硬件设计，包括具体章节实验例程实现的功能说明、使用到的硬件资源及其相关的硬件原理图，从而让读者清楚具体章节的实验例程要做什么？用那些硬件资源来做？这些硬件资源是如何进行连接的？便于在程序设计时编写驱动代码和应用代码。

程序设计，一般包括：驱动介绍、配置步骤、关键代码解析、main 函数讲解等及部分，一点一点地介绍程序代码是怎么来的和注意事项等，从而让读者掌握整个程序代码。

下载验证，属于实践环节，在程序设计完成之后，下载并验证设计的程序是否能按照预期工作，形成一个闭环的过程。

1.4 本书代码规范

为了提高读者编写代码的质量，本书对代码风格进行了统一，详细的代码规范说明文档，请参考 A 盘→1，入门资料→【正点原子】嵌入式单片机 C 代码规范与风格.pdf，对于初学者务必好好地学习一下这份文档。

下面总结几个代码编写规范的关键点：

- 1, 所有函数、变量名称，非特殊情况，一般使用小写字母；
- 2, 注释使用 Doxygen 风格，除屏蔽外，一律使用 “/* */” 的方式进行注释；
- 3, 代码统一使用 4 个空格进行缩进；
- 4, 每两个函数之间，一般有且只有一个空行；
- 5, 相对独立的程序块之间，使用一个空行隔开；
- 6, 全局变量的命名一般使用 “g_” 开头，全局指针变量的命名使用 “p_” 开头；
- 7, “if”、“for”、“while”、“do”、“case”、“switch”、“default” 等语句单独占一行，并且无论其有多少行执行语句，都加上 “{}”。

1.5 例程资源说明

M144Z-M4 最小系统板 STM32F407 版的配套资料中提供的标准例程多达 55 个，这些标准例程均是基于 ST 提供的 HAL 库进行编写的。提供的例程基本都是原创，并且拥有非常详细的注释，代码风格统一、内容循序渐进，非常适合初学者入门。

M144Z-M4 最小系统板 STM32F407 版的例程如下表所示：

编号	实验名称	编号	实验名称
1	跑马灯实验	18	内部温度传感器实验
2	按键输入实验	19-1	DAC 输出实验
3	外部中断实验	19-2	DAC 输出三角波实验
4	串口通信实验	19-3	DAC 输出正弦波实验
5	独立看门狗实验	20	IIC 实验
6	窗口看门狗实验	21	SPI 实验
7-1	基本定时器中断实验	22	触摸屏实验
8-1	通用定时器中断实验	23	FLASH 模拟 EEPROM 实验
8-2	通用定时器 PWM 输出实验	24	摄像头实验
8-3	通用定时器输入捕获实验	25	外部 SRAM 实验
8-4	通用定时器脉冲计数实验	26	内存管理实验
9-1	高级定时器输出指定个数 PWM 实验	27	SD 卡实验

9-2	高级定时器输出比较模式实验	28	FATFS 实验
9-3	高级定时器互补输出带死区控制实验	29	汉字显示实验
9-4	高级定时器 PWM 输入模式实验	30	图片显示实验
10	OLED 实验	31	照相机实验
11	TFTLCD (MCU 屏) 实验	32	视频播放器实验
12	USMART 调试实验	33-1	FPU 测试 (Julia 分形) 实验_开启硬件 FPU
13	RTC 实验	33-2	FPU 测试 (Julia 分形) 实验_关闭硬件 FPU
14	随机数发生器实验	34-1	DSP BasicMath 实验
15-1	PVD 电压监控实验	34-2	DSP FFT 实验
15-2	睡眠模式实验	35	手写识别实验
15-3	停止模式实验	36	T9 拼音输入法实验
15-4	待机模式实验	37	串口 IAP 实验
16	DMA 实验	38	USB 读卡器 (Slave) 实验
17-1	单通道 ADC 采集实验	39	USB 虚拟串口 (Slave) 实验
17-2	单通道 ADC 采集 (DMA 读取) 实验	40	FreeRTOS 移植实验
17-3	多通道 ADC 采集 (DMA 读取) 实验		

表 1.5.1 M144Z-M4 最小系统板 STM32F407 版例程表

从上表中可以看出，正点原子 M144Z-M4 最小系统板 STM32F407 版的例程基本上涵盖了 STM32F407ZGT6 芯片的所有片上资源，并且还扩展了很多有价值的例程，例如：USMART 调试实验、内存管理实验、T9 拼音输入法实验、手写识别实验、FreeRTOS 移植实验等。

并且按照上表的实验顺序编号可以看出，例程的安排都是循序渐进的，首先是最基础的跑马灯实验，然后一步一步地深入，从简单到复杂，这样更有利于读者学习和掌握，正也因此，正点原子的 M144Z-M4 最小系统板 STM32F407 版是非常适合初学者的。

1.6 学习资料查找

学习资料包括三个方面：

1, ST 官方资料

ST 官方 (<https://www.st.com/>) 在其官网上提供了 STM32 芯片相关的 IC 文档和软件，索引方式如下图所示 (若后续 ST 官网更新，索引方式可能会有差别)：

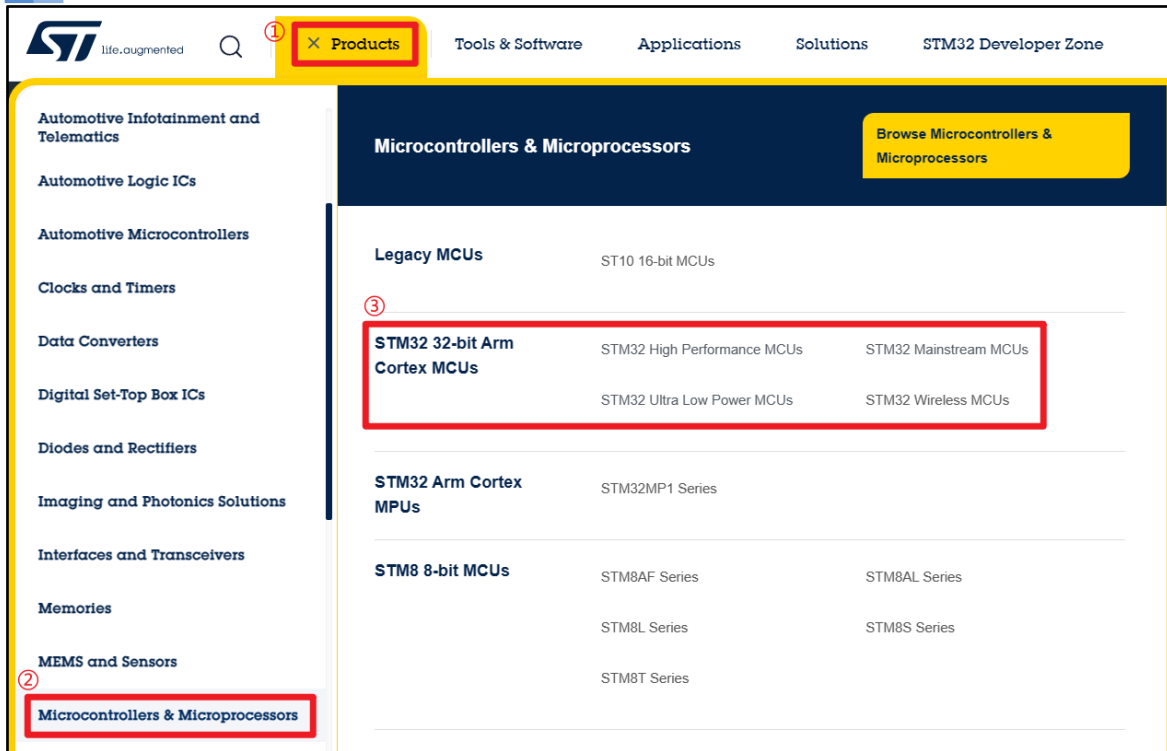


图 1.6.1 ST 官网微控制器芯片资料索引

随后根据芯片类型点击不同的标签，例如：STM32F1 系列对应“STM32 Mainstream MCUs”而 STM32F4、STM32F7、STM32H5 和 STM32H7 等则对应“STM32 High Performance MCUs”，随后点击页面中的“Product selector”便可看到该系列对应的所有芯片型号，如下图所示（以 STM32F103 系列为例）：

Overview Product selector Documentation CAD Resources Tools & Software										
Compare 29 total entries Export Bookmark Share Customize columns Expand filters										
Part Number Show Description	Status & Availability	Package	Core	Operating Frequency (MHz)	FPU	Co-Processor type	Co-Processor frequency max (MHz)	Flash Size (KB) Prog	Dual-bank Flash	Details
☐ STM32F103C4	ACTIVE	LQFP 48 7x7x1.4 mm	Arm Cortex-M3	72		-	-	16		
☐ STM32F103C6	ACTIVE	LQFP 48 7x7x1.4 mm, UFGFPN 48 7x7x0.55 mm	Arm Cortex-M3	72		-	-	32		
☐ STM32F103C8	ACTIVE	LQFP 48 7x7x1.4 mm	Arm Cortex-M3	72		-	-	64		
☐ STM32F103CB	ACTIVE	LQFP 48 7x7x1.4 mm, UFGFPN 48 7x7x0.55 mm	Arm Cortex-M3	72		-	-	128		
☐ STM32F103R4	ACTIVE	LQFP 64 10x10x1.4 mm, TFBGA 64 5x5x1.2 P 0.5 mm	Arm Cortex-M3	72		-	-	16		More details →

图 1.6.2 ST 官网 STM32F103 系列产品（部分截图）

点击芯片型号后便可根据指引进入芯片型号的产品页，在产品页中便可找到 ST 针对该型号芯片提供的所有资源，如下图所示（以 STM32F103VE 的数据手册为例）：

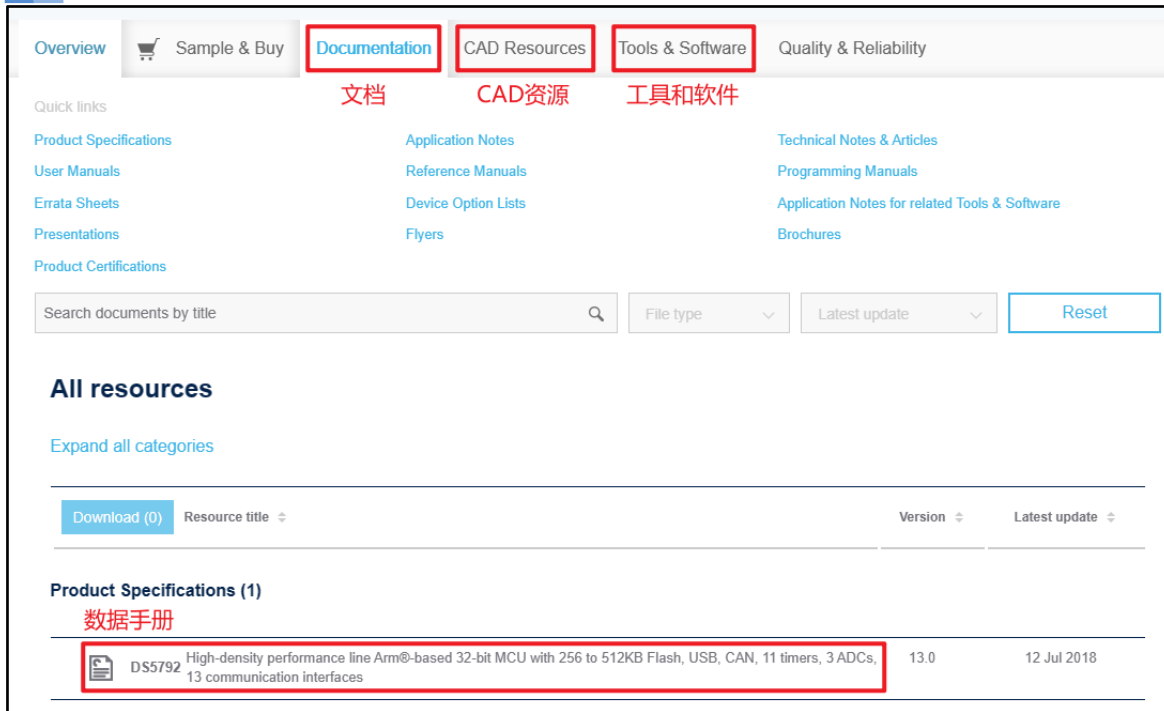


图 1.6.3 ST 官网 STM32F103VE 文档资源（部分截图）

2, 正点原子的学习资料

正点原子提供了大量的学习资料，为方便读者下载所有正点原子最新最全的学习资料，这些资料都放在正点原子文档中心（<http://www.openedv.com/docs/index.html>），如下图所示（正点原子文档中心会不时地更新，以保证为读者提供最新的学习资料）：



图 1.6.4 正点原子文档中心（部分截图）

在正点原子文档中心中，可以找到正点原子所有开发板、模块、产品等的详细资料下载链接。

3, 正点原子论坛

正点原子论坛（<http://www.openedv.com/forum.php>），即开源电子网，该论坛从 2010 年成立至今，已有十多年的时间，拥有数十万的注册用户和大量嵌入式相关的帖子，每天有数百人互动，是一个非常好的嵌入式学习交流平台。

第二章 STM32 简介

本章将为读者介绍 STM32 是什么？有什么资源？能做什么？以及如何选型等基础知识，让读者对 STM32 有一个基本的了解。

本章分为如下几个小节：

2.1 初识 STM32

2.2 STM32F407 资源简介

2.1 初识 STM32

STM32 是 ST（意法半导体）推出的基于 ARM Cortex®-M 内核 32 位通用 MCU，STM32 系列产片涵盖了高性能、实时性、数字信号处理、低功耗等多种应用场景，如下图所示（截取自 ST 官网，后续可能更新）：

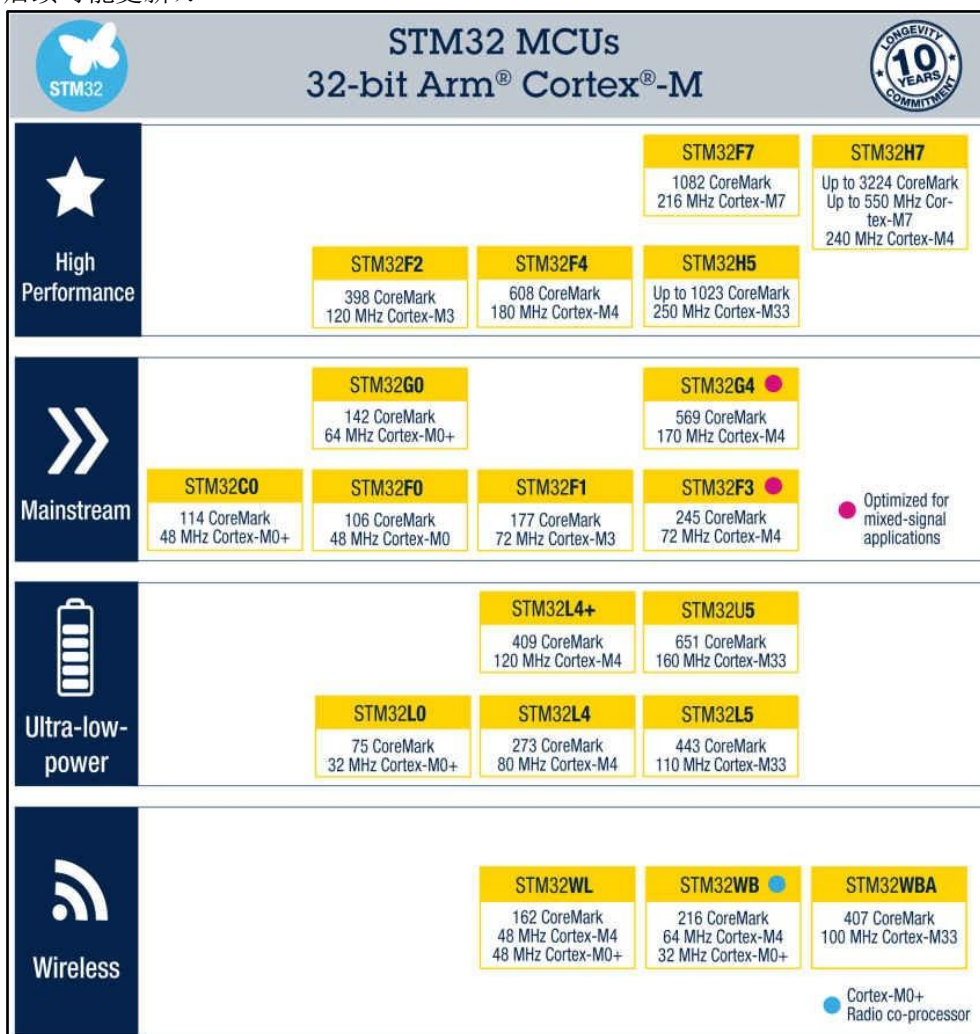


图 2.1.1 STM32 系列产品

2.2 STM32F407 资源简介

STM32F407 系列 MCU 下还分出了多款硬件规格不同的 MCU，正点原子 M144Z-M4 最小系统板 STM32F407 版使用的 MCU 芯片具体型号为 STM32F407ZGT6，该芯片的资源，如下表所示：

STM32F407ZGT6 资源					
内核	Cortex-M4	基本定时器	2	USART/UART	6
主频	168MHz	通用定时器	10	I2C	3
Flash	1024KB	高级定时器	2	SPI	3
SRAM	192+4KB	12 位 ADC	3	CAN	2
封装	LQFP144	12 位 DAC	1	USB OTG	2
IO 数量	114	ETH	1	SDIO	1
工作电压	1.8V~3.6V	DCMI	1	RTC	1

表 2.2.1 STM32F407ZGT6 内部资源表

上表也仅仅列出了 STM32F407ZGT6 内部常用的资源，但也可看出该 MCU 的内部资源是非常丰富的，本书会对 STM32F407ZGT6 内部的各个资源进行详细的使用介绍，并提供丰富的例程，供读者学习和参考。

关于 STM32F407ZGT6 更详细的芯片资源介绍，可以参考 ST 针对 STM32F407 系列 MCU 提供的数据手册（《stm32f407zg.pdf》），读者可以在 A 盘→8，STM32 参考资料中找到这份文档。

第三章 开发环境搭建

本章将介绍如何搭建 STM32 的开发环境。通过本章的学习，读者将了解到在进行 STM32 的开发过程中会涉及到的开发工具，例如 IDE、调试器、串口工具等。

本章分为如下几个小节：

- 3.1 常用开发工具简介
- 3.2 MDK 安装
- 3.3 仿真器驱动安装
- 3.4 USB 虚拟串口驱动安装

3.1 常用开发工具简介

在进行 STM32 开发的过程中需要使用到一些开发工具，例如 IDE、仿真器、串口调试助手等。常用的工具如下表所示：

工具	名称	说明
IDE	MDK	全名：MDK-ARM，是 Keil 公司（已被 ARM 收购）开发的一款 IDE，界面美观、简单易用，是 STM32 最常用的 IDE
	EWARM	IAR 公司开发的一款 IDE，支持 STM32，使用 EWARM 的人相对 MDK 少一些，习惯使用 IAR 的读者可以选择该软件开发 STM32
仿真器	CMSIS-DAP	ARM 公司开源的仿真器，支持 STM32 仿真调试，且自带虚拟串口功能。有高速和低速两个版本，具有免驱、速度快、价格便宜等优点
	ST-LINK	ST 公司开发的仿真器
	J-Link	Segger 公司开发的仿真器，支持 STM32 仿真调试，具有稳定、高速等优点，但价格昂贵
串口调试助手	ATK-XCOM	正点原子开发的串口调试助手，具有稳定、多功能、使用简单等优点
	SSCOM	大虾丁丁开发的串口调试助手，具有稳定、小巧、使用简单等优点

表 3.1.1 常用开发工具

读者可以根据自己的需求和喜好，选个合适的开发工具。上表中工具名称加粗的开发工具为本书推荐的 STM32 开发工具，即 IDE 推荐使用 MDK、仿真器推荐使用 CMSIS-DAP、串口调试助手推荐使用 ATK-XCOM，接下来介绍如何安装这些开发工具及其相关的软件驱动。

3.2 MDK 安装

注意：MDK 作为免费软件使用时，仅能用于非商业用途，若要商用，请读者自行联系 Keil 公司进行购买，本书使用 MDK 仅用于教学用途。

MDK 的安装分为两个步骤：1，安装 MDK 软件本体；2，安装对应芯片的设备包。

MDK 安装过程中涉及的安装包可以在 A 盘→6，软件资料→1，软件→MDK 中找到，如下图所示（如有压缩包，请解压后查看，且后续若软件更新，则提供的软件可能非最新版本）：

```
6, 软件资料/1, 软件/MDK/
|-- ARMCompiler_506_Windows_x86_b960.zip
|-- Keil.STM32F4xx_DFP.2.17.0.pack
|-- MDK536.EXE
'-- MDK538a.EXE
```

图 3.2.1 MDK 软件及 STM32F4 设备包

当然，读者也可以自行到软件提供方的官网自行下载，其中 MDK 软件本体由 Keil 提供，下载链接为 <https://www.keil.com/download/product/>，截止至本书编写的时候，MDK 的最新版本为 5.38a（MDK538a.EXE），由于 MDK 从 5.37 版本开始，不再随软件附带 Arm Compiler 5（AC5）编译器，并且后续进行 STM32 开发的时候需要使用 AC5，因此安装 5.37 及以后版本的 MDK 需

要读者再手动安装 AC5，AC5 安装包（ARMCompile_506_Windows_x86_b960.zip）由 ARM 提供，下载链接为 <https://developer.arm.com/downloads/view/ACOMP5>（可能需要注册并登入账号），当然读者也可以使用附带的 AC5 的 5.36 版本（MDK536.EXE）。设备包（Keil.STM32F4xx_DFP.2.17.0.pack）是由也是由 MDK 官方提供的，下载链接为 <https://www.keil.com/pack/>。

MDK 5.36 与 MDK 5.38a 的安装过程类似，本书以 MDK5.38a 为例展示如何安装 MDK 软件本体，首先打开 MDKxxx.EXE 安装包，接着一直点击“Next”按钮即可完成安装（需要同意许可协议，安装路径保持默认即可），安装完成后就可以在电脑桌面看到 MDK 的软件图标，如下图所示：



图 3.2.2 MDK 软件图标

MDK 软件本体安装好之后，还需要安装对应芯片的设备包，设备包的安装也很简单，仅需打开设备包安装程序，一路根据提示点击“Next”按钮即可完成安装，若读者后续要使用 MDK 软件基于其他芯片进行开发，也仅需安装对应芯片的设备包即可。

至此 MDK 5.36 软件安装完成，若读者安装 MDK 5.38a，则还需进行以下步骤安装 AC5。

首先解压 AC5 安装包，然后打开安装包中的“setup.exe”可执行程序，其路径如下图所示：

```
ARMCompiler_506_Windows_x86_b960/Installer/  
|-- data  
'-- setup.exe
```

图 3.2.3 AC5 安装程序路径

打开 AC5 的安装程序后，也是一样在同意许可协议后一路点击“Next”按钮，但务必要记住 AC5 的安装路径，如下图所示：

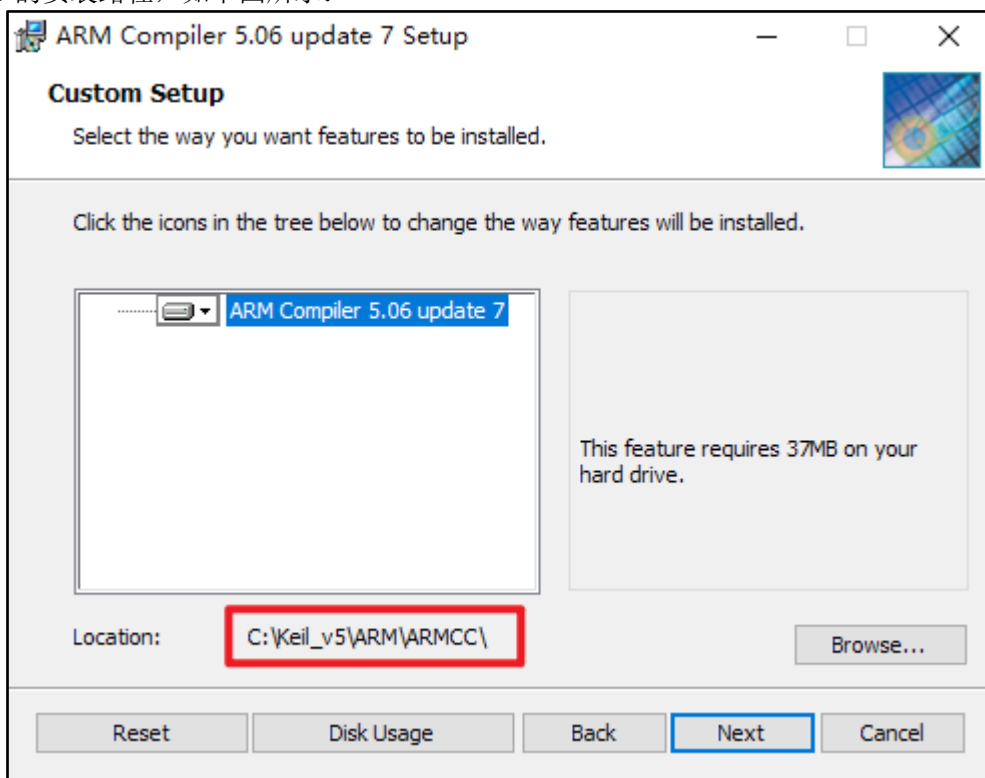


图 3.2.4 AC5 安装路径

如上图，AC5 将被安装在 MDK 默认的安装路径里的“ARMCC”文件夹中，虽然这个路径可以是任意的，但是务必要记下这个路径。

安装完成 AC5 后，最后还要配置 MDK，使得 MDK 能够找到并使用 AC5 编译器。由于 MDK

在没有打开工程的状态下无法配置编译器，因此需要打开任意一个工程，读者可以在标准例程文件夹（A 盘→4，程序源码→标准例程）中任意打开一个 MDK 工程，本书就以“跑马灯实验”为例，进入到标准例程→实验 1 跑马灯实验→Projects→MDK-ARM，可以看到该目录下有一个文件后缀为“uvprojx”的文件，该文件就是 MDK 的工程文件，如下图所示：

```
4, 程序源码/标准例程/实验1 跑马灯实验/Projects/MDK-ARM/
|-- atk_f407.uvoptx
|-- atk_f407.uvprojx
```

图 3.2.5 MDK 工程文件

找到 MDK 工程文件后，使用 MDK 打开该文件，打开后如下图所示：

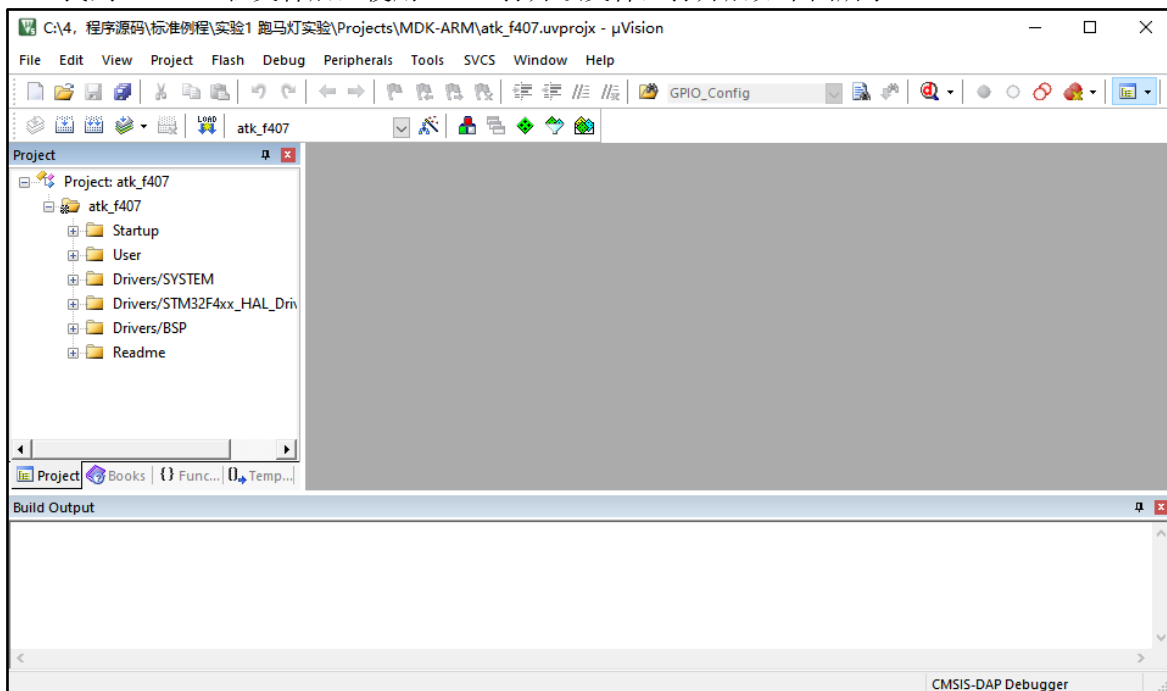


图 3.2.6 打开 MDK 工程

随后根据下图指示，打开工程项目管理窗口，如下图所示：

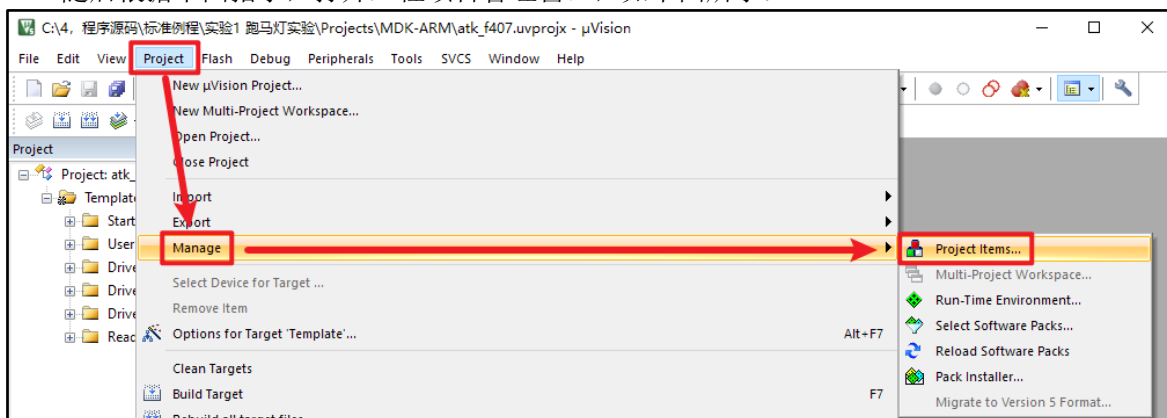


图 3.2.7 打开工程项目管理窗口

接着再根据下图指示，结合 AC5 的安装路径设置 MDK 使用的 AC5 编译器，如下图所示：

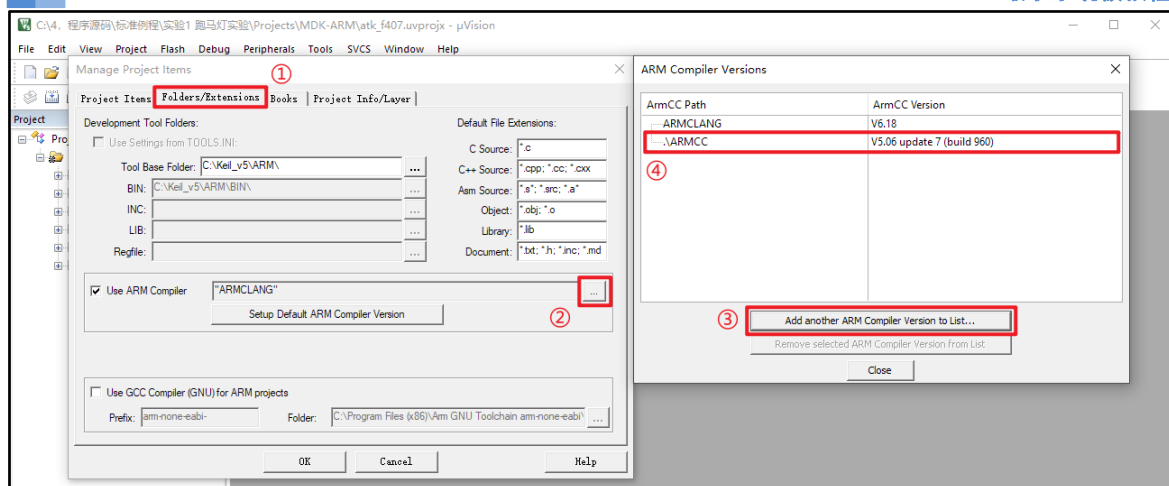


图 3.2.8 设置 MDK 使用 AC5 编译器

设置完成之后，点击工程项目管理窗口底部的“OK”按钮，关闭窗口即可，至此 MDK 安装完成。

3.3 仿真器驱动安装

STM32 可通过 CMSIS-DAP、J-Link 等仿真调试器进行程序的下载和仿真调试，本书推荐使用 CMSIS-DAP 仿真器，该仿真器在 Windows 操作系统下是免驱的，即无需安装驱动程序，即插即用，非常方便。

正点原子开发了两种规格的 CMSIS-DAP 仿真器：DAP 仿真器【高速版】ATK-HSDAP 和 DAP 仿真器【普速版】ATK-DAP，两款仿真器在使用上完全一样，只是高速版的速度更快，读者可根据实际需求进行选择。

3.4 USB 虚拟串口驱动安装

在进行 STM32 开发的过程中，会经常使用串口输出一些调试信息方便调试程序，这些调试信息可通过 PC 端的串口调试助手查看，不过 STM32 的串口并不能直接与 PC 连接。正点原子 STM32F407 最小系统板板载了 CH340C 芯片，该芯片能够将 STM32 的串口信号与 PC 端的 USB 信号进行转换，再配合 PC 端相关的软件驱动，能够实现在 PC 端虚拟出一个串口，使用该虚拟串口就能够与 STM32 的串口进行通信了。

因此接下来需要在 PC 端安装 CH340C 的驱动，该驱动由 CH340C 的官方沁恒提供，读者可前往沁恒的官网（<https://www.wch.cn/>）下载 CH340C 的驱动，也可在 A 盘→6，软件资料→1，软件→CH340 驱动文件夹下找到 CH340C 的驱动安装程序，如下图所示：

6， 软件资料/1， 软件/CH340驱动/
-- CH341SER.EXE

图 3.4.1 CH340C 驱动安装程序

打开 CH340C 驱动安装程序后，点击安装程序中的“安装”按钮，若提示“驱动安装成功”，则说明 CH340C 驱动安装成功，如下图所示：

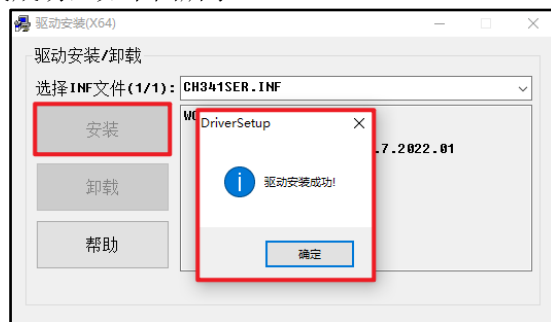


图 3.4.2 CH340C 驱动安装成功

成功安装 CH340C 驱动后，将正点原子 STM32F407 最小系统板的 USB_UART 接口通过具有数据传输功能的 USB 数据线与 PC 端的 USB 端口进行连接，随后便可在 PC 端的设备管理器中查看到 CH340C 虚拟出的串口，如下图所示：

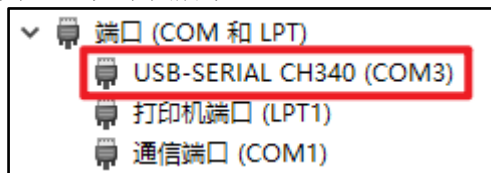


图 3.4.3 PC 端显示的虚拟串口

从上图中可以看到，CH340C 虚拟出的串口被 PC 分配了 COM3 的端口号，该端口号用于串口调试助手等上位机确定与之通信的串口端，值得一提的是，当 CH340C 与不同的 PC，甚至是与同一台 PC 的不同 USB 端口进行连接后，虚拟出的串口被 PC 分配到的端口号都有可能是不一样的，如 COM4、COM5 等，读者可以根据设备管理器中端口设备的名称来判断具体是哪一个端口号，如果同时连接了多个 CH340 系列的芯片，那么只能挨个测试了。

安装完 USB 虚拟串口驱动后，就能够使用串口调试助手，如正点原子开发的 ATK-XCOM 软件和板卡通过串口进行通信了。

第四章 STM32 初体验

本章并不涉及程序代码的编写，而是介绍如何编译工程、烧录程序以及进行程序的调试仿真，让读者体验 STM32 的开发流程，并会介绍一些 MDK 的使用技巧。通过本章的学习，读者将对 STM32 的开发流程及 MDK 的使用方法有一个大概的了解，为后续的深入学习打好基础。

本章分为如下几个小节：

- 4.1 使用 MDK 编译实验例程
- 4.2 使用串口烧录程序
- 4.3 使用 DAP 烧录及调试程序
- 4.4 MDK 使用技巧

4.1 使用 MDK 编译实验例程

在编写完程序代码后，需要对程序进行编译，只有在程序编译成功后，才能将编译出的二进制文件烧录至芯片中运行以及进行仿真调试等操作。

本书配套提供的实验例程在 A 盘→4，程序源码目录中，如下图所示：

4, 程序源码 /
`-- 标准例程

图 4.1.1 配套提供的实验例程源码

在上图“标准例程”文件夹中，就包含了基于 ST 提供的 HAL 库编写的各个实验例程，如下图所示：

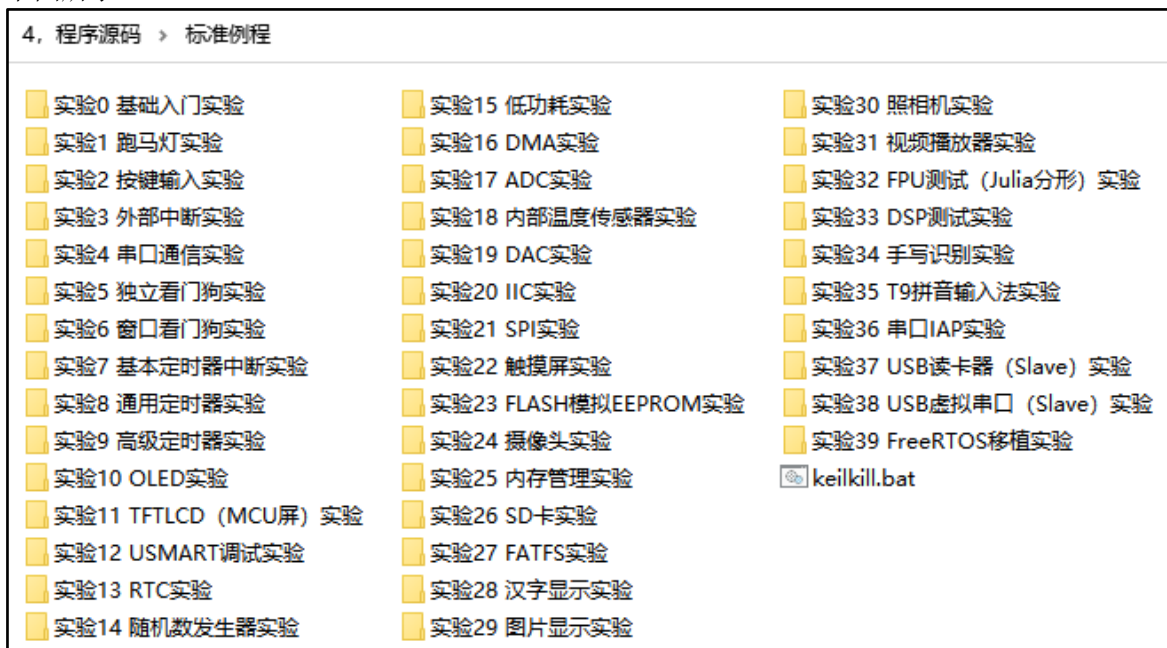


图 4.1.2 标准例程

从上图中可以看到，除了“实验 0 基础入门实验”是用于创建 MDK 工程等 MDK 基础使用例程外，共有 39 个实验，其中部分实验内有多多个实验例程，总的实验例程多大 54 个。

秉承着简单易懂的原则，本章以跑马灯实验例程（“实验 1 跑马灯实验”）作为示例例程，例程的根目录如下图所示：

```
实验1 跑马灯实验/
|-- Drivers
|-- Middlewares
|-- Output
|-- Projects
|-- User
|-- keilkill.bat
`-- readme.txt
```

图 4.1.3 跑马灯实验例程根目录

所有实验例程的目录结构都与上图中的目录结构类似，这会在后续创建 MDK 工程相关的章节中进行介绍（其实光看文件夹的名称，就能知道文件夹大概的作用了）。

接下来打开 MDK 工程文件，其路径为 Projects→MDK-ARM→atk_f407.uvprojx，如下图所示：

```
实验1 跑马灯实验/Projects/MDK-ARM/
|-- atk_f407.uvoptx
`-- atk_f407.uvprojx
```

图 4.1.4 跑马灯实验例程 MDK 工程文件

注意：打开 MDK 工程文件的前提是正确地安装了 MDK 软件。

使用 MDK 打开跑马灯的 MDK 工程文件后，如下图所示：

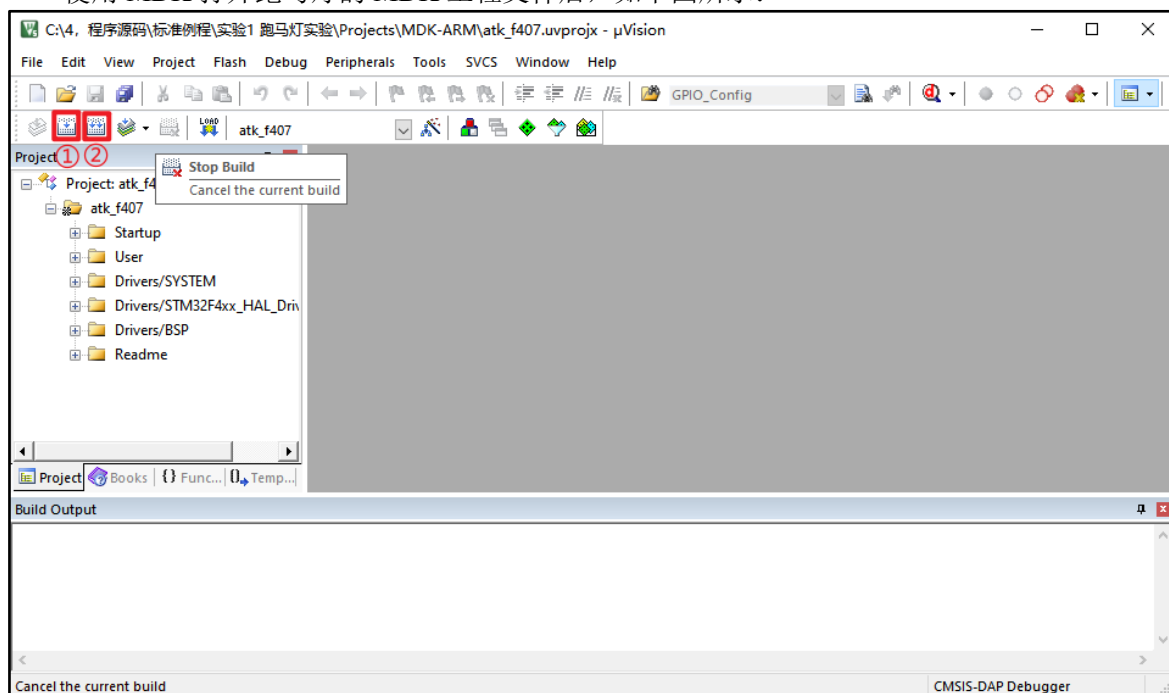


图 4.1.5 使用 MDK 软件打开跑马灯 MDK 工程

如上图所示，在 MDK 软件的左上方可以看到两个快捷按钮，这两个按钮都是用于编译工程的，区别如下：

①：Build 按钮，快捷键 F7，该按钮用于编译工程，并且该按钮比较“聪明”，使用该按钮编译工程时，只会编译在上次编译后被改动或需要重新编译的文件，并不会重新地编译整个工程，从而可以大大地缩短编译时间，推荐使用。

②：Rebuild 按钮，无快捷键，该按钮会重新地编译整个工程，因此整体上会比 Build 按钮耗时。

虽然 Build 按钮和 Rebuild 按钮存在一些差别，但是对于没有编译过的工程，这两个按钮的作用都是一样的，因此接下来按下任意一个编译按钮或直接点击快捷键 F7 编译跑马灯工程，编译完成后，MDK 软件底部的 Build Output 窗口会输出编译信息，如下图所示：

```
Build Output
compiling apm32f4xx_tmr.c...
linking...
Program Size: Code=3376 RO-data=436 RW-data=12 ZI-data=1836
FromELF: creating hex file...
\"..\\..\\Output\\atk_f407.axf\" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:01
```

图 4.1.6 编译结果输出信息

从上图中，就可以查看到本次编译的许多信息，这里介绍输出信息的后四行，如下：

Code: 程序代码占用的空间大小（3376Bytes）；

RO-data: 只读数据（一般是 const 关键字修饰的常量）占用的空间大小（436Bytes）；

RW-data: 有初值（初值不为 0）的可读写数据（Flash 中存放初值，取出至 RAM 中被访问）占用的空间（同时占用 Flash 和 RAM）大小（12Bytes）；

ZI-data: 无初值（初值为 0）的可读写数据占用的空间大小（1836Bytes）；

从上面的信息就可以看出，在不考虑动态内存分配的情况下，编写的程序将占用 3824Bytes（Code+RO-data+RW-data）的 Flash 空间和 1848Bytes（RW-data+ZI-data）的 RAM 空间；

创建了 Hex 文件（能够被单片机执行的文件，可直接烧录至单片机内部的 Flash 执行，使用串口烧录程序时使用）；

创建了 AXF 文件（不仅包含了能够被单片机执行的文件，还附加了其他的调试信息，能够进行仿真调试都靠它，使用调试器烧录程序时使用）；

编译结果有 0 个错误和 0 个警告；

编译耗时为 1 秒钟。

实验例程编译出的文件默认会输出在 Output 文件夹下，如下图所示：

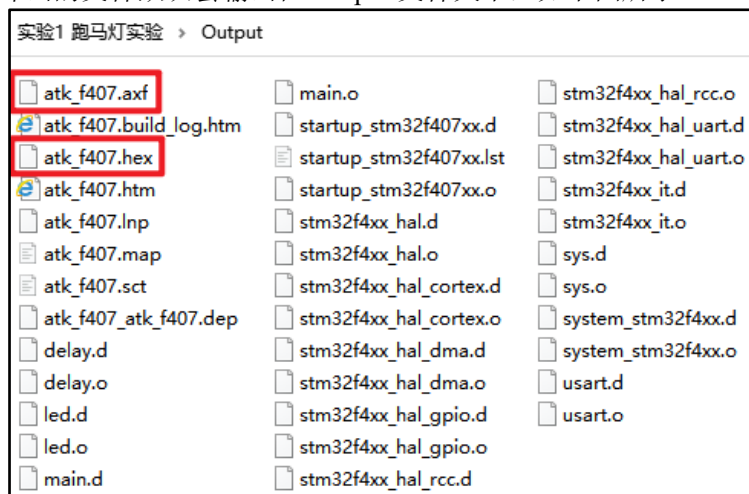


图 4.1.7 编译文件和编译过程产生的中间文件

从上图中可以看到，编译成功后，不仅生成了 Hex 和 AXF 文件，还生成了许多中间文件，对此，本书将在后续的 MAP 文件分析时进行介绍。

至此就完成了实验例程的编译，对于其他的实验例程，读者可以用同样的方式进行编译。

4.2 使用串口烧录程序

STM32 烧录程序有三种常见的方式，如下表所示：

烧录方式	使用接口	下载条件	优缺点
串口	串口 1 PA9/PA10	进入芯片内嵌的 BootLoader（上电时，BOOT1 引脚接 3.3V, BOOT0 引脚接 GND）	优点：仅需通过串口，即可通过上位机软件烧录程序，成本低 缺点：烧录速度慢，无法进行程序在线调试
SWD	SWD 口	直接下载	优点：仅需 2 个 IO 口就能进行程

	PA13/PA14		序烧录、仿真和在线调试，高速、高效、实用 缺点：需要额外的仿真器
JTAG	JTAG 口 PA13/PA14/PA15 PB3/(PB4)	直接下载	优点：能进行程序烧录、仿真和在线调试，高速、方便、实用 缺点：需要额外的仿真器，且占用 4~5 个 IO 口

表 4.2.1 STM32 常见程序烧录方式对比

通过上表对比，推荐读者在没有仿真器的情况下使用串口方式烧录程序，若有仿真器则使用 SWD 的方式烧录程序，同时也强烈建议读者购买一个仿真器（如 CMSIS-DAP 仿真器），可以极大的方便学习和开发。

上表中 BOOT0 和 BOOT1 引脚是 STM32F407 上的两个引脚，这两个引脚可以用于配置 STM32F407 上电或复位时的启动模式，具体如下表所示：

BOOT0	BOOT1	启动模式	访问方式
0	X	主闪存存储器（Flash）	Flash（0x08000000）被映射到启动空间（0x00000000），可以通过这两个地址访问到 Flash
1	0	系统存储器	系统存储器（0x1FFF0000）被映射到启动空间，可以通过这两个地址访问到系统存储器
1	1	内置 SRAM	内置 SRAM（0x20000000）被映射到启动空间

表 4.2.2 STM32F407 启动模式配置

从上表中可以看出，通过设置 BOOT[1:0]引脚，可以配置 STM32F407 从不同的地址运行程序，其中 Flash 中的程序就是用户烧录至芯片的用户程序，系统存储器中的程序是芯片厂家在芯片出厂前被固化（不可修改）到芯片内部的程序，也就是芯片内嵌的 BootLoader 程序。

通过芯片内嵌的 BootLoader 程序，用户可以非常轻松地将用户程序烧录到 Flash 中，即使某些在使用 Flash 启动时无法烧录程序的情况，例如，Flash 中的用户程序关闭了 SWJ-DP（Serial Wire/JTAG Debug Poet，串行/JTAG 调试端口）的情况。芯片内嵌 BootLoader 提供了多种烧录用户程序至 Flash 的接口，如：USART1、USART3、CAN2、USB OTG_FS 从设备模式，当然，也可以在 BootLoader 程序运行过程中通过仿真器烧录程序至 Flash。

本章就介绍如何使用串口 1（USART1）实现 STM32F407 的程序烧录。

从前文本小节的介绍中可知，若要使用串口 1 烧录程序，则需让 STM32F407 运行在芯片内嵌的 BootLoader 程序中，而进入芯片内嵌 BootLoader 的方法为：在芯片上电或复位时将 BOOT0 引脚置为高电平、BOOT1 引脚置为低电平，进入芯片内嵌 BootLoader 后就能够使用相关的上位机软件将事先编译出的 Hex 文件烧录置 Flash 中，烧录成功之后，还需要将 BOOT0 引脚置为低电平，然后重新上电或复位芯片，芯片才能运行烧录的用户程序，在这整个过程中，至少需要手动改变两次 BOOT0 引脚的电平和进行两次复位，比较繁琐。为此，STM32F407 最小系统板利用串口的 DTR 和 RST 信号，设计了一键下载电路。

一键下载电路使用串口的 DTR 信号控制 STM32F407 的 BOOT0 引脚，使用串口的 RST 信号控制 STM32F407 的 NRST 引脚（芯片复位引脚），再配合上位机软件（ATK-XISP），设置“DTR 的低电平复位，RST 高电平进 BootLoader”，这样就能够由软件自动控制 STM32F407 芯片的 BOOT0 引脚和复位，从而实现一键下载。一键下载电路的原理图级介绍，请参考《STM32F407 最小系统板硬件参考手册.pdf》。

在使用串口烧录程序之前，需要将芯片通过串口和 PC 进行连接，除了按照 3.4 小节中事先安装好 USB 虚拟串口的驱动并将板卡与 PC 进行连接，还需保证板卡上的 PA9 和 PA10 端子通过跳线帽分别与 RX 和 TX 端子进行连接，以保证板载 CH340C 芯片与 STM32F407ZGT6 MCU 通过串口相连接，如下图所示：

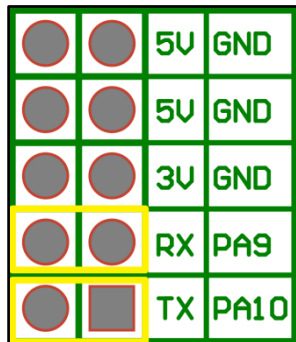


图 4.2.1 板卡设置

使用一键下载电路烧录程序的时候，需要使用到的上位机软件为 ATK-XISP，读者可以前往正点原子资料下载中心下载该软件，也可以在 A 盘→6，软件资料→1，软件→ATK-XISP 中找到该软件，如下图所示：

6, 软件资料/1, 软件/ATK-XISP/
--- ATK-XISP.exe

图 4.2.2 ATK-XISP 软件

打开 ATK-XISP 软件后，需根据实际情况进行一些配置，如下图所示：



图 4.2.3 ATK-XISP 配置

①：先在“串口”列表中根据实际情况选择连接至板卡的 COM 口，串口通信波特率可以选择最大值（若后续程序烧录失败，再适当降低串口通信波特率）；

②：选择 4.1 小节编译出的 Hex 文件；

③：勾选“编程后运行”和“校验”复选框，这样可以保证烧录程序的正确性，并在程序烧录完成后自动运行程序，省去了手动复位的操作；

④：务必正确设置为“DTR 低电平复位 RST 高电平进 BootLoader”，以保证能够配合一键下载电路控制 STM32F407 进入 BootLoader 烧录程序并且控制器复位。

设置好 ATK-XISP 软件之后，就能够点击“开始编程”按钮，即可一键将程序烧录至 STM32F407，烧录成功后，如下图所示：



图 4.2.4 烧录成功

从上图中可以看到，ATK-XISP 软件会输出烧录过程中的相关操作信息，例如如何控制 STM32F407 进入 BootLoader、烧录了多大的程序至 Flash、耗时多久等信息。同时，也能看到提示“运行成功”，这就表明 STM32F407 已经在运行 Flash 中的用户程序了，此时便可以看到 STM32F407 最小系统板上的 LED0 和 LED1 闪烁。

至此就完成了跑马灯实验例程程序的烧录，对于其他的实验例程，读者也可以使用同样的方式使用串口进行程序烧录。

4.3 使用 DAP 烧录及调试程序

在需要调试程序代码的时候，最好使用仿真器进行程序的烧录和调试，本节就将介绍，如何使用正点原子 DAP 仿真器给 STM32 进行程序的烧录和调试，对于其他型号的仿真器，具体的操作步骤也都是类似的。

正点原子 DAP 仿真器与 M144Z-M4 最小系统板 STM32F407 版的连接步骤如下：

- ①：通过板卡 USB 接口为板卡供电，也可以连接至 PC；
- ②：通过 USB 接口将正点原子 DAP 仿真器与 PC 进行连接，若连接正常，DAP 仿真器的蓝灯常亮；
- ③：通过 4P 排线将正点原子 DAP 的 SWD 接口与板卡的 SWD 接口相连。

4.3.1 使用 DAP 烧录程序

以第 4.1 小节中的跑马灯实验例程为例，在 MDK 软件界面下，点击按钮（快捷键 Alt+F7）打开“Options for Target”窗口，同时打开窗口顶部的“Debug”选项卡，如下图所示：

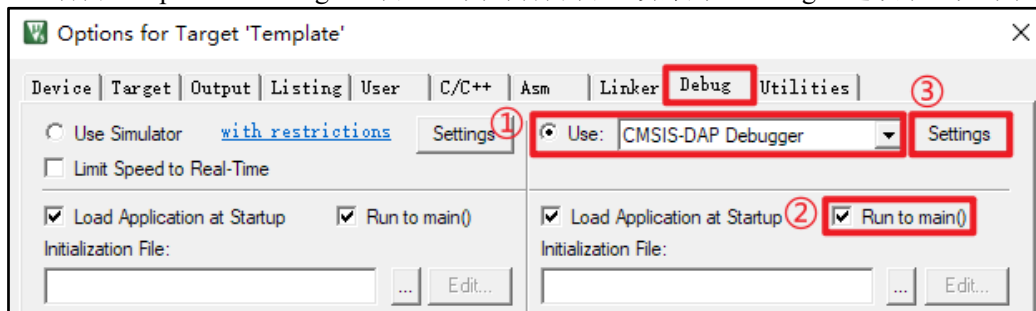


图 4.3.1.1 Debug 选项卡配置

如上图所示，打开“Debug”选项卡后，在①处根据使用的仿真器型号选择对应的调式工具，本章使用 DAP 仿真器，因此选择“CMSIS-DAP Debugger”，读者若使用其他仿真器，则对应选择调试工具即可。勾选②处的“Run to main()”复选框可以在开启调试后，程序自动运行到 main() 函数，否则程序会从启动文件中的“Reset_Handler”（复位异常）标号开始运行，读者可根据实际情况选择勾选或不勾选改复选框。接着再打开③处的“Settings”按钮，打开对应调试器的配置窗口，如下图所示：

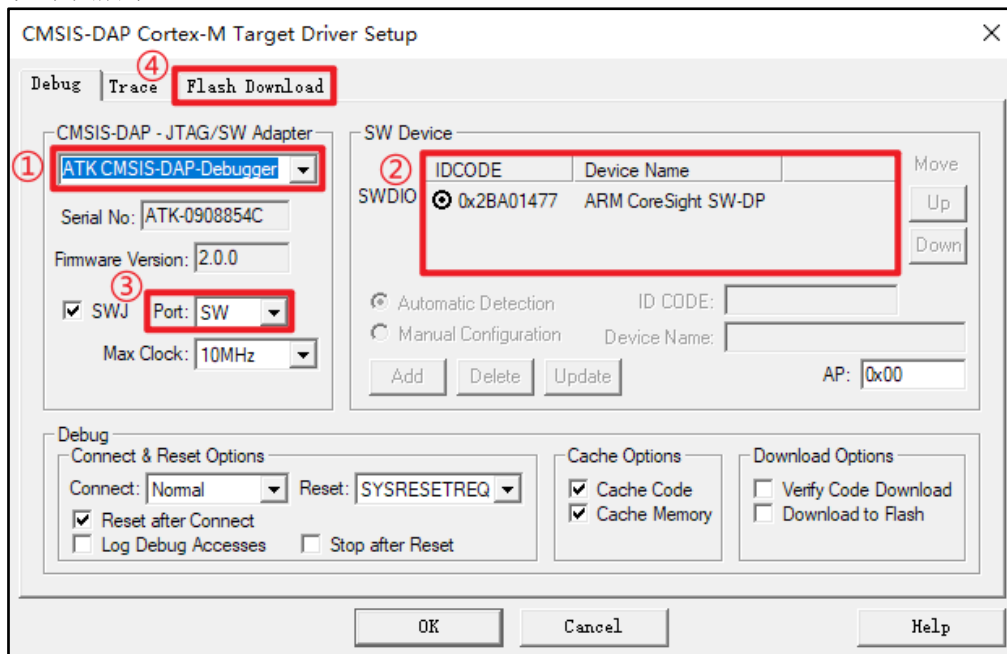


图 4.3.1.2 DAP 仿真器配置窗口

如上图所示，若 PC 上连接了多个 DAP 仿真器，则需要在①处选择对应的 DAP 仿真器，若 DAP 仿真器与板卡连接正常，则会在②处显示“IDCODE”和“Device Name”，若连接异常，则会显示“Error”，此时应该检查接线和供电。接着在③处的“Port”下拉框选择“SW”，该选项对应 4Pin 的 SWD 接口，另外一个选项为“JTAG”，其对应的是 20Pin 的 JTAG 接口。该窗口下的其他配置项保持默认，或根据实际情况进行配置。接着打开④处的“Flash Download”选项卡，如下图所示：

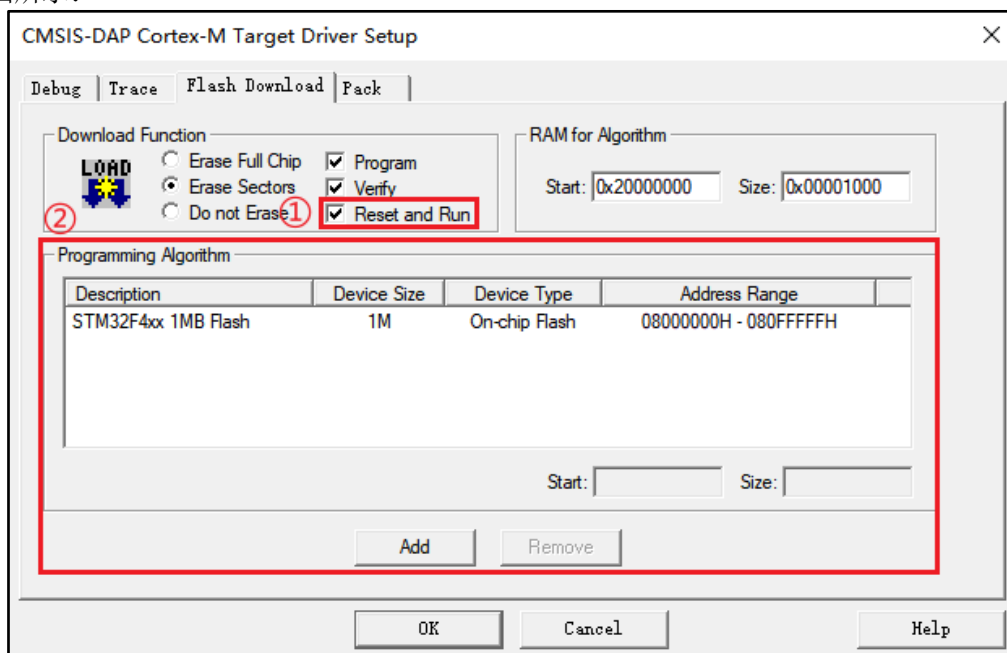


图 4.3.1.3 烧录选项配置

如上图所示，勾选①处的“Reset and Run”复选框，可以在烧录完成后自动完成复位并运行程序，若不勾选，则需要手动进行复位，程序才能运行。在②处可以选择烧录算法，程序代码一般是烧录到芯片内部的 Flash 中的，在烧录过程中，需要对芯片内部的 Flash 进行读写等操作，烧录算法就是用来告诉 MDK 软件，在烧录程序的时候应怎样读写芯片内部的 Flash。烧录算法一般有芯片厂商在芯片的设备包中提供，在安装设备包时，会一并安装烧录算法，在创建 MDK 工程时候，MDK 会自动选择芯片对应的烧录算法，若②处没有显示烧录算法，或烧录算法不符合实际使用的芯片，则需点击改窗口底部的“Add”按钮，添加对应的烧录算法。至此，使用 DAP 烧录程序的配置项均已配置完毕。

配置完成后，回到 MDK 软件主界面对工程进行编译，编译完成后，点击  按钮（快捷键 F8），随后 MDK 会自动将编译好的程序通过仿真器烧录到芯片上，同时会在“Build Output”窗口中提示烧录信息，如下图所示：

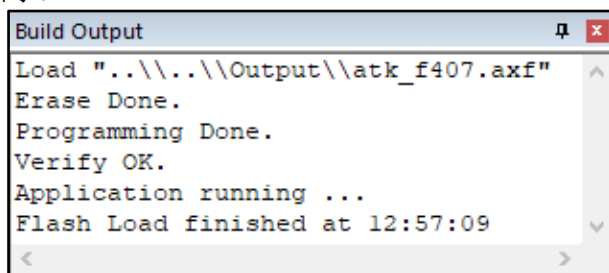


图 4.3.1.4 烧录结果输出信息

程序烧录完成后，就能看到 M144Z-M4 最小系统板 STM32F407 版上的 LED0 和 LED1 闪烁。

4.3.2 使用 DAP 调试程序

本小节依然以第 4.1 小节中的跑马灯实验例程为例，按照第 4.3.1 小节中的介绍配置好 MDK 软件中仿真器相关的配置并成功编译好程序后，点击  按钮（快捷键 Ctrl+F5）即可进入调试界面，进入调试界面前，若编译好的程序还未进行烧录操作，MDK 自会自动进行烧录操作，然后进入调试界面，调试界面如下图所示：

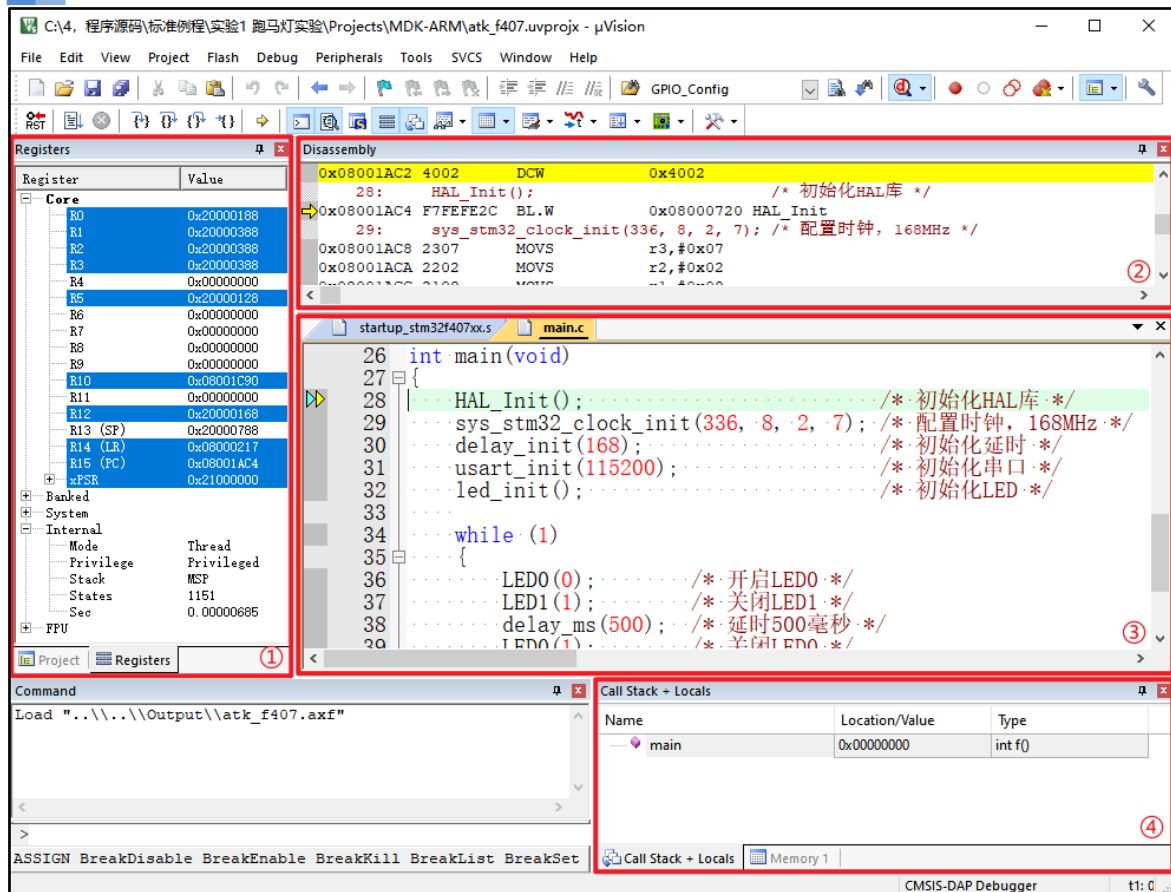


图 4.3.2.1 MDK 调试界面

上图中各个窗口的作用如所示：

①：Registers 窗口，该窗口显示了 Cortex-M4 内核寄存器 R0~R15 的值，并且还显示了内核当前的模式（Handler 模式或 Thread 模式）、当前使用的堆栈指针（MSP 或 PSP）等等一系列信息。Registers 窗口对查找 Bug 和掌握程序运行窗台有很大的帮助。

②：Disassembly 窗口，该窗口显示了反汇编代码和 C 语言代码的对比，包含了指令的保存地址、指令代码和指令等一系列信息，并且窗口的左侧有一个黄色的箭头用于指示程序当前将要执行的位置。Disassembly 窗口对查找 Bug 和优化程序有很大的帮助。

③：文本窗口，该窗口主要用于展示代码（C 代码或汇编代码），窗口左侧有一个青色和一个黄色的箭头，其中青色箭头用于指示当前光标所在的行，黄色箭头用于指示程序当前将要执行的位置。

④：Call Stack + Locals 窗口，该窗口用于显示当前函数的调用关系和函数局部变量的信息。该窗口在调试代码的候时非常有用的。

在调试界面中，还有一系列非常有用的工具，这些工具可通过 Debug 工具栏上的快捷按钮进行访问，Debug 工具栏，如下图所示：

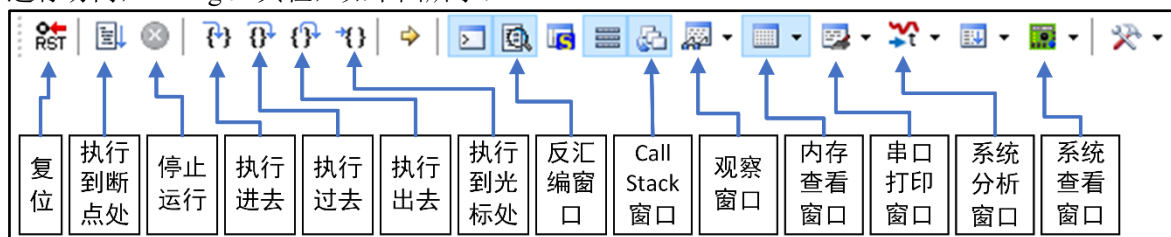


图 4.3.2.2 Debug 工具栏

复位：其功能等同于硬件上按复位按钮。按下该按钮之后，代码会重新从头开始执行（跳转到复位异常服务函数）。

执行到断点处：该按钮用来快速执行到断点处，有时候并不需要观看每步是怎么执行的，而

是想快速的执行到程序的某个地方看结果，这个按钮就可以实现这样的功能，但前提是在需要查看的地方设置了断点，否则程序将全速运行。

停止运行：该按钮在程序执行过程中变为有效，按下该按钮，可以使程序停止运行。

执行进去：该按钮用于跳转进当前将要执行的函数中单步执行。

执行过去：该按钮用于单步执行过当前将要执行的函数，而不跳转进函数里面。

执行出去：该按钮用于执行出当前函数，跳转到函数返回后的位置。

执行到光标处：该按钮用于执行到光标所在的位置，相当于在光标处打了一个临时的断点，当然，如果在执行到光标处前遇到了断点，程序也是会停止运行的。

Disassembly 窗口：该按钮用于打开或关闭 Disassembly 窗口。

Call Stack 窗口：该按钮用于打开或关闭 Call Stack + Locals 窗口。

Watch 窗口：该按钮用于打开或关闭 Watch 窗口，MDK 提供了两个 Watch 窗口，在调试界面，可以将想要观察的变量或表达式添加到 Watch 窗口中，便可以实时地查看变量或表达式的值等信息。

Memory 窗口：该按钮用于打开或关闭 Memory 窗口，MDK 提供了四个 Memory 窗口，在 Memory 窗口中可以查看一段指定起始地址内存的值。

Serial 窗口：该按钮用于打开或关闭 Serial 窗口，MDK 提供了四个 Serial 窗口，通过前三个 Serial 窗口可以查看 MCU 串口输出的数据，通过最后一个 Serial 窗口可以查看 MDK 提供的 Event Recorder 等工具标准输出的数据。

系统分析窗口：该按钮可以使用 MDK 提供的一些工具来查看系统运行的状态，不过部分功能对仿真器有一定要求。

系统查看窗口：该按钮可以用来查看 MCU 各个外设的寄存器值。

以上就介绍了 Debug 工具栏中比较常用的几个快捷按钮，这些快捷按钮对调试程序是很有帮助的。接下来就实际地演示一下简单的调试。

首先打开 Watch 1 窗口，并双击“Enter expression”添加一个变量名为“g_fac_us”的表达式，该变量是在 delay.c 中定义的全局变量，主要用于微秒级延时，此时可以看到这个变量的值显示为“<cannot evaluate>”，意思是无法计算，这时可以单击 main() 函数中调用的 delay_init() 函数（第 30 行）左侧灰色的部分来添加断点，如下图所示：

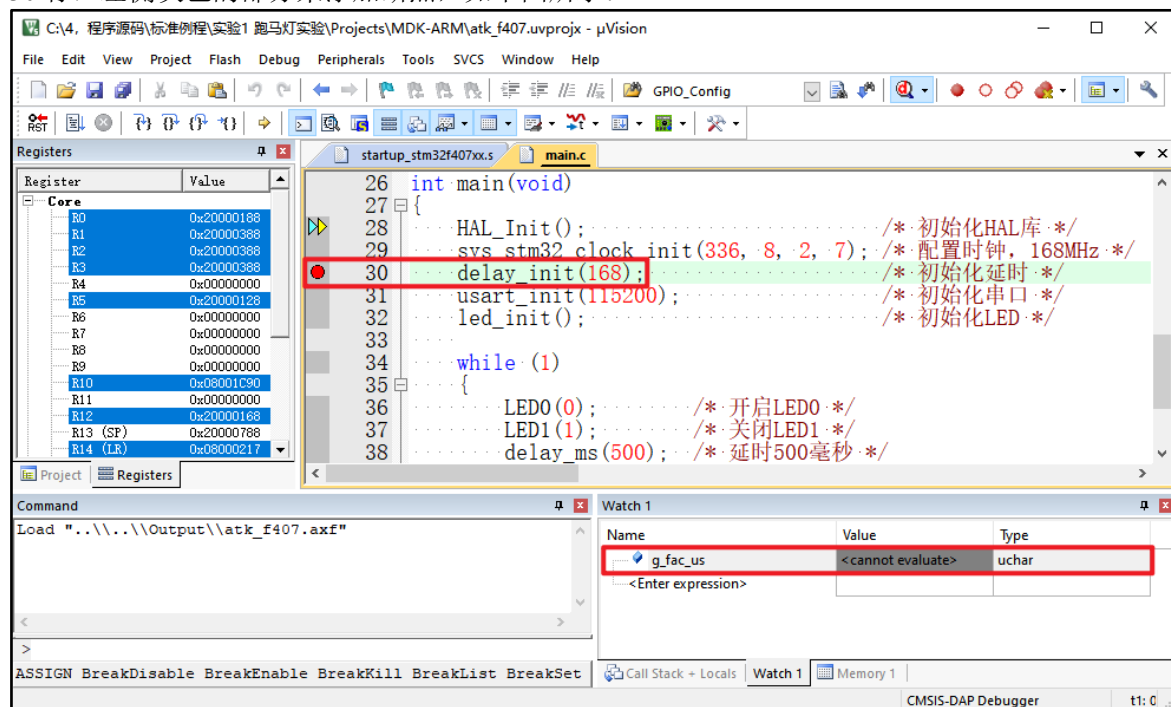


图 4.3.2.3 添加观察表达式和断点

随后点击 Debug 工具栏中的  按钮（快捷键 F5），即可运行到 delay_init() 函数调用前（断点处），然后点击 Debug 工具栏中的  按钮（快捷键 F11），即可跳转到 delay_init() 函数中，如下图

所示:

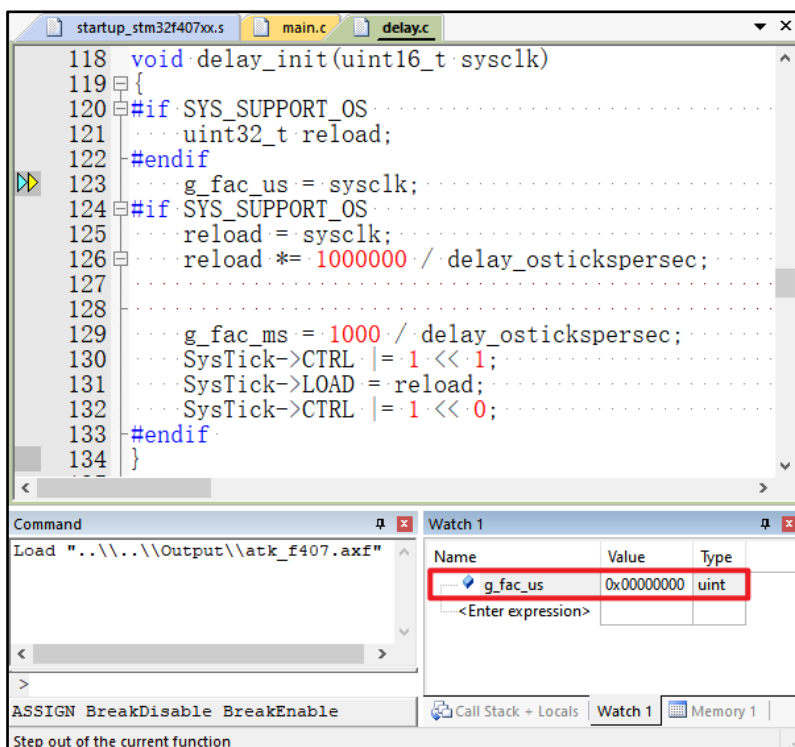


图 4.3.2.4 执行进入 delay_init()函数

因此函数 `delay_init()` 会对变量 `g_fac_us` 进行初始化, 因此可以看到 Watch 1 窗口中的表达式已经显示了具体的值, 接下来可以连续单击 Debug 工具栏上的  按钮 (快捷键 F10), 让程序单步执行到变量 `g_fac_us` 被赋值的地方 (大概第 123 行), 此时再次单击 Debug 工具栏上的  按钮, 随后便可在 Watch 1 窗口中看到变量 `g_fac_us` 被赋值后的结果, 也可将鼠标光标放置在变量 `g_fac_us` 上停留一会, MDK 会自动的提示该变量当前的值, 如下图所示:

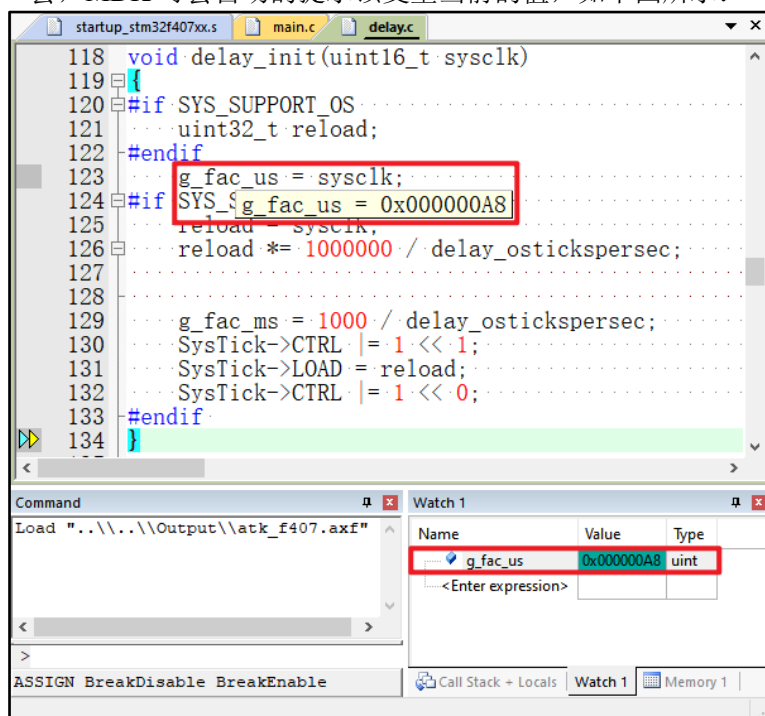


图 4.3.2.5 观察变量或表达式的值

接下来可以单击 Debug 工具栏上的  按钮 (快捷键 Ctrl+F11), 跳出 `delay_init()` 函数回到

main()函数，然后在 main()函数中 while 循环的第一个“LED0(0)”语句前打上断点单击 Debug 工具栏上的  按钮，让程序执行到该断点处，如下图所示：

```

27 {
28     HAL_Init(); /* 初始化HAL库 */
29     sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
30     delay_init(168); /* 初始化延时 */
31     usart_init(115200); /* 初始化串口 */
32     led_init(); /* 初始化LED */
33
34     while (1)
35     {
36         LED0(0); /* 开启LED0 */
37         LED1(1); /* 关闭LED1 */
38         delay_ms(500); /* 延时500毫秒 */
39         LED0(1); /* 关闭LED0 */
40         LED1(0); /* 开启LED1 */
41         delay_ms(500); /* 延时500毫秒 */
42     }
43 }

```

图 4.3.2.6 执行到“LED0(0)”语句处

此时，不断单击 Debug 工具栏上的  按钮，便可以看到板卡上的 LED0 和 LED1 依次闪烁（执行到 LED0()语句和 LED1()语句时，需多次单击  按钮才能跳到下一行并看到对应的板载 LED 状态发生变化，这是因为 LED0()语句和 LED1()语句是两个宏定义，这两个宏定义由多条语句组成，因此需要多次单击  按钮才能执行到下一行）。

此时，单击“LED0(0)”语句前的断点，将其取消掉，然后单击 Debug 工具栏上的  按钮，使程序全速执行，则可以看到板载 LED 自动地交替闪烁，随后打开 delay.c 文件并定位到 delay_us 函数（大概 144 行），并在该函数左侧任意有灰色的地方打上断点，可以看到程序马上停在了断点的位置，如下图所示：

```

143 void delay_us(uint32_t nus)
144 {
145     uint32_t ticks;
146     uint32_t told, tnow, tcnt = 0;
147     uint32_t reload = SysTick->LOAD; /* LOAD的值 */
148     ticks = nus * g_fac_us; /* 需要的节拍数 */
149
150     #if SYS_SUPPORT_OS /* 如果需要支持OS */
151     delay_osschedlock(); /* 锁定OS的任务调度器 */
152     #endif
153
154     told = SysTick->VAL; /* 刚进入时的计数器值 */
155     while (1)
156     {
157         tnow = SysTick->VAL;
158         if (tnow != told)
159         {

```

图 4.3.2.7 执行进入 delay_us()函数

此时，便可在 Call Stack + Locals 窗口查看到函数的调用情况，如下图所示：

Call Stack + Locals		
Name	Location/Value	Type
delay_us	0x080019D0	void f(uint)
nus	0x0007A120	param - uint
ticks	<not in scope>	auto - uint
told	<not in scope>	auto - uint
tnow	<not in scope>	auto - uint
tcnt	<not in scope>	auto - uint
reload	<not in scope>	auto - uint
delay_ms	0x080019CC	void f(ushort)
main	0x00000000	int f()

图 4.3.2.8 Call Stack + Locals 窗口

如上图所示，可以很清楚地查看到 `delay_us()` 函数是如何被调用的，从下往上看，`main()` 函数调用了 `delay_ms()` 函数，然后 `delay_ms()` 函数调用了 `delay_us()` 函数。

以上就是使用 DAP 调试程序的全部内容，这方便的内容也是比较重要的，掌握好调试程序的方法，对解决程序 Bug 是很有帮助的，并且对了解第三方库等的代码也很有帮助。

4.4 MDK 使用技巧

本小节将介绍 MDK 的一些使用技巧，这些技巧对提供开发效率是很有帮助的，读者可以实际操作一下，加深印象。

4.4.1 自定义编辑器

点击 MDK 软件顶部的  按钮，打开 MDK 配置窗口，并切换到顶部的 Editor 选项卡，如下图所示：

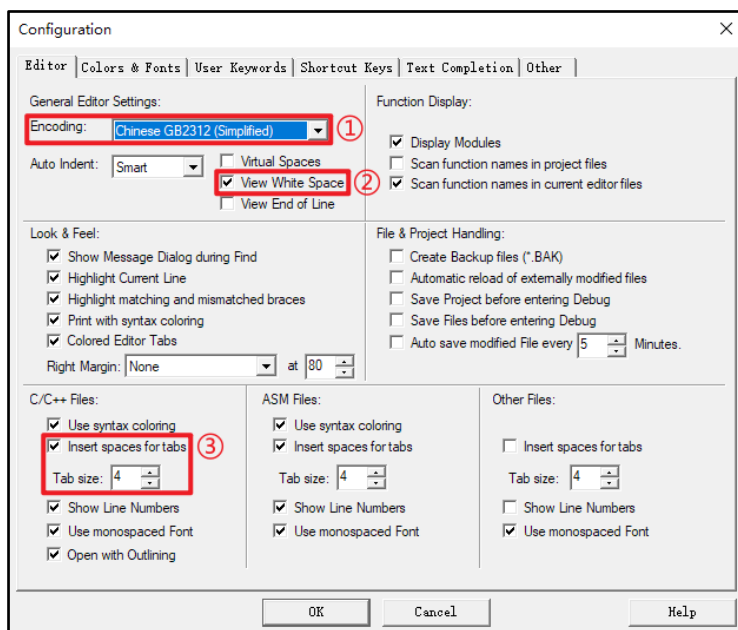


图 4.4.1.1 Editor 选项卡配置

①：“Encoding”用于设置在 MDK 中新建文件保存时使用的编码格式，推荐使用“Chinese GB2312(Simplified)”，该编码格式对中文的支持比较好。

②：勾选“View White Space”复选框可以在文本编辑器中以一个浅灰色的“•”代替一个空格进行显示，有助于代码对齐。

③：勾选“Insert spaces for tabs”复选框可以设置在文本编辑器输入时，使用空格来代替 TAB，TAB 键常用于代码对齐，但由于不同的文本编辑器对 TAB 长度的定义各不相同，因此使用 TAB 对齐可能在不同的文本编辑器中会出现不对齐的情况，因此可以使用空格来代替 TAB，以解决这个问题，同时可以设置使用多少个空格来代替一个 TAB，正点原子的编写代码均是以四个空格代替一个 TAB，建议读者进行相同的设置。

接着是 Colors & Fonts 选项卡，如下图所示：

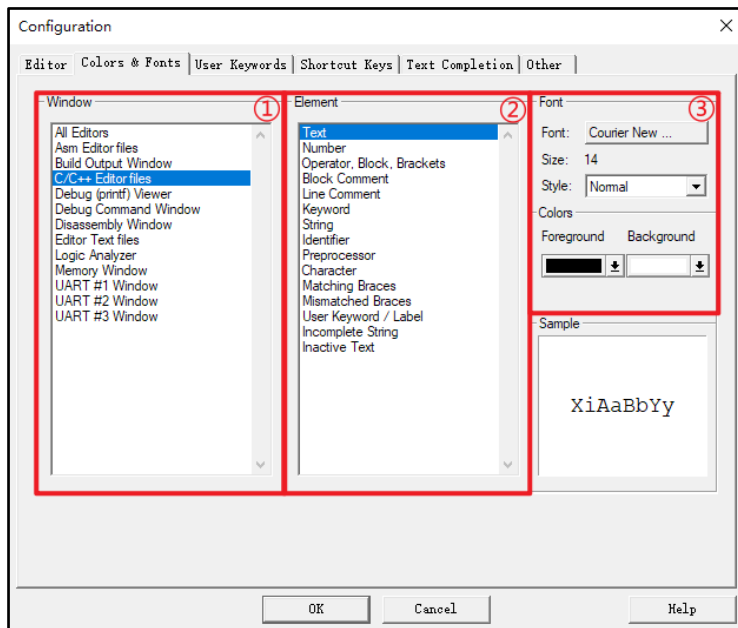


图 4.4.1.2 Colors & Fonts 选项卡配置

①: “Window” 窗口用于选择要修改的窗口对象，例如要修改的对象为 C/C++文本编辑器窗口，则选择“C/C++ Editor files”。

②: “Element” 窗口用于选择窗口对象中要修改的元素，例如要修改的对象为 C/C++文本编辑器窗口中的文本，则选择 “Text”。

③: “Font” 和 “Colors” 用于修改被选中窗口对象中元素的字体、字号、样式、前景色和背景色等参数，读者可以根据自己的喜好进行修改。

接着是 User Keywrds 选项卡，如下图所示：

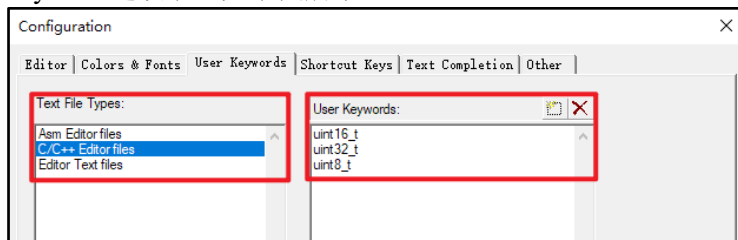


图 4.4.1.3 User Keywords 选项卡配置

该选项卡用于添加一些自定义的关键字，例如，在 C 语言中大约有 37 个关键字(C99 标准)，MDK 在 C/C++文本编辑器可以高亮显示这些关键字(可以在 Colors & Fonts 选项卡→C/C++ Editor files→Keyword 中进行自定义高亮的颜色)，让用户更容易地阅读代码，但是在代码中用 typedef 定义的变量类型名称或 struct 定义的结构体名称等，MDK 默认并不会高亮显示这些用户自定义的“关键字”，因此可以在 User Keyword 选项卡中添加自定义的“关键字”，如上图所示，就添加了三个自定义的关键字，上文提到的 delay_init()函数为例，该函数如使用到上述被添加的三个关键字，则都会被高亮，如下图所示：

自定义关键字前	自定义关键字后
<pre>void delay_init(uint16_t sysclk) { #if (SYS_SUPPORT_OS != 0) uint32_t reload; #endif SysTick->CTRL = 0; SysTick_ConfigCLKSource(SYSTICK_CLK_SOURCE_HCLK); g_fac_us = sysclk; #if (SYS_SUPPORT_OS != 0) reload = sysclk * (1000000 / delay_ostickspersec); #endif g_fac_ms = 1000 / delay_ostickspersec; SysTick->CTRL = SysTick_CTRL_TICKINT_Msk; SysTick->LOAD = reload - 1; SysTick->CTRL = SysTick_CTRL_ENABLE_Msk; } #endif</pre>	<pre>void delay_init(uint16_t sysclk) { #if (SYS_SUPPORT_OS != 0) uint32_t reload; #endif SysTick->CTRL = 0; SysTick_ConfigCLKSource(SYSTICK_CLK_SOURCE_HCLK); g_fac_us = sysclk; #if (SYS_SUPPORT_OS != 0) reload = sysclk * (1000000 / delay_ostickspersec); #endif g_fac_ms = 1000 / delay_ostickspersec; SysTick->CTRL = SysTick_CTRL_TICKINT_Msk; SysTick->LOAD = reload - 1; SysTick->CTRL = SysTick_CTRL_ENABLE_Msk; } #endif</pre>

图 4.4.1.4 自定义关键字效果

4.4.2 动态语法检测&代码自动补全

在 Configuration 窗口的 Text Completion 选项卡中可以配置动态语法检测和代码自动补全功能，如下图所示：

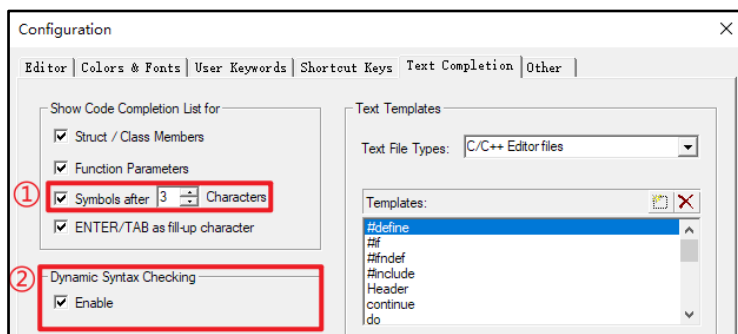


图 4.4.2.1 Text Completion 选项卡配置

①：用于使能代码自动补全功能，并可以配置在手动输入多少个字符后提示匹配自动补全的内容，实际的使用效果，如下图所示：

```
int main(void)
{
    NVIC_ConfigPriorityGroup(NVIC_PRIORITY_GROUP_3);
    sys_apm32_clock_init(336, 8, 2, 7);
    delay_init(168);
    usart_init(115200);
    led_init();
    del
    delay_init
    delay_ms
    delay_us
    LED0(0); /* LED0亮 */
    LED1(1); /* LED1灭 */
    delay_ms(500); /* 延时500毫秒 */
    LED0(1); /* LED0灭 */
    LED1(0); /* LED1亮 */
    delay_ms(500); /* 延时500毫秒 */
}
```

图 4.4.2.2 代码自动补全功能

②：用于使能动态语法检测，这可以帮助用户在编写代码时，实时地观察代码语法的问题，实际的使用效果，如下图所示：


```

26 int main(void)
27 {
28     NVIC_ConfigPriorityGroup(NVIC_PRIORITY_GROUP_3);
29     sys_apm32_clock_init(336, 8, 2, 7);
30     delay_init(168);
31     usart_init(115200);
32     led_init();
33     delay_ms(65536);
34     delay_ms(10);
35
36     while (1)
37     {
38         LED0(0); /* LED0亮 */
39         LED1(1); /* LED1灭 */
40         delay_ms(500); /* 延时500毫秒 */
41         LED0(1); /* LED0灭 */
42         LED1(0); /* LED1亮 */
43         delay_ms(500); /* 延时500毫秒 */
44     }

```

图 4.4.2.3 动态语法检测功能

如上图所示，动态语法检测功能可以对编写的代码进行提示，没有语法错误的代码则不会有任何提示，若语法有误，则会有警告（）和错误（）两种提示，如第 33 行有警告提示，是因为 delay_ms() 函数的形参是 16 位无符号整型，但实际传入的参数已经超出了 16 位无符号整型的范围，因此给出了警告的提示，警告并不会导致编译出错，但是在程序实际运行过程中可能会有意料之外的结果；再如第 34 行有错误提示，是因为 delay_ms() 函数调用后面少了一个分号（“;”），错误提示会直接导致程序编译失败。

4.4.3 代码编辑技巧

本小节介绍几个常用的 MDK 代码编辑技巧，希望对读者提高 MDK 代码编辑效率有所帮助。

1. TAB 键的妙用

在前面的章节中也提到了可以使用 TAB 键来进行代码对齐，那具体的使用方法呢？按下 TAB 键可以使被选中的代码段或光标后的代码向右侧对齐，同时也可以按下 Shift+TAB 键使被选中的代码段或光标后的代码向左侧对齐，如下图所示：

```

26 int main(void)
27 {
28     NVIC_ConfigPriorityGroup(NVIC_PRIORITY_GROUP_3);
29     sys_apm32_clock_init(336, 8, 2, 7);
30     delay_init(168);
31     usart_init(115200);
32     led_init();
33
34     while (1)
35     {
36         LED0(0); /* LED0亮 */
37         LED1(1); /* LED1灭 */
38         delay_ms(500); /* 延时500毫秒 */
39         LED0(1); /* LED0灭 */
40         LED1(0); /* LED1亮 */
41         delay_ms(500); /* 延时500毫秒 */
42     }
43 }

```

图 4.4.3.1 未对齐的代码

如上图所示，未对齐的代码一看就令人头大，若在每一行需要对齐的代码前手动敲入多个空格，那效率是非常低的，此时可以多行选中需要对齐的代码段，然后按以下 TAB 键，即可使被选中的代码段向右对齐，如下图所示：

```

26 int main(void)
27 {
28     ...NVIC_ConfigPriorityGroup(NVIC_PRIORITY_GROUP_3);
29     ...sys_apm32_clock_init(336, 8, 2, 7);
30     ...delay_init(168);
31     ...usart_init(115200);
32     ...led_init();
33
34     ...while(1)
35     ...{
36     ...LED0(0); /* LED0亮 */
37     ...LED1(1); /* LED1灭 */
38     ...delay_ms(500); /* 延时500毫秒 */
39     ...LED0(1); /* LED0灭 */
40     ...LED1(0); /* LED1亮 */
41     ...delay_ms(500); /* 延时500毫秒 */
42     ...}
43 }

```

图 4.4.3.2 用 TAB 键进行对齐

接着对于 while 语句块中的代码再进行一次 TAB 键对齐，最终对齐效果，如下图所示：

```

26 int main(void)
27 {
28     ...NVIC_ConfigPriorityGroup(NVIC_PRIORITY_GROUP_3);
29     ...sys_apm32_clock_init(336, 8, 2, 7);
30     ...delay_init(168);
31     ...usart_init(115200);
32     ...led_init();
33
34     ...while(1)
35     ...{
36     ...LED0(0); /* LED0亮 */
37     ...LED1(1); /* LED1灭 */
38     ...delay_ms(500); /* 延时500毫秒 */
39     ...LED0(1); /* LED0灭 */
40     ...LED1(0); /* LED1亮 */
41     ...delay_ms(500); /* 延时500毫秒 */
42     ...}
43 }

```

图 4.4.3.3 使用 TAB 键的最终对齐效果

2. 快速定位函数/变量被定义的地方

在编写或调试代码的时候，经常会需要查看函数或变量的定义，如果每个都手动查找，那效率就太低了，MDK 软件提供了一键跳转到函数或变量定义地方的功能，但该功能是可选的，需要开启该功能后才能使用，开启方式也很简单，在 Options for Target 窗口的 Output 选项卡中勾选 Browse Information 复选框即可开启该功能，如下图所示：

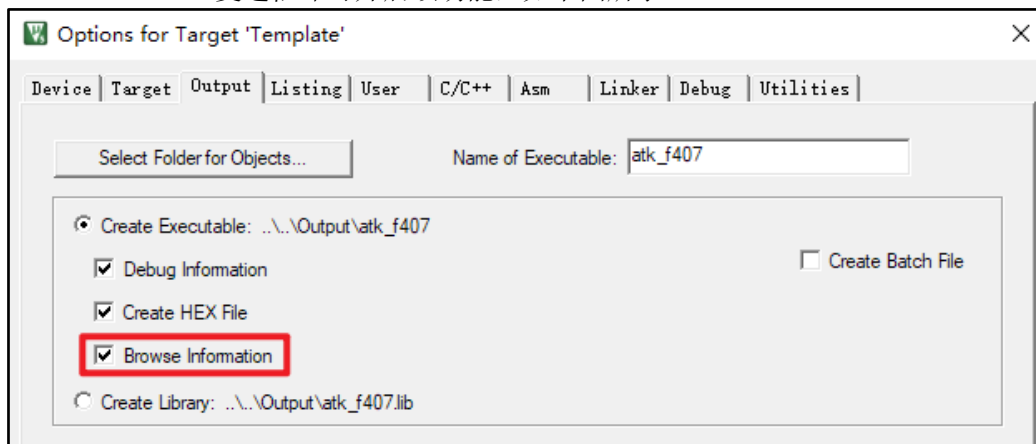


图 4.4.3.4 开启 Browse Information 功能

开启 Browse Information 功能后，还需进行一次编译才能使用跳转功能，使用方式也很简单，仅需单击或自定义选中函数或变量名，然后点击快捷键 F12 即可进行跳转，在查看完相关定义后，可以直接点击工具栏上的  按钮（快捷键 Ctrl+-），即可一步一步地返回。

3. 快速注释/取消注释

MDK 提供了一键注释或取消注释代码段的功能，使用方法也很简单，若要注释或取消一段

代码，则先选中代码段，然后点击工具栏上的（注释）或（取消注释），即可一键对代码段进行注释或取消注释。

4.4.4 其他小技巧

本小节再介绍几个 MDK 软件使用的小技巧。

①：右键#include 包含的头文件，弹出的菜单中有“Open document“xxx””的选项，可以快速地在文本编辑器中打开头文件，如下图所示：

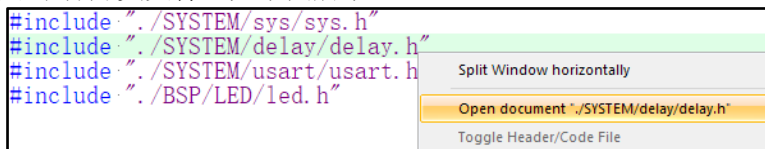


图 4.4.4.1 快速打开头文件

②：“Ctrl+H”、“Ctrl+F”和“Ctrl+Shift+F”的组合快捷键可一键打开替换、查找和跨文件查找的功能窗口，如下图所示：



图 4.4.4.3 替换、查找和跨文件查找的功能窗口

MDK 软件是一个非常强大的代码编辑和编辑工具，熟悉 MDK 的使用方法还是很有必要的。

第五章 STM32 基础知识入门

本章着重介绍 STM32 的一些基础知识，令读者对 STM32 有一个初步的了解，为后面的章节学习做铺垫。对于本章的内容，读者可以先只看一个大概，在后面需要使用到这部分知识点的时候，再回过头来仔细看。

本章分为如下几个小节：

5.1 C 语言基础知识复习

5.2 寄存器基础知识

5.3 STM32F407 系统架构

5.1 C 语言基础知识复习

本节介绍 C 语言的基础知识，对 C 语言比较熟练的读者，可以跳过本节，对于基础比较薄弱的读者，建议好好学习一下本节的内容。

由于 C 语言博大精深，不可能由一小节的内容就全讲明白，所以本节知识回顾在进行 STM32 开发时常用的几个 C 语言知识点，以便读者的更好的学习本书后续的内容，并编写相关的代码。

5.1.1 位操作

C 语言的位操作就是对基本类型变量进行位级别的操作。本节的内容比较简单，这里也就点到为止，不深入探讨。下面先讲解几种位操作运算符，然后介绍其相关的使用技巧。C 语言支持如下 6 种位操作的运算符：

运算符	含义	运算符	含义
&	按位与	~	按位取反
	按位或	<<	左移
^	按位异或	>>	右移

表 5.1.1.1 6 种位操作的运算符

下面介绍这些位操作运算符的使用技巧。

①：在不改变其他位的状况下，对某几个位进行设置

这个场景在单片机开发中经常使用，方法就是先对需要设置的位用&运算符进行清零操作，然后用|运算符设置。例如要设置 GPIOA 的 ODATA 寄存器 Bit6（第 6 位）为 1，则可以先使用&运算符对该寄存器的 Bit6 进行清零操作：

```
GPIOA->ODATA &= 0xFFFFFBBF; /* 将 Bit6 清 0 */
```

然后再使用|运算符对该寄存器的 Bit6 进行置 1 操作：

```
GPIOA->ODATA |= 0x00000040; /* 将 Bit6 置 1 */
```

②：移位操作提高代码的可读性

例如①中|操作使用到的 0x00000040，虽然通过换算，可以知道是将 Bit6 置 1，但是这样的表达可读性比较差，可以通过移位操作对其进行改进：

```
GPIOA->ODATA |= (1 << 6); /* 将 Bit6 置 1 */
```

这么一来，就可以非常直观地看出是将 Bit6 置 1 了。

③：按位取反操作使用技巧

②中使用移位操作改进①中仅使用|运算将 Bit6 置 1 的可读性，但要改进①中使用&运算将 Bit6 清 0 的可读性还需借助按位取反操作：

```
GPIOA->ODATA &= ~(1 << 6); /* 将 Bit6 清 0 */
```

这么一来，就可以非常直观地看出是将 Bit6 清 0 了。

④：按位异或操作使用技巧

按位异或可以很方便地对 Bit 位进行翻转，例如不考虑 LED 当前是何种状态，只要求控制 LED 翻转状态等情况（假设 LED 的亮灭状态由 PA6 输出的高低电平控制）：

```
GPIOA->ODATA ^= (1 << 6); /* Bit6 的值取反 */
```

这么一来，就可以很方便地操作 PA6 引脚输出相反的电平，而不用先读取 PA6 输出的电平状态然后才输出相反的电平。

5.1.2 define 宏定义

`define` 是 C 语言中的预处理命令，它用于宏定义，可以提高源代码的可读性，为编程提供方便，其常见的格式如下：

```
#define 标识符 字符串
```

“标识符”为所定义宏的名称；“字符串”可以是常数、表达式、格式串等。例如：

```
#define HSE_VALUE 8000000U
```

定义标识符 `HSE_VALUE` 的值为 `8000000U`，数字后的 `U` 是 `unsigned`（无符号）的意思，随后便可在程序代码中使用 `HSE_VALUE` 来代替 `8000000U`。

至于 `define` 宏定义的一些其他高级用法，例如宏定义带参数等，本章不过多介绍。

5.1.3 ifdef 条件编译

在单片机程序开发过程中，经常会遇到需要在满足某些条件时对一段代码进行编译，而当条件不满足或满足另一条件时编译另一段代码，这就可以使用条件编译，条件编译最常见的形式如下：

```
#ifdef 标识符
```

```
代码段 1
```

```
#else
```

```
代码段 2
```

```
#endif
```

如上的条件编译，当标识符被定义过（一般使用 `define` 进行定义），则代码段 1 会被编译，否则会编译代码段 2。

5.1.4 extern 外部申明

C 语言中 `extern` 关键字用于修饰变量或函数，以表示变量或函数定义在别的文件中，提示编译器遇到此变量或函数时，需在其他文件中寻找其定义。这里要注意的是，可以使用 `extern` 多次在不同文件中修饰同一个变量或函数，但该变量或函数只能被定义一次

5.1.5 typedef 类型别名

`typedef` 用于为现有类型创建一个新的名字，或称为类型别名，用来简化变量的定义。例如在编写程序时经常使用到的 `uint8_t`、`uint16_t` 和 `uint32_t` 等都是由 `typedef` 定义的类型别名，其定义如下：

```
typedef unsigned char    uint8_t;
typedef unsigned short int uint16_t;
typedef unsigned int     uint32_t;
```

这么一来就可以在编写程序代码的时候使用 `uint8_t` 等代替 `unsigned char` 等，极大地提高的代码的可读性可编写代码的效率。

5.1.6 struct 结构体

`struct` 用于定义结构体，结构体就是一堆变量的集合，结构体中的成员变量的作用一般都是相互关联的，定义结构体的形式如下：

```
struct 结构体名
```

```
{
```

```
    成员变量 1 的定义;
```

```
    成员变量 2 的定义;
```

```
    .....
};
```

例如：

```
struct lcd_device_struct
{
```

```
uint16_t width;
uint16_t height;
};
```

如上举例的结构体定义，一堆描述 LCD 屏幕的变量的集合，其中包含了 LCD 屏幕的宽度和高度。

结构体变量的定义如下：

```
struct lcd_device_struct lcd_device;
```

如上，就定义了一个名为 lcd_device 的结构体变量，那么怎么访问这个结构体变量中的成员变量呢？如下：

```
lcd_device.width = 240;
printf("LCD Height: %d\n", lcd_device.height);
```

如上就展示了结构体变量中成员变量的访问操作。

5.1.7 指针

指针是另一个变量的变量，指针变量在内存中保存的是另一个变量在内存中的地址，通过指针可以访问到另一个变量所在内存地址中的数据。

举一个定义指针的例子，如下：

```
char *p_str = "This is a string!";
```

如上，就定义一个名为 p_str 的指针变量，并将 p_str 指针指向了字符串 “This is a string!” 保存在内存中首地址，对于 STM32 来说，此时 p_str 的值就是一个 32 位的数，这个数就是一个内存地址，这个内存地址就是上述字符串保存在内存中的首地址。

通过 p_str 指针就可以访问到字符串 “This is a string!”，那具体是如何访问的呢？前面说 p_str 保存的是一个内存地址，那么就可以通过这个内存地址去内存中读取数据，通过 *p_str 就可以访问地址为 p_str 的内存数据，*(p_str + 1) 可以访问下一个内存地址中的数据。

知道了如何访问内存中的数据，但是读取到的数据要如何解析呢？这就有 p_str 指针的类型决定了。在这个例子中 p_str 是一个 char 类型的指针，那么访问 *p_str 就是访问地址为 p_str，大小为 sizeof(char)（一般为一个字节）的一段内存数据，在这个例子中就可以读取到字符 “T”，读取 *(p_str + 1) 就是 “h”，以此类推。

指针是 C 语言的精髓，但也是初学者望而生畏的一个知识点，若读者一时半会无法理解指针的用法也没关系，善用搜索引擎，网上有很多参考的学习资料。

5.2 寄存器基础知识

寄存器（Register）是一种特殊的内存，它主要用于实现控制和访问 MCU 的内核和各个片上外设。

对于 STM32 来说，寄存器一般都是 32 位的，但由于 MCU 上的内存资源十分宝贵，因此在一个 32 位寄存器中，会使用其中的 1 位或多位来控制或访问 MCU 的内核或各个片上外设的一种功能，但即使如此，STM32 上也还是有上百个寄存器，这实际上是因为 STM32 有很多的片上外设导致的，只要将这些寄存器按照功能分好类，学起来就不难了。

从大方向来区分，STM32 中的寄存器可分为两大类，分别为内核寄存器和外设寄存器，大类下还可以分出许多小类，如下表所示：

大类	小类	说明
内核寄存器	内核相关寄存器	R0~R15、xPSR 和特殊功能寄存器等
	中断相关寄存器	NVIC、SCB 相关的寄存器
	SysTick 寄存器	SysTick 相关寄存器
	DBGMCU 寄存器	DBGMCU 相关寄存器
外设寄存器	GPIO、UART、IIC、SPI 等片上外设	GPIO、UART、IIC、SPI、TIM、DMA、ADC、DAC、RTC、FSMC、RCC、PWR 等片上外设相关寄存器

表 5.2.1 STM32 寄存器分类

对于初学者来说仅需在学习 MCU 的各个片上外设时，再去学习该片上外设相关的寄存器即可。

5.3 STM32F407 系统架构

STM32F407 是 ST 公司基于 ARM 授权 Cortex-M4 内核设计的一款芯片，而 Cortex-M 内核采用了 ARM v7-M 架构，具有低成本、低功耗、实时性好、中断响应快、处理效率高等特点。

5.3.1 Cortex-M4 内核&芯片

ARM 公司提供内核（例如 Cortex-M4 内核）授权，完成的 MCU 芯片还需要很多其他的组件。芯片公司（例如 ST）在获取 Cortex-M4 内核授权后，就可以将其用于自己的芯片设计中，并添加存储器、外设、I/O 等其他组件。同于型号的芯片也会有不同的规格，实际当大多数就是存储器容量、外设资源等的差异。

5.3.2 STM32F407 系统框图

STM32F407 的系统框图，如下图所示：

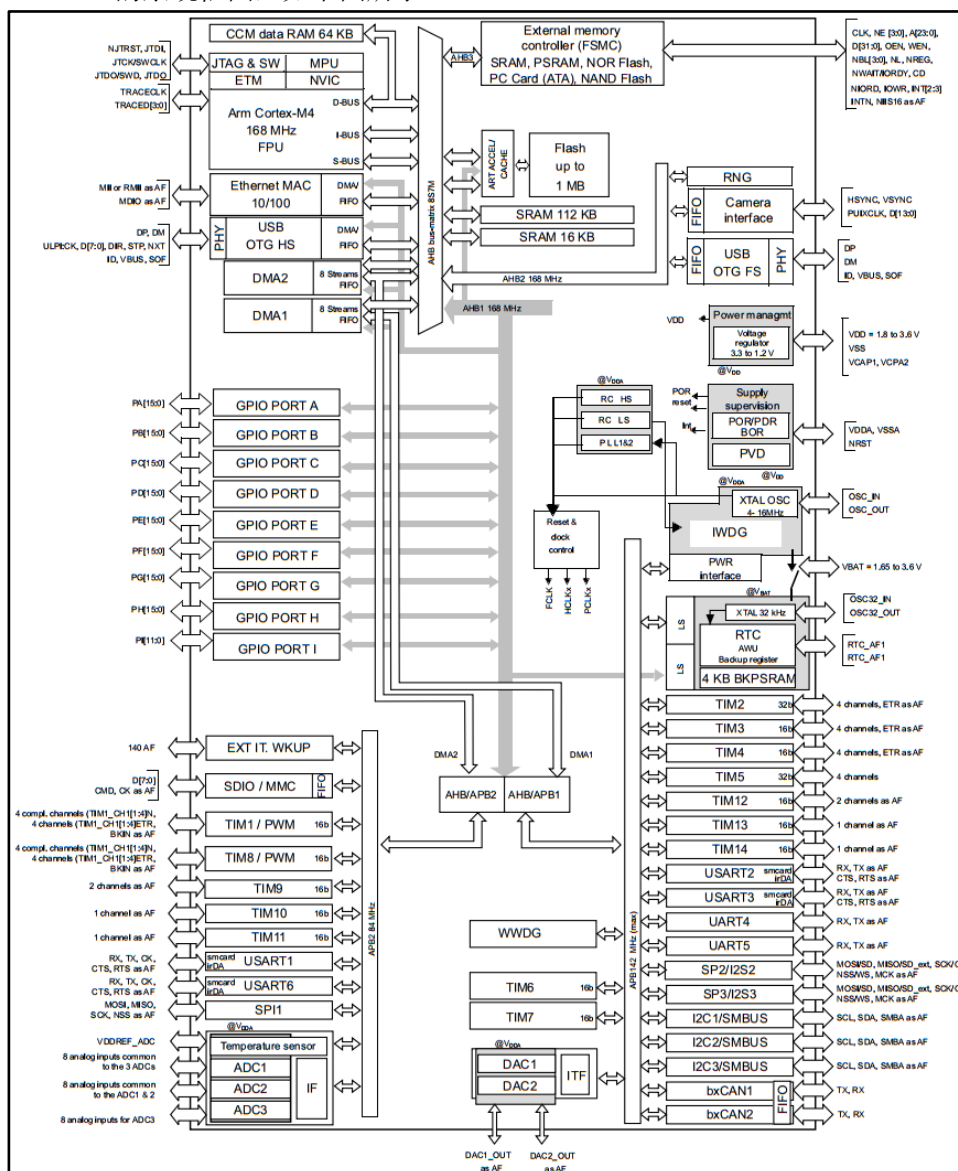


图 5.3.2.1 STM32F407 系统框图

从上图可以看出，STM32F407 就是在 ARM 提供的 Cortex-M4 内核上通过各种总线，挂载了各种各样的外设和存储器。

5.3.3 地址映射

STM32 是 32 位的 MCU，因此其地址总线的宽度也是 32 位，因此最大可访问的内存空间为 4GB ($2^{32}=4GB$)，STM32 就将这 4GB 的空间映射给各种寄存器和存储器。

STM32F407 地址映射图，如下图所示：

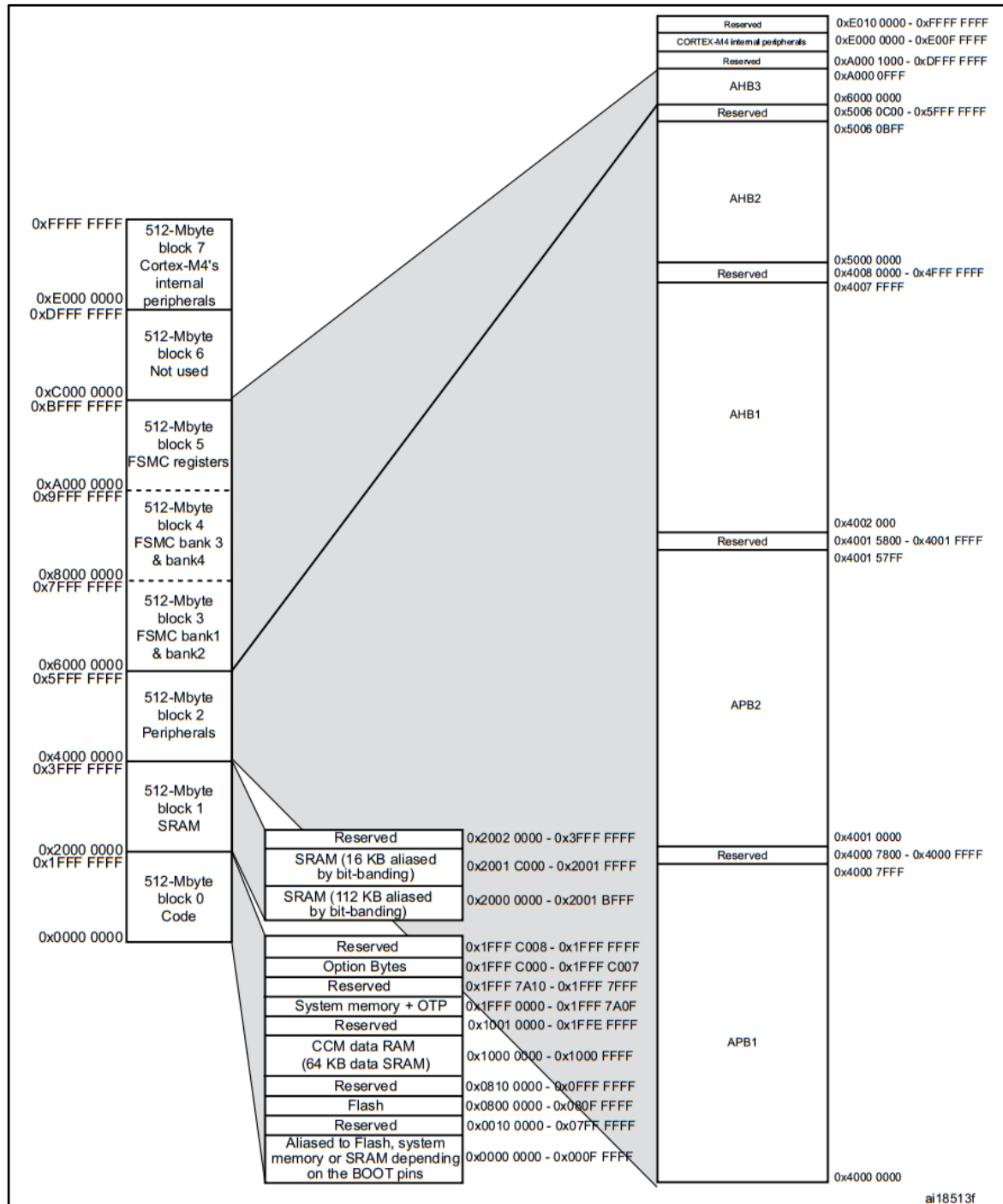


图 5.3.3.1 STM32F407 地址映射图

第六章 认识 HAL 库

编写程序控制 MCU 实际上就是操作 MCU 上的各个寄存器，但直接操作寄存器的开发方式不仅开发效率低且容易出错，为此 ST 官方为 STM32 提供了一个 HAL 库，这使得用户在进行程序代码编写时，仅需调用 HAL 库中提供的 API 函数即可完成对 MCU 上寄存器的访问操作，缩短开发时间、提高开发效率。

本章分为如下几个小节：

6.1 STM32CubeF4 简介

6.2 Drivers 文件夹介绍

6.3 ST HAL 库介绍

6.1 STM32CubeF4 简介

STM32CubeF4 是 ST 针对 STM32F4 系列 MCU 提供的软件包，包含了 HAL、LL 和 CMSIS，同时也包含了 USB、TCP/IP、文件系统、RTOS、图形界面库等一系列组件。

STM32CubeF4 由 ST 提供，可从 ST 官网下载，也可在 A 盘→8，STM32 参考资料中找到。STM32CubeF4 的文件结构，如下图所示：

```
STM32Cube_FW_F4_V1.27.1/
|-- Documentation
|-- Drivers
|-- Middlewares
|-- Package_license.html
|-- Package_license.md
|-- Projects
|-- README.md
|-- Release_Notes.html
|-- Utilities
|-- _htmresc
'-- package.xml
```

图 6.1.1 STM32CubeF4 文件结构

从上图中可以看到 STM32CubeF4 软件包中提供了许多文件，本文主要介绍 Drivers 中的驱动库。

6.2 Drivers 文件夹介绍

STM32CubeF4 的 Drivers 文件夹下包含了 ST 提供的各种驱动库，如下图所示：

```
STM32Cube_FW_F4_V1.27.1/Drivers/
|-- BSP
|-- CMSIS
'-- STM32F4xx_HAL_Driver
```

图 6.2.1 Libraries 文件夹

①：BSP

ST 官方 STM32F4 系列板卡的板级支持包。

②：CMSIS

CMSIS (Cortex Microcontroller Software Interface Standard, Cortex 微控制器软件接口标准) 是 Cortex-M 处理器系列的与供应商无关的硬件抽象层。使用 CMSIS 可以为处理器和外设实现一致且简单的软件接口，从而简化软件的重用、缩短微控制器开发人员的学习过程，并缩短新设备的上市时间。通过在所有 Cortex-M 芯片提供商产品中标准化软件接口，会使成本明显降低，尤

其是在创建新项目或将现有转迁到新设备时。

该文件夹为 ARM 的 Cortex 内核提供的一些标准接口，主要就是一些与编译器、内核等相关的头文件和 DSP 库。

③：STM32F4xx_HAL_Driver

ST 提供的针对 STM32F4 系列 MCU 的 HAL 驱动库。

6.3 ST HAL 库介绍

本书配套例程均使用 ST 提供的针对 STM32F4 系列 MCU 的 HAL 库进行开发，该库随 ST 针对的 STM32F4 系列 MCU 提供的 STM32CubeF4 软件包一起提供。该库由一系列的 C 源文件和头文件组成，并且在 HAL 库中也包含了 LL 库的一系列驱动文件，其中 HAL 库的驱动代码是硬件抽象层的驱动代码，而 LL 库是底层驱动代码。STM32F4xx_HAL_Driver 中的 HAL 和 LL 都是 ST 为了方便开发者使用 STM32 而开发的软件驱动库，如下图所示：

```
STM32F4xx_HAL_Driver/
|-- Inc
|   |-- stm32f4xx_hal.h
|   |-- stm32f4xx_hal_cortex.h
|   |-- stm32f4xx_hal_gpio.h
|   |-- stm32f4xx_hal_i2c.h
|   `-- stm32f4xx_hal_spi.h
`-- Src
    |-- stm32f4xx_hal.c
    |-- stm32f4xx_hal_cortex.c
    |-- stm32f4xx_hal_gpio.c
    |-- stm32f4xx_hal_i2c.c
    `-- stm32f4xx_hal_spi.c
```

图 6.3.1 HAL 库文件（部分）

HAL 驱动库中包含了许多文件，在后面的章节中，本书会逐一介绍本书配套例程中所使用的 HAL API 函数。

第七章 HAL 库版本 MDK 工程创建

在上一章的内容中简单介绍了 HAL 库，本章将详细讲解使用 HAL 库创建 MDK 工程的步骤，本章创建好的 MDK 工程存放路径为 A 盘→4，程序源码→标准例程→实验 0 基础入门实验→实验 0-2 新建工程实验-HAL 库版本，读者在创建工程时若遇到问题，可打开该工程进行对比。

本章分为如下几个小节：

7.1 HAL 库版本 MDK 工程创建

7.2 下载验证

7.1 HAL 库版本 MDK 工程创建

HAL 库版本 MDK 工程的创建可分为四个步骤，如下：

- ①：构建工程文件夹
- ②：添加文件到工程目录
- ③：创建 MDK 工程
- ④：配置 MDK 工程

7.1.1 构建工程文件夹

构建工程文件夹前需要先确定工程文件夹结构，建议初学者使用本书配套实验例程工程的文件夹结构，该工程文件夹结构也是正点原子所有实验例程的工程文件夹结构，读者可任意打开一个本书配到的实验例程根目录，如下图所示：

```
实验0-2 新建工程实验-HAL库版本/
|-- Drivers
|-- Middlewares
|-- Output
|-- Projects
|-- User
|-- keilkill.bat
'-- readme.txt
```

图 7.1.1.1 工程文件夹结构

该工程文件夹结构清晰，其中各个子文件夹的作用如下表所示：

名称	描述
Drivers	存放于硬件相关的驱动文件
Middlewares	存放正点原子提供的中间组件或第三方中间组件
Output	存放工程编译的输出文件
Projects	存放 MDK 等工程文件
User	存放用户编写的代码文件，例如 main.c 或其他应用程序

表 7.1.1.1 工程子文件夹介绍

接下来，就按照上面介绍的工程文件夹结构构件工程文件夹，如下图所示：

```
新建工程实验-HAL库版本/
|-- Drivers
|-- Middlewares
|-- Output
|-- Projects
|-- User
|-- keilkill.bat
'-- readme.txt
```

图 7.1.1.2 构建工程文件夹

上图中还另外添加了 keilkill.bat 的脚本文件和 readme.txt 文本文件，其中 keilkill.bat 脚本文

件用于清除工程文件夹中编译工程产生的中间文件，方便工程的存档和传输，readme.txt 文本文件主要是对该工程的描述，包括工程功能、使用的硬件资源等信息。

7.1.2 添加文件到工程目录

在该步骤中，涉及了五个文件夹，如下：

①：Drivers 文件夹

Drivers 文件夹用于存放与硬件相关的驱动文件，如下表所示：

名称	描述
CMSIS	STM32CubeF4 中的 CMSIS
STM32F4xx_HAL_Driver	STM32CubeF4 中的 STM32F4xx_HAL_Driver
SYSTEM	正点原子提供的系统级核心驱动代码
BSP	开发板板级支持包驱动代码

表 7.1.2.1 Drivers 文件夹介绍

其中 CMSIS 和 STM32F4xx_HAL_Driver 都可以从 STM32CubeF4 中获取；而 SYSTEM 文件夹是正点原子提供的系统级核心驱动代码，主要提供了系统时钟初始化、延时、串口收发等基础功能，读者可在本书配套的任意一个实验例程中获取到该文件夹；BSP 文件夹为开发板板级支持包驱动代码，在初次创建工程时留空即可，在后续的实验中会逐渐完善 BSP 文件夹。

添加好文件的 Drivers 文件夹，如下图所示：

```
新建工程实验-HAL库版本/Drivers/
|-- CMSIS
|-- STM32F4xx_HAL_Driver
`-- SYSTEM
```

图 7.1.2.1 Drivers 文件夹

②：Middlewares 文件夹

Middlewares 文件夹用于存放正点原子提供的中间组件（USMART、MALLOC 等）和第三方中间组件（FatFs、FreeRTOS 等）在初次创建工程时，并不会用到这些中间组件，因此留空即可。

③：Output 文件夹

Output 文件夹用户存放 MDK 工程编译后产生的二进制文件和编译过程文件等，这些文件均由 MDK 软件在进行工程编译的时候产生，留空即可。

④：Projects 文件夹

Projects 文件夹主要用于存放 MDK 的工程文件，在 Projects 文件夹中创建一个 MDK-ARM 的空文件夹，用于下文创建 MDK 工程时，存放 MDK 的工程文件，如下图所示：

```
新建工程实验-标准库版本/projects
`-- MDK-ARM
```

图 7.1.2.2 Projects 文件夹

⑤：User 文件夹

User 文件夹用于存放用户编写的应用代码，例如 main.c 等，main.c 文件中提供了 main() 函数（必须有该函数），对于初次创建的工程，可以从“实验 0-2 新建工程实验-HAL 库版本”的工程中复制。另外，在该文件夹下还会存放 stm32f4xx_hal_conf.h、stm32f4xx_it.c 和 stm32f4xx_it.h 这三个文件，其中 stm32f4xx_hal_conf.h 文件主要用于配置 HAL 库，其他两个文件主要是用于管理程序中的中断服务函数，这三个文件直接从“实验 0-2 新建工程实验-HAL 库版本”的工程中复制即可。

添加好文件的 User 文件夹，如下图所示：

```
新建工程实验-HAL库版本/User/
|-- main.c
|-- stm32f4xx_hal_conf.h
|-- stm32f4xx_it.c
`-- stm32f4xx_it.h
```

图 7.1.2.3 User 文件夹

7.1.3 创建 MDK 工程

在完成了以上两大步骤后，接下来可以开始创建 MDK 工程了，首先打开 MDK 软件，依次点击菜单栏中的 Project→New uVision Project，如下图所示：

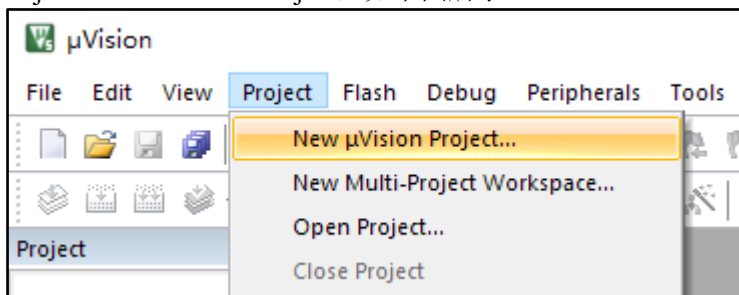


图 7.1.3.1 新建 MDK 工程

随后在弹出的窗口中选择工程文件的保存路径和工程文件名，如下图所示：

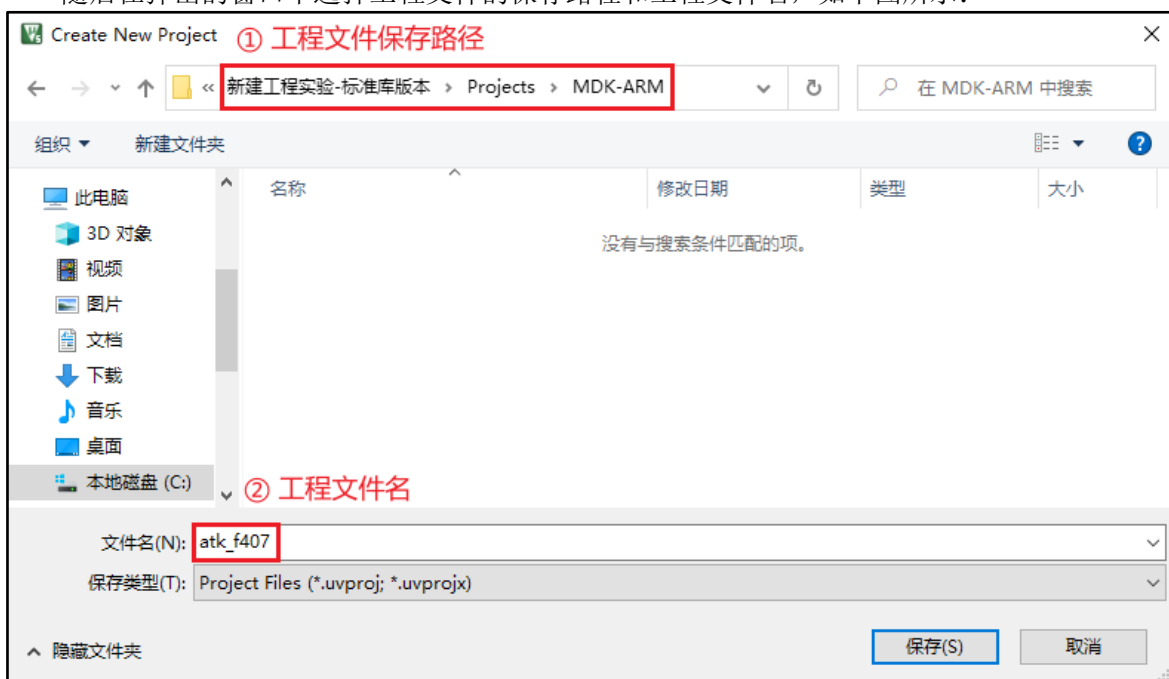


图 7.1.3.2 保存工程文件

点击保存后，会弹出选择设备的窗口，其实就是选择所使用的 MCU 型号，这里选择 STMicroelectronics→STM32F4 Series→STM32F407→STM32F407ZG→STM32F407ZGTx，如下图所示：

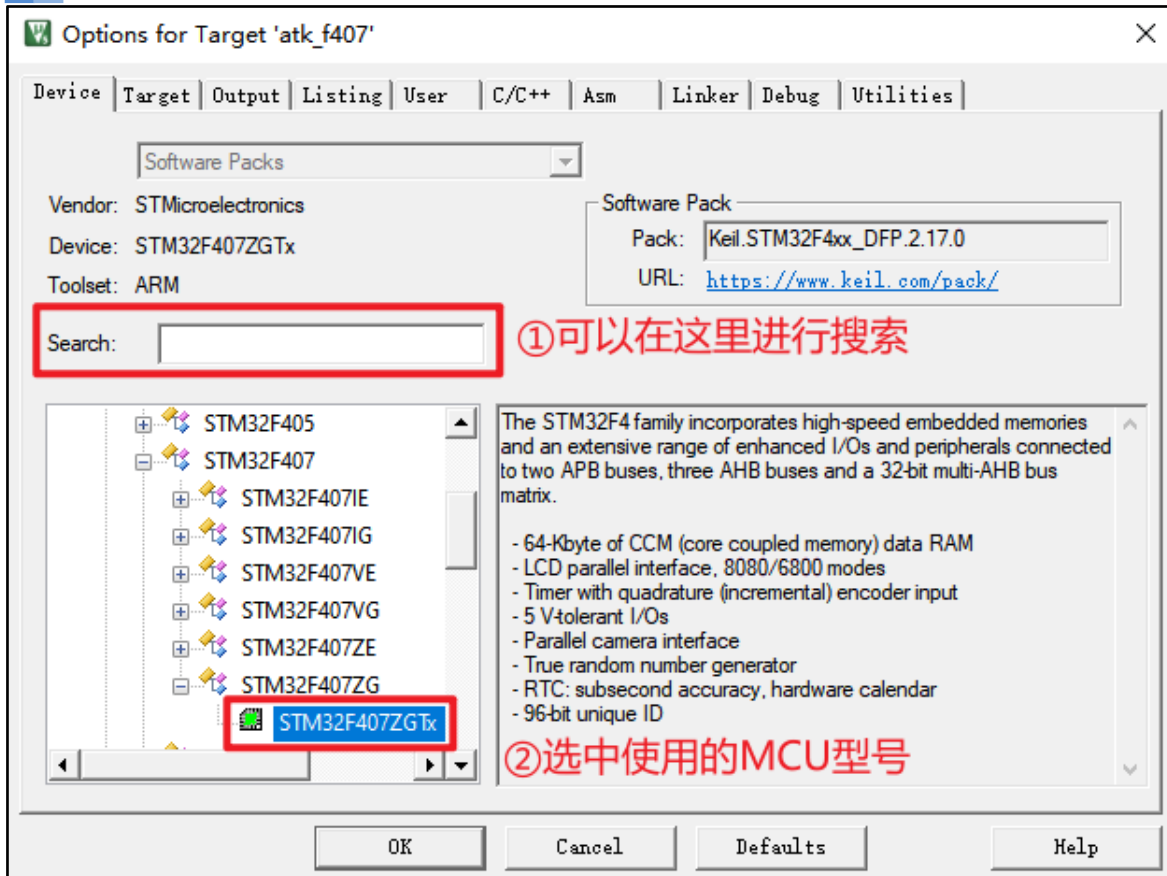


图 7.1.3.3 选择 MCU 型号

正确选择 MCU 型号并点击“OK”按钮后，会弹出 RTE（Run Time Environment，运行时环境，由 MDK 提供）的管理窗口，本文配套的所有实验例程均不使用 RTE，点击窗口下方的“Cancel”按钮关闭窗口即可。

至此，就能看到 MDK 工程的一个雏形了，如下图所示：

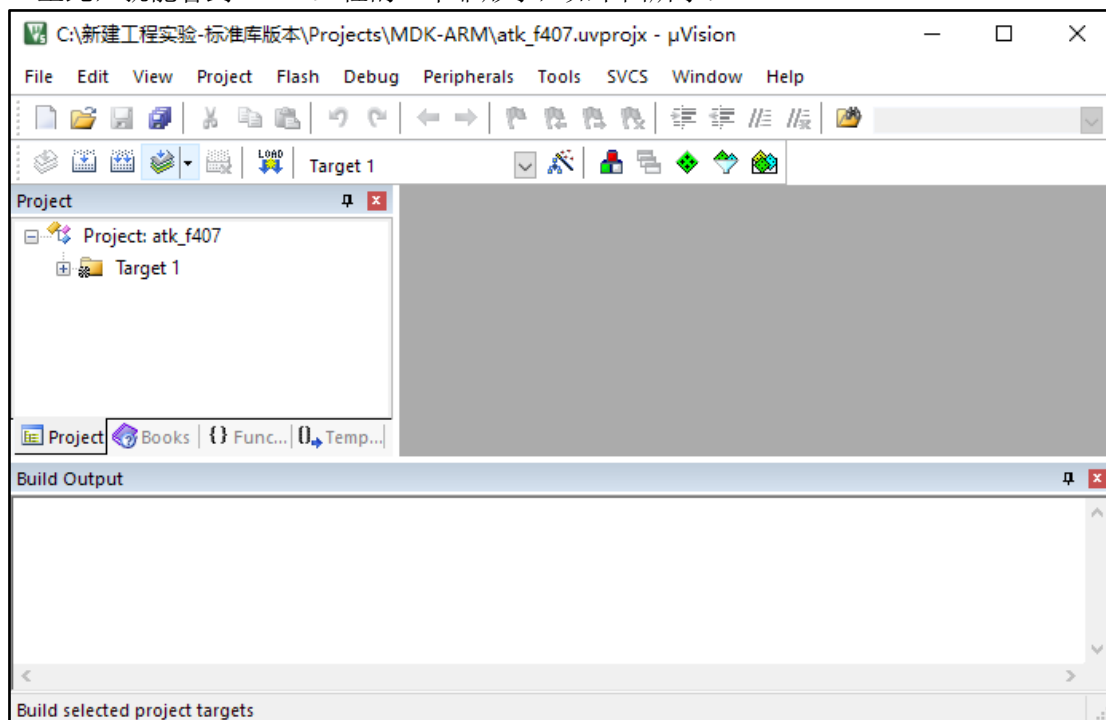


图 7.1.3.4 MDK 工程雏形

7.1.4 配置 MDK 工程

配置 MDK 工程主要就是为工程添加一些源码文件，以及对 MDK 工程进行一些配置等。

在添加源码文件之前，先创建一些分组，使工程的结构更加清晰明了。点击工具栏上的按钮打开工程项目管理窗口，在工程项目管理窗口中可以管理工程目标名和项目分组，按照下图所示，需改工程目标名并创建几个项目分组，如下图所示：

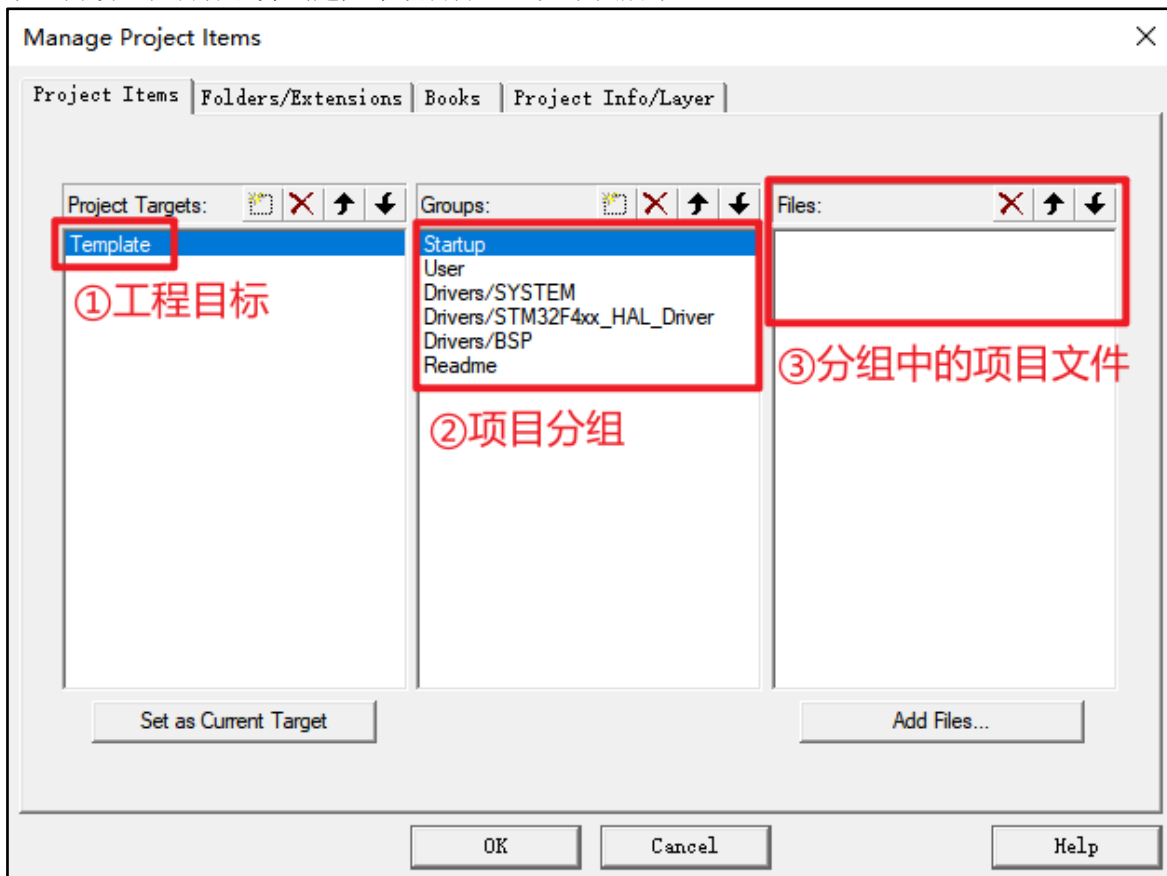


图 7.1.4.1 工程项目管理窗口

上图中共创建了五个项目分组，但还未向分组中添加项目文件，接下来分别介绍各个项目分组中应该添加那些项目文件。

①：Startup 项目分组

Startup 用于存放启动文件，对于 STM32F407ZG 型号的 MCU，其启动文件的路径为 Drivers→CMSIS→Device→ST→STM32F4xx→Source→Templates→arm→startup_stm32f407xx.s，添加项目文件后的 Startup 项目分组，如下图所示：

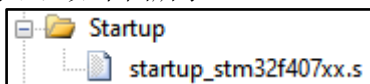


图 7.1.4.2 Startup 项目分组

②：User 项目分组

User 主要用于存放用户的应用代码及其相关的文件，如下图所示：

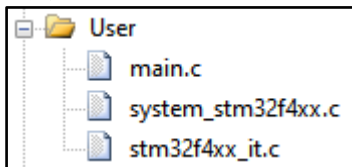


图 7.1.4.3 User 项目分组

main.c 文件的路径为 User→main.c;

system_stm32f4xx.c 文件的路径为 Drivers→CMSIS→Device→ST→STM32F4xx→Source→Te

mpfiles→system_stm32f4xx.c;

stm32f4xx_it.c 文件的路径为 User→stm32f4xx_it.c。

③: Drivers/SYSTEM 项目分组

Drivers/SYSTEM 用于存放 SYSTEM 文件夹中的 C 源文件，如下图所示：

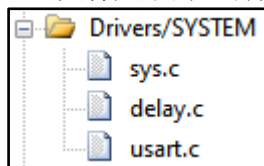


图 7.1.4.4 Drivers/SYSTEM 项目分组

sys.c、delay.c 和 usart.c 文件的路径均在 Drivers/SYSTEM 文件夹中。

④: Drivers/STM32F4xx_HAL_Driver 项目分组

Drivers/STM32F4xx_HAL_Driver 用于存放 HAL 库中的 C 源文件，如下图所示：

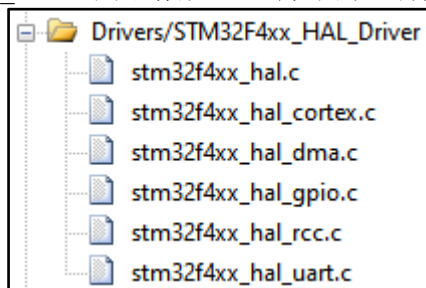


图 7.1.4.5 Drivers/STM32F4xx_HAL_Driver 项目分组

这部分的文件可以按需添加，以减少工程的编译时间。HAL 库 C 源文件的路径为 Drivers/STM32F4xx_HAL_Driver/src。

⑤: Readme 项目分组

Readme 用于存放工程根目录下的 readme.txt 文本文件，如下图所示：

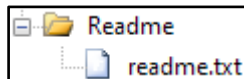


图 7.1.4.6 Readme 项目分组

至此，项目文件基本添加完毕，后续读者应根据实际需求再往工程中添加所需的项目文件。

接下来点击工具栏上的  按钮，打开 Options for Target 窗口，首先可以看到 Target 选项卡中的 ARM Compiler 默认选择了“Use default compiler version 6”即 AC6，由于 STM32F4xx_SDK 并不支持 AC6，因此要将该选项修改为“Use default compiler version 5”即 AC5，如下图所示：

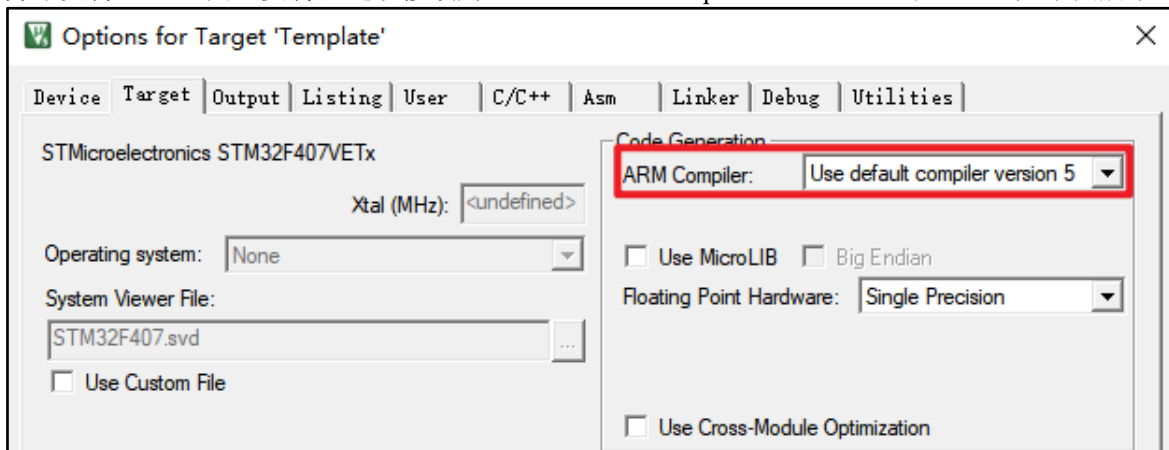


图 7.1.4.7 Target 选项卡

接着是 Output 选项卡，点击“Select Folder for Objects...”按钮指定编译输出的文件保存路径为工程根目录下的 Output 文件夹中，再勾选“Create HEX File”使工程成功编译后生成 Hex 文件，方便后续通过串口烧录程序，如下图所示：

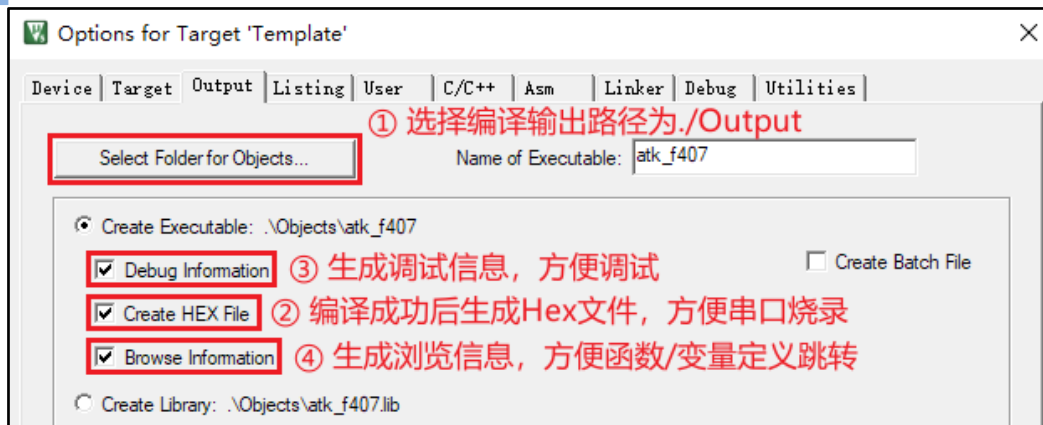


图 7.1.4.8 Output 选项卡

接着是 Listing 选项卡，点击“Select Folder for Listing...”按钮指定 map 等文件的输出目录为根文件夹下的 Output 文件夹，如下图所示：

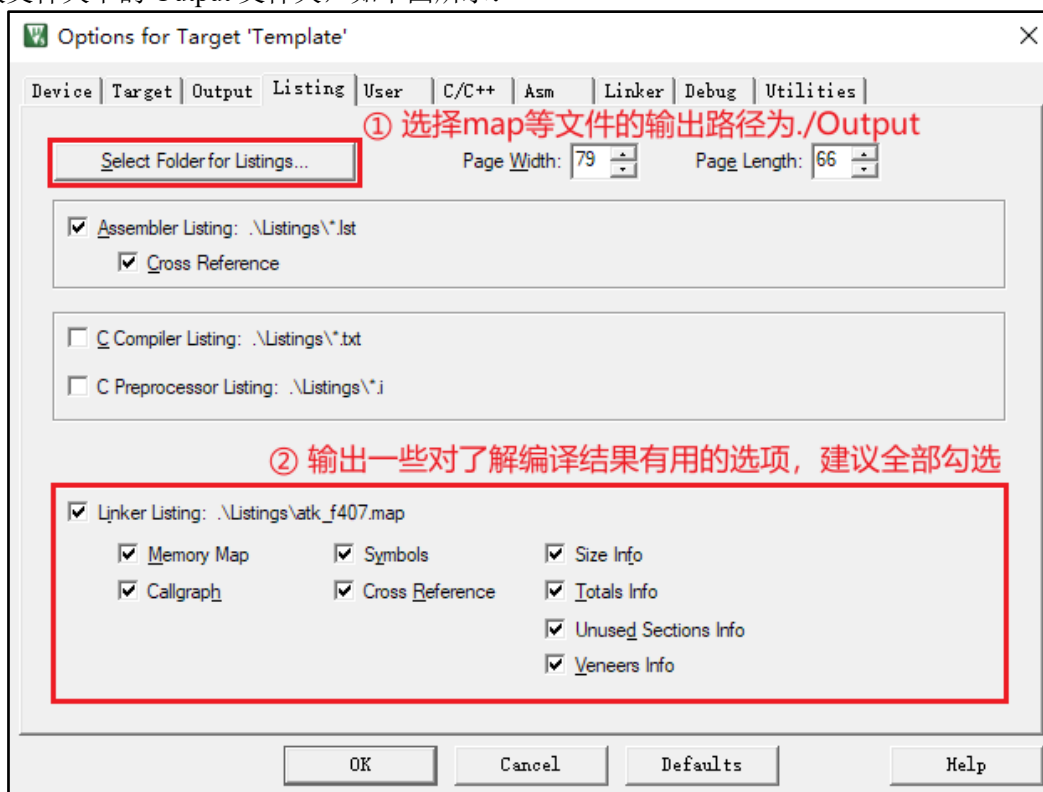


图 7.1.4.9 Listing 选项卡

接着是 User 选项卡，默认情况下，MDK 软件会在工程成功编译后自动进入调试模式，若想禁用该功能，则取消勾选 Start Debugging 复选框即可，如下图所示：

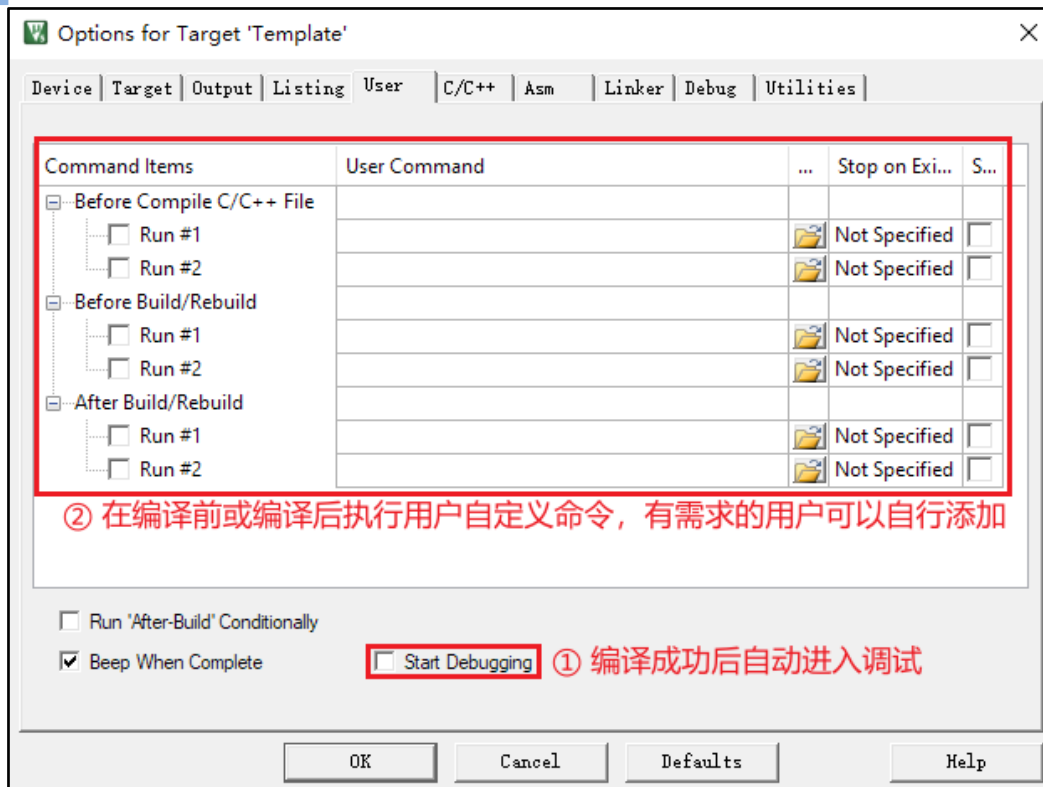


图 7.1.4.10 User 选项卡

接着是 C/C++选项卡, 为了正常使用 HAL 库在 Define 编辑框中添加“USE_HAL_DRIVER”和“STM32F407xx”的全局宏定义, 再接着就是通过 Include Paths 中添加头文件路径, 如下图所示:

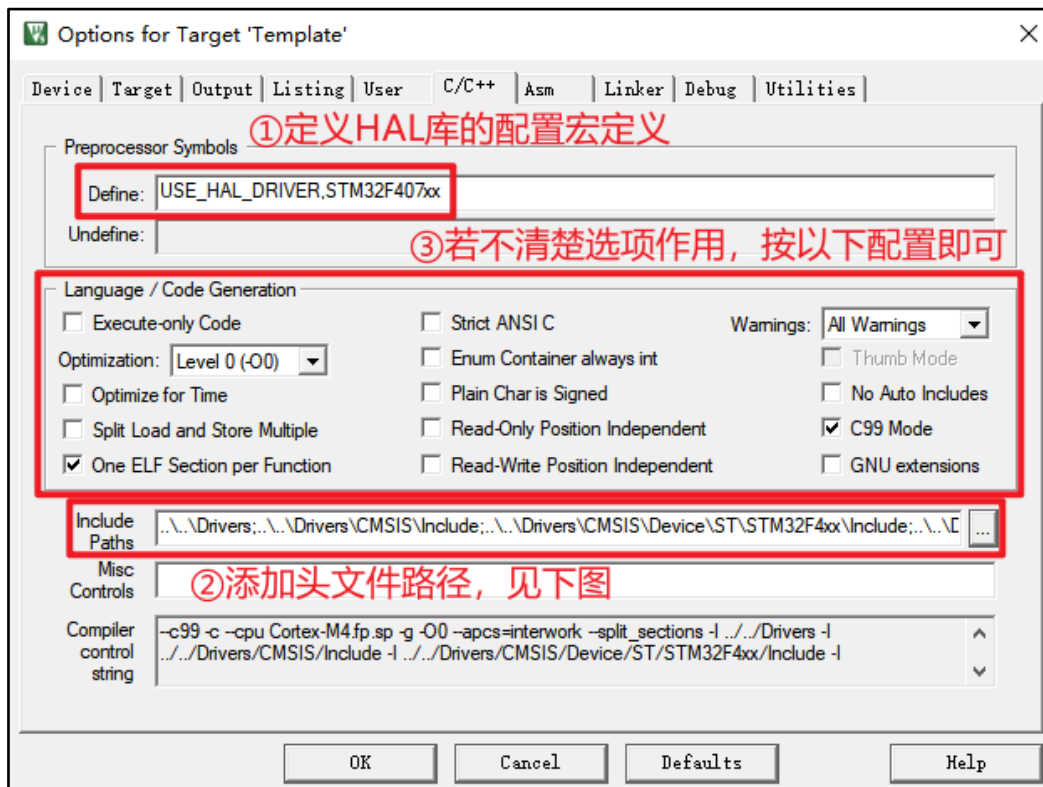


图 7.1.4.11 C/C++选项卡

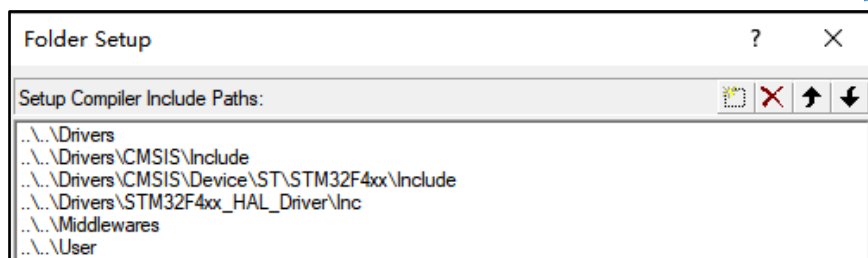


图 7.1.4.12 添加头文件路径

再接着就是与调试相关的配置，若读者需使用仿真器进行调试，请参考第 4.3 小节使用 DAP 烧录及调试程序。

至此，一个最基础的基于 HAL 库的 MDK 工程就创建好了，读者可以打开 main.c 文件查看本工程实现的功能，如下图所示：

```

21 #include "../SYSTEM/sys/sys.h"
22 #include "../SYSTEM/delay/delay.h"
23 #include "../SYSTEM/usart/usart.h"
24
25 int main(void)
26 {
27     HAL_Init();
28     sys_stm32_clock_init(336, 8, 2, 7);
29     delay_init(168);
30     usart_init(115200);
31
32     while (1)
33     {
34         printf("Hello, M100Z-M4 STM32F407!\r\n");
35         delay_ms(1000);
36     }
37 }

```

图 7.1.4.13 main.c 文件

可以看到，在 main() 函数中先进行了一些基本的初始化工作（主要是初始化系统时钟、延时功能和串口），接着在一个 while 循环中重复通过串口输出 “Hello, M144Z-M4 STM32F407\r\n” 字符串。

接下来点击工具栏中的  按钮尝试编译工程，若编译无误，则说明工程创建成功了，若有错误，建议读者仔细根据本小节的内容再好好检查一下创建的工程。

7.2 下载验证

点击 MDK 软件上方工具栏上的  按钮进行程序烧录，程序烧录成功后，便可通过串口调试助手（推荐使用 ATK-XCOM，该软件可在 A 盘→6，软件资料→1，软件→串口调试助手软件 (ATK-XOMC) 下找到）看到 USART1 不断输出 “Hello, M144Z-M4 STM32F407” 字符串，如下图所示：

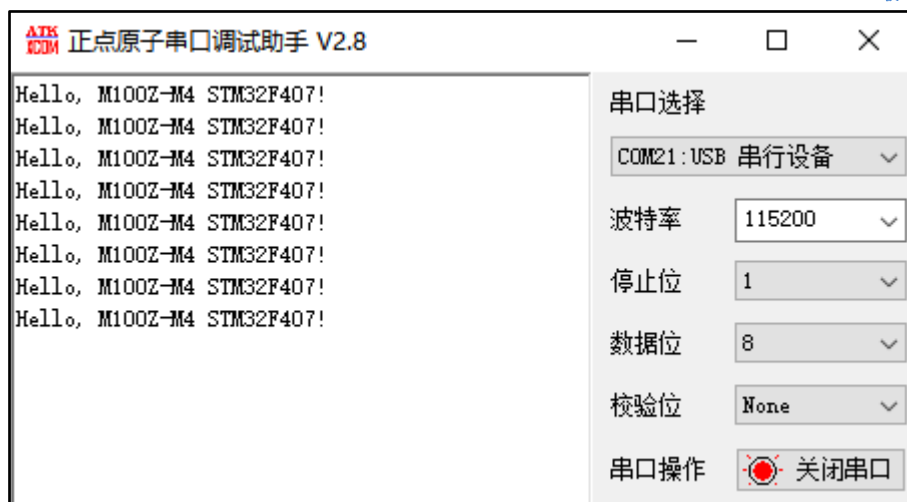


图 7.2.1 串口输出结果

第八章 STM32F407 时钟系统介绍

MCU 都是基于时序控制的系统，本章将为 STM32F407 的时钟系统作一个简单的介绍，帮助读者更全面、系统地认识 STM32F407 的时钟系统结构，并掌握 STM32F407 的时钟配置。

本章分为如下几个小节：

- 8.1 认识时钟树
- 8.2 配置系统主频
- 8.3 开启和关闭外设时钟

8.1 认识时钟树

从数字电路的知识可以知道：任意复杂的电路控制系统都可以经由门电路组成的组合电路实现。STM32 内部也是由多种多样的电路模块组合在一起实现的。若一个电路越复杂，在达到正确的输出结果前，它可能因为延时会有一些短暂的中间状态，而这些中间状态有时会导致输出结果会有一个短暂的错误，这叫做电路中的“毛刺现象”，如果电路需要运行得足够快，那么这些错误状态会被其它电路作为输入采样，最终形成一系列的系統错误。为了解决这个问题，在单片机系统中，设计时以时序电路控制替代纯粹的組合电路，在每一级输出结果前对各个信号进行采样，从而使得电路中某些信号即使出现延时也可以保证各个信号的同步，可以避免电路中发生的“毛刺现象”，达到精确控制输出的效果。

由于时序电路的重要性，因此在 MCU 设计时就设计了专门用于控制时序的电路，在芯片设计中称为时钟树设计。由此设计出来的时钟，可以精确控制单片机系统，这也是本小节要展开分析的。为什么是时钟树而不是时钟呢？一个 MCU 越复杂，时钟系统也会相应的变得复杂，如 STM32F407 的时钟系统就比较复杂，不像简单的 51 单片机，一个系统时钟就可以解决一切。对于 STM32F407 系列的芯片，正常工作的主频可以达到 168MHz，但并不是所有外设都需要系统时钟这么高的频率，比如看门狗以及 RTC 只需要几十 KHz 的时钟频率即可工作。同一个电路，时钟频率越快功耗就越大，同时抗电磁干扰能力也会越弱，所以对于较为复杂的 MCU 一般都是采取多时钟的方法来解决这些问题。

STM32 本身相对复杂，外设资源非常丰富，为了保证低功耗，STM32 上电或复位后默认不开启这些外设功能，即不为这些外设提供时钟。用户可以根据自己的需要决定 STM32 芯片要使用的功能，即为这些外设提供时钟，同时也要配置时钟源和时钟的频率。



上图就是 STM32F407 的时钟树框图

②：该部分为 PLL（锁相环），这一部分内部是较为复杂的模拟电路，主要用于对输入的时钟信号倍频后输出，输入的时钟源可以来自 HSI 或 HSE。

③：该部分主要用于选择系统时钟的时钟源，从图中可以看出，该选择器的输入有 HSE、HSI 和 PLL1，输出作用于 SYSCLK，而 SYSCLK 通过 AHB Prescaler 分频后输出，该输出将直

接或间接地作为 AHB、APB1 和 APB2 总线上外设的时钟源，其中 FCLK 就是 MCU 内核的时钟，这部分对整个系统的运行起着至关重要的作用。

④：AHB Prescaler 分频后的时钟再通过 APB1 Prescaler 和 APB2 Prescaler 分频后分别作为 APB1 和 APB2 总线上外设的时钟。

8.1.1 时钟源

STM32F407 有四个时钟源，分别为：LSI（内部低速时钟）、LSE（外部低速时钟）、HSI（内部高速时钟）、HSE（外部高速时钟）。按低速时钟源和高速时钟源可分为 LSI、LSE 和 HSI、HSE，按内部时钟源和外部时钟源可分为 LSI、HSI 和 LSE、HSE，其中内部时钟源 LSI 和 HSI 是 MCU 芯片内部的时钟源，芯片上电后即可输出时钟，无需借助外部电路。

①：外部高速时钟 HSE（High Speed External Clock）

该时钟信号由外部晶体、陶瓷谐振振荡器或外部时钟源产生，频率范围为 4MHz~26MHz，正点原子 M144Z-M4 最小系统板 STM32F407 版使用频率为 8MHz 的晶体振荡器产生该时钟信号。

②：外部低速时钟 LSE（Low Speed External Clock）

该时钟信号由外部晶体、陶瓷谐振振荡器或外部时钟源产生，频率为 32.768KHz，主要作为 RTC 的时钟源，正点原子 M144Z-M4 最小系统板 STM32F407 版使用频率为 32.768KHz 的晶体振荡器产生该时钟信号。

③：内部高速时钟 HSI（High Speed Internal Clock）

该时钟信号由内部 RC 振荡器产生，频率为 16MHz。

④：内部低速时钟 LSI（Low Speed Internal Clock）

该时钟信号由内部 RC 振荡器产生，频率为 20KHz~35KHz（受温度、电压影响），主要作为独立看门狗或 RTC 的时钟源。

MCU 上电时默认会使用 HSI 作为系统时钟启动，只有完成了相关的时钟配置，MCU 才会根据配置切换到对应的时钟源，因此同时了解这几个时钟源是很有必要的，下文将会有提到时钟配置方法的章节。

8.1.2 锁相环

锁相环（PLL）是自动控制系统中常用的一个反馈电路，在 STM32 主控中，锁相环主要有对输入时钟进行净化和倍频这两个作用，前者是利用锁相环电路的反馈机制实现的，后者是用于使 STM32 主控稳定地工作于更高的时钟频率。

在 STM32 中，锁相环的输出时钟可以配置为系统时钟的时钟源，如下图所示：

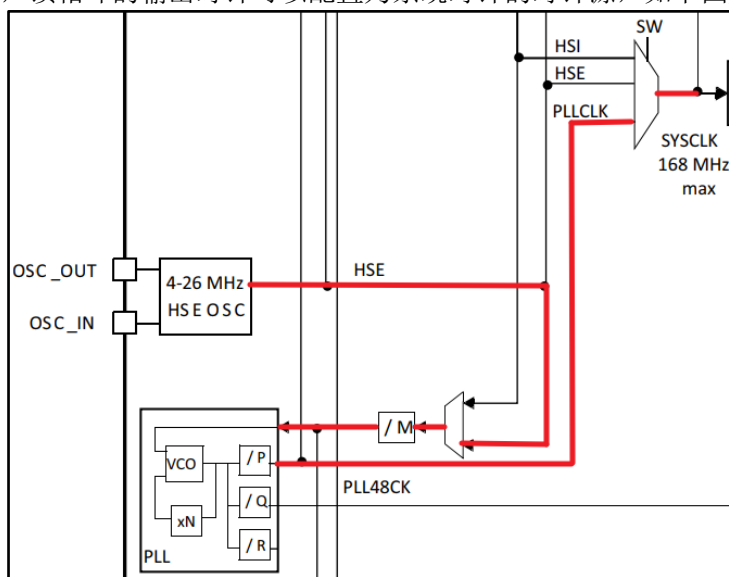


图 8.1.2.1 PLL1 输出时钟作为系统时钟示例

从上图中可以看出,HSE 的时钟 HSECLK 被作为 PLL 的输入时钟后经过 M 分频输入 PLL,随后在 PLL 中经过 N 倍频和 P 分频后作为 PLLCLK 输出,PLLCLK 被作为 SYSCLK。

上面过程中的配置基本可以由 PLL 配置寄存器 (RCC_PLLCFGR) 和时钟配置寄存器 (RCC_CFGR) 完成配置。

Bit 22 PLLSRC: Main PLL(PLL) and audio PLL (PLLI2S) entry clock source
Set and cleared by software to select PLL and PLLI2S clock source. This bit can be written only when PLL and PLLI2S are disabled.
0: HSI clock selected as PLL and PLLI2S clock entry
1: HSE oscillator clock selected as PLL and PLLI2S clock entry

图 8.1.2.2 PLL 配置寄存器 PLLSRC 位

PLLCLKS 位用于选择 PLL 的时钟源, 当该位写入 0 时, 使用 HSI 作为 PLL 的时钟源; 当该位写入 1 时, 使用 HSE 作为 PLL 的时钟源。对于示例, 将该配置为 1, 即使用频率为 8MHz 的 HSE 作为 PLL 的时钟源。

Bits 5:0 PLLM: Division factor for the main PLL (PLL) and audio PLL (PLLI2S) input clock
Set and cleared by software to divide the PLL and PLLI2S input clock before the VCO. These bits can be written only when the PLL and PLLI2S are disabled.

Caution: The software has to set these bits correctly to ensure that the VCO input frequency ranges from 1 to 2 MHz. It is recommended to select a frequency of 2 MHz to limit PLL jitter.

VCO input frequency = PLL input clock frequency / PLLM with $2 \leq \text{PLLM} \leq 63$

000000: PLLM = 0, wrong configuration
000001: PLLM = 1, wrong configuration
000010: PLLM = 2
000011: PLLM = 3
000100: PLLM = 4
...
111110: PLLM = 62
111111: PLLM = 63

图 8.1.2.3 PLL 配置寄存器 PLLM 位

PLLM 位用于对输入 PLL 的时钟信号进行分频, 可分频的范围为 2~63。对于示例, 将该位配置为 8, 即对输入的频率为 8MHz 的 HSE 进行 8 分频, 那么输入 PLL 的时钟频率为 1MHz。

Bits 14:6 PLLN: Main PLL (PLL) multiplication factor for VCO

Set and cleared by software to control the multiplication factor of the VCO. These bits can be written only when PLL is disabled. Only half-word and word accesses are allowed to write these bits.

Caution: The software has to set these bits correctly to ensure that the VCO output frequency is between 100 and 432 MHz.

VCO output frequency = VCO input frequency $\times$ PLLN with $50 \leq \text{PLLN} \leq 432$

00000000: PLLN = 0, wrong configuration

00000001: PLLN = 1, wrong configuration

...

000110010: PLLN = 50

...

001100011: PLLN = 99

001100100: PLLN = 100

...

110110000: PLLN = 432

110110001: PLLN = 433, wrong configuration

...

111111111: PLLN = 511, wrong configuration

Note: Multiplication factors ranging from 50 and 99 are possible for VCO input frequency higher than 1 MHz. However care must be taken that the minimum VCO output frequency respects the value specified above.

图 8.1.2.4 PLL 配置寄存器 PLLN 位

PLLN 用于对输入 PLL 的时钟信号进行倍频，可倍频的范围为 2~432。对于示例，将该位配置为 336，即可将输入 PLL 的 1MHz 时钟信号倍频为 336MHz。

Bits 17:16 PLLP: Main PLL (PLL) division factor for main system clock

Set and cleared by software to control the frequency of the general PLL output clock. These bits can be written only if PLL is disabled.

Caution: The software has to set these bits correctly not to exceed 180 MHz on this domain.

PLL output clock frequency = VCO frequency / PLLP with PLLP = 2, 4, 6, or 8

00: PLLP = 2

01: PLLP = 4

10: PLLP = 6

11: PLLP = 8

图 8.1.2.5 PLL 配置寄存器 PLLP 位

PLLP 用于对 PLL 待输出的时钟信号进行分频，可分频的值为 2 分频、4 分频、6 分频和 8 分频。对于示例，将该位写入 0，即对 336MHz 的待输出时钟信号进行 2 分频后输出 168MHz 的时钟信号。

Bits 1:0 SW: System clock switch

Set and cleared by software to select the system clock source.

Set by hardware to force the HSI selection when leaving the Stop or Standby mode or in case of failure of the HSE oscillator used directly or indirectly as the system clock.

00: HSI oscillator selected as system clock

01: HSE oscillator selected as system clock

10: PLL selected as system clock

11: not allowed

图 8.1.2.6 时钟配置寄存器 SW 位

SW 位用于选择 SYSCLK 的时钟源，可配置 HSICLK、HSECLK 或 PLLCLK 作为 SYSCLK 的时钟源。对于示例，将该位写入 2，即可选择频率为 168MHz 的 PLLCLK 作为 SYSCLK 时钟源。

8.1.3 系统时钟

STM32 的系统时钟 (SYSCLK) 为整个芯片的绝大多数外设和内核提供时钟信号, 对于相同稳定运行的系统, 时钟信号的频率越高, 系统运行的速度也就越快, 单位时间能够处理的事务也就越多。在上一小节中, 将频率为 8MHz 的 HSE 时钟信号通过 PLL 倍频为 168MHz 的时钟信号作为 SYSCLK 的时钟源。SYSCLK 时钟信号可直接或间接地作为 AHB 总线、APB1 总线、APB2 总线和内核的时钟源。

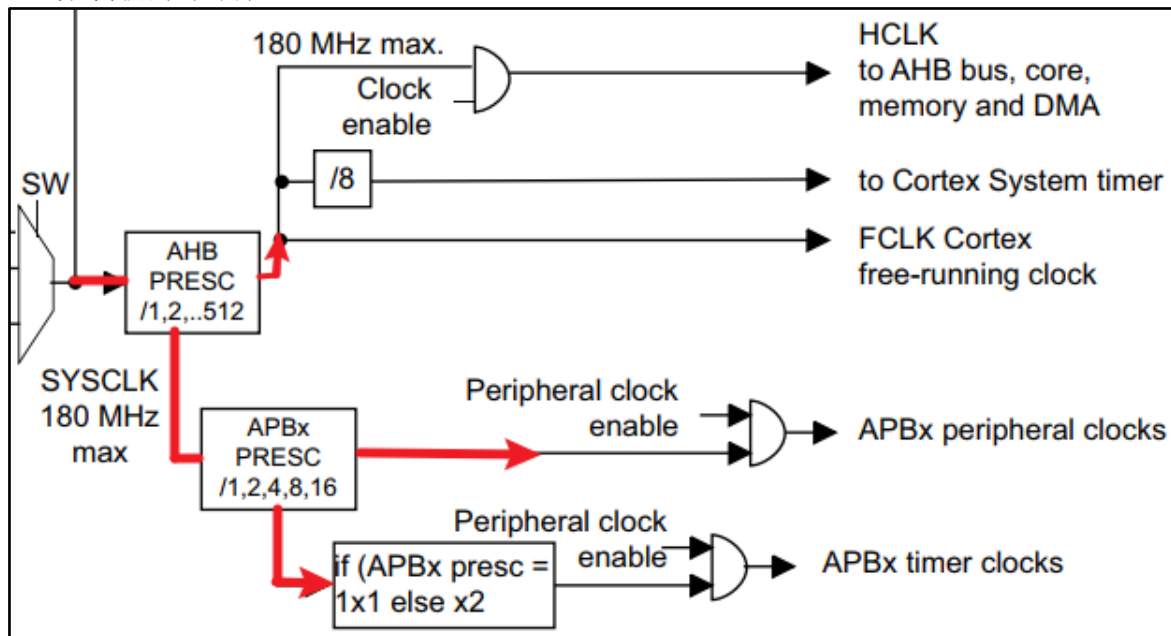


图 8.1.3.1 各总线和内核时钟配置示例

①: AHB 预分频器, 可对 SYSCLK 进行分频, 分频范围为 1~512。对于示例, AHB 预分频器选择不分频 (/1), 那么 AHB 总线、内核、APB1 总线和 APB2 总线输入的时钟信号频率都为 168MHz。

②: APB1 预分频器, 可对待输入 APB1 总线的时钟信号进行分频, 分频范围为 1~16, APB1 总线的时钟频率最大值为 42MHz。对于示例, APB1 预分频器选择 4, 即对频率为 168MHz 的时钟信号进行 4 分频为 42MHz 后作为 APB1 总线的时钟频率。

③: APB2 预分频器, 可对待输入 APB2 总线的时钟信号进行分频, 分频范围为 1~16, APB2 总线的时钟频率最大值为 84MHz。对于示例, APB2 预分频器选择 2, 即对频率为 168MHz 的时钟信号进行 2 分频为 84MHz 后作为 APB2 总线的时钟频率。

8.2 配置系统主频

STM32F407 在默认情况下, 使用频率为 16MHz 的 HSI 作为系统时钟的时钟源, 因此无需外部晶振也能够正常烧录和运行程序。

在上一小节中, 介绍了如何配置时钟树, 以让内核在 168MHz 的主频下运行, 168MHz 是 ST 官方针对 STM32F407 推荐的最大工作频率。

本节将介绍如何在代码中具体的对 STM32F407 的时钟进行配置。本书配置实验例程都是在 main() 函数中调用 sys_stm32_clock_init() 对 STM32F407 的时钟进行配置的, 如下所示:

```
int main(void)
{
    /* 省略其他无关代码 */

    sys_stm32_clock_init(336, 8, 2, 7); /* 配置系统时钟 */

    /* 省略其他无关代码 */
}
```

相信读者对 sys_stm32_clock_init() 函数传入的参数并不陌生, 这就是在上一小节示例中配置

PLL 配置寄存器中 PLLN、PLLM、PLLp 和 PLLQ 各位的值。

sys_stm32_clock_init()函数的代码比较多, 请读者在实验例程中查看本函数的具体实现, 本小节中仅列出该函数的关键代码, 如下所示:

```
uint8_t sys_stm32_clock_init( uint32_t pll_n,
                               uint32_t pll_m,
                               uint32_t pll_p,
                               uint32_t pll_q)
{
    /* 本函数的部分代码省略, 仅列出关键代码 */

    RCC_OscInitTypeDef rcc_osc_init = {0};
    RCC_ClkInitTypeDef rcc_clk_init = {0};

    /* 配置 PLL */
    rcc_osc_init.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    rcc_osc_init.HSEState = RCC_HSE_ON;
    rcc_osc_init.PLL.PLLState = RCC_PLL_ON;
    rcc_osc_init.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    rcc_osc_init.PLL.PLLN = pll_n;
    rcc_osc_init.PLL.PLLM = pll_m;
    rcc_osc_init.PLL.PLLP = pll_p;
    rcc_osc_init.PLL.PLLQ = pll_q;
    HAL_RCC_OscConfig(&rcc_osc_init);

    /* 配置时钟 */
    rcc_clk_init.ClockType = ( RCC_CLOCKTYPE_SYSCLK | \
                               RCC_CLOCKTYPE_HCLK | \
                               RCC_CLOCKTYPE_PCLK1 | \
                               RCC_CLOCKTYPE_PCLK2);
    rcc_clk_init.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    rcc_clk_init.AHBCLKDivider = RCC_SYSCLK_DIV1;
    rcc_clk_init.APB1CLKDivider = RCC_HCLK_DIV4;
    rcc_clk_init.APB2CLKDivider = RCC_HCLK_DIV2;
    HAL_RCC_ClockConfig(&rcc_clk_init, FLASH_LATENCY_5);

    return 0;
}
```

跟上一小节介绍的一致, AHB、APB2 和 APB1 总线的预分频器分别配置为不分频、2 分频和 4 分频; 根据函数的传入参数配置 PLL 配置寄存器中 PLLN、PLLM、PLLp 和 PLLQ 各位的值, 并将 PLL 的时钟源配置为 HSE; 将 PLLCLK 配置为 SYSCLK 的时钟源。上面列出的代码仅是一些关键代码, 除了列出代码的操作外还需要一些额外的操作, 例如使能 HSE 并等待 HSE 稳定等, 建议读者打开本书配套的任意实验例程, 在 sys.c 文件中详细地查看本函数的具体实现。

通过 sys_stm32_clock_init()函数就能够非常方便地配置系统主频了, 例如, 在考虑对 MCU 的性能需求和功耗的场景中, 可以适当降低 MCU 的主频, 来达到性能和功耗的平衡。

8.3 开启和关闭外设时钟

在前面的章节中, 分析了 STM32F407 的时钟系统和相关的配置步骤, 但是在使用某些外设之前, 还需要手动开启相应外设的时钟, 这是因为 MCU 在上电或复位后会默认关闭大部分外设的时钟, 以降低功耗, 因此, 在程序开发中, 对于某些开启的时钟但不再使用的外设, 也应将其时钟关闭。

STM32F407 外设的时钟由 RCC (Reset and Clock Control) 控制, RCC 中控制外设时钟的寄存器被按总线进行分类, 涉及了五个寄存器, 分别为 RCC_AHB1ENR、RCC_AHB2ENR、RCC_AHB3ENR、RCC_APB1ENR 和 RCC_APB2ENR, 这五个寄存器分别用于管理 AHB 总线、APB1 总线和 APB2 总线上外设的时钟。

下面以 GPIOA 为例, 介绍如何配置 RCC 中相应的寄存器来开启或关闭 GPIOA 的时钟。首先能够确定 GPIOA 是位于 AHB 总线上的外设。

接下来据需要在 RCC 的 RCC_AHB1ENR、RCC_AHB2ENR 和 RCC_AHB3ENR 寄存器中找到控制 GPIOA 时钟的寄存器, 这里有个小技巧, RCC_AHB1ENR、RCC_AHB2ENR 和

RCC_AHB3ENR 这三个寄存器中控制外设时钟的位，基本都是按照外设映射地址的高低来排序的，因为 GPIOA~GPIOI 的映射地址在 AHB 总线上是最低的，因此基本可以判断用于控制 GPIOA 时钟的寄存器为 RCC_AHB1ENR。在 RCC_AHB1ENR 寄存器中可以找到 GPIOAEN 位，该位就是用于控制 GPIOA 外设时钟使能和禁止的，如下图所示：

Bit 0 **GPIOAEN**: IO port A clock enable
Set and cleared by software.
0: IO port A clock disabled
1: IO port A clock enabled

图 8.3.3 RCC_AHB1ENR 寄存器 GPIOAEN 位

由上图可知，往 RCC_AHB1ENR 寄存器的 GPIOAEN 位写入 1 后，就能配置使能 GPIOA 外设的时钟，往 RCC_AHB1ENR 寄存器的 GPIOAEN 位写入 0 后，就能配置禁止 GPIOA 外设的时钟。

在 HAL 库中提供了用于开启和关闭外设时钟的宏定义，这些宏定义被定义在 stm32f4xx_hal_rcc.h 文件中，如以所示：

```
#define __HAL_RCC_GPIOA_CLK_ENABLE() \
do { \
    __IO uint32_t tmpreg = 0x00U; \
    SET_BIT(RCC->AHB1ENR, RCC_AHB1ENR_GPIOAEN); \
    /* Delay after an RCC peripheral clock enabling */ \
    tmpreg = READ_BIT(RCC->AHB1ENR, RCC_AHB1ENR_GPIOAEN); \
    UNUSED(tmpreg); \
} while(0U)

#define __HAL_RCC_GPIOA_CLK_DISABLE() (RCC->AHB1ENR &= ~(RCC_AHB1ENR_GPIOAEN))
```

上面的两个宏定义中，__HAL_RCC_GPIOA_CLK_ENABLE()用于使能 GPIOA 端口的时钟，而 __HAL_RCC_GPIOA_CLK_DISABLE()用于失能 GPIOA 端口的时钟。

细心的读者可能会发现 HAL 库提供的用于使能和失能外设时钟的宏定义，其命名规则大致为 __HAL_RCC_+外设名称+_CLK_+ENABLE 或 DISABLE()。

第九章 SYSTEM 文件夹介绍

SYSTEM 文件夹由正点原子提供，里面包含了一系列系统底层核心驱动代码，目的是为了 方便读者快速构建自己的工程。本章将介绍 SYSTEM 文件夹中的代码，也希望读者能够灵活地 使用 SYSTEM 文件夹中提供的各种函数来快速构建工程，并应用到实际的项目中。

SYSTEM 文件夹下包含了 sys、delay 和 usart 三个文件夹，分别包含了 sys.s、delay.c 和 usart.c 以及相关的头文件，这三个 C 源文件提供了系统时钟设置、延时和串口 1 调试等功能，通过这 些功能能够起到快速移植和辅助开发的作用。

本章分为如下几个小节：

9.1 sys 文件夹代码介绍

9.2 delay 文件夹代码介绍

9.3 usart 文件夹代码介绍

9.1 sys 文件夹代码介绍

sys 文件夹内包含了 sys.c 和 sys.h 两个文件，在 sys.c 文件中主要实现了如下几个函数：

```
void sys_nvic_set_vector_table(uint32_t baseaddr, uint32_t offset);
void sys_standby(void);
void sys_soft_reset(void);
uint8_t sys_stm32_clock_init(  uint32_t pll_n,
                                uint32_t pll_m,
                                uint32_t pll_p,
                                uint32_t pll_q);
void sys_wfi_set(void);
void sys_intx_disable(void);
void sys_intx_enable(void);
void sys_msr_msp(uint32_t addr);
```

这些函数主要作辅助开发的用途，都比较简单，看函数的注释也能明白函数的作用，最重要的 sys_stm32_clock_init()配置系统时钟函数也在前面的章节中分析了，这里便不再对 sys.c 文件 中的函数进行分析。

9.2 delay 文件夹代码介绍

delay 文件夹中包含了 delay.c 和 delay.h 两个文件，在 delay.h 文件中声明了以下一个函数：

```
/* 函数声明 */
void delay_init(uint16_t sysclk);      /* 初始化延时功能 */
void delay_us(uint32_t nus);          /* 微秒级延时 */
void delay_ms(uint16_t nms);          /* 毫秒级延时 */
```

以上三个函数均在 delay.c 文件中实现，分别用于初始化延时功能、微秒级延时和毫秒级延 时，该延时功能区分了裸机和操作系统的两种情况。

在裸机的情况下，该延时为阻塞延时，即 CPU 忙延时，说得再通俗一点就是“死等”，再延 时期间，CPU 只会不断地判断延时时间是否超时，而不会做其他任何事情。具体的实现是利用了 SysTick，SysTick 是一个 24 位的向下递减计数器，在初始化延时功能的时候，会配置 SysTick 的 计数频率为系统时钟频率，即 168MHz，也就是说 SysTick 的计数值减少 168 就耗时 1 微秒（不 精确），如果要延时 n 微秒，那么只要让 SysTick 从 n*168 开始计数，当 SysTick 的计数值为 0 时，就说明延时了 n 微秒。而毫秒级延时就是循环延时多个 1000 微秒。

在操作系统的情况下，一般操作系统都会提供一个毫秒级的非阻塞延时，那么就可以使用操 作系统提供的延时功能来进行毫秒级延时。而微秒级延时，则先将待延时的微秒数转换为毫秒数 进行毫秒级延时，若还有余数，则使用裸机情况下的微秒级延时进行延时。

9.3 usart 文件夹代码介绍

usart 文件夹中包含了 usart.c 和 usart.h 两个文件，这两个文件主要用于驱动 USART1 进行串

口数据的收发，方面调试。有关串口的相关知识，会在下文“串口通信实验”中具体讲解，本小节仅讲解 printf 函数的应用。

9.3.1 printf 函数支持

在学习 C 语言时，可以通过 printf 函数在终端里显示信息，以此做一些简单的调试，但对于单片机来说，是否有办法实现使用 printf 进行辅助调试呢？

C 标准库的 printf 为调试属性的函数，如果直接使用，会使单片机进入半主机模式（semihosting），这是一种调试模式，具体表现为直接下载代码后出现程序无法运行，但是在连接调试器进行 Debug 时程序反而能正常工作。半主机模式是 ARM 目标系统的一种机制，用于将输入或输出请求从应用程序代码通信到运行调试器的主机。例如，此机制可用于允许 C 标准库中的函数（如 printf() 和 scanf()）使用主机的屏幕和键盘，而不是在目标系统上设置屏幕和键盘。这很有用，因为开发硬件通常不具有最终系统的所有输入和输出设备，如屏幕、键盘等。半主机模式是通过一组定义好的软件指令（如 SVC 指令（以前称为 SWI 指令））来实现的，这些指令通过程序控制产生异常。应用程序调用相应的半主机调用，然后调试代理处理该异常。调试代理（这里的调试代理是仿真器）提供与主机之间的必需通信。也就是说使用半主机模式必须使用仿真器调试。

在独立环境下运行调试功能的函数，例如：printf 函数，printf 对字符 ch 处理后写入文件 f，最后使用 fputc 将文件 f 输出到显示设备。对于 PC 端的设备，fputc 通过复杂的源码，最终把字符显示到屏幕上。那我们需要做的，就是把 printf 调用的 fputc 函数重新实现，重定向 fputc 的输出端为串口，同时避免进入半主模式。

要避免半主机模式，主要有两种方式：一是使用 MicroLIB，即微库；另一种方法是确保 ARM 应用程序中没有链接 MicroLIB 的半主机相关函数。

先说微库，MicroLIB 是 ARM 为嵌入式设备开发的一套类似于标准 C 接口函数的精简代码库，用于替代默认 C 库，是专门针对专业嵌入式应用开发而设计的，特别适合那些对存储空间有特别要求的嵌入式应用程序，这些程序一般不在操作系统下运行。使用微库编写程序要注意其与默认 C 库之间存在的一些差异，如 main() 函数不能声明带参数，也无须返回；不支持 stdio，除了无缓冲的 stdin、stdout 和 stderr；微库不支持操作系统函数；微库不支持可选的单或两区存储模式；微库只提供分离的堆和栈两区存储模式等等，它裁减了很多函数，而且还有很多东西不支持。如果原来用标准库可以跑，选择 MicroLIB 后却突然不行了，是很常见的。与标准的 C 库不一样，微库重新实现了 printf，使用微库的情况下就不会进入半主机模式了。MDK 下使用微库的方法很简单，在“Target”下勾选“Use MicroLIB”即可，如下图所示：

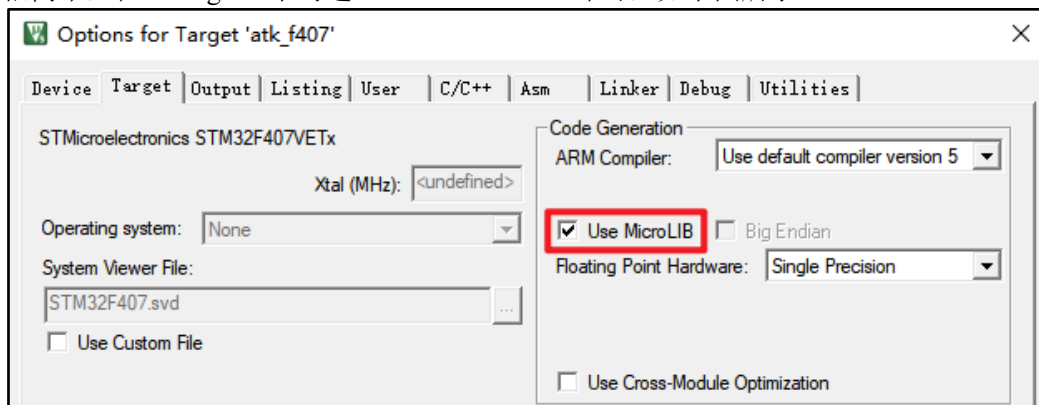


图 9.3.1.1 MDK 下 MicroLIB 开启方法

在 MDK 中，不管是否使用半主机模式，使用 printf、scanf、fopen 和 fread 等函数都需要自行填充底层函数，以 printf 函数为例，需要自行实现 fputc 函数，若启用微库，在初始化和使能串口 1 之后，只需实现 fputc 函数即可将每个传给 fputc 函数的字符 ch 重定向到串口 1，如果这时接上串口调试助手的话，可以看到串口的数据。实现的代码如下：

```
int fputc(int ch, FILE *f)
{
    while ((USART1->SR & 0X40) == 0);
```

```

USART1->DR = (uint8_t)ch;

return ch;
}

```

上面说到了微库的一些限制，使用时注意某些函数与标准库的区别就不会影响到我们代码的正常功能。如果不想使用微库，那就要用到我们提到的第二种方法：确保 ARM 应用程序中没有链接 MicroLIB 的半主机相关函数。只需在代码中添加不使用半主机的声明即可，对于 AC5 编译器版本，具体的实现代码如下：

```
#pragma import(__use_no_semihosting)
```

这样一来，MDK 就不会把标准库的这部分函数链接进最终编译出的程序中了。

同时，由于缺少 __FILE 结构体，因此额外进行定义，如下所示：

```

struct __FILE
{
    int handle;
};

```

如果用到原来半主机模式下的调试函数，需要重新实现它的一些依赖函数接口，对于 printf 函数需要实现的接口如下：

```

int _ttywrch(int ch)
{
    ch = ch;
    return ch;
}

void _sys_exit(int x)
{
    x = x;
}

char *_sys_command_string(char *cmd, int len)
{
    return NULL;
}

```

fputc 的重定向和上文的一样，重定向到串口 1 即可，如果硬件资源允许，或者有特殊需求，也可以重定向到 LCD 或者其它串口。

```

int fputc(int ch, FILE *f)
{
    while ((USART1->SR & 0X40) == 0);
    USART1->DR = (uint8_t)ch;

    return ch;
}

```

第二篇 入门篇

前面的章节介绍了开发 STM32 的工具和方法，本篇将介绍 STM32 的一些基础外设，并着重讲解如何使用 HAL 库为 STM32 的外设编写驱动程序。

本篇将采取一章一实例的方式，介绍 STM32 基础外设的使用，相信通过本篇的学习，读者便能够玩转 STM32，并应用到实际的项目开发中去。

第十章 跑马灯实验

跑马灯程序是嵌入式开发的一个经典程序，类似于学习 C 语言时，编写的“Hello World”程序。跑马灯本质上是控制单片机的 GPIO 输出高低电平，以此达到控制 LED 等亮灭状态的切换。通过本章的学习，读者将学习到 GPIO 输出模式的使用。

本章分为如下几个小节：

- 10.1 硬件设计
- 10.2 程序设计
- 10.3 下载验证

10.1 硬件设计

10.1.1 例程功能

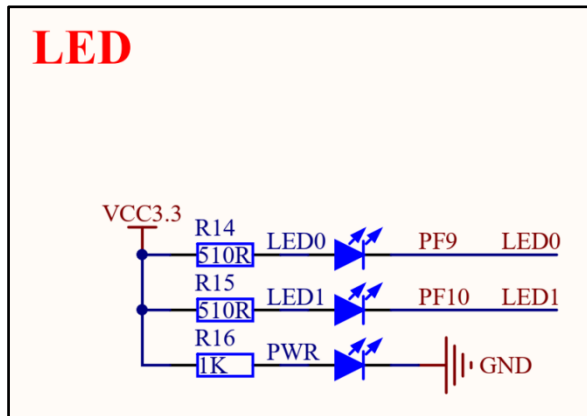
1. LED0 和 LED1 交替闪烁。

10.1.2 硬件资源

1. LED
 - LED0 - PF9
 - LED1 - PF10

10.1.3 原理图

本章实验用的两个 M144Z-M4 最小系统板 STM32F407 版板载 LED，分别为 LED0（红色）和 LED1（绿色），其与板载 MCU 的连接原理图，如下图所示：



10.1.3.1 LED 与 MCU 的连接原理图

从上面原理图中可以看出，LED0 和 LED1 的正极分别通过一个限流电阻连接到了电源正极，而负极分别与 MCU 的 PF9 引脚和 PF10 引脚相连接，因此只需通过控制 PF9 引脚或 PF10 引脚输出低电平，则能分别控制 LED0 和 LED1 点亮，反之，则熄灭。

10.2 程序设计

10.2.1 HAL 库的 GPIO 驱动

本章实验中要通过控制 GPIO 引脚输出高低电平来控制 LED 的亮灭状态，因此需要以下两个步骤：

- ①：配置 GPIO 引脚为输出模式
- ②：设置 GPIO 引脚输出电平

在 HAL 库中对应的驱动函数如下：

①：配置 GPIO 引脚

该函数用于配置 GPIO 引脚的功能和各项参数，其函数原型如下所示：

```
void HAL_GPIO_Init(GPIO_TypeDef *GPIOx, GPIO_InitTypeDef *GPIO_Init);
```

该函数的形参描述，如下表所示：

形参	描述
GPIOx	指向 GPIO 端口结构体的指针 例如：GPIOA、GPIOB 等（在 stm32f407xx.h 文件中有定义）
GPIO_Init	指向 GPIO 初始化结构体的指针 需自行定义，并根据 GPIO 的配置参数填充结构体中的成员变量

表 10.2.1.1 函数 HAL_GPIO_Init()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
无	无

表 10.2.1.2 函数 HAL_GPIO_Init()返回值描述

该函数使用 GPIO_InitTypeDef 类型的结构体变量传入 GPIO 引脚的配置参数，该结构体的定义如下所示：

```
typedef struct
{
    uint32_t Pin;          /* 待配置的引脚 */
    uint32_t Mode;         /* 模式 */
    uint32_t Pull;         /* 上拉/下拉 */
    uint32_t Speed;        /* 速度 */
    uint32_t Alternate;    /* 连接的外设 */
}GPIO_InitTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    GPIO_InitTypeDef gpio_init_struct = {0};

    /* 配置 PA0 引脚为输出模式 */
    gpio_init_struct.Pin = GPIO_PIN_0;
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;
    gpio_init_struct.Pull = GPIO_PULLUP;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &gpio_init_struct);
}
```

②：设置 GPIO 引脚输出电平

该函数用于设置 GPIO 引脚输出指定电平（高电平或低电平），其函数原型如下所示：

```
void HAL_GPIO_WritePin(GPIO_TypeDef* GPIOx,
                        uint16_t GPIO_Pin,
                        GPIO_PinState PinState);
```

该函数的形参描述，如下表所示：

形参	描述
GPIOx	指向 GPIO 端口结构体的指针 例如：GPIOA、GPIOB 等（在 stm32f407xx.h 文件中有定义）
GPIO_Pin	待设置的 GPIO 引脚号 例如：GPIO_PIN_0、GPIO_PIN_1 等（在 stm32f4xx_hal_gpio.h 文件中有定义）
PinState	GPIO 引脚输出的电平 GPIO_PIN_RESET：输出低电平 GPIO_PIN_SET：输出高电平

表 10.2.1.3 函数 HAL_GPIO_WritePin()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
无	无

表 10.2.1.4 函数 HAL_GPIO_WritePin()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 设置 PA0 引脚输出低电平 */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_RESET);

    /* 设置 PA0 引脚输出高电平 */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_SET);
}
```

③：翻转 GPIO 引脚输出电平（补充）

该函数用于翻转 GPIO 引脚的输出电平，其函数原型如下所示：

```
void HAL_GPIO_TogglePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
```

该函数的形参描述，如下表所示：

形参	描述
GPIOx	指向 GPIO 端口结构体的指针 例如：GPIOA、GPIOB 等（在 stm32f407xx.h 文件中有定义）
GPIO_Pin	待设置的 GPIO 引脚号 例如：GPIO_PIN_0、GPIO_PIN_1 等（在 stm32f4xx_hal_gpio.h 文件中有定义）

表 10.2.1.5 HAL_GPIO_TogglePin()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
无	无

表 10.2.1.6 HAL_GPIO_TogglePin()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 翻转 PA0 引脚输出电平 */
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_0);
}
```

10.2.2 LED 驱动

LED 驱动主要就是控制 GPIO 引脚输出高低电平，来控制 LED 亮起或熄灭。本章实验中，LED 的驱动代码包括 led.c 和 led.h 两个文件（本书配套实验例程中，器件或外设的驱动代码基本都由一个 C 源文件和一个对应的头文件组成）。

根据原理图可知，应当将 PF9 引脚和 PF10 引脚配置为通用输出模式，并在需要控制 LED0（LED1）亮起的时候，设置 PF9 引脚（PF10 引脚）输出低电平，在需要控制 LED0（LED1）熄灭的时候，设置 PF9 引脚（PF10 引脚）输出高电平。

LED 驱动中，对引脚的定义，如下所示：

```
#define LED0_GPIO_PORT      GPIOF
#define LED0_GPIO_PIN      GPIO_PIN_9
#define LED0_GPIO_CLK_ENABLE() do { __HAL_RCC_GPIOF_CLK_ENABLE(); } while (0)

#define LED1_GPIO_PORT      GPIOF
#define LED1_GPIO_PIN      GPIO_PIN_10
#define LED1_GPIO_CLK_ENABLE() do { __HAL_RCC_GPIOF_CLK_ENABLE(); } while (0)
```

在后续的实验代码中，基本都会使用上述的宏定义的方式定义 GPIO 引脚的各种信息（GPIO 端口、GPIO 引脚号、GPIO 端口时钟使能和 GPIO 复用功能等信息）。

LED 驱动中，操作引脚的定义，如下所示：

```
#define LED0(x) \
do { x ? \
    HAL_GPIO_WritePin(LED0_GPIO_PORT, LED0_GPIO_PIN, GPIO_PIN_SET) : \
    HAL_GPIO_WritePin(LED0_GPIO_PORT, LED0_GPIO_PIN, GPIO_PIN_RESET); \
} while (0)

#define LED1(x) \
do { x ? \
    HAL_GPIO_WritePin(LED1_GPIO_PORT, LED1_GPIO_PIN, GPIO_PIN_SET) : \
    HAL_GPIO_WritePin(LED1_GPIO_PORT, LED1_GPIO_PIN, GPIO_PIN_RESET); \
} while (0)

#define LED0_TOGGLE() \
do { \
    HAL_GPIO_TogglePin(LED0_GPIO_PORT, LED0_GPIO_PIN); \
} while (0)

#define LED1_TOGGLE() \
do { \
    HAL_GPIO_TogglePin(LED1_GPIO_PORT, LED1_GPIO_PIN); \
} while (0)
```

在后续实验代码中，基本都会使用上述的宏定义的方式定义 GPIO 引脚的操作（设置 GPIO 引脚输出高电平或低电平、翻转 GPIO 引脚输出电平和读取 GPIO 引脚输入电平等操作）。

LED 驱动中，LED 的初始化函数，如下所示：

```
/**
 * @brief 初始化 LED
 * @param 无
 * @retval 无
 */
void led_init(void)
{
    GPIO_InitTypeDef gpio_init_struct = {0};

    /* 使能 GPIO 端口时钟 */
    LED0_GPIO_CLK_ENABLE();
    LED1_GPIO_CLK_ENABLE();

    /* 配置 LED0 控制引脚 */
    gpio_init_struct.Pin = LED0_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;
    gpio_init_struct.Pull = GPIO_PULLDOWN;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(LED0_GPIO_PORT, &gpio_init_struct);

    /* 配置 LED1 控制引脚 */
    gpio_init_struct.Pin = LED1_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;
    gpio_init_struct.Pull = GPIO_PULLDOWN;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(LED1_GPIO_PORT, &gpio_init_struct);

    /* 关闭 LED0、LED1 */
    LED0(1);
    LED1(1);
}
```

LED 的初始化函数中，使能了 LED0 和 LED1 控制引脚的 GPIO 端口时钟，并将其配置为通用输出模式，最后默认将 LED 的状态设置为熄灭状态。

10.2.3 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    HAL_Init();                /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168);           /* 初始化延时 */
    usart_init(115200);        /* 初始化串口 */
    led_init();                /* 初始化 LED */

    while (1)
    {
        LED0(0);              /* 开启 LED0 */
        LED1(1);              /* 关闭 LED1 */
        delay_ms(500);         /* 延时 500 毫秒 */
        LED0(1);              /* 关闭 LED0 */
        LED1(0);              /* 开启 LED1 */
        delay_ms(500);         /* 延时 500 毫秒 */
    }
}
```

可以看到，在调用 LED 初始化之前，会先调用以下函数：

```
HAL_Init();                /* 初始化 HAL 库 */
sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
delay_init(168);           /* 初始化延时 */
usart_init(115200);        /* 初始化串口 */
```

- ①：第一行代码用于 HAL 库的初始化。
- ②：第二行代码用于配置系统时钟，因为大部分实验例程都无需考虑低功耗应用，因此大部分实验例程配置系统时钟的参数都与本实验一致，将系统主频配置为 168MHz。
- ③：第三行代码用于初始化延时功能，为后续的应用代码提供微秒级和毫秒级的延时的功能。
- ④：第四行代码用于初始化用于调试功能的 USART1，为后续的应用代码提供 printf 等功能，方面通过串口调试助手查看程序运行情况。
- ⑤：以上四行代码都是执行一些必要的初始化，基本在后续的每一个实验例程中都会看见，后续不再赘述。

执行完必要的初始化后，紧接着初始化 LED，然后在一个 while 循环中重复地控制板载 LED0 和 LED1 轮流亮起和熄灭，轮流时间为 500 毫秒，以此实现跑马灯的效果。

10.3 下载验证

在完成编译和烧录操作后，可以看到板子上的 LED0 和 LED1 轮流亮起和熄灭，轮流的时间大约为 500 毫秒，与预期的实验效果相符。

第十一章 按键输入实验

上一章中介绍了 GPIO 的输出模式，并用其控制 LED 的亮灭。在实际的应用尝试中，还会需要使用到 GPIO 的输入模式，来获取外部的输入信号，例如获取按键的状态。通过本章的学习，读者将学习到 GPIO 作为输入模式的使用。

本章分为如下几个小节：

- 11.1 硬件设计
- 11.2 程序设计
- 11.3 下载验证

11.1 硬件设计

11.1.1 例程功能

1. 按下 WKUP 按键，LED0 状态翻转。
2. 按下 KEY0 按键，LED1 状态翻转。

11.1.2 硬件资源

1. LED
 - LED0 - PF9
 - LED1 - PF10
2. 按键
 - WKUP - PA0
 - KEY0 - PE4

11.1.3 原理图

本章实验使用的两个 M144Z-M4 最小系统板 STM32F407 版板载按键，分别为 KEY0 按键和 WKUP 按键，其于板载 MCU 的连接原理图，如下图所示：

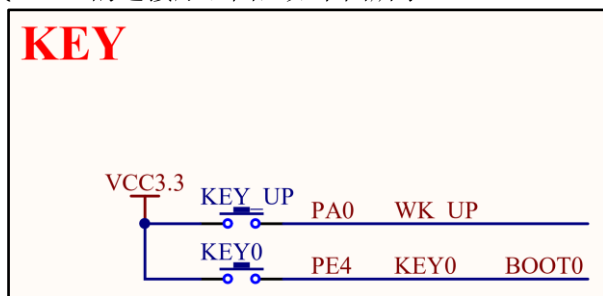


图 11.1.3.1 按键与 MCU 的连接原理图

从上面的原理图中可以看出，WKUP 按键和 KEY0 按键的一端连接到了电源正极，而另一端分别与 MCU 的 PA0 引脚和 PE4 引脚相连接，因此当任意一个按键被按下时，MCU 对应的引脚都能够读取到高电平的状态，而当松开按键后，MCU 对应的引脚读取到的电平状态却是确定的，因此用于读取 WKUP 按键和 KEY0 按键的 PA0 引脚和 PE4 引脚不仅要配置为输入模式，还需要配置成下拉，使对应引脚在悬空时被下拉在电源负极。

11.2 程序设计

11.2.1 HAL 库的 GPIO 驱动

本章实验中要通过读取 GPIO 引脚的输入状态来判断按键是否被按下，以此来控制对应 LED 的状态翻转，因此对于判断按键的状态，需要以下几个步骤：

①：配置 GPIO 引脚为输入模式和下拉

②：读取 GPIO 引脚的输入状态

在 HAL 库中对应的驱动函数如下：

①：配置 GPIO 引脚

请见第 10.2.1 小节中配置 GPIO 引脚的相关内容。

②：读取 GPIO 引脚输入电平

该函数用于读取 GPIO 引脚的输入电平（高电平或低电平），其函数原型如下所示：

```
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
```

该函数的形参描述，如下表所示：

形参	描述
GPIOx	指向 GPIO 端口结构体的指针 例如：GPIOA、GPIOB 等（在 stm32f407xx.h 文件中有定义）
GPIO_Pin	待设置的 GPIO 引脚号 例如：GPIO_PIN_0、GPIO_PIN_1 等（在 stm32f4xx_hal_gpio.h 文件中有定义）

表 11.2.1.1 函数 HAL_GPIO_ReadPin()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
GPIO_PIN_SET	GPIO 引脚输入的电平为高电平
GPIO_PIN_RESET	GPIO 引脚输入的电平为低电平

表 11.2.1.2 函数 HAL_GPIO_ReadPin()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    GPIO_PinState value;

    /* 读取 PA0 引脚输入电平 */
    value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0);

    if (value == GPIO_PIN_SET)
    {
        /* Do something. */
    }
    else
    {
        /* Do something. */
    }
}
```

11.2.2 按键驱动

按键驱动主要就是读取 GPIO 引脚的输入状态，以此判断按键是否被按下，本章实验中，按键的驱动代码包括 key.c 和 key.h 两个文件。

根据原理图可知，应当将 PA0 引脚和 PE4 引脚配置为输入模式和下拉，并在需要读取 WKUP 按键（KEY0 按键）状态的时候，读取 PA0 引脚（PE4 引脚）的输入电平，若读取到 PA0 引脚（PE4 引脚）的输入电平为高电平，则说明 WKUP 按键（KEY0 按键）被按下，反之，则说明 WKUP 按键（KEY0 按键）没有被按下。

按键驱动中，对引脚的定义，如下所示：

```
#define WKUP_GPIO_PORT      GPIOA
#define WKUP_GPIO_PIN      GPIO_PIN_0
#define WKUP_GPIO_CLK_ENABLE() do { __HAL_RCC_GPIOA_CLK_ENABLE(); } while (0)

#define KEY0_GPIO_PORT      GPIOE
#define KEY0_GPIO_PIN      GPIO_PIN_4
```



```
#define KEY0_GPIO_CLK_ENABLE() do { __HAL_RCC_GPIOE_CLK_ENABLE(); } while (0)
```

按键驱动中，操作引脚的定义，如下所示：

```
#define WKUP (
    (HAL_GPIO_ReadPin(WKUP_GPIO_PORT, WKUP_GPIO_PIN) == GPIO_PIN_RESET) ? \
    0 : \
    1)
#define KEY0 (
    (HAL_GPIO_ReadPin(KEY0_GPIO_PORT, KEY0_GPIO_PIN) == GPIO_PIN_RESET) ? \
    0 : \
    1)
```

按键驱动中，按键的初始化函数，如下所示：

```
/**
 * @brief  初始化按键
 * @param  无
 * @retval 无
 */
void key_init(void)
{
    GPIO_InitTypeDef gpio_init_struct = {0};

    /* 使能 GPIO 端口时钟 */
    WKUP_GPIO_CLK_ENABLE();
    KEY0_GPIO_CLK_ENABLE();

    /* 配置 WKUP 控制引脚 */
    gpio_init_struct.Pin = WKUP_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_INPUT;
    gpio_init_struct.Pull = GPIO_PULLDOWN;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(WKUP_GPIO_PORT, &gpio_init_struct);

    /* 配置 KEY0 控制引脚 */
    gpio_init_struct.Pin = KEY0_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_INPUT;
    gpio_init_struct.Pull = GPIO_PULLDOWN;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(KEY0_GPIO_PORT, &gpio_init_struct);
}
```

按键的初始化函数中，使能了 KEY0 按键和 WKUP 按键对应引脚的 GPIO 端口时钟，并将其配置为输入模式和下拉。

按键的驱动中，扫描按键状态的函数，如下所示：

```
/**
 * @brief  扫描按键
 * @note   按键响应具有优先级：WKUP > KEY0
 * @param  mode: 扫描模式
 * @arg    0: 不支持连续按
 * @arg    1: 支持连续按
 * @retval 按键键值
 * @arg    NONE_PRES: 没有按键按下
 * @arg    WKUP_PRES: WKUP 按键按下
 * @arg    KEY0_PRES: KEY0 按键按下
 */
uint8_t key_scan(uint8_t mode)
{
    static uint8_t key_release = 1;
    uint8_t key_value = NONE_PRES;

    if (mode != 0)
    {
        key_release = 1;
    }

    if ((key_release == 1) && ((WKUP == 1) || (KEY0 == 1)))
```

```

{
    delay_ms(10);
    key_release = 0;

    if (KEY0 == 1)
    {
        key_value = KEY0_PRES;
    }

    if (WKUP == 1)
    {
        key_value = WKUP_PRES;
    }
}
else if ((WKUP == 0) && (KEY0 == 0))
{
    key_release = 1;
}

return key_value;
}

```

以上函数代码就实现了按键扫描，且具有按键消抖的功能。

11.2.3 实验应用代码

本章实验的应用代码，如下所示：

```

int main(void)
{
    uint8_t key;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */

    while (1)
    {
        key = key_scan(0); /* 扫描按键 */

        switch (key)
        {
            case WKUP_PRES: /* WKUP 按键被按下 */
            {
                LED0_TOGGLE(); /* 翻转 LED0 状态 */
                break;
            }
            case KEY0_PRES: /* KEY0 按键被按下 */
            {
                LED1_TOGGLE(); /* 翻转 LED1 状态 */
                break;
            }
        }

        delay_ms(10);
    }
}

```

可以看到应用代码中，在初始化完 LED 和按键后，就进入了一个 while 循环，在循环中，每隔 10 毫秒就调用 key_scan() 函数扫描以此按键的状态，如果扫描到 KEY0 按键或 WKUP 按键被按下，则反转对应 LED 的亮灭状态。

11.3 下载验证

在完成编译和烧录操作后，可以看到板子上的 LED0 和 LED1 默认是处于熄灭的状态，若此时按下并释放一次 WKUP 按键，则能够看到 LED0 的亮灭状态发生了一次翻转，同样的，若此时按下并释放一次 KEY0 按键，则能够看到 LED1 的亮灭状态发生了一次翻转，与预期的实验现象效果相符。

第十二章 外部中断实验

前两章介绍了 GPIO 在输出模式和输入模式下最基本的操作，本章将介绍如何将 GPIO 引脚作为外部中断输入来使用。通过本章的学习，读者将学习到 GPIO 作为外部中断输入的使用。

本章分为如下几个小节：

- 12.1 硬件设计
- 12.2 程序设计
- 12.3 下载验证

12.1 硬件设计

12.1.1 例程功能

1. 按下 WKUP 按键，LED0 状态翻转。
2. 按下 KEY0 按键，LED1 状态翻转。

12.1.2 硬件资源

1. LED
 - LED0 - PF9
 - LED1 - PF10
2. 按键
 - WKUP - PA0
 - KEY0 - PE4

12.1.3 原理图

本章实验使用的两个 M144Z-M4 最小系统板 STM32F407 版板载按键，分别为 KEY0 按键和 WKUP 按键，其于板载 MCU 的连接原理图，如下图所示：

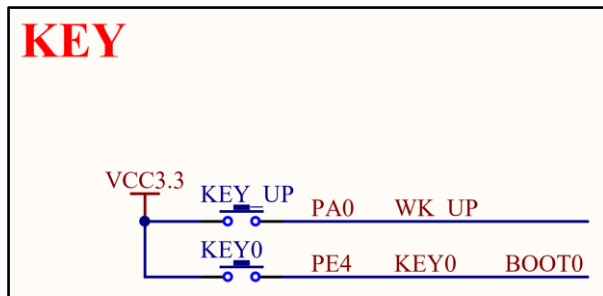


图 12.1.3.1 按键与 MCU 的连接原理图

从上面的原理图中可以看出，WKUP 按键和 KEY0 按键的一端连接到了电源正极，而另一端分别与 MCU 的 PA0 引脚和 PE4 引脚相连接，并且在上一小节的介绍中，也说明了对于该硬件设计，PA0 引脚和 PE4 引脚应当配置为下拉，这样一来，在按键被按下的时候，PA0 引脚或 PE4 引脚就会从原来的低电平状态变为高电平状态，在这期间就会有一个上升沿的跳变，因此可以使用该上升沿信号作为中断的触发源。

12.2 程序设计

12.2.1 HAL 库的 GPIO 驱动

针对本章的实验要求，仅需将对应的 GPIO 引脚配置为下拉并且上升沿触发中断模式即可，请见第 10.2.1 小节中配置 GPIO 引脚的相关内容。

①：配置 EXTI 中断

该函数用于配置 NVIC（嵌套向量中断控制器）中外设的中断优先级，因此不仅能用于配置 EXTI 的中断，还能配置其他外设等的中断，其函数原型如下所示：

```
void HAL_NVIC_SetPriority( IRQn_Type IRQn,
                          uint32_t PreemptPriority,
                          uint32_t SubPriority);
```

该函数的形参描述，如下表所示：

形参	描述
IRQn	NVIC 中断请求 例如：EXTI0_IRQn、USART1_IRQn 等（在 stm32f407xx.h 文件中有定义）
PreemptPriority	抢占优先级
SubPriority	子优先级

表 12.2.1.1 函数 HAL_NVIC_SetPriority()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
无	无

表 12.2.1.2 函数 HAL_NVIC_SetPriority()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 配置 EXTI0 的中断优先级 */
    HAL_NVIC_SetPriority(EXTI0_IRQn, 0, 0);
}
```

②：使能 EXTI 中断

该函数用于使能 NVIC（嵌套向量中断控制器）中外设的中断优先级，因此不仅能用于使能 EXTI 的中断，还能使能其他外设等的中断，其函数原型如下所示：

```
void HAL_NVIC_EnableIRQ(IRQn_Type IRQn);
```

该函数的形参描述，如下表所示：

形参	描述
IRQn	NVIC 中断请求 例如：EXTI0_IRQn、USART1_IRQn 等（在 stm32f407xx.h 文件中有定义）

表 12.2.1.3 函数 HAL_NVIC_EnableIRQ()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
无	无

表 12.2.1.4 函数 HAL_NVIC_EnableIRQ()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 使能 EXTI0 中断 */
    HAL_NVIC_EnableIRQ(EXTI0_IRQn);
}
```

12.2.2 外部中断驱动

本实验的外部中断驱动主要就是配置 GPIO 引脚作为 EINT 的外部中断源，并在其对应的中断回调函数中处理按键被按下后执行的操作。

外部中断驱动中，对 GPIO 引脚、SYSCFG、EINT 的相关定义，如下所示：

```
#define WKUP_INT_GPIO_PORT      GPIOA
#define WKUP_INT_GPIO_PIN      GPIO_PIN_0
```

```
#define WKUP_INT_GPIO_CLK_ENABLE() \
    do { \
        __HAL_RCC_GPIOA_CLK_ENABLE(); \
    } while (0)

#define WKUP_INT_IRQn EXTIO_IRQn
#define WKUP_INT_IRQHandler EXTIO_IRQHandler
#define KEY0_INT_GPIO_PORT GPIOE
#define KEY0_INT_GPIO_PIN GPIO_PIN_4
#define KEY0_INT_GPIO_CLK_ENABLE() \
    do { \
        __HAL_RCC_GPIOE_CLK_ENABLE(); \
    } while (0)

#define KEY0_INT_IRQn EXTIO4_IRQn
#define KEY0_INT_IRQHandler EXTIO4_IRQHandler
```

外部中断驱动中，外部中断的初始化函数，如下所示：

```
/**
 * @brief 初始化外部中断
 * @param 无
 * @retval 无
 */
void exti_init(void)
{
    GPIO_InitTypeDef gpio_init_struct = {0};

    /* 使能 GPIO 端口时钟 */
    WKUP_INT_GPIO_CLK_ENABLE();
    KEY0_INT_GPIO_CLK_ENABLE();

    /* 配置 WKUP 控制引脚 */
    gpio_init_struct.Pin = WKUP_INT_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_IT_RISING;
    gpio_init_struct.Pull = GPIO_PULLDOWN;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(WKUP_INT_GPIO_PORT, &gpio_init_struct);

    /* 配置 KEY0 控制引脚 */
    gpio_init_struct.Pin = KEY0_INT_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_IT_RISING;
    gpio_init_struct.Pull = GPIO_PULLDOWN;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(KEY0_INT_GPIO_PORT, &gpio_init_struct);

    /* 配置中断优先级并使能中断 */
    HAL_NVIC_SetPriority(WKUP_INT_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(WKUP_INT_IRQn);
    HAL_NVIC_SetPriority(KEY0_INT_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(KEY0_INT_IRQn);
}
```

在外部中断的初始化函数中，除了使能按键对应 GPIO 端口的时钟，还配置了对应的 EXTI 中断。

在外部中断的初始化函数中，开启了两个 EXTI 的中断，因此需要编写这两个 EXTI 中断对应的中断回调函数，如下所示：

```
/**
 * @brief WKUP 按键外部中断服务函数
 * @param 无
 * @retval 无
 */
void WKUP_INT_IRQHandler(void)
{
    HAL_GPIO_EXTI_IRQHandler(WKUP_INT_GPIO_PIN);
}

/**
```

```

* @brief    KEY0 按键外部中断服务函数
* @param    无
* @retval    无
*/
void KEY0_INT_IRQHandler(void)
{
    HAL_GPIO_EXTI_IRQHandler(KEY0_INT_GPIO_PIN);
}

/**
* @brief    HAL 库外部中断回调函数
* @param    GPIO_Pin: 外部中断线对应的引脚
* @retval    无
*/
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    /* 机械按键消抖（仅演示，切勿在实际工程的中断服务函数种进行阻塞延时） */
    delay_ms(20);

    switch (GPIO_Pin)
    {
        case WKUP_INT_GPIO_PIN:    /* WKUP 按键对应引脚发生中断 */
        {
            LED0_TOGGLE();    /* 翻转 LED0 状态 */
            break;
        }
        case KEY0_INT_GPIO_PIN:    /* KEY0 按键对应引脚发生中断 */
        {
            LED1_TOGGLE();    /* 翻转 LED1 状态 */
            break;
        }
    }
}

```

在按键被按下的一瞬间，其与 MCU 连接的 GPIO 引脚会收到一个上升沿的信号，就会触发 EXTI 的中断，并运行对应中断回调函数。

12.2.3 实验应用代码

本章实验的应用代码，如下所示：

```

int main(void)
{
    HAL_Init();    /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7);    /* 配置时钟，168MHz */
    delay_init(168);    /* 初始化延时 */
    usart_init(115200);    /* 初始化串口 */
    led_init();    /* 初始化 LED */
    exti_init();    /* 初始化外部中断 */

    while (1)
    {
    }
}

```

本实验的应用代码很简单，在初始化完 LED 和外部表中断后，就进入一个什么都不做的 while 循环，这是因为按键控制 LED 亮灭状态翻转的操作都在对应 EXTI 的中断回调函数中完成了。

12.3 下载验证

在完成编译和烧录操作后，可以看到板子上的 LED0 和 LED1 默认是处于熄灭的状态，若此时按下 WKUP 按键，则能够看到 LED0 的亮灭状态发生了一次翻转，同样的，若此时按下 KEY0 按键，则能够看 LED1 的亮灭状态发生了一次翻转，与预期的实验现象效果相符。

第十三章 串口通信实验

本章将介绍使用串口进行数据的收发操作，具体实现 STM32F407 与上位机软件的数据通信，STM32F407 将接受自上位机软件的数据原原本本地发送回给上位机软件。通过本章的学习，读者将学习到 USART 和 GPIO 引脚复用的使用。

本章分为如下几个小节：

- 13.1 硬件设计
- 13.2 程序设计
- 13.3 下载验证

13.1 硬件设计

13.1.1 例程功能

1. 回显串口接收到的数据
2. 每间隔一定时间，串口发送一段提示信息
3. LED0 闪烁，提示程序正在运行

13.1.2 硬件资源

1. USART1
 - USART1_TX - PA9
 - USART1_RX - PA10
2. LED
 - LED0 - PF9

13.1.3 原理图

本章实验使用的 USART1 通过跳线帽与板载的 USB 转串口芯片连接，其原理图如下图所示：

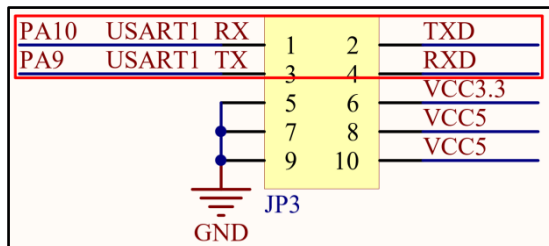


图 13.1.3.1 USART1 与 USB 转串口芯片跳线帽连接

因此在进行本章实验之前，需确保上图位置中正确安装了跳线帽

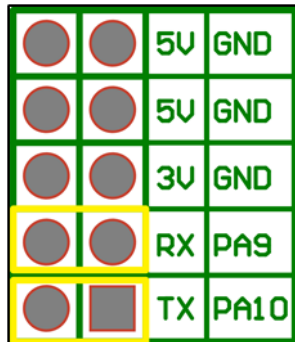
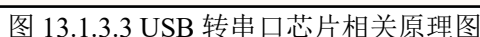


图 13.1.3.2 串口通信跳线帽位置

板载的 USB 转串口芯片的 USB 接口通过板载的 USB UART 端口引出，其原理图如下图所示：



13.2.1 HAL 库的 GPIO 驱动

请见第 10.2.1 小节中配置 GPIO 引脚的相关内容。

13.2.2 HAL 库的 UART 驱动

该函数的形参描述，如下表所示：

表 13.2.2.1 函数 HAL_UART_Init()形参描述

该函数的返回值描述，如下表所示：

表 13.2.2.2 函数 HAL_UART_Init()返回值描述

```
typedef struct
```

```

{
    uint32_t BaudRate;      /* 波特率 */
    uint32_t WordLength;    /* 位宽 */
    uint32_t StopBits;      /* 停止位 */
    uint32_t Parity;        /* 校验位 */
    uint32_t Mode;          /* 模式 */
    uint32_t HwFlowCtl;     /* 硬件流控 */
    uint32_t OverSampling;  /* 过采样 */
} UART_InitTypeDef;

```

该函数的使用示例，如下所示：

```

#include "stm32f4xx_hal.h"

void example_fun(void)
{
    UART_HandleTypeDef uart_handle = {0};

    /* 初始化 USART1 */
    uart_handle.Instance = USART1;
    uart_handle.Init.BaudRate = 115200;
    uart_handle.Init.WordLength = UART_WORDLENGTH_8B;
    uart_handle.Init.StopBits = UART_STOPBITS_1;
    uart_handle.Init.Parity = UART_PARITY_NONE;
    uart_handle.Init.Mode = UART_MODE_TX_RX;
    uart_handle.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    uart_handle.Init.OverSampling = UART_OVERSAMPLING_16;
    HAL_UART_Init(&uart_handle);
}

```

①：UART 中断接收数据

该函数用于 UART 中断接收数据，其函数原型如下所示：

```

HAL_StatusTypeDef HAL_UART_Receive_IT( UART_HandleTypeDef *huart,
                                       uint8_t *pData,
                                       uint16_t Size)

```

该函数的形参描述，如下表所示：

形参	描述
huart	指向 UART 句柄的指针
pData	指向数据接收缓冲区的指针
Size	接收数据的大小

表 13.2.2.3 函数 HAL_UART_Receive_IT()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 13.2.2.4 函数 HAL_UART_Receive_IT()返回值描述

该函数的使用示例，如下所示：

```

#include "stm32f4xx_hal.h"

void example_fun(void)
{
    uint8_t buf[10];

    /* UART 中断接收数据 */
    HAL_UART_Receive_IT(&uart_handle, buf, sizeof(buf));
}

```

13.2.3 串口通讯驱动

本实验的串口通信驱动主要是配置 USART1 并完成一些相关的初始化操作，并支持将 printf 函数重定向到 USART1 进行输出，同时向应用层提供了一个数据接收缓冲区和接收完成标志，通过这些，应用层就能够很方便地使用 USART1 进行数据传输了，本章实验中，串口通讯的驱动代码包括 usart.c 和 usart.h 两个文件。

串口通讯驱动中，对 GPIO、USART 的相关宏定义，如下所示：

```
#define USART_TX_GPIO_PORT      GPIOA
#define USART_TX_GPIO_PIN      GPIO_PIN_9
#define USART_TX_GPIO_AF       GPIO_AF7_USART1
#define USART_TX_GPIO_CLK_ENABLE() \
do { \
    __HAL_RCC_GPIOA_CLK_ENABLE(); \
} while (0)

#define USART_RX_GPIO_PORT      GPIOA
#define USART_RX_GPIO_PIN      GPIO_PIN_10
#define USART_RX_GPIO_AF       GPIO_AF7_USART1
#define USART_RX_GPIO_CLK_ENABLE() \
do { \
    __HAL_RCC_GPIOA_CLK_ENABLE(); \
} while (0)

#define USART_UX                USART1
#define USART_UX_IRQn           USART1_IRQn
#define USART_UX_IRQHandler     USART1_IRQHandler
#define USART_UX_CLK_ENABLE() \
do { \
    __HAL_RCC_USART1_CLK_ENABLE(); \
} while (0)
```

串口通讯驱动中，USART1 的初始化函数，如下所示：

```
/**
 * @brief  串口 X 初始化函数
 * @param  baudrate: 波特率，根据自己需要设置波特率值
 * @note   注意：必须设置正确的时钟源，否则串口波特率就会设置异常。
 *         这里的 USART 的时钟源在 sys_stm32_clock_init() 函数中已经设置过了。
 * @retval 无
 */
void usart_init(uint32_t baudrate)
{
    g_uart1_handle.Instance = USART_UX;                /* USART1 */
    g_uart1_handle.Init.BaudRate = baudrate;            /* 波特率 */
    g_uart1_handle.Init.WordLength = UART_WORDLENGTH_8B; /* 字长为 8 位数据格式 */
    g_uart1_handle.Init.StopBits = UART_STOPBITS_1;    /* 一个停止位 */
    g_uart1_handle.Init.Parity = UART_PARITY_NONE;     /* 无奇偶校验位 */
    g_uart1_handle.Init.HwFlowCtl = UART_HWCONTROL_NONE; /* 无硬件流控 */
    g_uart1_handle.Init.Mode = UART_MODE_TX_RX;        /* 收发模式 */
    HAL_UART_Init(&g_uart1_handle);                    /* HAL_UART_Init() 会使能 UART1 */

    /* 该函数会开启接收中断:标志位 UART_IT_RXNE，并且设置接收缓冲以及接收缓冲接收最大数据量 */
    HAL_UART_Receive_IT(&g_uart1_handle, (uint8_t *)g_rx_buffer, RXBUFFERSIZE);
}
```

在串口通信的初始化函数中，在使能 USART1 收发引脚的 GPIO 端口时钟和 USART1 外设时钟后，配置了 USART1 收发引脚的 GPIO 复用功能和配置其为复用功能模式，并在最后配置并使能了 USART1，同时也使能并配置了 USART1 的接收缓冲区非空中断，这么一来，USART1 的中断回调函数就会在 USART1 接收到数据的时候被调用。

串口通讯驱动中，USART1 的接收完成回调函数，如下所示：

```
/**
 * @brief  Rx 传输回调函数
 * @param  huart: UART 句柄类型指针
 * @retval 无
 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance == USART_UX)
    {
        if((g_usart_rx_sta & 0x8000) == 0)
        {

```

```

        if(g_usart_rx_sta & 0x4000)
        {
            if(g_rx_buffer[0] != 0x0a)
            {
                g_usart_rx_sta = 0;
            }
            else
            {
                g_usart_rx_sta |= 0x8000;
            }
        }
        else
        {
            if(g_rx_buffer[0] == 0x0d)
            {
                g_usart_rx_sta |= 0x4000;
            }
            else
            {
                g_usart_rx_buf[g_usart_rx_sta & 0X3FFF] = g_rx_buffer[0] ;
                g_usart_rx_sta++;
                if(g_usart_rx_sta > (USART_REC_LEN - 1))
                {
                    g_usart_rx_sta = 0;
                }
            }
        }
    }
    HAL_UART_Receive_IT(    &g_uart1_handle,
                           (uint8_t *)g_rx_buffer,
                           RXBUFFERSIZE);
}

```

在 USART1 的接收完成回调函数中主要用于读取 USART1 接收到的数据，并将其逐一存入接收的缓冲区，并在接收到“回车”和“换行”后标志数据接收完成。

13.2.4 实验应用代码

本章实验的应用代码，如下所示：

```

int main(void)
{
    uint16_t len;
    uint16_t times = 0;

    HAL_Init();                                /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7);        /* 配置时钟，168MHz */
    delay_init(168);                            /* 初始化延时 */
    usart_init(115200);                          /* 初始化串口 */
    led_init();                                  /* 初始化 LED */

    while (1)
    {
        if (g_usart_rx_sta & 0x8000)          /* 接收到数据 */
        {
            len = g_usart_rx_sta & 0x3FFF;
            printf("\r\n 您发送的消息为: \r\n");
            HAL_UART_Transmit( &g_uart1_handle,
                               (uint8_t *)g_usart_rx_buf,
                               len,
                               HAL_MAX_DELAY);

            printf("\r\n\r\n");
            g_usart_rx_sta = 0;
        }
        else

```

```
{
    if ((times % 5000) == 0)
    {
        printf("\r\n 正点原子 M144Z-M4 最小系统板 STM32F407 版 \
            串口通信实验\r\n");
        printf("正点原子@ALIENTEK\r\n\r\n\r\n");
    }
    if ((times % 200) == 0)
    {
        printf("请输入数据，以回车键结束\r\n");
    }
    if ((times % 30) == 0)
    {
        LED0_TOGGLE();
    }
    times++;
    delay_ms(10);
}
}
```

本实验的实验代码很简单，在完成初始化后，就不断地通过串口通信驱动提供的数据接收完成标志判断数据是否接收完毕，若还未完成数据接收，则每间隔一段时间就使用 `printf` 函数通过 USART1 打印一段提示信息，若数据接收完毕，则将数据原原本本的使用 `printf` 函数通过 USART1 打印出去，实现数据的回显。

13.3 下载验证

在完成编译和烧录操作后，需要将引出 USART1 引脚的排针通过串口转 TTL 模块等工具连接至 PC。接着打开 PC 上的 ATK-XCOM 串口调试助手软件，选择好正确的 COM 端口和相关的配置后，就能看到串口调试助手上每间隔一段时间就打印一次“请输入数据，以回车键结束”，接下来就可以根据提示通过串口调试助手发送一段任意的数据（以回车换行结束），随后立马就能看到串口调试助手上显示发送出去的数据，这就是本实验实现的数据回显。

第十四章 独立看门狗实验

本章介绍 STM32F407 独立看门狗（IWDG）的使用，独立看门狗能够帮助 CPU 在进入错误状态或程序跑飞时进行复位。通过本章的学习，读者将学习到 IWDG 的使用。

本章分为如下几个小节：

- 14.1 硬件设计
- 14.2 程序设计
- 14.3 下载验证

14.1 硬件设计

14.1.1 例程功能

1. 若每间隔 1 秒内按下一次 WKUP 按键，则 LED0 常亮，否则 LED0 闪烁

14.1.2 硬件资源

1. LED
LED0 - PF9
2. 按键
WKUP - PA0
3. IWDG

14.1.3 原理图

本章实验使用的独立看门狗为 STM32F407 的片上资源，因此并没有相应的连接原理图。

14.2 程序设计

14.2.1 HAL 库的 IWDG 驱动

在使用 IWDG 前，需要先对其进行初始化，在初始化操作中要开启 IWDG 并配置 IWDG 的预分频系数和重装载值，预分频系数和重装载值就决定了 IWDG 单次溢出的时间，因此这两个值应该根据实际的应用场景妥善进行设置，若 IWDG 的溢出时间太长，则对异常情况的反应将变得迟钝，但若设置地太短，则会误触发复位，因此需要妥善设置。使用 IWDG 的具体步骤如下：

- ①：初始化 IWDG
 - ②：在 IWDG 溢出前不断地进行“喂狗”操作
- 在 HAL 库中对应的驱动函数如下：

①：初始化 IWDG

该函数用于初始化 IWDG，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_IWDG_Init(IWDG_HandleTypeDef *hiwdg);
```

该函数的形参描述，如下表所示：

形参	描述
hiwdg	指向 IWDG 句柄的指针

表 14.2.1.1 函数 HAL_IWDG_Init()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 14.2.1.2 函数 HAL_IWDG_Init()返回值描述

该函数需要传入 IWDG 的句柄指针，该句柄中就包含了 IWDG 的初始化配置参数结构体，

该结构体的定义如下所示：

```
typedef struct
{
    uint32_t Prescaler; /* 分频系数 */
    uint32_t Reload;    /* 重装载值 */
} IWDG_InitTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f34xx_hal.h"

void example_fun(void)
{
    IWDG_HandleTypeDef iwdg_handle = {0};

    /* 初始化 IWDG */
    iwdg_handle.Instance = IWDG;
    iwdg_handle.Init.Prescaler = IWDG_PRESCALER_64;
    iwdg_handle.Init.Reload = 500;
    HAL_IWDG_Init(&iwdg_handle);
}
```

②：重装载 IWDG

该函数用于重装载 IWDG，也就是所谓的“喂狗”，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_IWDG_Refresh(IWDG_HandleTypeDef *hiwdg);
```

该函数的形参描述，如下表所示：

形参	描述
hiwdg	指向 IWDG 句柄的指针

表 14.2.1.3 函数 HAL_IWDG_Refresh()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 14.2.1.4 函数 HAL_IWDG_Refresh()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 重装载 IWDG（喂狗） */
    HAL_IWDG_Refresh();
}
```

14.2.2 看门狗驱动

本章实验的看门狗驱动主要负责向应用层提供 IWDG 初始化和喂狗的操作函数，本章实验中，看门狗的驱动代码包括 wdg.c 和 wdg.h 两个文件。

看门狗驱动中，IWDG 的初始化函数，如下所示：

```
/**
 * @brief 初始化独立看门狗
 * @param prescaler: 预分频器系数
 * @arg IWDG_PRESCALER_4 : 4 分频
 * @arg IWDG_PRESCALER_8 : 8 分频
 * @arg IWDG_PRESCALER_16 : 16 分频
 * @arg IWDG_PRESCALER_32 : 32 分频
 * @arg IWDG_PRESCALER_64 : 64 分频
 * @arg IWDG_PRESCALER_128 : 128 分频
 * @arg IWDG_PRESCALER_256 : 256 分频
 * @param reload: 自动重装载值 (0~0xFFF)
 * @retval 无
 */
void iwdg_init(uint32_t prescaler, uint32_t reload)
```

```
{
    g_iwdg_handler.Instance = IWDG;
    g_iwdg_handler.Init.Prescaler = prescaler;
    g_iwdg_handler.Init.Reload = reload;
    HAL_IWDG_Init(&g_iwdg_handler);
}
```

IWDG 的初始化函数中使能了 IWDG 并配置其预分频系数和重装载值，并进行了一次“喂狗”防止 IWDG 一使能就溢出引发复位。

看门狗驱动中，IWDG 的“喂狗”函数，如下所示：

```
/**
 * @brief   喂狗独立看门狗
 * @param   无
 * @retval   无
 */
void iwdg_feed(void)
{
    HAL_IWDG_Refresh(&g_iwdg_handler);
}
```

该函数很简单，就是重装载 IWDG 的计数值。

14.2.3 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    uint8_t key;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟，168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    iwdg_init(IWDG_PRESCALER_64, 500); /* 初始化独立看门狗，溢出时间约 1 秒 */

    delay_ms(100);
    LED0(0);

    while (1)
    {
        key = key_scan(1);
        if (key == WKUP_PRES) /* WKUP 按键被按下 */
        {
            iwdg_feed(); /* 喂狗独立看门狗 */
        }

        delay_ms(10);
    }
}
```

可以看到应用代码中，LED 初始化后，LED0 会处于默认的熄灭状态 100 毫秒，随后初始化 IWDG 并点亮 LED0，接着在 while 循环中重复判断 WKUP 按键是否被按下，若按下则进行“喂狗”操作，若在 IWDG 溢出前都为按下 WKUP 按键，则 IWDG 会触发复位，复位会导致 LED0 熄灭大约 100 毫秒（便于观察）。

14.3 下载验证

在完成编译和烧录操作后，可以看到板子上的 LED0 每间隔一段时间（大约 1 秒）就闪烁一次，这是因为 IWDG 不断地溢出，导致的复位。接下来若以时间间隔小于 1 秒（大约）的速度频繁地按下 WKUP 按键，则可以在 IWDG 溢出前及时“喂狗”，具体的现象为 LED0 不再闪烁。

第十五章 窗口看门狗实验

本章介绍 STM32F407 窗口看门狗（WWDG）的使用，窗口看门狗与独立看门狗一样能够帮助 CPU 在进入错误状态或程序跑飞时进行复位，不过窗口看门狗相对于独立看门狗限制了“喂狗”的最小间隔，若两次“喂狗”的间隔太短，一样会产生复位。通过本章的学习，读者将学习到 WWDG 的使用。

本章分为如下几个小节：

- 15.1 硬件设计
- 15.2 程序设计
- 15.3 下载验证

15.1 硬件设计

15.1.1 例程功能

- 1. LED1 闪烁

15.1.2 硬件资源

- 1. LED
 - LED0 - PF9
- 2. WWDG

15.1.3 原理图

本章实验使用的窗口看门狗为 STM32F407 的片上资源，因此并没有相应的连接原理图。

15.2 程序设计

15.2.1 HAL 库的 WWDG 驱动

本章实验使用到了 WWDG 的提前唤醒中断，提前唤醒中断指的是 WWDG 在“喂狗”超时即将进行复位前由 WWDG 产生的中断，本章实验就在 WWDG 的提前唤醒中断服务函数中进行“喂狗”。在使用 WWDG 前，需要先对其进行初始化，在初始化中，需要使能 WWDG 并配置 WWDG 的预分频系数和窗口值，还要进行使能 WWDG 中断的相关操作，具体的步骤如下：

- ①：初始化 WWDG
- ②：在 WWDG 提前唤醒回调函数对其进行“喂狗”

在 HAL 库中对应的驱动函数如下：

①：初始化 WWDG

该函数用于初始化 WWDG，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_WWDG_Init(WWDG_HandleTypeDef *hwwdg);
```

该函数的形参描述，如下表所示：

形参	描述
hwwdg	指向 WWDG 句柄的指针

表 15.2.1.1 函数 HAL_WWDG_Init()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 15.2.1.2 函数 HAL_WWDG_Init()返回值描述

该函数需要传入 WWDG 的句柄指针，该句柄中就包含了 WWDG 的初始化配置参数结构体，该结构体的定义如下所示：

```
typedef struct
{
    uint32_t Prescaler; /* 分频系数 */
    uint32_t Window; /* 窗口值 */
    uint32_t Counter; /* 计数值 */
    uint32_t EWIMode; /* 提前唤醒模式 */
} WWDG_InitTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    WWDG_HandleTypeDef wwdg_handle = {0};

    /* 初始化 WWDG */
    wwdg_handle.Instance = WWDG;
    wwdg_handle.Init.Prescaler = WWDG_PRESCALER_8;
    wwdg_handle.Init.Window = 0x5F;
    wwdg_handle.Init.Counter = 0x7F;
    wwdg_handle.Init.EWIMode = WWDG_EWI_ENABLE;
    HAL_WWDG_Init(&wwdg_handle);
}
```

②：重装载 WWDG 的计数值

该函数用于重装载 WWDG 的计数值，也就是所谓的“喂狗”，其函数原型如下：

```
HAL_StatusTypeDef HAL_WWDG_Refresh(WWDG_HandleTypeDef *hwwdg);
```

该函数的形参描述，如下表所示：

形参	描述
hwwdg	指向 WWDG 句柄的指针

表 15.2.1.3 函数 HAL_WWDG_Refresh()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 15.2.1.4 函数 HAL_WWDG_Refresh()返回值描述

该函数的使用示例，如下表所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 重装载 WWDG 的计数值（喂狗） */
    HAL_WWDG_Refresh();
}
```

15.2.2 看门狗驱动

本章实验的看门狗驱动主要负责向应用层提供 WWDG 的初始化函数，并实现 WWDG 的提前唤醒中断服务函数，在 WWDG 的提前唤醒中断服务函数中执行“喂狗”操作。本章实验中，看门狗的驱动代码包括 wdt.c 和 wdt.h 两个文件。

看门狗驱动中 WWDG 的初始化函数，如下所示：

```
/**
 * @brief 初始化窗口看门狗
 * @param counter: 计数器值
 * @param window: 窗口值
 * @param prescaler: 预分频器系数
 * @arg WWDG_PRESCALER_1: 1 分频
 * @arg WWDG_PRESCALER_2: 2 分频
 * @arg WWDG_PRESCALER_4: 4 分频
 * @arg WWDG_PRESCALER_8: 8 分频
 * @retval 无
```

```

*/
void wwdg_init(uint32_t counter, uint32_t window, uint32_t prescaler)
{
    g_wwdg_handler.Instance = WWDG;
    g_wwdg_handler.Init.Prescaler = prescaler;
    g_wwdg_handler.Init.Window = window;
    g_wwdg_handler.Init.Counter = counter;
    g_wwdg_handler.Init.EWIMode = WWDG_EWI_ENABLE;
    HAL_WWDG_Init(&g_wwdg_handler);
}

```

从上面的代码中可以看出，WWDG 的初始化函数就是调用了函数 HAL_WWDG_Init()函数来初始化 WWDG。

看门狗驱动中，WWDG 提前唤醒回调函数，如下所示：

```

/**
 * @brief    HAL 库窗口看门狗提前唤醒中断回调函数
 * @param    hwwdg: 窗口看门狗句柄
 * @retval    无
 */
void HAL_WWDG_EarlyWakeupCallback(WWDG_HandleTypeDef *hwwdg)
{
    /* 喂狗窗口看门狗 */
    HAL_WWDG_Refresh(&g_wwdg_handler);
    LED1_TOGGLE();
}

```

可以看到，在窗口看门狗的提前唤醒回调函数中，对 WWDG 进行了喂狗操作，同时还对 LED1 的状态进行了以此翻转（方便观察）。

15.2.3 实验应用代码

本章实验的应用代码，如下所示：

```

int main(void)
{
    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟，168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    wwdg_init(0x7F, 0x5F, WWDG_PRESCALER_8); /* 初始化窗口看门狗 */

    while (1)
    {

    }
}

```

可以看到应用代码中，LED 初始化后会初始化窗口看门狗，若窗口看门狗正常被“喂狗”，则可以看到 LED1 闪烁。

15.3 下载验证

在完成编译和烧录操作后，可以看 LED1 不断闪烁，说明 WWDG “喂狗”正常。

第十六章 基本定时器中断实验

STM32F407 内部有多种多个定时器 (TIM)，本章介绍 STM32F407 基本定时器的使用，基本定时器是一类只能实现定时功能、没有外部接口的定时器。通过本章的学习，读者将学习到基本定时器的使用。

本章分为如下几个小节：

- 16.1 硬件设计
- 16.2 程序设计
- 16.3 下载验证

16.1 硬件设计

16.1.1 例程功能

- 1. LED1 以 1Hz 的频率闪烁
- 2. LED0 闪烁，提示程序正在运行

16.1.2 硬件资源

- 1. LED
 - LED0 - PF9
 - LED1 - PF10
- 2. TIM6

16.1.3 原理图

本章实验使用的 TIM6 为 STM32F407 的片上资源，因此并没有相应的连接原理图。

16.2 程序设计

16.2.1 HAL 库的 TIM 驱动

本章实验将配置 TIM6 每间隔 500 毫秒产生一次中断，并在其中断服务函数中改变一次 LED1 的状态，具体的步骤如下：

- ①：初始化 TIM6
 - ②：开启 TIM6 中断模式计数
- 在 HAL 库中对应的驱动函数如下：

①：初始化 TIM

该函数用于初始化 TIM 的各项参数，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_TIM_Base_Init(TIM_HandleTypeDef *htim);
```

该函数的形参描述，如下表所示：

形参	描述
htim	指向 TIM 句柄的指针

表 16.2.1.1 函数 HAL_TIM_Base_Init()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 16.2.1.2 函数 HAL_TIM_Base_Init()返回值描述

该函数需要传入 TIM 的句柄指针，该句柄中就包含了 TIM 的初始化配置参数结构体，该结构体的定义如下所示：

```
typedef struct
```

```
{
    uint32_t Prescaler;          /* 分频系数 */
    uint32_t CounterMode;        /* 计数模式 */
    uint32_t Period;             /* 重装载值 */
    uint32_t ClockDivision;      /* 计数分频 */
    uint32_t RepetitionCounter;   /* 重复计数器 */
    uint32_t AutoReloadPreload;   /* 自动重装载 */
} TIM_Base_InitTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    TIM_HandleTypeDef tim_handle = {0};

    /* 初始化 TIM6 */
    tim_handle.Instance = TIM6;
    tim_handle.Init.Prescaler = 168 - 1;
    tim_handle.Init.Period = 1000 - 1;
    HAL_TIM_Base_Init(&tim_handle);
}
```

②：开启 TIM 中断模式计数

该函数用于开启 TIM 中断模式计数，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_TIM_Base_Start_IT(TIM_HandleTypeDef *htim);
```

该函数的形参描述，如下表所示：

形参	描述
htim	指向 TIM 句柄的指针

表 16.2.1.3 函数 HAL_TIM_Base_Start_IT()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 16.2.1.4 函数 HAL_TIM_Base_Start_IT()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 开启 TIM6 中断模式计数 */
    HAL_TIM_Base_Start_IT(&tim_handle);
}
```

16.2.2 基本定时器驱动

本章实验的基本定时器驱动主要负责向应用层提供基本定时器的初始化函数，并实现基本定时器的中断回调函数。本章实验中，基本定时器的驱动代码包括 btmr.c 和 btmr.h 两个文件。

基本定时器驱动中，对 TIM 的相关宏定义，如下所示：

```
#define BTIM_TIMX_INT          TIM6
#define BTIM_TIMX_INT_IRQn     TIM6_DAC_IRQn
#define BTIM_TIMX_INT_IRQHandler TIM6_DAC_IRQHandler
#define BTIM_TIMX_INT_CLK_ENABLE() do { \
                                   __HAL_RCC_TIM6_CLK_ENABLE(); \
                                   } while (0)
```

基本定时器驱动中 TIM6 的初始化函数，如下所示：

```
/**
 * @brief  初始化基本定时器
 * @param  arr: 自动重装载值
 * @param  psc: 预分频系数
 * @retval 无
```



```

*/
void btim_timx_int_init(uint16_t arr, uint16_t psc)
{
    g_timx_handle.Instance = BTIM_TIMX_INT;
    g_timx_handle.Init.Prescaler = psc;
    g_timx_handle.Init.Period = arr;
    HAL_TIM_Base_Init(&g_timx_handle);
    HAL_TIM_Base_Start_IT(&g_timx_handle);
}

```

从 TIM6 的初始化代码中可以看到，该函数调用了函数 HAL_TIM_Base_Init() 和函数 HAL_TIM_Base_Start_IT() 分别用来初始化 TIM6 和开启了 TIM6 的中断模式计数。

基本定时器驱动中 TIM6 的计数溢出回调函数，如下所示：

```

/**
 * @brief    HAL 库基本定时器超时中断回调函数
 * @param    无
 * @retval    无
 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == BTIM_TIMX_INT)
    {
        LED1_TOGGLE();
    }
}

```

从上面的代码中可以看出，在 TIM6 每次计数溢出后都会翻转一次 LED1 的状态。

16.2.3 实验应用代码

本章实验的应用代码，如下所示：

```

int main(void)
{
    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟，168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    btim_timx_int_init(5000 - 1, 8400 - 1); /* 初始化基本定时器，溢出频率为 2Hz */

    while (1)
    {
        LED0_TOGGLE();
        delay_ms(200);
    }
}

```

从上面的代码中可以看到，TIM6 的自动重装载配置为(5000-1)，TIM6 的预分频器数值配置为(8400-1)，并且 TIM6 的时钟频率为 84MHz，因此 TIM6 的计数频率为 10KHz，且 TIM6 计数 5000 次溢出一次，因此溢出频率为 2Hz，又因为 TIM6 每溢出一次都会改变一次 LED1 的状态，因此 LED1 的闪烁频率为 1Hz。

16.3 下载验证

在完成编译和烧录操作后，可以看到板子上的 LED0 和 LED1 都在闪烁，但闪烁频率不同，LED0 每间隔 200 毫秒改变一次状态，LED1 在 TIM6 的中断回调函数中被改变状态，其闪烁的频率约为 1Hz。

第十七章 通用定时器中断实验

本章介绍 STM32F407 通用定时器的使用，通用定时器相较于基本定时器，拥有输入捕获和输出比较等功能，这些功能可以用来测量脉冲宽度、频率和占空比，并且可以产生并输出波形等。通过本章的学习，读者将学习到通用定时器的基本使用。

本章分为如下几个小节：

- 17.1 硬件设计
- 17.2 程序设计
- 17.3 下载验证

17.1 硬件设计

17.1.1 例程功能

1. LED1 以 1Hz 的频率闪烁
2. LED0 闪烁，提示程序正在运行

17.1.2 硬件资源

1. LED
 - LED0 - PF9
 - LED1 - PF10
2. TIM3

17.1.3 原理图

本章实验使用的 TIM3 为 STM32F407 的片上资源，因此没有对应的连接原理图。

17.2 程序设计

17.2.1 HAL 库的 TIM 驱动

本章实验仅是使用通用 TIM3 代替上一章中基本 TIM6，实现通用定时器的一些基本功能，具体的原理和使用的 HAL 库函数都是一样的，因此请参考第 16.2.1 小节中对 HAL 库中 TIM 驱动的相关介绍。

17.2.2 通用定时器驱动

本章实验仅是使用通用 TIM3 代替上一章中基本 TIM6，实现通用定时器的一些基本功能，具体的驱动代码都是一样的，因此请参考第 16.2.2 小节中基本定时器驱动的相关介绍。

17.2.3 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    HAL_Init();                /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟，168MHz */
    delay_init(168);           /* 初始化延时 */
    usart_init(115200);        /* 初始化串口 */
    led_init();                /* 初始化 LED */
    gtim_timx_int_init(5000 - 1, 8400 - 1); /* 初始化通用定时器，溢出频率为 2Hz */

    while (1)
```

```
{  
    LED0_TOGGLE();  
    delay_ms(200);  
}
```

与上一章实验一样，TIM3 的计数频率为 10KHz，溢出频率为 2Hz，因此 LED1 的闪烁频率也为 1Hz。

17.3 下载验证

在完成编译和烧录操作后，可以看到板子上的 LED0 和 LED1 都在闪烁，但闪烁的频率不同，LED0 每间隔 200 毫秒改变一次状态，LED1 在 TIM3 的中断回调函数中被改变状态，其闪烁的频率约为 1Hz。

第十八章 通用定时器 PWM 输出实验

本章将介绍使用 STM32F407 的通用定时器输出 PWM。通过本章的学习，读者将学习到通用定时器输出比较的使用。

本章分为如下几个小节：

- 18.1 硬件设计
- 18.2 程序设计
- 18.3 下载验证

18.1 硬件设计

18.1.1 例程功能

- 1. PB4 引脚输出频率为 2KHz 占空比不断变化的 PWM
- 2. LED0 闪烁，提示程序正在运行

18.1.2 硬件资源

- 1. LED
 - LED0 - PF9
- 2. TIM3
 - CH1 - PB4

18.1.3 原理图

本章实验使用的 TIM3 为 STM32F407 的片上资源，因此没有对应的连接原理图。

18.2 程序设计

18.2.1 HAL 库的 TIM 驱动

本章实验将使用 TIM3 从 CH1（PB4 引脚）输出 PWM，因此需要配置通用定时器从指定通道输出 PWM，具体的步骤如下：

- ①：初始化定时器 PWM
- ②：配置定时器 PWM 输出通道
- ③：开始定时器 PWM 输出

在 HAL 库中对应的驱动函数如下：

①：初始化定时器 PWM

该函数用于初始化定时器 PWM，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_TIM_PWM_Init(TIM_HandleTypeDef *htim);
```

该函数的形参描述，如下表所示：

形参	描述
htim	指向 TIM 句柄的指针

表 18.2.1.1 函数 HAL_TIM_PWM_Init()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 18.2.1.2 函数 HAL_TIM_PWM_Init()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
```

```
{
    TIM_HandleTypeDef tim_handle = {0};

    /* 初始化 TIM3 PWM */
    tim_handle.Instance = TIM3;
    tim_handle.Init.Prescaler = 84 - 1;
    tim_handle.Init.Period = 500 - 1;
    HAL_TIM_PWM_Init(&tim_handle);
}
```

②：配置定时器 PWM 输出通道

该函数用于配置定时器的 PWM 输出通道，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_TIM_PWM_ConfigChannel(    TIM_HandleTypeDef *htim,
                                                TIM_OC_InitTypeDef *sConfig,
                                                uint32_t Channel);
```

该函数的形参描述，如下表所示：

形参	描述
htim	指向 TIM 句柄的指针
sConfig	指向输出比较通道配置结构体的指针 需自行定义，并根据输出比较通道配置参数填充结构体中的成员变量
Channel	定时器通道 例如：TIM_CHANNEL_1、TIM_CHANNEL_2 等（在 stm32f4xx_hal_tim.h 文件中有定义）

表 18.2.1.3 函数 HAL_TIM_PWM_ConfigChannel()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 18.2.1.4 函数 HAL_TIM_PWM_ConfigChannel()返回值描述

该函数使用 TIM_OC_InitTypeDef 类型结构体指针传入了定时器输出比较通道的配置参数，该结构体的定义如下所示：

```
typedef struct
{
    uint32_t OCMODE;          /* 输出比较模式 */
    uint32_t Pulse;           /* 占空比 */
    uint32_t OCPolarity;      /* 输出极性 */
    uint32_t OCNPolarity;     /* 互补输出极性 */
    uint32_t OCFastMode;      /* 快速模式 */
    uint32_t OCIdleState;     /* 空闲状态 */
    uint32_t OCNIdleState;    /* 互补空闲状态 */
} TIM_OC_InitTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    TIM_OC_InitTypeDef tim_oc_init_struct = {0};

    /* 配置定时器通道 1 PWM 输出 */
    tim_oc_init_struct.OCMode = TIM_OCMode_PWM1;
    tim_oc_init_struct.Pulse = 250 - 1;
    tim_oc_init_struct.OCPolarity = TIM_OCPOLARITY_HIGH;
    HAL_TIM_PWM_ConfigChannel(&tim_handle, &tim_oc_init_struct, TIM_CHANNEL_1);
}
```

③：开始定时器 PWM 输出

该函数用于开始定时器的 PWM 输出，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_TIM_PWM_Start(TIM_HandleTypeDef *htim, uint32_t Channel);
```

该函数的形参描述，如下表所示：

形参	描述
htim	指向 TIM 句柄的指针
Channel	定时器通道 例如：TIM_CHANNEL_1、TIM_CHANNEL_2 等（在 stm32f4xx_hal_tim.h 文件中有定义）

表 18.2.1.5 函数 HAL_TIM_PWM_Start()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 18.2.1.6 函数 HAL_TIM_PWM_Start()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 开启定时器通道1PWM输出 */
    HAL_TIM_PWM_Start(&tim_handle, TIM_CHANNEL_1);
}
```

18.2.2 通用定时器驱动

本章实验的通用定时器驱动主要负责向应用层提供通用定时器的初始化函数。本章实验中，通用定时器的驱动代码包括 gtim.c 和 gtim.h 两个文件。

通用定时器驱动中，对 TIM、GPIO 的相关宏定义，如下所示：

```
#define GTIM_TIMX_PWM          TIM3
#define GTIM_TIMX_PWM_CLK_ENABLE() \
do { \
    __HAL_RCC_TIM3_CLK_ENABLE(); \
} while (0)
#define GTIM_TIMX_PWM_CHY      TIM_CHANNEL_1
#define GTIM_TIMX_PWM_CHY_GPIO_PORT      GPIOB
#define GTIM_TIMX_PWM_CHY_GPIO_PIN      GPIO_PIN_4
#define GTIM_TIMX_PWM_CHY_GPIO_AF      GPIO_AF2_TIM3
#define GTIM_TIMX_PWM_CHY_GPIO_CLK_ENABLE() \
do { \
    __HAL_RCC_GPIOB_CLK_ENABLE(); \
} while (0)
```

通用定时器驱动中 TIM3 的初始化函数，如下所示：

```
/**
 * @brief  初始化通用定时器 PWM
 * @param  arr: 自动重装载值
 * @param  psc: 预分频系数
 * @retval 无
 */
void gtim_timx_pwm_chy_init(uint16_t arr, uint16_t psc)
{
    TIM_OC_InitTypeDef timx_oc_pwm_struct = {0};

    /* 初始化定时器 PWM */
    g_timx_pwm_handle.Instance = GTIM_TIMX_PWM;
    g_timx_pwm_handle.Init.Prescaler = psc;
    g_timx_pwm_handle.Init.CounterMode = TIM_COUNTERMODE_UP;
    g_timx_pwm_handle.Init.Period = arr;
    HAL_TIM_PWM_Init(&g_timx_pwm_handle);

    /* 配置定时器 PWM 输出通道 */
    timx_oc_pwm_struct.OCMode = TIM_OCMode_PWM1;
    timx_oc_pwm_struct.Pulse = (arr + 1) >> 1;
    timx_oc_pwm_struct.OCpolarity = TIM_OCPOLARITY_HIGH;
    HAL_TIM_PWM_ConfigChannel(&g_timx_pwm_handle,
```

```

                                &timx_oc_pwm_struct,
                                GTIM_TIMX_PWM_CHY);

    /* 开始定时器 PWM 输出 */
    HAL_TIM_PWM_Start(&g_timx_pwm_handle, GTIM_TIMX_PWM_CHY);
}

```

从 TIM3 的初始化代码中可以看到，初始化函数中初始化了 TIM3 的 PWM 并配置了 TIM3 的 PWM 输出通道 1，最后开始 TIM3 的 PWM 输出。

18.2.3 实验应用代码

本章实验的应用代码，如下所示：

```

int main(void)
{
    uint16_t compare = 0;
    uint8_t counter = 0;

    HAL_Init();                                /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7);        /* 配置时钟，168MHz */
    delay_init(168);                            /* 初始化延时 */
    usart_init(115200);                         /* 初始化串口 */
    led_init();                                 /* 初始化 LED */
    gtim_timx_pwm_chy_init(500 - 1, 84 - 1);    /* 初始化通用定时器 PWM */

    while (1)
    {
        if (compare++ >= 300)
        {
            compare = 100;
            __HAL_TIM_SET_COMPARE(&g_timx_pwm_handle, GTIM_TIMX_PWM_CHY, compare);

            if (counter++ == 20)
            {
                counter = 0;
                LED0_TOGGLE();
            }

            delay_ms(10);
        }
    }
}

```

从上面的代码中可以看到，在初始化完 TIM3 输出 PWM 后，就不断地改变 TIM3 通道 1 的比较值，以达到改变 PWM 占空比。

18.3 下载验证

在完成编译和烧录操作后，可以观察到 PB4 引脚输出频率为 2KHz，占空比不断变化的 PWM。

第十九章 通用定时器输入捕获实验

本章将介绍使用 STM32F407 通用定时器的输入捕获功能。通过本章的学习，读者将学习到通用定时器输入捕获的使用。

本章分为如下几个小节：

- 19.1 硬件设计
- 19.2 程序设计
- 19.3 下载验证

19.1 硬件设计

19.1.1 例程功能

- 1. 按下并松开 WKUP 按键，串口输出 WKUP 被按下的时长
- 2. LED0 闪烁，提示程序正在运行

19.1.2 硬件资源

- 1. LED
LED0 - PF9
- 2. 按键
WKUP - PA0
- 3. TIM5
CH1 - PA0
- 4. USART1
USART1_TX - PA9
USART1_RX - PA10

19.1.3 原理图

本章实验使用的 TIM5 为 STM32F407 的片上资源，因此没有对应的连接原理图。

19.2 程序设计

19.2.1 HAL 库的 TIM 驱动

本章实验将使用 TIM5 的通道 1 捕获 WKUP 按键被按下的高电平脉冲宽度，具体的步骤如下：

- ①：初始化定时器输入捕获
 - ②：配置定时器输入捕获通道
 - ③：开始定时器输入捕获
- 在 HAL 库中对应的驱动函数如下：

①：初始化定时器输入捕获

该函数用于初始化定时器输入捕获，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_TIM_IC_Init(TIM_HandleTypeDef *htim);
```

该函数的形参描述，如下表所示：

形参	描述
htim	指向 TIM 句柄的指针

表 19.2.1.1 函数 HAL_TIM_IC_Init()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
-----	----

HAL_StatusTypeDef	HAL 状态
-------------------	--------

表 19.2.1.2 函数 HAL_TIM_IC_Init()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    TIM_HandleTypeDef tim_handle = {0};

    /* 初始化 TIM5 输入捕获 */
    tim_handle.Instance = TIM5;
    tim_handle.Init.Prescaler = 84 - 1;
    tim_handle.Init.Period = 0xFFFF;
    HAL_TIM_IC_Init(&tim_handle);
}
```

②：配置定时器输入捕获通道

该函数用于配置定时器的输入捕获通道，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_TIM_IC_ConfigChannel(    TIM_HandleTypeDef *htim,
                                              TIM_IC_InitTypeDef *sConfig,
                                              uint32_t Channel);
```

该函数的形参描述，如下表所示：

形参	描述
htim	指向 TIM 句柄的指针
sConfig	指向输入捕获通道配置结构体的指针 需自行定义，并根据输入捕获通道配置参数填充结构体中的成员变量
Channel	定时器通道 例如：TIM_CHANNEL_1、TIM_CHANNEL_2 等（在 stm32f4xx_hal_tim.h 文件中有定义）

表 19.2.1.3 函数 HAL_TIM_IC_ConfigChannel()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 19.2.1.4 函数 HAL_TIM_IC_ConfigChannel()返回值描述

该函数使用 TIM_IC_InitTypeDef 类型结构体指针传入了定时器输入捕获通道的配置参数，该结构体的定义如下所示：

```
typedef struct
{
    uint32_t ICPolarity;    /* 输入捕获极性 */
    uint32_t ICSelection;   /* 输入 */
    uint32_t ICPrescaler;   /* 输入分频系数 */
    uint32_t ICFilter;      /* 输入滤波 */
} TIM_IC_InitTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    TIM_IC_InitTypeDef tim_ic_init_struct = {0};

    /* 配置定时器通道 1 输入捕获 */
    tim_ic_init_struct.ICPolarity = TIM_ICPOLARITY_RISING;
    tim_ic_init_struct.ICSelection = TIM_ICSELECTION_DIRECTTI;
    tim_ic_init_struct.ICPrescaler = TIM_ICPSC_DIV1;
    tim_ic_init_struct.ICFilter = 0;
    HAL_TIM_IC_ConfigChannel(&tim_handle, &tim_ic_init_struct, TIM_CHANNEL_1);
}
```

③：开始定时器中断模式输入捕获

该函数用于开始定时器中断模式输入捕获，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_TIM_IC_Start_IT( TIM_HandleTypeDef *htim,
                                         uint32_t Channel);
```

该函数的形参描述，如下表所示：

形参	描述
htim	指向 TIM 句柄的指针
Channel	定时器通道 例如：TIM_CHANNEL_1、TIM_CHANNEL_2 等（在 stm32f4xx_hal_tim.h 文件中有定义）

表 19.2.1.5 函数 HAL_TIM_IC_Start_IT()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 19.2.1.6 函数 HAL_TIM_IC_Start_IT()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 开启定时器通道 1 中断模式输入捕获 */
    HAL_TIM_IC_Start_IT(&tim_handle, TIM_CHANNEL_1);
}
```

19.2.2 通用定时器驱动

本章实验的通用定时器驱动主要负责向应用层提供通用定时器的初始化函数，并实现通用定时器的中断回调函数。本章实验中，通用定时器的驱动代码包括 gtim.c 和 gtim.h 两个文件。

通用定时器驱动中，对 TIM、GPIO 的相关宏定义，如下所示：

```
#define GTIM_TIMX_CAP          TIM5
#define GTIM_TIMX_CAP_IRQn     TIM5_IRQn
#define GTIM_TIMX_CAP_IRQHandler TIM5_IRQHandler
#define GTIM_TIMX_CAP_CLK_ENABLE() \
do {                             \
    __HAL_RCC_TIM5_CLK_ENABLE(); \
} while (0)
#define GTIM_TIMX_CAP_CHY      TIM_CHANNEL_1
#define GTIM_TIMX_CAP_CHY_GPIO_PORT GPIOA
#define GTIM_TIMX_CAP_CHY_GPIO_PIN GPIO_PIN_0
#define GTIM_TIMX_CAP_CHY_GPIO_AF GPIO_AF2_TIM5
#define GTIM_TIMX_CAP_CHY_GPIO_CLK_ENABLE() \
do {                             \
    __HAL_RCC_GPIOA_CLK_ENABLE(); \
} while (0)
```

通用定时器驱动中 TIM5 的初始化函数，如下所示：

```
/**
 * @brief  初始化通用定时器输入捕获
 * @param  arr: 自动重装载值
 * @param  psc: 预分频系数
 * @retval 无
 */
void gtim_timx_cap_chy_init(uint16_t arr, uint16_t psc)
{
    TIM_IC_InitTypeDef timx_ic_cap_struct = {0};

    /* 初始化定时器输入捕获 */
    g_timx_cap_handle.Instance = GTIM_TIMX_CAP;
    g_timx_cap_handle.Init.Prescaler = psc;
    g_timx_cap_handle.Init.CounterMode = TIM_COUNTERMODE_UP;
```

```
g_timx_cap_handle.Init.Period = arr;
HAL_TIM_IC_Init(&g_timx_cap_handle);

/* 配置定时器输入捕获通道 */
timx_ic_cap_struct.ICPolarity = TIM_ICPOLARITY_RISING;
timx_ic_cap_struct.ICSelection = TIM_ICSELECTION_DIRECTTI;
timx_ic_cap_struct.ICPrescaler = TIM_ICPSC_DIV1;
timx_ic_cap_struct.ICFilter = 0;
HAL_TIM_IC_ConfigChannel(&g_timx_cap_handle,
                        &timx_ic_cap_struct,
                        GTIM_TIMX_CAP_CHY);

/* 开始定时器中断模式输入捕获 */
HAL_TIM_ENABLE_IT(&g_timx_cap_handle, TIM_IT_UPDATE);
HAL_TIM_IC_Start_IT(&g_timx_cap_handle, GTIM_TIMX_CAP_CHY);
}
```

从 TIM5 的初始化代码中可以看到,初始化函数中初始化了 TIM5 的输入捕获并配置了 TIM5 的输入捕获通道 1,最后开始 TIM5 的中断模式输入捕获。

19.2.3 实验应用代码

本章实验的应用代码,如下所示:

```
int main(void)
{
    uint32_t total = 0;
    uint8_t t = 0;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    gtim_timx_cap_chy_init(0xFFFF, 84 - 1); /* 初始化通用定时器输入捕获 */

    while (1)
    {
        /* 捕获完成 */
        if ((g_timx_chy_cap_sta & 0x80) != 0)
        {
            /* 计算总捕获到的计数值 */
            total = g_timx_chy_cap_sta & 0x3F;
            total *= 0xFFFF;
            total += g_timx_chy_cap_val;
            printf("High: %dus\r\n", total);

            /* 开启下一次输入捕获 */
            g_timx_chy_cap_sta = 0;
        }

        if (++t == 20)
        {
            t = 0;
            LED0_TOGGLE();
        }

        delay_ms(10);
    }
}
```

从上面的代码中可以看到, TIM5 的预分频系数被配置为(84-1),同时因为 TIM5 的时钟频率为 84MHz,因此 TIM5 的计数频率为 1MHz,因此在 TIM5 成功捕获到外部输入信号的高电平后,可以直接计算出捕获到高电平的脉宽时间。

19.3 下载验证

在完成编译和烧录操作后，短暂按下并抬起 WKUP 按键，可以通过串口调试助手观察到捕获到的 WKUP 按键被按下的时间。

第二十章 通用定时器脉冲计数实验

本章将介绍使用 STM32F407 通用定时器对输入脉冲的个数进行计数。通过本章的学习，读者将学习到通用定时器从模式的使用。

本章分为如下几个小节：

- 20.1 硬件设计
- 20.2 程序设计
- 20.3 下载验证

20.1 硬件设计

20.1.1 例程功能

- 1. 串口输出 WKUP 按键被按下的次数，按下 KEY0 可清零计数值
- 2. LED0 闪烁，提示程序正在运行

20.1.2 硬件资源

- 1. LED
 - LED0 - PF9
- 2. 按键
 - WKUP - PA0
 - KEY0 - PE4
- 3. TIM2
 - CH1 - PA0
- 4. USART1
 - USART1_TX - PA9
 - USART1_RX - PA10

20.1.3 原理图

本章实验使用的 TIM2 为 STM32F407 的片上资源，因此没有对应的连接原理图。

20.2 程序设计

20.2.1 HAL 库的 TIM 驱动

本章实验将使用 WKUP 按键按下时产生的上升沿作为 TIM2 计数的触发源，因此除了像第十九章实验配置定时器输入捕获外，该需要配置通用定时器的从模式，具体的步骤如下：

①：配置定时器从模式

在 HAL 库中对应的驱动函数如下：

①：配置定时器从模式

该函数用于配置定时器从模式，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_TIM_SlaveConfigSynchro(  
    TIM_HandleTypeDef *htim,  
    TIM_SlaveConfigTypeDef *sSlaveConfig);
```

该函数的形参描述，如下表所示：

形参	描述
htim	指向 TIM 句柄的指针
sSlaveConfig	指向从模式配置结构体的指针 需自行定义，并根据从模式配置参数填充结构体中的成员变量

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

该函数使用 `TIM_SlaveConfigTypeDef` 类型结构体指针传入了定时器从模式的配置参数，该结构的定义如下所示：

该函数的使用示例，如下所示：

20.2.2 通用定时器驱动

本章实验的通用定时器驱动主要负责向应用层提供通用定时器的初始化、获取和清零计数值等函数，并实现通用定时器的中断回调函数。本章实验中，通用定时器的驱动代码包括 `gtim.c` 和 `gtim.h` 两个文件。

通用定时器驱动中，对 TIM、GPIO 的相关宏定义，如下所示：

通用定时器驱动中 TIM2 的初始化函数，如下所示：

103


```
TIM_SlaveConfigTypeDef tim_slave_config_struct = {0};

/* 初始化定时器输入捕获 */
g_timx_cnt_handle.Instance = GTIM_TIMX_CNT;
g_timx_cnt_handle.Init.Prescaler = psc;
g_timx_cnt_handle.Init.CounterMode = TIM_COUNTERMODE_UP;
g_timx_cnt_handle.Init.Period = 0xFFFF;
HAL_TIM_IC_Init(&g_timx_cnt_handle);

/* 配置定时器从模式 */
tim_slave_config_struct.SlaveMode = TIM_SLAVEMODE_EXTERNAL1;
tim_slave_config_struct.InputTrigger = TIM_TS_TI1FP1;
tim_slave_config_struct.TriggerPolarity = TIM_TRIGGERPOLARITY_RISING;
tim_slave_config_struct.TriggerPrescaler = TIM_TRIGGERPRESCALER_DIV1;
tim_slave_config_struct.TriggerFilter = 0;
HAL_TIM_SlaveConfigSynchro(&g_timx_cnt_handle, &tim_slave_config_struct);

/* 开始定时器输入捕获 */
HAL_TIM_ENABLE_IT(&g_timx_cnt_handle, TIM_IT_UPDATE);
HAL_TIM_IC_Start(&g_timx_cnt_handle, GTIM_TIMX_CNT_CHY);
}
```

从 TIM2 的初始化代码中可以看到,初始化函数中初始化了 TIM2 的输入捕获并配置了 TIM2 的从模式,最后开始 TIM2 的中断模式输入捕获。

通用定时器驱动中,获取和清零计数值的函数,如下所示:

```
/**
 * @brief 获取通用定时器脉冲计数值
 * @param 无
 * @retval 脉冲计数值
 */
uint32_t gtim_timx_cnt_chy_get_count(void)
{
    uint32_t total;

    /* 计算总脉冲计数值 */
    total = g_timx_chy_cnt_ofcnt * 0xFFFF;
    total += __HAL_TIM_GET_COUNTER(&g_timx_cnt_handle);

    return total;
}

/**
 * @brief 重启通用定时器脉冲计数
 * @param 无
 * @retval 无
 */
void gtim_timx_cnt_chy_restart(void)
{
    __HAL_TIM_DISABLE(&g_timx_cnt_handle);
    g_timx_chy_cnt_ofcnt = 0;
    __HAL_TIM_SET_COUNTER(&g_timx_cnt_handle, 0);
    __HAL_TIM_ENABLE(&g_timx_cnt_handle);
}
```

从上面的代码中可以看到,获取计数值时,就是对定时器当前的计数值和溢出次数进行计算得出的,清零计数值就是清零计数值和溢出次数。定时器的溢出次数是在 TIM2 的更新中断中累加的。

通用定时器驱动中, TIM2 的超时回调函数,如下所示:

```
/**
 * @brief HAL 库定时器超时中断回调函数
 * @param 无
 * @retval 无
 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
```

```
{
    /* 脉冲计数定时器 */
    if (htim->Instance == GTIM_TIMX_CNT)
    {
        /* 更新脉冲计数定时器溢出次数 */
        g_timx_chy_cnt_ofcnt++;
    }
}
```

TIM2 的超时回调函数就是用于在 TIM2 发生更新中断时，统计 TIM2 的溢出次数。

20.2.3 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    uint8_t key;
    uint32_t count;
    uint32_t count_prev = 0;
    uint8_t t = 0;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    gtim_timx_cnt_chy_init(0); /* 初始化通用定时器脉冲计数 */
    gtim_timx_cnt_chy_restart(); /* 重启通用定时器脉冲计数 */

    while (1)
    {
        key = key_scan(0);
        if (key == KEY0_PRES)
        {
            /* 重启通用定时器脉冲计数 */
            gtim_timx_cnt_chy_restart();
        }

        /* 获取通用定时器脉冲计数值 */
        count = gtim_timx_cnt_chy_get_count();
        /* 脉冲计数值有更新 */
        if (count_prev != count)
        {
            count_prev = count;
            printf("Cnt: %d\r\n", count);
        }

        if (++t == 20)
        {
            t = 0;
            LED0_TOGGLE();
        }

        delay_ms(10);
    }
}
```

从上面的代码中可以看到，TIM2 的预分频系数被配置为 0，因此每个脉冲都会触发 TIM2 的计数值加 1。完成相关初始化后，便不断地扫描按键和获取 TIM2 的计数值，若 KEY0 按键被按下，则会清零 TIM2 的计数值；若 WKUP 按键被按下，则会触发 TIM2 的计数值加 1，同时会将新的计数值通过串口输出至串口调试助手。

20.3 下载验证

在完成编译和烧录操作后，连续按下 WKUP 按键，模拟产生多个脉冲，可以通过串口调试助手观察到 WKUP 按键被按下的次数，同时按下 KEY0 按键，可清零统计值。

第二十一章 高级定时器输出指定个数 PWM 实验

本章将介绍使用 STM32F407 高级定时器输出指定个数的 PWM。通过本章的学习，读者将学习到高级定时器重复计数器的使用。

本章分为如下几个小节：

- 21.1 硬件设计
- 21.2 程序设计
- 21.3 下载验证

21.1 硬件设计

21.1.1 例程功能

- 1. 按下 KEY0 按键，PC6 引脚输出 5 个频率为 2Hz 的 PWM
- 2. LED0 闪烁，提示程序正在运行

21.1.2 硬件资源

- 1. LED
LED0 - PF9
- 2. 按键
KEY0 - PE4
- 3. TIM8
CH1 - PC6

21.1.3 原理图

本章实验使用的 TIM8 为 STM32F407 的片上资源，因此没有对应的连接原理图。

21.2 程序设计

21.2.1 HAL 库的 TIM 驱动

本章实验将使用 KEY0 按键控制 TIM8 通过通道 1（PC6 引脚）输出指定个数的 PWM，其具体的配置步骤如下：

- ①：初始化定时器 PWM
- ②：配置定时器 PWM 输出通道
- ③：开始定时器 PWM 输出

在 HAL 库中对应的驱动函数如下：

- ①：初始化定时器 PWM 输出

请见第 18.2.1 小节中初始化定时器 PWM 的相关内容。

- ②：配置定时器 PWM 输出通道

请见第 18.2.1 小节中配置定时器 PWM 输出通道的相关内容。

- ③：开始定时器 PWM 输出

请见第 18.2.1 小节中开始定时器 PWM 输出的相关内容。

21.2.2 高级定时器驱动

本章实验的高级定时器驱动主要负责向应用层提供高级定时器的初始化和输出指定个数 PWM 的函数，并实现高级定时器的中断回调函数。本章实验中，高级定时器的驱动代码包括 atim.c 和 atim.h 两个文件。

高级定时器驱动中，对 TIM、GPIO 的相关宏定义，如下所示：

```
#define ATIM_TIMX_NPWM TIM8
#define ATIM_TIMX_NPWM_IRQn TIM8_UP_TIM13_IRQn
#define ATIM_TIMX_NPWM_IRQHandler TIM8_UP_TIM13_IRQHandler
#define ATIM_TIMX_NPWM_CLK_ENABLE() \
do { \
    __HAL_RCC_TIM8_CLK_ENABLE(); \
} while (0)
#define ATIM_TIMX_NPWM_CHY TIM_CHANNEL_1
#define ATIM_TIMX_NPWM_CHY_GPIO_PORT GPIOC
#define ATIM_TIMX_NPWM_CHY_GPIO_PIN GPIO_PIN_6
#define ATIM_TIMX_NPWM_CHY_GPIO_AF GPIO_AF3_TIM8
#define ATIM_TIMX_NPWM_CHY_GPIO_CLK_ENABLE() \
do { \
    __HAL_RCC_GPIOC_CLK_ENABLE(); \
} while (0)
```

高级定时器驱动中 TIM8 的初始化函数，如下所示：

```
/**
 * @brief 初始化高级定时器输出指定个数 PWM
 * @param arr: 自动重装载值
 * @param psc: 预分频系数
 * @retval 无
 */
void atim_timx_chy_npwm_init(uint16_t arr, uint16_t psc)
{
    GPIO_InitTypeDef gpio_init_struct;
    TIM_OC_InitTypeDef timx_oc_init_struct = {0};

    /* 使能相关时钟 */
    ATIM_TIMX_NPWM_CHY_GPIO_CLK_ENABLE();
    ATIM_TIMX_NPWM_CLK_ENABLE();

    /* 配置 PWM 输出引脚 */
    gpio_init_struct.Pin = ATIM_TIMX_NPWM_CHY_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;
    gpio_init_struct.Pull = GPIO_PULLUP;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
    gpio_init_struct.Alternate = ATIM_TIMX_NPWM_CHY_GPIO_AF;
    HAL_GPIO_Init(ATIM_TIMX_NPWM_CHY_GPIO_PORT, &gpio_init_struct);

    /* 配置中断优先级并使能中断 */
    HAL_NVIC_SetPriority(ATIM_TIMX_NPWM_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(ATIM_TIMX_NPWM_IRQn);

    /* 初始化定时器 PWM */
    g_timx_npwm_handle.Instance = ATIM_TIMX_NPWM;
    g_timx_npwm_handle.Init.Prescaler = psc;
    g_timx_npwm_handle.Init.CounterMode = TIM_COUNTERMODE_UP;
    g_timx_npwm_handle.Init.Period = arr;
    g_timx_npwm_handle.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
    g_timx_npwm_handle.Init.RepetitionCounter = 0;
    HAL_TIM_PWM_Init(&g_timx_npwm_handle);

    /* 配置定时器 PWM 输出通道 */
    timx_oc_init_struct.OCMode = TIM_OCMODE_PWM1;
    timx_oc_init_struct.Pulse = (arr + 1) >> 1;
    timx_oc_init_struct.OCpolarity = TIM_OCPOLARITY_HIGH;
    HAL_TIM_PWM_ConfigChannel(&g_timx_npwm_handle,
                             &timx_oc_init_struct,
                             ATIM_TIMX_NPWM_CHY);

    /* 开始定时器 PWM 输出 */
    __HAL_TIM_ENABLE_IT(&g_timx_npwm_handle, TIM_IT_UPDATE);
    HAL_TIM_PWM_Start(&g_timx_npwm_handle, ATIM_TIMX_NPWM_CHY);
}
```

从 TIM8 的初始化代码中可以看到，初始化函数中初始化了 TIM8 的 PWM 并配置了 TIM8 的 PWM 输出通道 1，最后开始 TIM8 的 PWM 输出。

高级定时器驱动中，开启高级定时器输出指定个数 PWM 的函数，如下所示：

```
/**
 * @brief 设置高级定时器输出指定个数 PWM
 * @paramn pwm: 指定 PWM 个数
 * @retval 无
 */
void atim_timx_chy_npwm_set(uint32_t npwm)
{
    if (npwm == 0)
    {
        return;
    }

    /* 产生更新事件来输出指定个数 PWM */
    g_timx_chy_npwm_remain = npwm;
    HAL_TIM_GenerateEvent(&g_timx_npwm_handle, TIM_EVENTSOURCE_UPDATE);
    __HAL_TIM_ENABLE(&g_timx_npwm_handle);
}
```

该函数记录下了需要产生的 PWM 个数，因为会在 TIM8 的更新中断中处理输出 PWM，因此手动产生了一次更新事件，由于初始化函数中开启了 TIM8 的更新中断，因此随后会进入 TIM 的中断服务函数中。

高级定时器驱动中，TIM8 的中断回调函数，如下所示：

```
/**
 * @brief 输出指定个数 PWM 高级定时器中断回调函数
 * @param 无
 * @retval 无
 */
void ATIM_TIMX_NPWM_IRQHandler(void)
{
    uint16_t npwm = 0;

    /* 定时器更新事件中断 */
    if (__HAL_TIM_GET_FLAG(&g_timx_npwm_handle, TIM_FLAG_UPDATE) != RESET)
    {
        /* 每次最多输出 256 个 PWM (TIMx_RCR 只有[7:0]有效) */
        if (g_timx_chy_npwm_remain > 256)
        {
            g_timx_chy_npwm_remain -= 256;
            npwm = 256;
        }
        else if ((g_timx_chy_npwm_remain % 256) != 0)
        {
            npwm = g_timx_chy_npwm_remain % 256;
            g_timx_chy_npwm_remain = 0;
        }

        if (npwm != 0)
        {
            /* 利用高级定时器的重复计数器来输出 PWM */
            ATIM_TIMX_NPWM->RCR = npwm - 1;
            HAL_TIM_GenerateEvent(&g_timx_npwm_handle, TIM_EVENTSOURCE_UPDATE);
            __HAL_TIM_ENABLE(&g_timx_npwm_handle);
        }
        else
        {
            ATIM_TIMX_NPWM->CR1 &= ~(TIM_CR1_CEN);
        }

        /* 清除更新事件标志 */
        __HAL_TIM_CLEAR_IT(&g_timx_npwm_handle, TIM_IT_UPDATE);
    }
}
```

```
}
}
```

因为高级定时器的重复计数寄存器只有低八位有效，因此需要在需要产生大于 255 个 PWM 的情况下做特殊处理，随后将需要产生的 PWM 个数值减 1 写入 TIM8 的重复计数寄存器，并且由于写入重复计数寄存器的新值只有在下一次更新事件发生时才生效，因此需要手动产生一次更新事件，最后使能 TIM8，这么一来，TIM8 就会持续地输出 PWM 直到重复计数寄存器的数值为 0 后，产生一次更新中断，若此时已经输出完了指定个数的 PWM，那么就会关闭高级定时器。

21.2.3 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    uint8_t key;
    uint8_t t = 0;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟，168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    atim_timx_chy_npwm_init(5000-1, 16800-1); /* 初始化高级定时器输出指定个数 PWM */

    while (1)
    {
        key = key_scan(0);
        if (key == KEY0_PRES)
        {
            /* 设置高级定时器输出 5 个 PWM */
            atim_timx_chy_npwm_set(5);
        }

        if (++t == 20)
        {
            t = 0;
            LED0_TOGGLE();
        }

        delay_ms(10);
    }
}
```

从上面的代码中可以看到，TIM8 的自动重装载值配置为(5000-1)，TIM8 的预分频器数值配置为(16800-1)，并且 TIM8 的时钟频率为 168MHz，因此 TIM8 的计数频率为 10KHz，且 TIM8 每计数 5000 次溢出一次，因此溢出频率为 2Hz，也就是输出 PWM 的频率为 2Hz。

初始化完 TIM8 后，就重复地扫描按键，若 KEY0 按键被按下，则输出 5 个 PWM。

21.3 下载验证

在完成编译和烧录操作后，按下 KEY0 按键，此时可以通过示波器或外接 LED 的方式观察 PC6 引脚输出了 5 个频率为 2Hz，占空比为 50%的 PWM。

第二十二章 高级定时器输出比较模式实验

本章将介绍使用 STM32F407 的高级定时器输出多个频率、占空比相同但相位不同的 PWM。通过本章的学习，读者将学习到高级定时器输出比较的匹配时输出翻转模式的使用。

本章分为如下几个小节：

- 22.1 硬件设计
- 22.2 程序设计
- 22.3 下载验证

22.1 硬件设计

22.1.1 例程功能

1. PC6 和 PC7 引脚输出频率同为 500Hz，但相位相差 25%的 PWM
2. LED0 闪烁，提示程序正在运行

22.1.2 硬件资源

1. LED
LED0 - PF9
2. TIM8
CH1 - PC6
CH2 - PC7

22.1.3 原理图

本章实验使用的 TIM8 为 STM32F407 的片上资源，因此没有对应的连接原理图。

22.2 程序设计

22.2.1 HAL 库的 TIM 驱动

本章实验将使用 TIM8 输出四个频率、占空比相同，但相位不同的 PWM，其具体步骤如下：

- ①：初始化定时器输出比较
- ②：配置定时器输出比较通道
- ③：开始定时器输出比较

在 HAL 库中对应的驱动函数如下：

①：初始化定时器输出比较

该函数用于初始化定时器输出比较，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_TIM_OC_Init(TIM_HandleTypeDef *htim);
```

该函数的形参描述，如下表所示：

形参	描述
htim	指向 TIM 句柄的指针

表 22.2.1.1 函数 HAL_TIM_OC_Init()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 22.2.1.2 函数 HAL_TIM_OC_Init()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"
```

```
void example_fun(void)
```

```
{
    TIM_HandleTypeDef tim_handle = {0};

    /* 初始化 TIM8 输出比较 */
    tim_handle.Instance = TIM8;
    tim_handle.Init.Prescaler = 168 - 1;
    tim_handle.Init.CounterMode = TIM_COUNTERMODE_UP;
    tim_handle.Init.Period = 1000 - 1;
    tim_handle.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
    tim_handle.Init.RepetitionCounter = 0;
    HAL_TIM_OC_Init(&tim_handle);
}
```

②：配置定时器输出比较通道

该函数用于配置定时器的输出比较通道，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_TIM_OC_ConfigChannel(TIM_HandleTypeDef *htim,
                                            TIM_OC_InitTypeDef *sConfig,
                                            uint32_t Channel);
```

该函数的形参描述，如下表所示：

形参	描述
htim	指向 TIM 句柄的指针
sConfig	指向输出比较通道配置结构体的指针 需自行定义，并根据输出比较通道配置参数填充结构体中的成员变量
Channel	定时器通道 例如：TIM_CHANNEL_1、TIM_CHANNEL_2 等（在 stm32f4xx_hal_tim.h 文件中有定义）

表 22.2.1.3 函数 HAL_TIM_OC_ConfigChannel()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 22.2.1.4 函数 HAL_TIM_OC_ConfigChannel()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    TIM_OC_InitTypeDef tim_oc_init_struct = {0};

    /* 配置定时器通道 1 输出比较 */
    tim_oc_init_struct.OCMode = TIM_OCMODE_TOGGLE;
    tim_oc_init_struct.Pulse = 500 - 1;
    tim_oc_init_struct.OCpolarity = TIM_OCPOLARITY_HIGH;
    HAL_TIM_OC_ConfigChannel(&tim_handle, &tim_oc_init_struct, TIM_CHANNEL_1);
}
```

③：开始定时器输出比较

该函数用于开始定时器的输出比较，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_TIM_OC_Start(TIM_HandleTypeDef *htim, uint32_t Channel);
```

该函数的形参描述，如下表所示：

形参	描述
htim	指向 TIM 句柄的指针
Channel	定时器通道 例如：TIM_CHANNEL_1、TIM_CHANNEL_2 等（在 stm32f4xx_hal_tim.h 文件中有定义）

表 22.2.1.5 函数 HAL_TIM_OC_Start()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
-----	----

HAL_StatusTypeDef	HAL 状态
-------------------	--------

表 22.2.1.6 函数 HAL_TIM_OC_Start()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 开启定时器通道 1 输出比较 */
    HAL_TIM_OC_Start(&tim_handle, TIM_CHANNEL_1);
}
```

22.2.2 高级定时器驱动

本章实验的高级定时器驱动主要负责向应用层提供高级定时器的初始化函数。本章实验中，高级定时器的驱动代码包括 `atim.c` 和 `atim.h` 两个文件。

高级定时器驱动中，对 TIM、GPIO 的相关宏定义，如下所示：

```
#define ATIM_TIMX_COMP_TIM8
#define ATIM_TIMX_COMP_CLK_ENABLE() \
    do { \
        __HAL_RCC_TIM8_CLK_ENABLE(); \
    } while (0)
#define ATIM_TIMX_COMP_CH1_GPIO_PORT GPIOC
#define ATIM_TIMX_COMP_CH1_GPIO_PIN GPIO_PIN_6
#define ATIM_TIMX_COMP_CH1_GPIO_AF GPIO_AF3_TIM8
#define ATIM_TIMX_COMP_CH1_GPIO_CLK_ENABLE() \
    do { \
        __HAL_RCC_GPIOC_CLK_ENABLE(); \
    } while (0)
#define ATIM_TIMX_COMP_CH2_GPIO_PORT GPIOC
#define ATIM_TIMX_COMP_CH2_GPIO_PIN GPIO_PIN_7
#define ATIM_TIMX_COMP_CH2_GPIO_AF GPIO_AF3_TIM8
#define ATIM_TIMX_COMP_CH2_GPIO_CLK_ENABLE() \
    do { \
        __HAL_RCC_GPIOC_CLK_ENABLE(); \
    } while (0)
```

高级定时器驱动中 TIM8 的初始化函数，如下所示：

```
/**
 * @brief 初始化高级定时器输出比较
 * @param arr: 自动重装载值
 * @param psc: 预分频系数
 * @retval 无
 */
void atim_timx_comp_init(uint16_t arr, uint16_t psc)
{
    GPIO_InitTypeDef gpio_init_struct;
    TIM_OC_InitTypeDef timx_oc_init_struct = {0};

    /* 使能相关时钟 */
    ATIM_TIMX_COMP_CH1_GPIO_CLK_ENABLE();
    ATIM_TIMX_COMP_CH2_GPIO_CLK_ENABLE();
    ATIM_TIMX_COMP_CLK_ENABLE();

    /* 配置输出比较通道 1 输出引脚 */
    gpio_init_struct.Pin = ATIM_TIMX_COMP_CH1_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;
    gpio_init_struct.Pull = GPIO_PULLUP;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
    gpio_init_struct.Alternate = ATIM_TIMX_COMP_CH1_GPIO_AF;
    HAL_GPIO_Init(ATIM_TIMX_COMP_CH1_GPIO_PORT, &gpio_init_struct);

    /* 配置输出比较通道 2 输出引脚 */
    gpio_init_struct.Pin = ATIM_TIMX_COMP_CH2_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;
```

```

gpio_init_struct.Pull = GPIO_PULLUP;
gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
gpio_init_struct.Alternate = ATIM_TIMX_COMP_CH2_GPIO_AF;
HAL_GPIO_Init(ATIM_TIMX_COMP_CH2_GPIO_PORT, &gpio_init_struct);

/* 初始化定时器输出比较 */
g_timx_comp_handle.Instance = ATIM_TIMX_COMP;
g_timx_comp_handle.Init.Prescaler = psc;
g_timx_comp_handle.Init.CounterMode = TIM_COUNTERMODE_UP;
g_timx_comp_handle.Init.Period = arr;
g_timx_comp_handle.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
g_timx_comp_handle.Init.RepetitionCounter = 0;
HAL_TIM_OC_Init(&g_timx_comp_handle);

/* 配置定时器通道 1 输出比较 */
timx_oc_init_struct.OCMode = TIM_OCMODE_TOGGLE;
timx_oc_init_struct.Pulse = ((arr + 1) >> 2) * 1;
timx_oc_init_struct.OCpolarity = TIM_OCPOLARITY_HIGH;
HAL_TIM_OC_ConfigChannel(&g_timx_comp_handle,
                        &timx_oc_init_struct,
                        TIM_CHANNEL_1);

/* 配置定时器通道 2 输出比较 */
timx_oc_init_struct.OCMode = TIM_OCMODE_TOGGLE;
timx_oc_init_struct.Pulse = ((arr + 1) >> 2) * 2;
timx_oc_init_struct.OCpolarity = TIM_OCPOLARITY_HIGH;
HAL_TIM_OC_ConfigChannel(&g_timx_comp_handle,
                        &timx_oc_init_struct,
                        TIM_CHANNEL_2);

/* 开始定时器输出比较 */
HAL_TIM_OC_Start(&g_timx_comp_handle, TIM_CHANNEL_1);
HAL_TIM_OC_Start(&g_timx_comp_handle, TIM_CHANNEL_2);
}

```

在 TIM8 的初始化函数中，初始化了 TIM8 的输出比较同时配置了 TIM8 的输出比较通道 1 和通道 2，最后开始了 TIM8 的输出比较通道 1 和通道 2。

22.2.3 实验应用代码

本章实验的应用代码，如下所示：

```

int main(void)
{
    uint8_t t = 0;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    uart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    atim_timx_comp_init(1000 - 1, 168 - 1); /* 初始化高级定时器输出比较 */

    while (1)
    {
        if (++t == 20)
        {
            t = 0;
            LED0_TOGGLE();
        }

        delay_ms(10);
    }
}

```

从上面的代码中可以看到，TIM8 的自动重载值配置为(1000-1)，TIM8 的预分频器数值配

置为(168-1)，并且 TIM8 的时钟频率为 168MHz，因此 TIM8 的计数频率为 1MHz，且 TIM8 每计数 1000 次溢出一次，因此溢出频率为 1KHz，但 TIM8 的两个通道配置为了匹配时输出翻转模式，因此两个通道输出的 PWM 频率应为 500Hz。

22.3 下载验证

在完成编译和烧录操作后，可以通过示波器观察 PC6 引脚和 PC7 引脚输出的两路 PWM，可以发现这两路 PWM 的频率都为 500Hz、占空比都为 50%，但相位两两相差了 25%。

第二十三章 高级定时器互补输出带死区控制实验

本章将介绍使用 STM32F407 输出带死区和刹车控制的两路互补 PWM。通过本章的学习，读者将学习到高级定时器的互补输出、死区插入和刹车的功能的使用。

本章分为如下几个小节：

- 23.1 硬件设计
- 23.2 程序设计
- 23.3 下载验证

23.1 硬件设计

23.1.1 例程功能

1. 若 PA6 引脚输入低电平，则 PA8 和 PA7 引脚输出频率为 1KHz，占空比为 30%的互补 PWM
2. 若 PA6 引脚输入高电平，则 PA8 和 PA7 停止输出 PWM
3. LED0 闪烁，提示程序正在运行

23.1.2 硬件资源

1. LED
LED0 - PF9
2. TIM1
CH1 - PA8
CH1N - PA7
BKIN - PA6

23.1.3 原理图

本章实验使用的 TIM1 为 STM32F407 的片上资源，因此没有对应的连接原理图。

23.2 程序设计

23.2.1 HAL 库的 TIM 驱动

本章实验将使用 TIM1 的通道 1 和通道 1 的互补通道输出两路带死区的互补 PWM，同时还使用到了刹车功能，其具体的配置步骤如下：

- ①：初始化定时器 PWM
- ②：配置定时器 PWM 输出通道
- ③：开始定时器 PWM 输出
- ④：开始定时器互补通道 PWM 输出

在 HAL 库中对应的驱动函数如下：

①：初始化定时器 PWM 输出

请见第 18.2.1 小节中初始化定时器 PWM 的相关内容。

②：配置定时器 PWM 输出通道

请见第 18.2.1 小节中配置定时器 PWM 输出通道的相关内容。

③：配置定时器刹车死区时间

该函数用于配置定时器的刹车死区时间，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_TIMEx_ConfigBreakDeadTime(  
    TIM_HandleTypeDef *htim,  
    TIM_BreakDeadTimeConfigTypeDef *sBreakDeadTimeConfig);
```

该函数的形参描述，如下表所示：

形参	描述
htim	指向 TIM 句柄的指针
sBreakDeadTimeConfig	指向刹车死区时间配置结构体的指针 需自行定义，并根据刹车死区时间配置参数填充结构体中的成员变量

表 23.2.1.1 函数 HAL_TIMEx_ConfigBreakDeadTime()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 23.2.1.2 函数 HAL_TIMEx_ConfigBreakDeadTime()返回值描述

该函数使用 TIM_BreakDeadTimeConfigTypeDef 类型结构体指针传入了定时器刹车死区时间的配置参数，该结构体的定义如下所示：

```
typedef struct
{
    uint32_t OffStateRunMode; /* 运行模式 */
    uint32_t OffStateIDLEMode; /* 空闲模式 */
    uint32_t LockLevel; /* Lock 等级 */
    uint32_t DeadTime; /* 死区时间 */
    uint32_t BreakState; /* 刹车状态 */
    uint32_t BreakPolarity; /* 刹车极性 */
    uint32_t BreakFilter; /* 刹车滤波 */
    uint32_t AutomaticOutput; /* 刹车自动恢复 */
} TIM_BreakDeadTimeConfigTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    TIM_BreakDeadTimeConfigTypeDef tim_break_dead_time_config_struct = {0};

    /* 配置定时器刹车死区时间 */
    tim_break_dead_time_config_struct.OffStateRunMode = TIM_OSSR_DISABLE;
    tim_break_dead_time_config_struct.OffStateIDLEMode = TIM_OSSI_DISABLE;
    tim_break_dead_time_config_struct.LockLevel = TIM_LOCKLEVEL_OFF;
    tim_break_dead_time_config_struct.DeadTime = 100;
    tim_break_dead_time_config_struct.BreakState = TIM_BREAK_ENABLE;
    tim_break_dead_time_config_struct.BreakPolarity = TIM_BREAKPOLARITY_HIGH;
    tim_break_dead_time_config_struct.BreakFilter = 0;
    tim_break_dead_time_config_struct.AutomaticOutput =
        TIM_AUTOMATICOUTPUT_ENABLE;

    HAL_TIMEx_ConfigBreakDeadTime( &tim_handle,
        &tim_break_dead_time_config_struct);
}
```

④：开始定时器 PWM 输出

请见第 18.2.1 小节中开始定时器 PWM 输出的相关内容。

⑤：开始定时器互补通道 PWM 输出

该函数用于开始定时器的互补通道 PWM 输出，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_TIMEx_PWMN_Start( TIM_HandleTypeDef *htim,
    uint32_t Channel);
```

该函数的形参描述，如下表所示：

形参	描述
htim	指向 TIM 句柄的指针
Channel	定时器通道 例如：TIM_CHANNEL_1、TIM_CHANNEL_2 等（在 stm32f4xx_hal_tim.h 文件中有定义）

表 23.2.1.3 函数 HAL_TIMEx_PWMN_Start()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 23.2.1.4 函数 HAL_TIMEx_PWMN_Start()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 开启定时器通道 1 互补通道 PWM 输出 */
    HAL_TIMEx_PWMN_Start(&tim_handle, TIM_CHANNEL_1);
}
```

23.2.2 高级定时器驱动

本章实验的高级定时器驱动主要负责向应用层提供高级定时器的初始化函数和 PWM 占空比、死区时间的配置函数。本章实验中，高级定时器的驱动代码包括 atim.c 和 atim.h 两个文件。

高级定时器驱动中，对 TIM、GPIO 相关的宏定义，如下所示：

```
#define ATIM_TIMX_CPLM_TIM1
#define ATIM_TIMX_CPLM_CLK_ENABLE() \
do { \
    __HAL_RCC_TIM1_CLK_ENABLE(); \
} while (0)
#define ATIM_TIMX_CPLM_CHY TIM_CHANNEL_1
#define ATIM_TIMX_CPLM_CHY_GPIO_PORT GPIOA
#define ATIM_TIMX_CPLM_CHY_GPIO_PIN GPIO_PIN_8
#define ATIM_TIMX_CPLM_CHY_GPIO_AF GPIO_AF1_TIM1
#define ATIM_TIMX_CPLM_CHY_GPIO_CLK_ENABLE() \
do { \
    __HAL_RCC_GPIOA_CLK_ENABLE(); \
} while (0)
#define ATIM_TIMX_CPLM_CHYN_GPIO_PORT GPIOA
#define ATIM_TIMX_CPLM_CHYN_GPIO_PIN GPIO_PIN_7
#define ATIM_TIMX_CPLM_CHYN_GPIO_AF GPIO_AF1_TIM1
#define ATIM_TIMX_CPLM_CHYN_GPIO_CLK_ENABLE() \
do { \
    __HAL_RCC_GPIOA_CLK_ENABLE(); \
} while (0)
#define ATIM_TIMX_CPLM_BKIN_GPIO_PORT GPIOA
#define ATIM_TIMX_CPLM_BKIN_GPIO_PIN GPIO_PIN_6
#define ATIM_TIMX_CPLM_BKIN_GPIO_AF GPIO_AF1_TIM1
#define ATIM_TIMX_CPLM_BKIN_GPIO_CLK_ENABLE() \
do { \
    __HAL_RCC_GPIOA_CLK_ENABLE(); \
} while (0)
```

高级定时器驱动中 TIM8 的初始化函数，如下所示：

```
/**
 * @brief 初始化高级定时器互补输出带死区控制
 * @param arr: 自动重装载值
 * @param psc: 预分频系数
 * @retval 无
 */
void atim_timx_cplm_init(uint16_t arr, uint16_t psc)
{
    GPIO_InitTypeDef gpio_init_struct;
    TIM_OC_InitTypeDef timx_oc_struct = {0};

    /* 使能相关时钟 */
    ATIM_TIMX_CPLM_CHY_GPIO_CLK_ENABLE();
    ATIM_TIMX_CPLM_CHYN_GPIO_CLK_ENABLE();
    ATIM_TIMX_CPLM_BKIN_GPIO_CLK_ENABLE();
    ATIM_TIMX_CPLM_CLK_ENABLE();
}
```

```

/* 配置输出比较通道 1 输出引脚 */
gpio_init_struct.Pin = ATIM_TIMX_CPLM_CHY_GPIO_PIN;
gpio_init_struct.Mode = GPIO_MODE_AF_PP;
gpio_init_struct.Pull = GPIO_PULLUP;
gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
gpio_init_struct.Alternate = ATIM_TIMX_CPLM_CHY_GPIO_AF;
HAL_GPIO_Init(ATIM_TIMX_CPLM_CHY_GPIO_PORT, &gpio_init_struct);

/* 配置输出比较通道 1 互补输出引脚 */
gpio_init_struct.Pin = ATIM_TIMX_CPLM_CHYN_GPIO_PIN;
gpio_init_struct.Mode = GPIO_MODE_AF_PP;
gpio_init_struct.Pull = GPIO_PULLUP;
gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
gpio_init_struct.Alternate = ATIM_TIMX_CPLM_CHYN_GPIO_AF;
HAL_GPIO_Init(ATIM_TIMX_CPLM_CHYN_GPIO_PORT, &gpio_init_struct);

/* 配置输出比较通道刹车输入引脚 */
gpio_init_struct.Pin = ATIM_TIMX_CPLM_BKIN_GPIO_PIN;
gpio_init_struct.Mode = GPIO_MODE_AF_PP;
gpio_init_struct.Pull = GPIO_PULLUP;
gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
gpio_init_struct.Alternate = ATIM_TIMX_CPLM_BKIN_GPIO_AF;
HAL_GPIO_Init(ATIM_TIMX_CPLM_BKIN_GPIO_PORT, &gpio_init_struct);

/* 初始化定时器 PWM 输出 */
g_timx_cplm_handle.Instance = ATIM_TIMX_CPLM;
g_timx_cplm_handle.Init.Prescaler = psc;
g_timx_cplm_handle.Init.CounterMode = TIM_COUNTERMODE_UP;
g_timx_cplm_handle.Init.Period = arr;
g_timx_cplm_handle.Init.ClockDivision = TIM_CLOCKDIVISION_DIV4;
g_timx_cplm_handle.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
g_timx_cplm_handle.Init.RepetitionCounter = 0;
HAL_TIM_PWM_Init(&g_timx_cplm_handle);

/* 配置定时器 PWM 输出通道 */
timx_oc_struct.OCMode = TIM_OCMODE_PWM1;
timx_oc_struct.Pulse = (arr + 1) >> 1;
timx_oc_struct.OCpolarity = TIM_OCPOLARITY_LOW;
timx_oc_struct.OCNPolarity = TIM_OCNPOLARITY_LOW;
timx_oc_struct.OCIdleState = TIM_OCIDLESTATE_SET;
timx_oc_struct.OCNIdleState = TIM_OCNIDLESTATE_SET;
HAL_TIM_PWM_ConfigChannel(&g_timx_cplm_handle,
                          &timx_oc_struct,
                          ATIM_TIMX_CPLM_CHY);

/* 配置定时器刹车死区时间 */
g_break_dead_time_config_struct.OffStateRunMode = TIM_OSSR_DISABLE;
g_break_dead_time_config_struct.OffStateIDLEMode = TIM_OSSI_DISABLE;
g_break_dead_time_config_struct.LockLevel = TIM_LOCKLEVEL_OFF;
g_break_dead_time_config_struct.BreakState = TIM_BREAK_ENABLE;
g_break_dead_time_config_struct.BreakPolarity = TIM_BREAKPOLARITY_HIGH;
g_break_dead_time_config_structAutomaticOutput = TIM_AUTOMATICOUTPUT_ENABLE;
HAL_TIMEx_ConfigBreakDeadTime(&g_timx_cplm_handle,
                               &g_break_dead_time_config_struct);

/* 开始定时器 PWM 输出 */
HAL_TIM_PWM_Start(&g_timx_cplm_handle, ATIM_TIMX_CPLM_CHY);
HAL_TIMEx_PWMN_Start(&g_timx_cplm_handle, ATIM_TIMX_CPLM_CHY);
}

```

从上面的代码中可以看出，初始化函数初始化了 TIM1 的 PWM 输出，同时配置了 TIM1 的 PWM 输出通道，并且也配置了 TIM1 的刹车和死区时间，最后开始了 TIM1 的 PWM 输出。

高级定时器驱动中配置 PWM 占空比和死区时间的函数，如下所示：

```
/**
```

```

* @brief    设置高级定时器互补输出带死区控制
* @param    ccr: 输出比较值
* @param    dtg: 死区时间
* @retval   无
*/
void atim_timx_cplm_set(uint16_t ccr, uint8_t dtg)
{
    g_break_dead_time_config_struct.DeadTime = dtg;      /* 死区时间 */
    HAL_TIMEx_ConfigBreakDeadTime(&g_timx_cplm_handle, /* 配置死区时间参数 */
    &g_break_dead_time_config_struct);
    __HAL_TIM_MOE_ENABLE(&g_timx_cplm_handle);          /* 使能主输出 */
    ATIM_TIMX_CPLM->CCR1 = ccr;                          /* 设置比较寄存器 */
}

```

从上面的代码中可以看出，该函数配置了 TIM1 的死区时间和输出比较值，因为配置 PWM 的占空比就是配置对应通道的输出比较值。

23.2.3 实验应用代码

本章实验的应用代码，如下所示：

```

int main(void)
{
    uint8_t t = 0;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    atim_timx_cplm_init(1000 - 1, 168 - 1); /* 初始化高级定时器互补输出带死区控制 */
    atim_timx_cplm_set(300 - 1, 100); /* 设置高级定时器互补输出带死区控制 */

    while (1)
    {
        if (++t == 20)
        {
            t = 0;
            LED0_TOGGLE();
        }

        delay_ms(10);
    }
}

```

从上面的代码中可以看到，TIM1 的自动重装载值配置为(1000-1)，TIM1 的预分频器数值配置为(168-1)，并且 TIM1 的时钟频率为 168MHz，因此 TIM1 的计数频率为 1MHz，且 TIM1 每计数 1000 次溢出一次，因此溢出频率为 1KHz，因此 TIM1 通道 1 及其互补通道输出两路互补 PWM 的频率也应该为 1KHz。

随后配置了 TIM1 通道 1 的比较值为(300-1)，与自动重装载值的比值为 30%，因此输出 PWM 的占空比也应该为 30%，同时配置了死区时间为 100，因为 TIM1 的时钟频率为 168MHz，因此死区时间在配置为 100 的情况下，对应的具体时间为 $(100 \times 1 / 168 \text{MHz}) \text{ns} \approx 595 \text{ns}$ 。

23.3 下载验证

在完成编译和烧录操作后，可以通过示波器观察 PA8 引脚和 PA7 引脚输出的两路 PWM，可以发现，这两路 PWM 为互补 PWM，且频率为 1KHz、占空比为 30%、死区时间大约为 595ns。应为是禁了刹车和自动输出功能，因此将 PA6 引脚接入有效的高电平后，可以看到两路 PWM 都被禁止输出了，撤销 PA6 引脚接入的高电平后，可以看到两路 PWM 有自动恢复输出了。

第二十四章 高级定时器 PWM 输入模式实验

本章将介绍使用 STM32F407 的高级定时器检测输入 PWM 的占空比和周期。通过本章的学习，读者将学习到高级定时器 PWM 输入模式的使用。

本章分为如下几个小节：

24.1 硬件设计

24.2 程序设计

24.3 下载验证

24.1 硬件设计

24.1.1 例程功能

1. PC6 引脚输出频率为 10KHz 的 PWM，串口输出 PC6 引脚输入 PWM 的周期和高电平脉宽
2. LED0 闪烁，提示程序正在运行

24.1.2 硬件资源

1. LED
LED0 - PF9
2. TIM8
CH1 - PC6
3. TIM3
CH1 - PB4
4. USART1
USART1_TX - PA9
USART1_RX - PA10

24.1.3 原理图

本章实验使用的 TIM8 为 STM32F407 的片上资源，因此没有对应的连接原理图。

24.2 程序设计

24.2.1 HAL 库的 TIM 驱动

本章实验将使用 TIM8 的通道 1(PC6 引脚)在 PWM 输入模式下捕获 TIM3 从通道 1(PB4)引脚输出的 PWM（有关通用定时器输出 PWM 的相关内容，请见第十八章“通用定时器 PWM 输出实验”），将分别捕获输入 PWM 信号的上升沿和下降沿，以此来计算输入 PWM 信号的占空比和周期，要是实现以上功能都依赖于高级定时器的 PWM 输入模式，其具体的配置步骤如下：

- ①：初始化定时器输入捕获
- ②：配置定时器从模式
- ③：配置定时器输入捕获通道
- ④：开始定时器输入捕获

在 HAL 库中对应的驱动函数如下：

①：初始化定时器输入捕获

请见第 19.2.1 小节中初始化定时器输入捕获的相关内容。

②：配置定时器从模式

请见第 20.2.1 小节中配置定时器从模式的相关内容。

③：配置定时器输入捕获通道

请见第 19.2.1 小节中配置定时器输入捕获通道的相关内容。

④：开始定时器输入捕获

请见第 19.2.1 小节中开始定时器输入捕获的相关内容。

24.2.2 高级定时器驱动

本章实验的高级定时器驱动主要负责向应用层提供高级定时器的初始化函数，并实现高级定时器的中断服务函数。本章实验中，高级定时器的驱动代码包括 `atim.c` 和 `atim.h` 两个文件。

高级定时器驱动中，对 TIM、GPIO 相关的宏定义，如下所示：

```
#define ATIM_TIMX_PWMIN TIM8
#define ATIM_TIMX_PWMIN_IRQn TIM8_CC_IRQn
#define ATIM_TIMX_PWMIN_IRQHandler TIM8_CC_IRQHandler
#define ATIM_TIMX_PWMIN_CLK_ENABLE() \
do { \
    __HAL_RCC_TIM8_CLK_ENABLE(); } \
while (0)
#define ATIM_TIMX_PWMIN_CHY TIM_CHANNEL_1
#define ATIM_TIMX_PWMIN_CHY_GPIO_PORT GPIOC
#define ATIM_TIMX_PWMIN_CHY_GPIO_PIN GPIO_PIN_6
#define ATIM_TIMX_PWMIN_CHY_GPIO_AF GPIO_AF3_TIM8
#define ATIM_TIMX_PWMIN_CHY_GPIO_CLK_ENABLE() \
do { \
    __HAL_RCC_GPIOC_CLK_ENABLE(); \
} while (0)
```

高级定时器驱动中 TIM8 的初始化函数，如下所示：

```
/**
 * @brief 初始化高级定时器 PWM 输入
 * @param psc: 预分频系数
 * @retval 无
 */
void atim_timx_pwmin_chy_init(uint16_t psc)
{
    GPIO_InitTypeDef gpio_init_struct;
    TIM_SlaveConfigTypeDef tim_slave_config_struct = {0};
    TIM_IC_InitTypeDef tim_ic_init_struct = {0};

    /* 使能相关时钟 */
    ATIM_TIMX_PWMIN_CLK_ENABLE();
    ATIM_TIMX_PWMIN_CHY_GPIO_CLK_ENABLE();

    /* 配置 PWM 输入引脚 */
    gpio_init_struct.Pin = ATIM_TIMX_PWMIN_CHY_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;
    gpio_init_struct.Pull = GPIO_PULLDOWN;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
    gpio_init_struct.Alternate = ATIM_TIMX_PWMIN_CHY_GPIO_AF;
    HAL_GPIO_Init(ATIM_TIMX_PWMIN_CHY_GPIO_PORT, &gpio_init_struct);

    /* 初始化定时器输入捕获 */
    g_timx_pwmin_handle.Instance = ATIM_TIMX_PWMIN;
    g_timx_pwmin_handle.Init.Prescaler = psc;
    g_timx_pwmin_handle.Init.CounterMode = TIM_COUNTERMODE_UP;
    g_timx_pwmin_handle.Init.Period = 0xFFFF;
    HAL_TIM_IC_Init(&g_timx_pwmin_handle);

    /* 配置定时器从模式 */
    tim_slave_config_struct.SlaveMode = TIM_SLAVEMODE_RESET;
    tim_slave_config_struct.InputTrigger = TIM_TS_TIF1;
    tim_slave_config_struct.TriggerPolarity = TIM_TRIGGERPOLARITY_RISING;
    tim_slave_config_struct.TriggerFilter = 0;
    HAL_TIM_SlaveConfigSynchro(&g_timx_pwmin_handle, &tim_slave_config_struct);

    /* 配置定时器输入捕获通道 */
}
```

```
tim_ic_init_struct.ICPolarity = TIM_ICPOLARITY_RISING;
tim_ic_init_struct.ICSelection = TIM_ICSELECTION_DIRECTTI;
tim_ic_init_struct.ICPrescaler = TIM_ICPSC_DIV1;
tim_ic_init_struct.ICFilter = 0;
HAL_TIM_IC_ConfigChannel( &g_timx_pwm_in_handle,
                          &tim_ic_init_struct,
                          TIM_CHANNEL_1);

/* 配置定时器输入捕获通道 */
tim_ic_init_struct.ICPolarity = TIM_ICPOLARITY_FALLING;
tim_ic_init_struct.ICSelection = TIM_ICSELECTION_INDIRECTTI;
tim_ic_init_struct.ICPrescaler = TIM_ICPSC_DIV1;
tim_ic_init_struct.ICFilter = 0;
HAL_TIM_IC_ConfigChannel( &g_timx_pwm_in_handle,
                          &tim_ic_init_struct,
                          TIM_CHANNEL_2);

/* 配置中断优先级并使能中断 */
HAL_NVIC_SetPriority(ATIM_TIMX_PWM_IN_IRQn, 0, 0);
HAL_NVIC_EnableIRQ( ATIM_TIMX_PWM_IN_IRQn );

/* 开始定时器输入捕获 */
HAL_TIM_IC_Start_IT(&g_timx_pwm_in_handle, TIM_CHANNEL_1);
HAL_TIM_IC_Start_IT(&g_timx_pwm_in_handle, TIM_CHANNEL_2);
}
```

从上面的代码中可以看出，在配置定时器输入捕获通道时，配置了 IC1 捕获上升沿，IC2 捕获下降沿，并同时开启了捕获比较 1 和 2 的中断，这么一来就可以在中断回调函数中获取通道 1 和通道 2 的捕获比较寄存器值来计算输入 PWM 信号的周期和占空比了。

高级定时器驱动中，TIM8 的中断回调函数，如下所示：

```
/**
 * @brief    PWM 输入高级定时器中断回调函数
 * @param    无
 * @retval    无
 */
void ATIM_TIMX_PWM_IN_IRQHandler(void)
{
    /* 输入捕获通道 1 捕获到上升沿 */
    if ( __HAL_TIM_GET_FLAG(&g_timx_pwm_in_handle, TIM_FLAG_CC1) != RESET)
    {
        /* 获取捕获到 PWM 的周期 */
        g_timx_chy_pwm_in_cval =
            HAL_TIM_ReadCapturedValue(&g_timx_pwm_in_handle, TIM_CHANNEL_1) + 1;
        g_timx_chy_pwm_in_sta = 1;

        __HAL_TIM_CLEAR_FLAG(&g_timx_pwm_in_handle, TIM_FLAG_CC1);
    }

    /* 输入捕获通道 2 捕获到下降沿 */
    if ( __HAL_TIM_GET_FLAG(&g_timx_pwm_in_handle, TIM_FLAG_CC2) != RESET)
    {
        /* 获取捕获到 PWM 的高电平脉宽 */
        g_timx_chy_pwm_in_hval =
            HAL_TIM_ReadCapturedValue(&g_timx_pwm_in_handle, TIM_CHANNEL_2) + 1;

        __HAL_TIM_CLEAR_FLAG(&g_timx_pwm_in_handle, TIM_FLAG_CC2);
    }
}
```

从上面的代码中可以看出，在捕获比较通道 1 中断中获取通道 1 的捕获比较寄存器值就是 TIM8 在输入 PWM 信号的两个上升沿之间的计数值，通过该值可以计算出输入 PWM 信号的周期；在捕获比较通道 2 中断中获取通道 2 的捕获比较寄存器值就是 TIM8 在输入 PWM 信号的上升沿和下降沿之间的计数值，通过该值可以计算出输入 PWM 信号的高电平占空比。

24.2.3 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    uint8_t t = 0;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟，168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    gtim_timx_pwm_chy_init(100 - 1, 84 - 1); /* 初始化通用定时器 PWM */
    atim_timx_pwmmin_chy_init(168 - 1); /* 初始化高级定时器 PWM 输入 */

    while (1)
    {
        /* 捕获成功 */
        if (g_timx_chy_pwmmin_sta == 1)
        {
            g_timx_chy_pwmmin_sta = 0;

            printf("高电平时间: %d us\r\n", g_timx_chy_pwmmin_hval);
            printf("PWM 周期: %d us\r\n", g_timx_chy_pwmmin_cval);
            printf("PWM 频率: %d Hz\r\n", 1000000 / g_timx_chy_pwmmin_cval);
            printf("\r\n");
        }

        if (++t == 20)
        {
            t = 0;
            LED0_TOGGLE();
        }

        delay_ms(10);
    }
}
```

从上面的代码中可以看出，会配置通用定时器输出一个频率为 100Hz、占空比为 50% 的 PWM，该 PWM 信号用于作为本实验中高级定时器的 PWM 输入信号。随后会初始化高级定时器 TIM8，TIM8 的预分频计数器值配置为(168-1)，并且 TIM8 的时钟频率为 168MHz，因此 TIM8 的计数频率为 1MHz，即 1 个计数值对应 1 微秒，方便后面的时间计算。

初始化完成后，就等待高级定时器中断函数中的捕获成功标志为真，捕获成功后，便将捕获到的 PWM 输入信号的周期和占空比输出值串口调试助手。

24.3 下载验证

在完成编译和烧录操作后，将 TIM3 通道 1 输出的 PWM 信号接入 TIM8 的 PWM 输入引脚，即将 PB4 引脚与 PC6 引脚进行短接，随后便可在串口调试助手上看到，捕获到 PWM 输入信号的高电平时间为 5000 微秒、PWM 周期为 10000 微秒，即 PWM 的占空比为 50%，并且也能看到 PWM 输入信号的频率为 100Hz。

第二十五章 OLED 实验

本章将介绍使用 STM32F407 驱动 OLED 模块进行显示。通过本章的学习，读者将学习到使用 GPIO 模拟 8080 时序以及 OLED 模块的驱动。

本章分为如下几个小节：

- 25.1 硬件设计
- 25.2 程序设计
- 25.3 下载验证

25.1 硬件设计

25.1.1 例程功能

- 1. OLED 上不断刷新显示字符
- 2. LED0 闪烁，提示程序正在运行

25.1.2 硬件资源

- 1. LED
 - LED0 - PF9
- 2. ATK-MD0096 模块

25.1.3 原理图

本章实验使用了一个正点原子 0.96 寸 OLED 模块，该模块需通过 OLED 模块延长线与板载的 OLED 接口进行连接，该接口也可与摄像头模块进行连接，该接口与板载 MCU 的连接原理图，如下图所示：

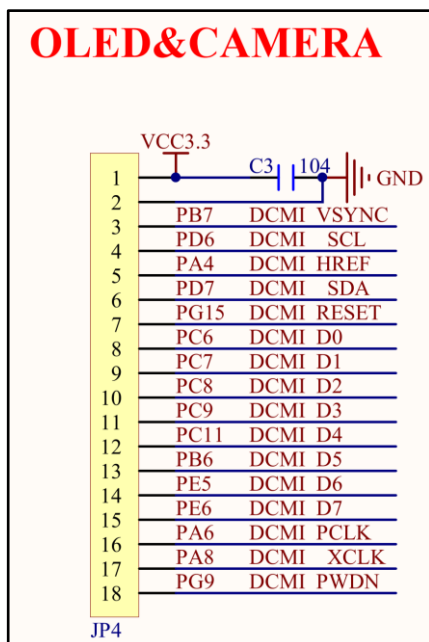


图 25.1.3.1 OLED 模块与 MCU 的连接原理图

25.2 程序设计

25.2.1 HAL 库的 GPIO 驱动

本章实验通过控制与 OLED 模块连接的 GPIO 模拟 8080 时序，实现配置 OLED 模块并在 OLED 模块上显示指定字符，对于 GPIO，主要涉及 GPIO 的配置和写操作，需要以下几个步骤：

- ①：配置 GPIO 引脚为通用输出模式
- ②：在与 OLED 模块通讯时，根据需求控制指定 GPIO 引脚输出指定电平

在 HAL 库中对应的驱动函数如下：

①：配置 GPIO 引脚

请见第 10.2.1 小节中配置 GPIO 引脚的相关内容。

②：设置 GPIO 引脚输出电平

使用 HAL 库提供的驱动函数设置 GPIO 引脚输出电平的方式，请见第 10.2.1 小节中设置 GPIO 引脚输出电平的相关内容。虽然 HAL 库已经提供了完整的 GPIO 驱动函数，但在大多数场景下，调用 HAL 库中 GPIO 驱动函数的效率并没有直接操作 GPIO 寄存器的方式高，例如，本章实验中将以直接操作寄存器的方式写与 OLED 模块通讯时模拟的 8080 时序通讯的数据总线，具体的方式是操作 GPIO 端口的 GPIO 端口输出数据寄存器（GPIOx_ODR），其使用示例，如下所示：

```
#include "stm32f4xx.h"

void example_fun(void)
{
    /* 设置 PA0 输出高电平 */
    GPIOA->ODR |= (1 << 0);

    /* 设置 PB1 输出低电平 */
    GPIOB->ODR &= ~(1 << 1);
}
```

25.2.2 OLED 驱动

本章介绍使用 GPIO 模拟 8080 时序驱动 OLED 模块，本质就是控制 GPIO 输出高低电平。OLED 驱动主要负责向应用层提供 OLED 的初始化函数，和各种 OLED 显示的操作函数。本章实验中，OLED 的驱动代码包括 oled.c、oled.h 和字体文件 oledfont.h 三个文件。

OLED 驱动中，对 GPIO 相关的宏定义，如下所示：

```
#define OLED_SPI_RST_PORT    GPIOG
#define OLED_SPI_RST_PIN    GPIO_PIN_15
#define OLED_SPI_RST_CLK_ENABLE() \
do { \
    __HAL_RCC_GPIOG_CLK_ENABLE(); \
} while (0)

#define OLED_SPI_CS_PORT    GPIOB
#define OLED_SPI_CS_PIN    GPIO_PIN_7
#define OLED_SPI_CS_CLK_ENABLE() \
do { \
    __HAL_RCC_GPIOB_CLK_ENABLE(); \
} while (0)

#define OLED_SPI_RS_PORT    GPIOB
#define OLED_SPI_RS_PIN    GPIO_PIN_6
#define OLED_SPI_RS_CLK_ENABLE() \
do { \
    __HAL_RCC_GPIOB_CLK_ENABLE(); \
} while (0)

#define OLED_RST(x) do { (x) ? \
    HAL_GPIO_WritePin(OLED_SPI_RST_PORT, OLED_SPI_RST_PIN, GPIO_PIN_SET); \
    HAL_GPIO_WritePin(OLED_SPI_RST_PORT, OLED_SPI_RST_PIN, GPIO_PIN_RESET); \
}
```

```

    } while (0)

#define OLED_CS(x) do { (x) ?
    HAL_GPIO_WritePin(OLED_SPI_CS_PORT, OLED_SPI_CS_PIN, GPIO_PIN_SET); \
    HAL_GPIO_WritePin(OLED_SPI_CS_PORT, OLED_SPI_CS_PIN, GPIO_PIN_RESET); \
    } while (0)

#define OLED_RS(x) do { (x) ?
    HAL_GPIO_WritePin(OLED_SPI_RS_PORT, OLED_SPI_RS_PIN, GPIO_PIN_SET); \
    HAL_GPIO_WritePin(OLED_SPI_RS_PORT, OLED_SPI_RS_PIN, GPIO_PIN_RESET); \
    } while (0)

#define OLED_WR(x) do { (x) ?
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET); \
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_RESET); \
    } while (0)

#define OLED_RD(x) do { (x) ?
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_11, GPIO_PIN_SET); \
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_11, GPIO_PIN_RESET); \
    } while (0)

```

OLED 驱动中，OLED 的初始化函数，如下所示：

```

/**
 * @brief  初始化 OLED
 * @param  无
 * @retval 无
 */
void oled_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;

    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();
    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOD_CLK_ENABLE();
    __HAL_RCC_GPIOE_CLK_ENABLE();
    __HAL_RCC_GPIOG_CLK_ENABLE();

    gpio_init_struct.Pin = GPIO_PIN_4;
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;
    gpio_init_struct.Pull = GPIO_PULLUP;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &gpio_init_struct);

    gpio_init_struct.Pin = GPIO_PIN_6 |
                           GPIO_PIN_7;
    HAL_GPIO_Init(GPIOB, &gpio_init_struct);

    gpio_init_struct.Pin = GPIO_PIN_6 |
                           GPIO_PIN_7 |
                           GPIO_PIN_8 |
                           GPIO_PIN_9 |
                           GPIO_PIN_11;
    HAL_GPIO_Init(GPIOC, &gpio_init_struct);

    gpio_init_struct.Pin = GPIO_PIN_6 |
                           GPIO_PIN_7;
    HAL_GPIO_Init(GPIOD, &gpio_init_struct);

    gpio_init_struct.Pin = GPIO_PIN_5 |
                           GPIO_PIN_6;
    HAL_GPIO_Init(GPIOE, &gpio_init_struct);

    gpio_init_struct.Pin = GPIO_PIN_15;
    HAL_GPIO_Init(GPIOG, &gpio_init_struct);
}

```

```

OLED_WR(1);
OLED_RD(1);

OLED_CS(1);
OLED_RS(1);

OLED_RST(0);
delay_ms(100);
OLED_RST(1);

oled_wr_byte(0xAE, OLED_CMD);
oled_wr_byte(0xD5, OLED_CMD);
oled_wr_byte(0x50, OLED_CMD);
oled_wr_byte(0xA8, OLED_CMD);
oled_wr_byte(0x3F, OLED_CMD);
oled_wr_byte(0xD3, OLED_CMD);
oled_wr_byte(0x00, OLED_CMD);
oled_wr_byte(0x40, OLED_CMD);
oled_wr_byte(0x8D, OLED_CMD);
oled_wr_byte(0x14, OLED_CMD);
oled_wr_byte(0x20, OLED_CMD);
oled_wr_byte(0x02, OLED_CMD);
oled_wr_byte(0xA1, OLED_CMD);
oled_wr_byte(0xC8, OLED_CMD);
oled_wr_byte(0xDA, OLED_CMD);
oled_wr_byte(0x12, OLED_CMD);
oled_wr_byte(0x81, OLED_CMD);
oled_wr_byte(0xEF, OLED_CMD);
oled_wr_byte(0xD9, OLED_CMD);
oled_wr_byte(0xF1, OLED_CMD);
oled_wr_byte(0xDB, OLED_CMD);
oled_wr_byte(0x30, OLED_CMD);
oled_wr_byte(0xA4, OLED_CMD);
oled_wr_byte(0xA6, OLED_CMD);
oled_wr_byte(0xAF, OLED_CMD);
oled_clear();
}

```

从上面的代码中可以看出，OLED 的初始化函数，就是配置与 OLED 模块连接的 GPIO 引脚为输出模式，同时根据通信协议控制 GPIO 输出高低电平，以完成配置 OLED 模块的操作。

OLED 驱动中，向 OLED 写一字节数据的函数如下所示：

```

/**
 * @brief 向 OLED 写入一个字节
 * @param data: 要输出的数据
 * @param cmd : 命令/数据标志
 * @arg OLED_CMD : 命令
 * @arg OLED_DATA: 数据
 * @retval 无
 */
static void oled_wr_byte(uint8_t data, uint8_t cmd)
{
    oled_data_out(data);
    OLED_RS(cmd);
    OLED_CS(0);
    OLED_WR(0);
    OLED_WR(1);
    OLED_CS(1);
    OLED_RS(1);
}

```

从上面的代码中可以看出，向 OLED 写一个字节就是按照 8080 时序来控制 GPIO 完成的，其中操作数据总线的函数 oled_data_out() 是通过直接操作 GPIO 寄存器完成的，其函数如下所示：

```

/**
 * @brief 通过拼凑的方法向 OLED 输出一个 8 位数据
 * @param data: 要输出的数据

```

```

* @retval 无
*/
static void oled_data_out(uint8_t data)
{
    GPIOC->ODR &= ~(0xF<<6);          /* GPIOC[9:6]清零 */
    GPIOC->ODR |= ((uint16_t)data&0xF)<<6; /* data[3:0]-->GPIOC[9:6] */

    GPIOC->ODR &= ~(0x1<<11);          /* GPIOC[11]清零 */
    GPIOC->ODR |= ((uint16_t)(data>>4)&0x1)<<11; /* data[4]-->GPIOC[11] */

    GPIOB->ODR &= ~(0x1<<6);          /* GPIOB[6]清零 */
    GPIOB->ODR |= ((uint16_t)(data>>5)&0x1)<<6; /* data[5]-->GPIOB[6] */

    GPIOB->ODR &= ~(0x3<<5);          /* GPIOB[9:8]清零 */
    GPIOB->ODR |= ((uint16_t)(data>>6)&0x3)<<5; /* data[7:6]-->GPIOB[9:8] */
}

```

通过上面介绍的驱动函数就能够往 OLED 模块写入命令或数据了，而在 OLED 模块的显示屏上显示出特定的字符或图案，都是通过 OLED 模块规定的特定命令来完成的，想深究的读者可以查看正点原子 ATK-MD0096 模块的用户手册或查看实际使用的 OLED 模块相关的文档。

25.2.3 实验应用代码

本章实验的应用代码，如下所示：

```

int main(void)
{
    uint8_t t = ' ';

    HAL_Init();          /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168);      /* 初始化延时 */
    usart_init(115200);   /* 初始化串口 */
    led_init();           /* 初始化 LED */
    oled_init();          /* 初始化 OLED */

    oled_show_string(0, 0, "ALIENTEK", 24);
    oled_show_string(0, 24, "0.96' OLED TEST", 16);
    oled_show_string(0, 40, "ATOM 2022/10/15", 12);
    oled_show_string(0, 52, "ASCII:", 12);
    oled_show_string(64, 52, "CODE:", 12);
    oled_refresh_gram();

    while (1)
    {
        oled_show_char(36, 52, t, 12, 1); /* 显示 ASCII 字符 */
        oled_show_num(94, 52, t, 3, 12); /* 显示 ASCII 字符的码值 */
        oled_refresh_gram();              /* 更新显存到 OLED */

        if (++t > '~')
        {
            t = ' ';
        }

        LED0_TOGGLE();
        delay_ms(500);
    }
}

```

从上面的代码中可以看出，在初始化完 OLED 后，便使用函数 `oled_show_string()` 将一些字符串写入到本地的显存中，然后使用函数 `oled_refresh_gram()` 将本地显存中的数据刷新值 OLED 上进行显示，最后在 `while` 循环中遍历 ASCII 表中的部分字符，将其轮流显示值 OLED 上，同时显示字符对应的 ASCII 码值。

25.3 下载验证

在完成编译和烧录操作后，先将开发板断电，随后将 OLED 模块通过 OLED 延长线与开发板进行连接，最后再给开发板供电（在插拔开发板上的接插件和模块时，要求必须断电操作，否则容易烧毁硬件）。程序运行后，可以看到 OLED 上显示了预期的字符串，同时不断地遍历显示 ASCII 表中的部分字符，即字符对应的 ASCII 码值。

第二十六章 TFTLCD（MCU 屏）实验

本章将介绍使用 STM32F407 驱动 TFTLCD（MCU 屏）进行显示。通过本章的学习，读者将学习到 FSMC 的使用。

本章分为如下几个小节：

- 26.1 硬件设计
- 26.2 程序设计
- 26.3 下载验证

26.1 硬件设计

26.1.1 例程功能

- 1. TFTLCD 上显示实验信息，并不断刷新底色
- 2. LED0 闪烁，提示程序正在运行

26.1.2 硬件资源

- 1. LED
 - LED0 - PF9
- 2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块

26.1.3 原理图

本章实验使用了正点原子的 TFTLCD 模块（兼容正点原子 2.8/3.5/4.3/7/10 寸的 TFTLCD 模块），该模块需通过 LCD 转接板与板载的 TFTLCD 接口进行连接，该接口与板载 MCU 的连接原理图，如下图所示：

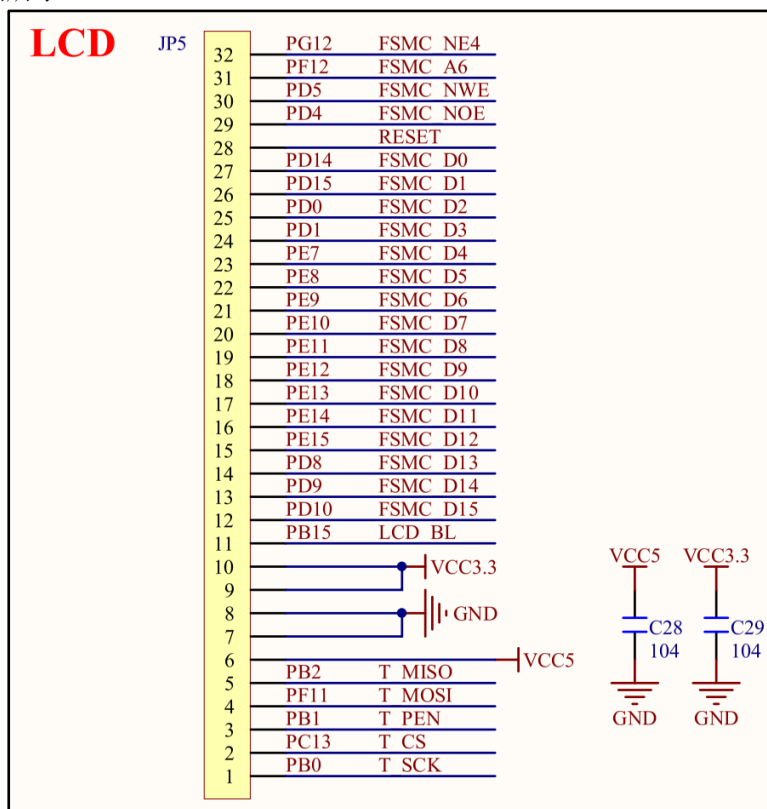


图 26.1.3.1 TFTLCD 模块与 MCU 的连接原理图

从上图中可以看到，TFTLCD 模块连接至 FSMC 的存储块 1 的区域块 4，并且使用了地址线 A6 作为 TFTLCD 模块命令或数据的选择信号，可计算出配置好 FSMC 后，TFTLCD 模块的命令访问地址被映射到 0x6C00007E，TFTLCD 模块的数据访问地址被映射到 0x6C000080。

26.2 程序设计

26.2.1 HAL 库的 FSMC 驱动

本章实验通过 FSMC 驱动 8080 并口的 TFTLCD 模块，通过 FSMC 可以将 TFTLCD 模块的命令和数据寄存器映射为两个地址，往这两个地址写入或读取数据就可直接与 TFTLCD 模块进行通讯，因此需要对 FSMC 做相应的配置，具体的步骤如下：

①：配置 FSMC 存储块 1 的区域块 1

在 HAL 库中对应的驱动函数如下：

①：配置 FSMC 存储块 1 的区域块

该函数用于配置 FSMC 存储块 1 的区域块，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_SRAM_Init( SRAM_HandleTypeDef *hsram,
                                  FMC_NORSRAM_TimingTypeDef *Timing,
                                  FMC_NORSRAM_TimingTypeDef *ExtTiming);
```

该函数的形参描述，如下表所示：

形参	描述
hsram	指向 SRAM 句柄的指针
Timing	指向 SRAM 读时序配置结构体的指针 需自行定义，并根据 SRAM 读时序配置参数填充结构体中的成员变量
ExtTiming	指向 SRAM 写时序配置结构体的指针 需自行定义，并根据 SRAM 写时序配置参数填充结构体中的成员变量

表 26.2.1.1 函数 HAL_SRAM_Init()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 26.2.1.2 函数 HAL_SRAM_Init()返回值描述

该函数使用 FMC_NORSRAM_TimingTypeDef 类型的结构体变量传入 FSMC 存储块 1 区域块的读写时序配置参数，该结构体的定义如下所示：

```
typedef struct
{
    uint32_t AddressSetupTime; /* 地址建立时间 */
    uint32_t AddressHoldTime; /* 地址保持时间 */
    uint32_t DataSetupTime; /* 数据建立时间 */
    uint32_t BusTurnAroundDuration; /* 总线转换时间 */
    uint32_t CLKDivision; /* 时钟分频系数 */
    uint32_t DataLatency; /* 数据延时 */
    uint32_t AccessMode; /* 访问模式 */
} FMC_NORSRAM_TimingTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    SRAM_HandleTypeDef sram_handle = {0};
    FMC_NORSRAM_TimingTypeDef read_timing = {0};
    FMC_NORSRAM_TimingTypeDef write_timing = {0};

    /* 填充读数据结构体 */
    read_timing.AddressSetupTime = 0xFF;
```

```

read_timing.AddressHoldTime = 0xFF;
read_timing.DataSetupTime = 0xFF;
read_timing.AccessMode = FSMC_ACCESS_MODE_A;

/* 填充写时序结构体 */
write_timing.AddressSetupTime = 0xFF;
write_timing.AddressHoldTime = 0xFF;
write_timing.DataSetupTime = 0xFF;
write_timing.AccessMode = FSMC_ACCESS_MODE_A;

/* 配置 FSMC 存储块 1 的区域块 */
sram_handle.Instance = FSMC_NORSRAM_DEVICE;
sram_handle.Extended = FSMC_NORSRAM_EXTENDED_DEVICE;
sram_handle.Init.NSBank = FSMC_NORSRAM_BANK1;
sram_handle.Init.DataAddressMux = FSMC_DATA_ADDRESS_MUX_DISABLE;
sram_handle.Init.MemoryType = FSMC_MEMORY_TYPE_SRAM;
sram_handle.Init.MemoryDataWidth = FSMC_NORSRAM_MEM_BUS_WIDTH_16;
sram_handle.Init.BurstAccessMode = FSMC_BURST_ACCESS_MODE_DISABLE;
sram_handle.Init.WaitSignalPolarity = FSMC_WAIT_SIGNAL_POLARITY_LOW;
sram_handle.Init.WrapMode = FSMC_WRAP_MODE_DISABLE;
sram_handle.Init.WaitSignalActive = FSMC_WAIT_TIMING_BEFORE_WS;
sram_handle.Init.WriteOperation = FSMC_WRITE_OPERATION_ENABLE;
sram_handle.Init.WaitSignal = FSMC_WAIT_SIGNAL_DISABLE;
sram_handle.Init.ExtendedMode = FSMC_EXTENDED_MODE_ENABLE;
sram_handle.Init.AsynchronousWait = FSMC_ASYNCHRONOUS_WAIT_DISABLE;
sram_handle.Init.WriteBurst = FSMC_WRITE_BURST_DISABLE;
sram_handle.Init.PageSize = FSMC_PAGE_SIZE_NONE;
HAL_SRAM_Init(&sram_handle, &read_timing, &write_timing);
}

```

26.2.2 TFTLCD 驱动

本章实验的 TFTLCD 驱动主要负责向应用层提供 TFTLCD 的初始化和各种 TFTLCD 显示的操作函数。本章实验中，TFTLCD 的驱动代码包括 lcd.c、lcd_ex.c、lcd.h 和字体文件 lcdfont.h 四个文件。

由于 TFTLCD 模块需要使用到大量的 GPIO 引脚，因此对于 GPIO 的相关定义，请读者自行查看 lcd.c 和 lcd.h 这两个文件。

TFTLCD 驱动中，TFTLCD 的初始化函数，如下所示：

```

/**
 * @brief 初始化 LCD
 * @param 无
 * @retval 无
 */
void lcd_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;
    FSMC_NORSRAM_TimingTypeDef fsmc_read_timing_struct = {0};
    FSMC_NORSRAM_TimingTypeDef fsmc_write_timing_struct = {0};

    /* 使能时钟 */
    LCD_BL_GPIO_CLK_ENABLE();

    /* 配置 LCD BL 引脚 */
    gpio_init_struct.Pin = LCD_BL_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;
    gpio_init_struct.Pull = GPIO_PULLUP;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(LCD_BL_GPIO_PORT, &gpio_init_struct);

    /* 配置 FSMC 读时序 */
    fsmc_read_timing_struct.AddressSetupTime = 15;
    fsmc_read_timing_struct.AddressHoldTime = 0;
    fsmc_read_timing_struct.DataSetupTime = 60;
}

```

```
fsmc_read_timing_struct.AccessMode = FSMC_ACCESS_MODE_A;

/* 配置 FSMC 写时序 */
fsmc_write_timing_struct.AddressSetupTime = 9;
fsmc_write_timing_struct.AddressHoldTime = 0;
fsmc_write_timing_struct.DataSetupTime = 9;
fsmc_write_timing_struct.AccessMode = FSMC_ACCESS_MODE_A;

/* 配置 FSMC */
g_sram_handle.Instance = FSMC_NORSRAM_DEVICE;
g_sram_handle.Extended = FSMC_NORSRAM_EXTENDED_DEVICE;
g_sram_handle.Init.NSBank = FSMC_NORSRAM_BANK4;
g_sram_handle.Init.DataAddressMux = FSMC_DATA_ADDRESS_MUX_DISABLE;
g_sram_handle.Init.MemoryType = FSMC_MEMORY_TYPE_SRAM;
g_sram_handle.Init.MemoryDataWidth = FSMC_NORSRAM_MEM_BUS_WIDTH_16;
g_sram_handle.Init.BurstAccessMode = FSMC_BURST_ACCESS_MODE_DISABLE;
g_sram_handle.Init.WaitSignalPolarity = FSMC_WAIT_SIGNAL_POLARITY_LOW;
g_sram_handle.Init.WrapMode = FSMC_WRAP_MODE_DISABLE;
g_sram_handle.Init.WaitSignalActive = FSMC_WAIT_TIMING_BEFORE_WS;
g_sram_handle.Init.WriteOperation = FSMC_WRITE_OPERATION_ENABLE;
g_sram_handle.Init.WaitSignal = FSMC_WAIT_SIGNAL_DISABLE;
g_sram_handle.Init.ExtendedMode = FSMC_EXTENDED_MODE_ENABLE;
g_sram_handle.Init.AsynchronousWait = FSMC_ASYNCHRONOUS_WAIT_DISABLE;
g_sram_handle.Init.WriteBurst = FSMC_WRITE_BURST_DISABLE;
g_sram_handle.Init.PageSize = FSMC_PAGE_SIZE_NONE;
HAL_SRAM_Init( &g_sram_handle,
               &fsmc_read_timing_struct,
               &fsmc_write_timing_struct);

delay_ms(50);

/* 尝试读取 ILI9341 的 ID */
lcd_wr_regno(0xD3);
lcddev.id = lcd_rd_data(); /* 无效数据 */
lcddev.id = lcd_rd_data(); /* 读取到 0x00 */
lcddev.id = lcd_rd_data() << 8; /* 读取到 0x93 */
lcddev.id |= lcd_rd_data(); /* 读取到 0x41 */
if (lcddev.id != 0x9341) /* 尝试读取 ST7789 的 ID */
{
    lcd_wr_regno(0x04);
    lcddev.id = lcd_rd_data(); /* 无效数据 */
    lcddev.id = lcd_rd_data(); /* 读取到 0x85 */
    lcddev.id = lcd_rd_data() << 8; /* 读取到 0x85 */
    lcddev.id |= lcd_rd_data(); /* 读取到 0x52 */
    if (lcddev.id == 0x8552) /* 8552 的 ID 记为 0x7789 */
    {
        lcddev.id = 0x7789;
    }
    if (lcddev.id != 0x7789) /* 尝试读取 NT35310 的 ID */
    {
        lcd_wr_regno(0xD4);
        lcddev.id = lcd_rd_data(); /* 无效数据 */
        lcddev.id = lcd_rd_data(); /* 读取到 0x01 */
        lcddev.id = lcd_rd_data() << 8; /* 读取到 0x53 */
        lcddev.id |= lcd_rd_data(); /* 读取到 0x10 */
        if (lcddev.id != 0x5310) /* 尝试读取 NT35510 的 ID */
        {
            lcd_wr_regno(0xD3);
            lcddev.id = lcd_rd_data(); /* 无效数据 */
            lcddev.id = lcd_rd_data(); /* 读取到 0x01 */
            lcddev.id = lcd_rd_data() << 8; /* 读取到 0x53 */
            lcddev.id |= lcd_rd_data(); /* 读取到 0x10 */
            if (lcddev.id != 0x7796) /* 尝试读取 NT35510 的 ID */
            {

```

```

/* 发送密钥 (厂家提供) */
lcd_write_reg(0xF000, 0x0055);
lcd_write_reg(0xF001, 0x00AA);
lcd_write_reg(0xF002, 0x0052);
lcd_write_reg(0xF003, 0x0008);
lcd_write_reg(0xF004, 0x0001);

lcd_wr_regno(0xC500);
lcddev.id = lcd_rd_data() << 8;          /* 读取到 0x80 */
lcd_wr_regno(0xC501);
lcddev.id |= lcd_rd_data();              /* 读取到 0x00 */
delay_ms(5);
if(lcddev.id==0x8000)lcddev.id=0x5510;
if (lcddev.id != 0x5510)                  /* 尝试读取 SSD1963 的 ID */
{
    lcd_wr_regno(0xD3);
    lcddev.id = lcd_rd_data();
    lcddev.id = lcd_rd_data() << 8;      /* 读取到 0x55 */
    lcddev.id |= lcd_rd_data();          /* 读取到 0x10 */
    if (lcddev.id == 0x9806)
    {
        lcd_wr_regno(0xA1);
        lcddev.id = lcd_rd_data();
        lcddev.id = lcd_rd_data() << 8; /* 读取到 0x98 */
        lcddev.id |= lcd_rd_data();     /* 读取到 0x06 */
    }

    if (lcddev.id == 0x5761)              /* 将 SSD1963 的 ID 记为 0x1963 */
    {
        lcddev.id = 0x1963;
    }
}
}
}

if (lcddev.id == 0x7789)
{
    lcd_ex_st7789_reginit(); /* 执行 ST7789 初始化 */
}
else if (lcddev.id == 0x7796)
{
    lcd_ex_st7796_reginit(); /* 执行 ST7796 初始化 */
}
else if (lcddev.id == 0x9341)
{
    lcd_ex_ili9341_reginit(); /* 执行 ILI9341 初始化 */
}
else if (lcddev.id == 0x5310)
{
    lcd_ex_nt35310_reginit(); /* 执行 NT35310 初始化 */
}
else if (lcddev.id == 0x5510)
{
    lcd_ex_nt35510_reginit(); /* 执行 NT35510 初始化 */
}
else if (lcddev.id == 0x9806)
{
    lcd_ex_ili9806_reginit(); /* 执行 ILI9806 初始化 */
}
else if (lcddev.id == 0x1963)
{
    lcd_ex_ssd1963_reginit(); /* 执行 SSD1963 初始化 */
}

```

```

        lcd_ssd_backlight_set(100); /* 背光设置为最亮 */
    }

    /* 初始化完成后, 提速 */
    if ( (lcddev.id == 0x9341) ||
        (lcddev.id == 0x1963) ||
        (lcddev.id == 0x7789) ||
        (lcddev.id == 0x7796))
    {
        fsmc_write_timing_struct.AddressSetupTime = 3;
        fsmc_write_timing_struct.DataSetupTime = 3;
        FSMC_NORSRAM_Extended_Timing_Init( g_sram_handle.Extended,
                                            &fsmc_write_timing_struct,
                                            g_sram_handle.Init.NSBank,
                                            g_sram_handle.Init.ExtendedMode);
    }
    else if ( (lcddev.id == 0x5310) ||
              (lcddev.id == 0x5510) ||
              (lcddev.id == 0x9806))
    {
        fsmc_write_timing_struct.AddressSetupTime = 2;
        fsmc_write_timing_struct.DataSetupTime = 2;
        FSMC_NORSRAM_Extended_Timing_Init( g_sram_handle.Extended,
                                            &fsmc_write_timing_struct,
                                            g_sram_handle.Init.NSBank,
                                            g_sram_handle.Init.ExtendedMode);
    }

    lcd_display_dir(0); /* 默认设置为竖屏 */
    LCD_BL(1);          /* 点亮背光 */
    lcd_clear(WHITE);   /* 清屏 */
}

```

从上的代码中可以看出, 本章实验的 TFTLCD 驱动是兼容了正点原子的多款 TFTLCD 模块的, 因此在初始化完 FSMC 后, 会与 TFTLCD 进行通讯, 确定 TFTLCD 的型号, 然后根据型号针对性地对 TFTLCD 模块进行配置。

TFTLCD 驱动中与 TFTLCD 模块通讯的函数, 如下所示:

```

/**
 * @brief   LCD 写数据
 * @param   data: 要写入的数据
 * @retval  无
 */
void lcd_wr_data(volatile uint16_t data)
{
    data = data;
    LCD->LCD_RAM = data;
}

/**
 * @brief   LCD 写寄存器编号或地址
 * @param   regno: 寄存器编号或地址
 * @retval  无
 */
void lcd_wr_regno(volatile uint16_t regno)
{
    regno = regno;
    LCD->LCD_REG = regno;
}

/**
 * @brief   LCD 写寄存器
 * @param   regno: 寄存器编号
 * @param   data : 要写入的数据
 * @retval  无
 */

```

```

*/
void lcd_write_reg(uint16_t regno, uint16_t data)
{
    LCD->LCD_REG = regno;
    LCD->LCD_RAM = data;
}

/**
 * @brief    LCD 读数据
 * @param    无
 * @retval   读取到的数据
 */
static uint16_t lcd_rd_data(void)
{
    volatile uint16_t ram;

    ram = LCD->LCD_RAM;

    return ram;
}

```

从上面的代码中可以看出，与 TFTLCD 的通讯都是通过 LCD 这一结构体对象来完成的，对于 LCD 结构体的相关定义，如下所示：

```

typedef struct
{
    volatile uint16_t LCD_REG;
    volatile uint16_t LCD_RAM;
} LCD_TypeDef;

#define LCD_FSMC_NEX    4
#define LCD_FSMC_AX    6
#define LCD_BASE        \
    (uint32_t)((0x60000000+(0x4000000*(LCD_SMC_NEX-1))|(((1<LCD_SMC_AX)*2)-2))
#define LCD              ((LCD_TypeDef *)LCD_BASE)

```

从 LCD 结构体的相关定义中可以看出，与 TFTLCD 模块的通讯地址是与 TFTLCD 连接的 FSMC 存储块 1 的区域块和 TFTLCD 模块命令、数据选择信号所连接的 FSMC 地址线是有关的。通过上面宏定义的计算，可以算出 LCD_BASE 宏定义的值 0x6C00007E，因此访问 LCD->LCD_REG 就是访问 0x6C00007E 这一地址，访问 LCD->LCD_RAM 就是访问 0x6C000080 这一地址，这两个地址也就是通过 FSMC 映射的 TFTLCD 命令和数据寄存器的访问地址。

通过上面介绍的驱动函数就能够与 TFTLCD 模块进行通讯了，而在 TFTLCD 模块的显示屏上显示出特定的图案或字符或设置 TFTLCD 模块的显示方向等等的操作都是能够通过 TFTLCD 模块规定的特定命令来完成的，想深究的读者可以产看正点原子 TFTLCD 模块的用户手册或查看实际使用的 TFTLCD 模块的相关文档。

26.2.3 实验应用代码

本章实验的应用代码，如下所示：

```

int main(void)
{
    uint8_t x = 0;
    uint8_t lcd_id[13];

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    lcd_init(); /* 初始化 LCD */

    sprintf((char *)lcd_id, "LCD ID: %04X", lcddev.id);
    while (1)
    {

```

```
switch (x)
{
    case 0:
    {
        lcd_clear(WHITE);
        break;
    }
    case 1:
    {
        lcd_clear(BLACK);
        break;
    }
    case 2:
    {
        lcd_clear(BLUE);
        break;
    }
    case 3:
    {
        lcd_clear(RED);
        break;
    }
    case 4:
    {
        lcd_clear(MAGENTA);
        break;
    }
    case 5:
    {
        lcd_clear(GREEN);
        break;
    }
    case 6:
    {
        lcd_clear(CYAN);
        break;
    }
    case 7:
    {
        lcd_clear(YELLOW);
        break;
    }
    case 8:
    {
        lcd_clear(BRRED);
        break;
    }
    case 9:
    {
        lcd_clear(GRAY);
        break;
    }
    case 10:
    {
        lcd_clear(LGRAY);
        break;
    }
    case 11:
    {
        lcd_clear(BROWN);
        break;
    }
}

lcd_show_string(10, 40, 240, 32, 32, "STM32", RED);
lcd_show_string(10, 80, 240, 24, 24, "TFTLCD TEST", RED);
lcd_show_string(10, 110, 240, 16, 16, "ATOM@ALIENTEK", RED);
```



```
lcd_show_string(10, 130, 240, 16, 16, (char *)lcd_id, RED);

if (++x == 12)
{
    x = 0;
}

LED0_TOGGLE();
delay_ms(1000);
}
```

从上面的代码中可以看出，在初始化完 LCD 后，便在 LCD 上显示一些本实验的相关信息，随后便每间隔 1000 毫秒就更换一次 LCD 屏幕显示的背景色。

26.3 下载验证

在完成编译和烧录操作后，先将开发板断电，随后将 TFTLCD 模块通过 LCD 转接板与开发板进行连接，最后再给开发板供电（在插拔开发板上的接插件和模块时，要求必须断电操作，否则容易烧毁硬件）。程序运行后，可以看到 LCD 上显示了本实验的相关信息，同时 LCD 显示的背景色被间隔 1000 毫秒就切换一次。

第二十七章 USART 调试实验

USART 调试组件是正点原子开发的一款灵巧的串口调试交互组件，其功能类似 Linux 的 Shell，支持通过串口调用程序中的任意函数，对调试代码有很大的帮助。通过本章的学习，读者将学习到 USART 调试组件的使用。

本章分为如下几个小节：

- 27.1 硬件设计
- 27.2 程序设计
- 27.3 下载验证

27.1 硬件设计

27.1.1 例程功能

1. 通过 USART 调用程序中的函数，实现对 LCD、LED 和延时操作
2. LED0 闪烁，指示程序正在运行

27.1.2 硬件资源

1. LED
 - LED0 - PF9
 - LED1 - PF10
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. USART1
 - USART1_TX - PA9
 - USART1_RX - PA10
4. TIM4

27.1.3 原理图

本章实验使用的 USART 组件为应用软件，因此没有对应的连接原理图。

27.2 程序设计

27.2.1 USART 组件

使用 USART 组件需要先进行相应的移植操作，USART 组件的移植也非常简单，只需要实现 `usmart_port.c` 文件中的 5 个函数即可完成移植。

第一个函数为 `usmart_get_input_string()`，该函数用于 USART 获取串口输入的数据流，该函数的实现，如下所示：

```
/**
 * @brief  获取输入数据流 (字符串)
 * @note   USART 通过解析该函数返回的字符串以获取函数名及参数等信息
 * @param  无
 * @retval
 *   @arg  0, 没有接收到数据
 *   @arg  其他, 数据流首地址 (不能是 0)
 */
char *usmart_get_input_string(void)
{
    uint8_t len;
    char *pbuf = 0;

    if (g_usart_rx_sta & 0x8000) /* 串口接收完成? */
```

```

    {
        len = g_usart_rx_sta & 0x3fff; /* 得到此次接收到的数据长度 */
        g_usart_rx_buf[len] = '\0'; /* 在末尾加入结束符. */
        pbuf = (char*)g_usart_rx_buf;
        g_usart_rx_sta = 0; /* 开启下一次接收 */
    }

    return pbuf;
}

```

从上面的代码中可以看出，该函数就是从 SYSTEM 文件夹 USART 驱动中获取 USART1 输入的数据。

剩余的四个函数，在宏 USMART_ENTIMX_SCAN 开启后才需要定义，该宏用于使能 runtime 统计等功能。

第二个函数为 usmart_timx_reset_time()，该函数用于复位 runtime，该函数的实现，如下所示：

```

/**
 * @brief 复位 runtime
 * @note 需要根据所移植到的 MCU 的定时器参数进行修改
 * @param 无
 * @retval 无
 */
void usmart_timx_reset_time(void)
{
    /* 清除中断标志位 */
    __HAL_TIM_CLEAR_FLAG(&g_timx_usmart_handle, TIM_FLAG_UPDATE);
    /* 将重载值设置到最大 */
    __HAL_TIM_SET_AUTORELOAD(&g_timx_usmart_handle, 0xFFFF);
    /* 清空定时器的 CNT */
    __HAL_TIM_SET_COUNTER(&g_timx_usmart_handle, 0);
    usmart_dev.runtime = 0;
}

```

该函数复位了 runtime 功能和用于 runtime 功能的相关 TIM。

第三个函数为 usmart_timx_get_time()，用于 runtime 功能获取时间，该函数的实现，如下所示：

```

/**
 * @brief 获得 runtime 时间
 * @note 需要根据所移植到的 MCU 的定时器参数进行修改
 * @param 无
 * @retval 执行时间,单位:0.1ms,最大延时时间为定时器 CNT 值的 2 倍*0.1ms
 */
uint32_t usmart_timx_get_time(void)
{
    /* 在运行期间,产生了定时器溢出 */
    if (__HAL_TIM_GET_FLAG(&g_timx_usmart_handle, TIM_FLAG_UPDATE) == SET)
    {
        usmart_dev.runtime += 0xFFFF;
    }
    usmart_dev.runtime += __HAL_TIM_GET_COUNTER(&g_timx_usmart_handle);
    return usmart_dev.runtime;
}

```

应为该函数能够处理一次定时器的溢出情况，因此能获取到的时间上限为定时器计数最大值的两倍。

第四个函数为 usmart_timx_init()，用于初始化用于 runtime 功能的定时器，该函数的实现，如下所示：

```

/**
 * @brief 定时器初始化函数
 * @param arr:自动重载值
 * @param psc:定时器分频系数
 * @retval 无
 */
void usmart_timx_init(uint16_t arr, uint16_t psc)

```

```
{
    USART_TIMX_CLK_ENABLE();

    g_timx_usmart_handle.Instance = USART_TIMX;
    g_timx_usmart_handle.Init.Prescaler = psc;
    g_timx_usmart_handle.Init.CounterMode = TIM_COUNTERMODE_UP;
    g_timx_usmart_handle.Init.Period = arr;
    g_timx_usmart_handle.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    HAL_TIM_Base_Init(&g_timx_usmart_handle);
    HAL_TIM_Base_Start_IT(&g_timx_usmart_handle);
    HAL_NVIC_SetPriority(USART_TIMX_IRQn, 3, 3);
    HAL_NVIC_EnableIRQ(USART_TIMX_IRQn);
}
```

可以看到，该函数初始化了一个用于 runtime 功能的定时器，同时使能了该定时器的更新中断。

第五个函数就是用于 runtime 功能的定时器的中断服务函数，该函数的实现，如下所示：

```
/**
 * @brief USART 定时器中断服务函数
 * @param 无
 * @retval 无
 */
void USART_TIMX_IRQHandler(void)
{
    /* 溢出中断 */
    if(__HAL_TIM_GET_IT_SOURCE(&g_timx_usmart_handle, TIM_IT_UPDATE) == SET)
    {
        usmart_dev.scan(); /* 执行 usmart 扫描 */
        __HAL_TIM_SET_COUNTER(&g_timx_usmart_handle, 0); /* 清空定时器的 CNT */
        __HAL_TIM_SET_AUTORELOAD(&g_timx_usmart_handle, 100); /* 恢复原来的设置 */
    }

    __HAL_TIM_CLEAR_IT(&g_timx_usmart_handle, TIM_IT_UPDATE); /* 清除中断标志位 */
}
```

以上就是移植 USART 组件时需要实现的五个函数，至此 USART 组件的移植也就基本完成了，接下来便可在 usart_config.c 文件中的 usmart_nametable 数组中添加需要调试的函数。

27.2.2 实验应用代码

本章实验的应用代码，如下所示：

```
/**
 * @brief LED 状态设置
 * @param 无
 * @retval 无
 */
void led_set(uint8_t sta)
{
    LED1(sta);
}

/**
 * @brief 测试函数参数调用
 * @param 无
 * @retval 无
 */
void test_fun(void (*ledset)(uint8_t), uint8_t sta)
{
    ledset(sta);
}

int main(void)
{
    HAL_Init(); /* 初始化 HAL 库 */
}
```

```
sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
delay_init(168); /* 初始化延时 */
usart_init(115200); /* 初始化串口 */
usmart_dev.init(84); /* 初始化 USMART */
led_init(); /* 初始化 LED */
lcd_init(); /* 初始化 LCD */

lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
lcd_show_string(30, 70, 200, 16, 16, "USMART TEST", RED);
lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

while (1)
{
    LED0_TOGGLE();
    delay_ms(500);
}
```

从上面的代码中可以看出，main()函数中初始化了 USMART 组件，并且另外定义了两个函数，分别为：函数 led_set()和函数 test_fun()，这两个函数都被添加到了 usmart_config.c 文件的 usmart_nametab 数组中，用于测试 USMART 组件，同时也添加了 LCD 操作和延时等函数，读者也可以添加自行编写的函数进行测试和调试。

27.3 下载验证

在完成编译和烧录操作后，便可通过串口调试助手“体验”USMART 组件，例如通过串口调试助手发送“led_set(0)\r\n”或“led_set(1)\r\n”，即可看到板载的 LED1 亮起或熄灭，也可使用同样的方式调用 LCD 的操作函数操作 LCD 进行显示。

第二十八章 RTC 实验

本章介绍 STM32F407 实时时钟（RTC）的使用，实时时钟能为系统提供一个准确的时间，即时系统复位或主电源断电，RTC 依然能够运行，因此 RTC 也经常用于各种低功耗场景。通过本章的学习，读者将学习到 RTC 的使用。

本章分为如下几个小节：

- 28.1 硬件设计
- 28.2 程序设计
- 28.3 下载验证

28.1 硬件设计

28.1.1 例程功能

1. 通过 LCD 实时显示 RTC 时间，并可通过 USMART 设置 RTC 时间和闹钟
2. RTC 闹钟串口提示
3. RTC 周期唤醒改变 LED1 状态，周期的 1 秒
4. LED0 闪烁，指示程序正在运行

28.1.2 硬件资源

1. LED
 - LED0 - PF9
 - LED1 - PF10
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. USART1
 - USART1_TX - PA9
 - USART1_RX - PA10
4. TIM4
5. RTC

28.1.3 原理图

本章实验使用的 RTC 为 STM32F407 的片上资源，因此没有相应的连接原理图。

28.2 程序设计

28.2.1 HAL 库的 PWR 驱动

本章实验要使用到 RTC，因此需要对 RTC 及其相关的寄存器进行配置，但是为了防止误操作，系统复位后备份区域（指 RTC、备份寄存器）是被禁止写访问的，备份区域的写访问是由 PWR 进行配置的，具体的配置步骤如下：

①：使能访问备份区域

在 HAL 库中对应的驱动函数如下：

①：使能访问备份区域

该函数用于使能访问备份区域，其函数原型如下所示：

```
void HAL_PWR_EnableBkUpAccess(void);
```

该函数的形参描述，如下表所示：

形参	描述
无	无

表 28.2.1.1 函数 HAL_PWR_EnableBkUpAccess() 形参描述

该函数的返回值描述，如下表所示：

返回值	描述
无	无

表 28.2.1.2 函数 HAL_PWR_EnableBkUpAccess()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 使能访问备份区域 */
    HAL_PWR_EnableBkUpAccess();
}
```

28.2.2 HAL 库的 RTC 驱动

本章实验使用了 RTC，RTC 最基本的操作就是设置和获取时间和日期，同时还需要读写 RTC 备份寄存器保存是否已经在初始化过程中设置过时间的标志，其具体的步骤如下：

- ①：初始化 RTC
- ②：读取 RTC 备份寄存器判断是否进行设置过时间
- ③：若未设置过时间，则设置 RTC 的时间
- ④：若未设置过时间，则设置 RTC 的日期
- ⑤：将设置过时间的标志写入 RTC 备份寄存器
- ⑥：读取 RTC 的时间
- ⑦：读取 RTC 的日期

在 HAL 库中对应的驱动函数如下：

①：初始化 RTC

该函数用于初始化 RTC，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_RTC_Init(RTC_HandleTypeDef *hrtc);
```

该函数的形参描述，如下表所示：

形参	描述
hrtc	指向 RTC 句柄的指针

表 28.2.2.1 函数 HAL_RTC_Init()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 28.2.2.2 函数 HAL_RTC_Init()返回值描述

该函数需要传入 RTC 的句柄指针，该句柄中就包含了 RTC 的初始化配置参数结构体，该结构体的定义如下所示：

```
typedef struct
{
    uint32_t HourFormat;          /* 时间格式 */
    uint32_t AsynchPrediv;        /* 异步预分频器系数 */
    uint32_t SynchPrediv;         /* 同步预分频器系数 */
    uint32_t OutPut;              /* 信号输出 */
    uint32_t OutPutPolarity;      /* 输出极性 */
    uint32_t OutPutType;          /* 输出引脚模式 */
} RTC_InitTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    RTC_HandleTypeDef rtc_handle = {0};

    /* 初始化 RTC */
}
```



```
rtc_handle.Instance = RTC;
rtc_handle.Init.HourFormat = RTC_HOURFORMAT_24;
rtc_handle.Init.AsynchPrediv = 0x7F;
rtc_handle.Init.SynchPrediv = 0xFF;
rtc_handle.Init.OutPut = RTC_OUTPUT_DISABLE;
rtc_handle.Init.OutPutPolarity = RTC_OUTPUT_POLARITY_HIGH;
rtc_handle.Init.OutPutType = RTC_OUTPUT_TYPE_OPENDRAIN;
HAL_RTC_Init(&rtc_handle);
}
```

②：读取 RTC 备份寄存器

该函数用于读取 RTC 备份寄存器，其函数原型如下所示：

```
uint32_t HAL_RTCEx_BKUPRead(RTC_HandleTypeDef *hrtc, uint32_t BackupRegister);
```

该函数的形参描述，如下表所示：

形参	描述
hrtc	指向 RTC 句柄的指针
BackupRegister	RTC 备份寄存器编号

表 28.2.2.3 函数 HAL_RTCEx_BKUPRead()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
uint32_t 类型数据	RTC 备份寄存器值

表 28.2.2.4 函数 HAL_RTCEx_BKUPRead()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    uint32_t rtc_backup_reg0;

    /* 读取 RTC 备份寄存器 0 */
    rtc_backup_reg0 = HAL_RTCEx_BKUPRead(&rtc_handle, RTC_BKP_DR0);

    /* Do something. */
}
```

③：配置 RTC 时间

该函数用于配置 RTC 的时间，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_RTC_SetTime(RTC_HandleTypeDef *hrtc,
                                   RTC_TimeTypeDef *sTime,
                                   uint32_t Format);
```

该函数的形参描述，如下表所示：

形参	描述
hrtc	指向 RTC 句柄的指针
sTime	指向 RTC 时间结构体的指针 需自行定义，并根据 RTC 的时间填充结构体中的成员变量
Format	时间格式 例如：RTC_FORMAT_BIN 和 RTC_FORMAT_BCD（在 stm32f4xx_hal_rtc.h 文件中有定义）

表 28.2.2.5 函数 HAL_RTC_SetTime()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 28.2.2.6 函数 HAL_RTC_SetTime()返回值描述

该函数使用 RTC_TimeTypeDef 类型的结构体变量传入 RTC 的时间参数，该结构体的定义如下所示：

```
typedef struct
{
    uint8_t Hours; /* 时 */
}
```

```
uint8_t Minutes;      /* 分 */
uint8_t Seconds;      /* 秒 */
uint8_t TimeFormat;    /* 上下午 */
uint32_t SubSeconds;   /* 亚秒 */
} RTC_TimeTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    RTC_TimeTypeDef time = {0};

    /* 配置 RTC 时间 */
    RTC_TimeTypeDef.Hours = 0;
    RTC_TimeTypeDef.Minutes = 0;
    RTC_TimeTypeDef.Seconds = 0;
    RTC_TimeTypeDef.TimeFormat = RTC_HOURFORMAT12_AM;
    RTC_TimeTypeDef.SubSeconds = 0;
    HAL_RTC_SetTime(&rtc_handle, &time, RTC_FORMAT_BIN);
}
```

④：配置 RTC 日期

该函数用于配置 RTC 的日期，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_RTC_SetDate( RTC_HandleTypeDef *hrtc,
                                    RTC_DateTypeDef *sDate,
                                    uint32_t Format);
```

该函数的形参描述，如下表所示：

形参	描述
hrtc	指向 RTC 句柄的指针
sDate	指向 RTC 日期结构体的指针 需自行定义，并根据 RTC 的日期填充结构体中的成员变量
Format	时间格式 例如：RTC_FORMAT_BIN 和 RTC_FORMAT_BCD（在 stm32f4xx_hal_rtc.h 文件中有定义）

表 28.2.2.7 函数 HAL_RTC_SetDate()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 28.2.2.8 函数 HAL_RTC_SetDate()返回值描述

该函数使用 RTC_DateTypeDef 类型的结构体变量传入 RTC 的日期参数，该结构体的定义如下所示：

```
typedef struct
{
    uint8_t WeekDay; /* 星期 */
    uint8_t Month;    /* 月 */
    uint8_t Date;     /* 日 */
    uint8_t Year;     /* 年 */
} RTC_DateTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    RTC_DateTypeDef date = {0};

    /* 配置 RTC 日期 */
    date.WeekDay = RTC_WEEKDAY_SATURDAY;
    date.Month = RTC_MONTH_JANUARY;
    date.Date = 1;
```

```
date.Year = 0;
HAL_RTC_SetDate(&rtc_handle, &date, RTC_FORMAT_BIN);
}
```

⑤：写入 RTC 备份寄存器

该函数用于写入 RTC 备份寄存器，其函数原型如下所示：

```
void HAL_RTCEx_BKUPWrite(  RTC_HandleTypeDef *hrtc,
                           uint32_t BackupRegister,
                           uint32_t Data);
```

该函数的形参描述，如下表所示：

形参	描述
hrtc	指向 RTC 句柄的指针
BackupRegister	RTC 备份寄存器编号
Data	写入 RTC 备份寄存器的值

表 28.2.2.9 函数 HAL_RTCEx_BKUPWrite()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
无	无

表 28.2.2.10 函数 HAL_RTCEx_BKUPWrite()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 往 RTC 备份寄存器 0 写入 0x50505050 */
    RTC_WriteBackup(&rtc_handle, RTC_BKP_DR0, 0x50505050);
}
```

⑥：读取 RTC 时间

该函数用于读取 RTC 的时间，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_RTC_GetTime( RTC_HandleTypeDef *hrtc,
                                    RTC_TimeTypeDef *sTime,
                                    uint32_t Format);
```

该函数的形参描述，如下表所示：

形参	描述
hrtc	指向 RTC 句柄的指针
sTime	指向 RTC 时间结构体的指针 需自行定义，并根据 RTC 的时间填充结构体中的成员变量
Format	时间格式 例如：RTC_FORMAT_BIN 和 RTC_FORMAT_BCD（在 stm32f4xx_hal_rtc.h 文件中有定义）

表 28.2.2.11 函数 HAL_RTC_GetTime()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 28.2.2.12 函数 HAL_RTC_GetTime()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    RTC_TimeTypeDef time;

    /* 读取 RTC 的时间 */
    HAL_RTC_GetTime(&rtc_handle, &time, RTC_FORMAT_BIN);

    /* Do something. */
}
```

⑦：读取 RTC 日期

该函数用于读取 RTC 的日期，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_RTC_GetDate( RTC_HandleTypeDef *hrtc,
                                   RTC_DateTypeDef *sDate,
                                   uint32_t Format);
```

该函数的形参描述，如下表所示：

形参	描述
hrtc	指向 RTC 句柄的指针
sDate	指向 RTC 日期结构体的指针 需自行定义，并根据 RTC 的日期填充结构体中的成员变量
Format	时间格式 例如：RTC_FORMAT_BIN 和 RTC_FORMAT_BCD（在 stm32f4xx_hal_rtc.h 文件中有定义）

表 28.2.2.13 函数 HAL_RTC_GetDate()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 28.2.2.14 函数 HAL_RTC_GetDate()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    RTC_DateTypeDef date;

    /* 读取 RTC 的日期 */
    HAL_RTC_GetDate(&hrtc_handle, &date, RTC_FORMAT_BIN);

    /* Do something. */
}
```

本实验还使能了 RTC 的周期性唤醒功能，该功能是低功耗唤醒的一个不错的方案，其具体的使用步骤如下：

①：设置 RTC 中断唤醒定时器

在 HAL 库中对应的驱动函数如下：

①：设置 RTC 中断唤醒定时器

该函数用于设置 RTC 中断唤醒定时器，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_RTCEX_SetWakeUpTimer_IT( RTC_HandleTypeDef *hrtc,
                                                uint32_t WakeUpCounter,
                                                uint32_t WakeUpClock);
```

该函数的形参描述，如下表所示：

形参	描述
hrtc	指向 RTC 句柄的指针
WakeUpCounter	唤醒计数器
WakeUpClock	唤醒时钟

表 28.2.2.15 函数 HAL_RTCEX_SetWakeUpTimer_IT()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 28.2.2.16 函数 HAL_RTCEX_SetWkaeUpTimer_IT()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
```

```
/* 设置 RTC 中断唤醒计数器 */
HAL_RTCEx_SetWakeUpTimer_IT(    &rtc_handle,
                                0,
                                RTC_WAKEUPCLOCK_CK_SPRE_16BITS);
}
```

本实验同时也使能了 RTC 的闹钟功能，闹钟功能可以在 RTC 时间到达设定值时触发中断，其具体的使用步骤如下：

①：设置 RTC 中断闹钟

在 HAL 库中对应的驱动函数如下：

①：设置 RTC 中断闹钟

该函数用于设置 RTC 的中断闹钟，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_RTC_SetAlarm_IT( RTC_HandleTypeDef *hrtc,
                                       RTC_AlarmTypeDef *sAlarm,
                                       uint32_t Format);
```

该函数的形参描述，如下表所示：

形参	描述
hrtc	指向 RTC 句柄的指针
sAlarm	指向 RTC 闹钟结构体的指针 需自行定义，并根据 RTC 闹钟的配置参数充结构体中的成员变量
Format	时间格式 例如：RTC_FORMAT_BIN 和 RTC_FORMAT_BCD（在 stm32f4xx_hal_rtc.h 文件中有定义）

表 28.2.2.23 函数 HAL_RTC_SetAlarm_IT()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 28.2.2.24 函数 HAL_RTC_SetAlarm_IT()返回值描述

该函数使用 RTC_AlarmTypeDef 类型的结构体变量传入 RTC 闹钟的配置参数，该结构体的定义如下所示：

```
typedef struct
{
    RTC_TimeTypeDef AlarmTime;    /* 闹钟时间 */
    uint32_t AlarmMask;          /* 闹钟掩码 */
    uint32_t AlarmSubSecondMask; /* 闹钟亚秒掩码 */
    uint32_t AlarmDateWeekDaySel; /* 日闹钟或星期闹钟 */
    uint8_t AlarmDateWeekDay;    /* 闹钟日期 */
    uint32_t Alarm;              /* 闹钟对象 */
} RTC_AlarmTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    RTC_AlarmTypeDef alarm = {0};

    /* 设置 RTC 闹钟中断 */
    alarm.AlarmTime.Hours = 0;
    alarm.AlarmTime.Minutes = 0;
    alarm.AlarmTime.Seconds = 0;
    alarm.AlarmTime.TimeFormat = RTC_HOURFORMAT12_AM;
    alarm.AlarmTime.SubSeconds = 0;
    alarm.AlarmMask = RTC_ALARMMASK_NONE;
    alarm.AlarmSubSecondMask = RTC_ALARMSUBSECONDMASK_NONE;
    alarm.AlarmDateWeekDaySel = RTC_ALARMDATEWEEKDAYSEL_WEEKDAY;
    alarm.AlarmDateWeekDay = RTC_WEEKDAY_MONDAY;
    alarm.Alarm = RTC_ALARM_A;
```

```
HAL_RTC_SetAlarm_IT(&rtc_handle, &alarm, RTC_FORMAT_BIN);
}
```

28.2.4 RTC 驱动

本章实验的 RTC 驱动主要负责向应用层提供 RTC 的初始化和配置自动唤醒及闹钟的函数。本章实验中，RTC 的驱动代码包括 `rtc.c` 和 `rtc.h` 两个文件。

RTC 驱动中，RTC 的初始化函数，如下所示：

```
/**
 * @brief    初始化 RTC
 * @param    无
 * @retval    初始化结果
 * @arg      0: 初始化成功
 * @arg      1: 初始化失败
 */
uint8_t rtc_init(void)
{
    uint16_t flag;

    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_RCC_RTC_ENABLE();
    HAL_PWR_EnableBkUpAccess();

    /* 初始化 RTC */
    g_rtc_handle.Instance = RTC;
    g_rtc_handle.Init.HourFormat = RTC_HOURFORMAT_24;
    g_rtc_handle.Init.AsynchPrediv = 0x7F;
    g_rtc_handle.Init.SynchPrediv = 0xFF;
    g_rtc_handle.Init.OutPut = RTC_OUTPUT_DISABLE;
    g_rtc_handle.Init.OutPutPolarity = RTC_OUTPUT_POLARITY_HIGH;
    g_rtc_handle.Init.OutPutType = RTC_OUTPUT_TYPE_OPENDRAIN;

    /* 从后备寄存器读取 RTC 初始化标志 */
    flag = rtc_read_bkr(0);

    if (HAL_RTC_Init(&g_rtc_handle) != HAL_OK)
    {
        return 1;
    }

    /* RTC 第一次初始化 */
    if ((flag != 0x5051) && (flag != 0x5050))
    {
        /* 设置 RTC 时间和日期信息 */
        rtc_set_time(8, 0, 0, 0);
        rtc_set_date(23, 4, 23, 7);
    }

    return 0;
}
```

从上面的代码中可以看出，RTC 的初始化函数中，会先读取 RTC 备份寄存器来判断是否初始化过 RTC 然后再初始化 RTC，RTC 初始化完成后，若是第一次初始化 RTC，则设置 RTC 的时间，否则不设置 RTC 的时间。

RTC 驱动中设置、获取 RTC 时间、日期的四个函数，如下所示：

```
/**
 * @brief    设置 RTC 时间信息
 * @param    hour: 时
 * @param    minute: 分
 * @param    second: 秒
 * @param    ampm: 上下午
 * @arg      0: 上午
 * @arg      1: 下午
```

```

* @retval 设置结果
* @arg 0: 设置成功
* @arg 1: 设置失败
*/
uint8_t rtc_set_time(uint8_t hour, uint8_t minute, uint8_t second, uint8_t ampm)
{
    RTC_TimeTypeDef rtc_time_struct = {0};

    rtc_time_struct.Hours = hour;
    rtc_time_struct.Minutes = minute;
    rtc_time_struct.Seconds = second;
    rtc_time_struct.TimeFormat = ampm;
    rtc_time_struct.DayLightSaving = RTC_DAYLIGHTSAVING_NONE;
    rtc_time_struct.StoreOperation = RTC_STOREOPERATION_RESET;
    if (HAL_RTC_SetTime(&g_rtc_handle, &rtc_time_struct, RTC_FORMAT_BIN) !=
        HAL_OK)
    {
        return 1;
    }

    return 0;
}

/**
* @brief 设置 RTC 日期信息
* @param year: 年
* @param month: 月
* @param date: 日
* @param week: 星期
* @retval 设置结果
* @arg 0: 设置成功
* @arg 1: 设置失败
*/
uint8_t rtc_set_date(uint8_t year, uint8_t month, uint8_t date, uint8_t week)
{
    RTC_DateTypeDef rtc_date_struct = {0};

    rtc_date_struct.WeekDay = week;
    rtc_date_struct.Month = month;
    rtc_date_struct.Date = date;
    rtc_date_struct.Year = year;
    if (HAL_RTC_SetDate(&g_rtc_handle, &rtc_date_struct, RTC_FORMAT_BIN) !=
        HAL_OK)
    {
        return 1;
    }

    return 0;
}

/**
* @brief 获取 RTC 时间信息
* @param hour: 时
* @param minute: 分
* @param second: 秒
* @param ampm: 上下午
* @arg 0: 上午
* @arg 1: 下午
* @retval 无
*/
void rtc_get_time( uint8_t *hour,
                  uint8_t *minute,
                  uint8_t *second,
                  uint8_t *ampm)

```



```
{
    RTC_TimeTypeDef rtc_time_struct = {0};

    HAL_RTC_GetTime(&g_rtc_handle, &rtc_time_struct, RTC_FORMAT_BIN);

    *hour = rtc_time_struct.Hours;
    *minute = rtc_time_struct.Minutes;
    *second = rtc_time_struct.Seconds;
    *ampm = rtc_time_struct.TimeFormat;
}

/**
 * @brief 获取 RTC 日期信息
 * @param year: 年
 * @param month: 月
 * @param date: 日
 * @param week: 星期
 * @retval 无
 */
void rtc_get_date(uint8_t *year, uint8_t *month, uint8_t *date, uint8_t *week)
{
    RTC_DateTypeDef rtc_date_struct = {0};

    HAL_RTC_GetDate(&g_rtc_handle, &rtc_date_struct, RTC_FORMAT_BIN);

    *year = rtc_date_struct.Year;
    *month = rtc_date_struct.Month;
    *date = rtc_date_struct.Date;
    *week = rtc_date_struct.WeekDay;
}
```

以上四个获取、设置 RTC 时间、日期的函数，均是对 HAL 库中 RTC 驱动的简单封装。

RTC 驱动中，配置 RTC 唤醒中断及其对应的唤醒定时器事件回调函数，如下所示：

```
/**
 * @brief 设置 RTC 周期性唤醒中断
 * @param clock: 唤醒时钟
 * @param count: 唤醒计数器
 * @retval 无
 */
void rtc_set_wakeup(uint8_t clock, uint8_t count)
{
    HAL_NVIC_SetPriority(RTC_WKUP_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(RTC_WKUP_IRQn);

    HAL_RTCEx_SetWakeUpTimer_IT(&g_rtc_handle, count, clock);
}

/**
 * @brief HAL 库 RTC 唤醒中断回调函数
 * @param hrtc: RTC 句柄
 * @retval 无
 */
void HAL_RTCEx_WakeUpTimerEventCallback(RTC_HandleTypeDef *hrtc)
{
    LED1_TOGGLE();
}
```

从上面的代码中可以看出，在 RTC 的唤醒定时器事件回调函数中翻转了 LED1 的亮灭状态，因此 RTC 唤醒时间将会周期性地发生，也就能看到 LED1 周期性地改变状态。

RTC 驱动中，配置 RTC 闹钟中断及其对应的闹钟事件回调函数，如下所示：

```
/**
 * @brief 设置 RTC 闹钟时间信息
 * @param week: 星期
 * @param hour: 时
```

```

* @param  minute: 分
* @param  second: 秒
* @retval 无
*/
void rtc_set_alarm(uint8_t week, uint8_t hour, uint8_t minute, uint8_t second)
{
    RTC_AlarmTypeDef rtc_alarm_struct = {0};

    HAL_NVIC_SetPriority(RTC_Alarm_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(RTC_Alarm_IRQn);

    /* 设置闹钟中断 */
    rtc_alarm_struct.AlarmTime.Hours = hour;
    rtc_alarm_struct.AlarmTime.Minutes = minute;
    rtc_alarm_struct.AlarmTime.Seconds = second;
    rtc_alarm_struct.AlarmTime.TimeFormat = RTC_HOURFORMAT12_AM;
    rtc_alarm_struct.AlarmTime.SubSeconds = 0;
    rtc_alarm_struct.AlarmMask = RTC_ALARMMASK_NONE;
    rtc_alarm_struct.AlarmSubSecondMask = RTC_ALARMSUBSECONDMASK_NONE;
    rtc_alarm_struct.AlarmDateWeekDaySel = RTC_ALARMDATEWEEKDAYSEL_WEEKDAY;
    rtc_alarm_struct.AlarmDateWeekDay = week;
    rtc_alarm_struct.Alarm = RTC_ALARM_A;
    HAL_RTC_SetAlarm_IT(&g_rtc_handle, &rtc_alarm_struct, RTC_FORMAT_BIN);
}

/**
* @brief  HAL 库 RTC 闹钟 A 中断回调函数
* @param  hrtc: RTC 句柄
* @retval 无
*/
void HAL_RTC_AlarmAEventCallback(RTC_HandleTypeDef *hrtc)
{
    printf("Alarm A!\r\n");
}

```

从上面的代码中可以看到，在 RTC 的闹钟中断中通过串口打印了闹钟的提示。

28.2.5 实验应用代码

本章实验的应用代码，如下所示：

```

int main(void)
{
    uint8_t t = 0;
    char tbuf[40];
    uint8_t hour, minute, second, ampm;
    uint8_t year, month, date, week;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟，168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    usmart_dev_init(84); /* 初始化 USMART */
    led_init(); /* 初始化 LED */
    lcd_init(); /* 初始化 LCD */
    rtc_init(); /* 初始化 RTC */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "RTC TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    /* 设置 RTC 周期性唤醒中断 */
    rtc_set_wakeup(RTC_WAKEUPCLOCK_CK_SPRE_16BITS, 0);

    while (1)

```

```

{
    if ((t % 10) == 0)
    {
        /* 获取 RTC 时间信息 */
        rtc_get_time(&hour, &minute, &second, &ampm);
        sprintf(tbuf, "Time:%02d:%02d:%02d", hour, minute, second);
        lcd_show_string(30, 120, 210, 16, 16, tbuf, RED);
        /* 获取 RTC 日期信息 */
        rtc_get_date(&year, &month, &date, &week);
        sprintf(tbuf, "Date:20%02d-%02d-%02d", year, month, date);
        lcd_show_string(30, 140, 210, 16, 16, tbuf, RED);
        sprintf(tbuf, "Week:%d", week);
        lcd_show_string(30, 160, 210, 16, 16, tbuf, RED);
    }

    if (++t == 20)
    {
        t = 0;
        LED0_TOGGLE();
    }

    delay_ms(10);
}

```

从上面的代码中可以看到，在初始化完 RTC 后，配置了 RTC 周期性地唤醒，且唤醒周期为 1Hz，因此应该能看到 LED1 以 0.5Hz 的频率闪烁，随后便每间隔 100 毫秒获取一次 RTC 的时间和日期，并在 LCD 上进行显示。

本实验同时也使用的 USMART 调试组件，并在 usart_config.c 文件中添加了 RTC 驱动中相关的函数，以便调试。

28.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上实时地显示着 RTC 的时间，并且可以看到 LED1 在 RTC 周期性唤醒的驱动下以 0.5Hz 的频率闪烁着，此时可以通过串口调试助手调用 USMART 调试组件的 rtc_set_alarma()函数来设置 RTC 的闹钟，当通过 LCD 观察到 RTC 的时间达到设置的闹钟时间后，可以看到串口调试助手上打印了“ALARMA!\r\n”的字符串提示。

第二十九章 硬件随机数实验

本章介绍 STM32F407 随机数（RNG）的使用，RNG 能提供 32 位的随机数。通过本章的学习，读者将学习到 RNG 的使用。

本章分为如下几个小节：

- 29.1 硬件设计
- 29.2 程序设计
- 29.3 下载验证

29.1 硬件设计

29.1.1 例程功能

1. TFTLCD 上不断显示 0-9 之间的随机数
2. 按下 KEY0 按键后，TFTLCD 上显示一次 32 位宽度的随机数
3. LED0 闪烁，提示程序正在运行

29.1.2 硬件资源

1. LED
LED0 - PF9
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
KEY0 - PE4
4. RNG

29.1.3 原理图

本章实验使用的 RNG 为 STM32F407 的片上资源，因此没有对应的连接原理图。

29.2 程序设计

29.2.1 HAL 库的 RNG 驱动

本章实验要使用 RNG 生成随机数，其使用方式时非常简单的，具体的使用步骤如下：

- ①：初始化 RNG
 - ②：读取 RNG 生成的 32 位随机数
- 在 HAL 库中对应的驱动函数如下：

①：初始化 RNG

该函数用于初始化 RNG，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_RNG_Init(RNG_HandleTypeDef *hrng);
```

该函数的形参描述，如下表所示：

形参	描述
hrng	指向 RNG 句柄的指针

表 29.2.1.1 函数 HAL_RNG_Init()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 29.2.1.2 函数 HAL_RNG_Init()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"
```

```
void example_fun(void)
{
    RNG_HandleTypeDef rng_handle = {0};

    /* 初始化 RNG */
    rng_handle.Instance = RNG;
    HAL_RNG_Init(&rng_handle);
}
```

②：读取 32 位随机数

该函数用于读取 RNG 生成的 32 位随机数，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_RNG_GenerateRandomNumber(    RNG_HandleTypeDef *hrng,
                                                    uint32_t *random32bit);
```

该函数的形参描述，如下表所示：

形参	描述
hrng	指向 RNG 句柄的指针
random32bit	32 位随机数

表 29.2.1.3 函数 HAL_RNG_GenerateRandomNumber()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 29.2.1.4 函数 HAL_RNG_GenerateRandomNumber()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    uint32_t random;

    /* 获取 RNG 产生的 32 位随机数 */
    HAL_RNG_GenerateRandomNumber(&rng_handle, &random);

    /* Do something. */
}
```

29.2.2 RNG 驱动

本章实验的 RNG 驱动主要负责向应用层提供 RNG 的初始化及获取随机数的函数。本章实验中，RNG 的驱动代码包括 rng.c 和 rng.h 两个文件。

RNG 驱动中，RNG 的初始化函数，如下所示：

```
/**
 * @brief  初始化随机数发生器
 * @param  无
 * @retval 初始化结果
 * @arg    0：初始化成功
 * @arg    1：初始化失败
 */
uint8_t rng_init(void)
{
    g_rng_handle.Instance = RNG;

    if (HAL_RNG_DeInit(&g_rng_handle) != HAL_OK)
    {
        return 1;
    }

    if (HAL_RNG_Init(&g_rng_handle) != HAL_OK)
    {
        return 1;
    }
}
```

```
    return 0;
}
```

RNG 驱动中提供了两个获取随机数的函数，一个是直接获取 RNG 生成的 32 位随机数，另一个是将 RNG 生成的 32 位随机数进行处理得到一个在指定范围内的随机数，这两个函数如下所示：

```
/**
 * @brief  获取随机数
 * @param  无
 * @retval 随机数
 */
uint32_t rng_get_random(void)
{
    uint32_t random;

    if (HAL_RNG_GenerateRandomNumber(&g_rng_handle, &random) != HAL_OK)
    {
        return 0;
    }

    return random;
}

/**
 * @brief  获取指定范围的随机数
 * @param  min: 指定范围的最小值
 * @param  max: 指定范围的最大值
 * @retval 随机数
 */
int32_t rng_get_random_range(int32_t min, int32_t max)
{
    uint32_t random;

    if (HAL_RNG_GenerateRandomNumber(&g_rng_handle, &random) != HAL_OK)
    {
        return 0;
    }

    return ((int32_t)random % (max - min + 1) + min);
}
```

29.2.3 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    uint8_t t = 0;
    uint8_t key;
    uint32_t random;
    int32_t random_range;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    lcd_init(); /* 初始化 LCD */
    rng_init(); /* 初始化随机数发生器 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "RNG TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
}
```

```

lcd_show_string(30, 130, 200, 16, 16, "KEY0:Get Random Num", RED);
lcd_show_string(30, 150, 200, 16, 16, "Random Num:", RED);
lcd_show_string(30, 170, 200, 16, 16, "Random Num[0-9]:", RED);

while (1)
{
    key = key_scan(0);
    if (key == KEY0_PRES)
    {
        /* 获取随机数 */
        random = rng_get_random();
        lcd_show_num(30 + 8 * 11, 150, random, 10, 16, BLUE);
    }

    if (++t == 20)
    {
        t = 0;
        LED0_TOGGLE();

        /* 获取指定范围的随机数 */
        random_range = rng_get_random_range(0, 9);
        lcd_show_num(30 + 8 * 16, 170, random_range, 1, 16, BLUE);
    }

    delay_ms(10);
}
}

```

从上面的代码中可以看出，在进行完相关的初始化后，便会在 LCD 屏幕上不断显示一个 0-9 的随机数，若按下 KEY0 按键，则会在 LCD 屏幕上显示 RNG 生成的 32 位随机数。

29.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上不断地刷新显示一个 0-9 的随机数，此时若按下 KEY0 按键，则可以看到 LCD 屏幕上刷新显示了一个 32 位的随机数。

第三十章 PVD 电压监控实验

本章介绍 STM32F407 电源电压监测器（PVD）的使用，PVD 可以设置一个电压阈值，当监测到电源电压低于该阈值后，可以触发中断，以完成一些紧急处理。通过本章的学习，读者将学习到 PVD 的使用。

本章分为如下几个小节：

- 30.1 硬件设计
- 30.2 程序设计
- 30.3 下载验证

30.1 硬件设计

30.1.1 例程功能

1. TFTLCD 上显示实验信息
2. 供电不足时，LED1 亮起，LCD 上提示 “PVD Low Voltage!”
3. 供电正常时，LED1 熄灭，LCD 上提示 “PVD Voltage OK! ”
4. LED0 闪烁，提示程序正在运行

30.1.2 硬件资源

1. LED
 - LED0 - PF9
 - LED1 - PF10
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. PVD

30.1.3 原理图

本章实验使用的 PVD 为 STM32F407 的片上资源，因此没有对应的连接原理图。

30.2 程序设计

30.2.1 HAL 库的 PWR 驱动

本章实验使用的 PVD 为 PWR 的子模块，因此对 PVD 的操作函数都由 HAL 库中的 PWR 驱动提供，使用 PVD 的具体步骤如下：

- ①：配置 PVD
- ②：使能 PVD

在 HAL 库中对应的驱动函数如下：

①：配置 PVD

该函数用于配置 PVD，其函数原型如下所示：

```
void HAL_PWR_ConfigPVD(PWR_PVDTypeDef *sConfigPVD);
```

该函数的形参描述，如下表所示：

形参	描述
sConfigPVD	指向 PVD 配置结构体的指针 需自行定义，并根据 PVD 的配置参数充结构体中的成员变量

表 30.2.1.1 函数 HAL_PWR_ConfigPVD()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
-----	----

无	无
---	---

表 30.2.1.2 函数 HAL_PWR_ConfigPVD()返回值描述

该函数使用 PWR_PVDTypeDef 类型的结构体变量传入 PVD 的配置参数，该结构体的定义如下所示：

```
typedef struct
{
    uint32_t PVDLevel; /* 等级 */
    uint32_t Mode;      /* 模式 */
}PWR_PVDTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    PWR_PVDTypeDef config = {0};

    /* 配置 PVD */
    config.PVDLevel = PWR_PVDLEVEL_7;
    config.Mode = PWR_PVD_MODE_IT_RISING_FALLING;
    HAL_PWR_ConfigPVD(&config);
}
```

②：使能 PVD

该函数用于使能 PVD，其函数原型如下所示：

```
void HAL_PWR_EnablePVD(void);
```

该函数的形参描述，如下表所示：

形参	描述
无	无

表 30.2.1.3 函数 HAL_PWR_EnablePVD()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
无	无

表 30.2.1.4 函数 HAL_PWR_EnablePVD()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 使能 PVD */
    HAL_PWR_EnablePVD();
}
```

30.2.2 PWR 驱动

本章实验的 PWR 驱动主要负责向应用层提供 PVD 的初始化函数，并实现 PVD 的中断回调函数。本章实验中，PWR 的驱动代码包括 pwr.c 和 pwr.h 两个文件。

PWR 驱动中，初始化 PVD 的函数，如下所示：

```
/**
 * @brief 初始化 PVD
 * @param pl: 电压等级
 * @retval 无
 */
void pwr_pvd_init(uint32_t pl)
{
    PWR_PVDTypeDef pwr_pvd_struct = {0};

    __HAL_RCC_PWR_CLK_ENABLE();

    HAL_NVIC_SetPriority(PVD_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(PVD_IRQn);
}
```

```

/* 配置并使能 PVD */
pwr_pvd_struct.PVDLevel = pl;
pwr_pvd_struct.Mode = PWR_PVD_MODE_IT_RISING_FALLING;
HAL_PWR_ConfigPVD(&pwr_pvd_struct);
HAL_PWR_EnablePVD();
}

```

从上面的代码中可以看到，PVD 的初始化函数中，根据函数传入的参数配置了 PVD 的电压阈值，并开启了 PVD 的相关中断和使能 PVD。

PWR 驱动中，PVD 的中断回调函数，如下所示：

```

/**
 * @brief   HAL 库 PVD 中断回调函数
 * @param   无
 * @retval   无
 */
void HAL_PWR_PVDCallback(void)
{
    if (__HAL_PWR_GET_FLAG(PWR_FLAG_PVDO) != RESET)
    {
        /* 电压低于设置的电压等级，进行相应提示 */
        LED1(0);
        lcd_show_string(30, 130, 200, 16, 16, "PVD Low Voltage!", RED);
    }
    else
    {
        /* 电压不低于设置的电压等级，进行相应提示 */
        LED1(1);
        lcd_show_string(30, 130, 200, 16, 16, "PVD Voltage OK! ", BLUE);
    }
}

```

从 PVD 的中断回调函数中可以看到，当 PVD 监测到电源电压小于设定的电压阈值时，会在 LCD 上显示电压低的提示并点亮 LED1，在 PVD 监测到电源电压恢复至设定的电压阈值时，会在 LCD 上显示电压正常的提示并熄灭 LED1。

虽然 PVD 在监测到电源电压低于设定的电压阈值时，会有相应的操作，但是由于电压过低可能导致无法观察到部分操作的现象。

30.2.3 实验应用代码

本章实验的应用代码，如下所示：

```

int main(void)
{
    uint8_t t = 0;

    HAL_Init();                /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟，168MHz */
    delay_init(168);           /* 初始化延时 */
    usart_init(115200);        /* 初始化串口 */
    led_init();                /* 初始化 LED */
    lcd_init();                /* 初始化 LCD */
    pwr_pvd_init(PWR_PVDLEVEL_7); /* 初始化 PVD */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "PVD TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    lcd_show_string(30, 130, 200, 16, 16, "PVD Voltage OK! ", BLUE);

    while (1)
    {
        if (++t == 20)
        {

```

```
t = 0;
LED0_TOGGLE();

}

delay_ms(10);

}
```

本实验的应用代码很简单，主要就是配置了 PVD 的阈值电压为 2.7V。

30.3 下载验证

在完成编译和烧录操作后，若系统的供电正常，便可以在 LCD 上看到“PVD Voltage OK!”的提示，并且 LED1 也处于熄灭状态；若系统的供电低于设置的 PVD 电压阈值 2.7V 时，便可以在 LCD 上看到“PVD Low Voltage!”的提示，并且 LED1 也会亮起（系统供电过低时，可能出现跟多意想不到的结果，因此可能会看不到部分现象）。

正常情况下开发板的供电都是正常的，若读者强制影响开发板的供电，可能导致不可逆的硬件损坏。

第三十一章 睡眠模式实验

本章介绍 STM32F407 低功耗模式中的睡眠模式，通过调用 WFI 命令进入睡眠模式后，ARM Corten-M4 内核将停止以降低功耗，在该睡眠模式下可以被任意中断唤醒。通过本章的学习，读者将学习到低功耗模式中睡眠模式的使用。

本章分为如下几个小节：

- 31.1 硬件设计
- 31.2 程序设计
- 31.3 下载验证

31.1 硬件设计

31.1.1 例程功能

1. TFTLCD 上显示实验信息
2. LED0 闪烁时，按下 KEY0 按键后，LED1 亮起，LED0 停止闪烁
3. LED0 停止闪烁时，按下 WKUP 按键后，LED1 熄灭，LED0 闪烁

31.1.2 硬件资源

1. LED
 - LED0 - PF9
 - LED1 - PF10
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
 - WKUP - PA0
 - KEY0 - PE4

31.1.3 原理图

本章实验介绍 STM32F407 低功耗模式中的睡眠模式，不涉及连接原理图。

31.2 程序设计

31.2.1 HAL 库的 PWR 驱动

HAL 库中提供了配置进入睡眠模式的驱动函数，其配置步骤如下：

①：进入睡眠模式

在 HAL 库中对应的驱动函数如下：

①：进入睡眠模式

该函数用于进入睡眠模式，其函数原型如下所示：

```
void HAL_PWR_EnterSLEEPMode(uint32_t Regulator, uint8_t SLEEPEntry);
```

该函数的形参描述，如下表所示：

形参	描述
Regulator	睡眠模式下调压器的工作状态 例如：PWR_MAINREGULATOR_ON 和 PWR_LOWPOWER REGULATOR_ON（在 stm32f4xx_hal_pwr.h 文件中有定义）
SLEEPEntry	进入睡眠模式的方式 例如：PWR_SLEEPENTRY_WFI、PWR_SLEEPENTRY_WFE （在 stm32f4xx_hal_pwr.h 文件中有定义）

表 31.2.1.1 函数 HAL_PWR_EnterSLEEPMode()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
无	无

表 31.2.1.2 函数 HAL_PWR_EnterSLEEPMode()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 进入睡眠模式 */
    HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON, PWR_SLEEPENTRY_WFI);
}
```

31.2.2 PWR 驱动

本章实验的 PWR 驱动主要负责向应用层提供睡眠模式唤醒按键的初始化和进入睡眠模式的函数，同时实现唤醒按键的中断回调函数。本章实验中，PWR 的驱动代码包括 pwr.c 和 pwr.h 两个文件。

PWR 驱动中，睡眠模式唤醒按键的相关宏定义，如下所示：

```
#define PWR_WKUP_INT_GPIO_PORT GPIOA
#define PWR_WKUP_INT_GPIO_PIN GPIO_PIN_0
#define PWR_WKUP_INT_GPIO_CLK_ENABLE() \
do { \
    __HAL_RCC_GPIOA_CLK_ENABLE(); \
} while (0)
#define PWR_WKUP_INT_IRQn EXTI0_IRQn
#define PWR_WKUP_INT_IRQHandler EXTI0_IRQHandler
```

PWR 驱动中，睡眠模式唤醒按键的初始化函数，如下所示：

```
/**
 * @brief 进入睡眠模式
 * @param 无
 * @retval 无
 */
void pwr_enter_sleep(void)
{
    GPIO_InitTypeDef gpio_init_struct = {0};

    __HAL_RCC_PWR_CLK_ENABLE();
    PWR_WKUP_INT_GPIO_CLK_ENABLE();

    HAL_NVIC_SetPriority(PWR_WKUP_INT_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(PWR_WKUP_INT_IRQn);

    /* 配置唤醒引脚 */
    gpio_init_struct.Pin = PWR_WKUP_INT_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_IT_RISING;
    gpio_init_struct.Pull = GPIO_PULLDOWN;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(PWR_WKUP_INT_GPIO_PORT, &gpio_init_struct);

    /* 关闭 SysTick 中断 */
    SysTick->CTRL &= ~SysTick_CTRL_TICKINT_Msk;

    /* 进入睡眠模式 */
    HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON, PWR_SLEEPENTRY_WFI);
}
```

因为进入睡眠模式后，能够被任意的中断唤醒，因此睡眠模式唤醒按键的初始化只需要配置好按键的外部中断即可。

PWR 驱动中，睡眠模式唤醒按键对应的中断回调函数，如下所示：

```
/**
 * @brief HAL 库外部中断中断回调函数
```

```

* @param  无
* @retval 无
*/
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == PWR_WKUP_INT_GPIO_PIN)
    {
        /* 重新开启 SysTick 中断 */
        SysTick->CTRL |= SysTick_CTRL_TICKINT_Msk;
    }
}

```

因为睡眠唤醒按键的中断只是为了唤醒睡眠模式，因此并不需要作太多事务，仅需将 SysTick 的中断重新打开，保证 HAL 库能正常运行即可。

31.2.3 实验应用代码

本章实验的应用代码，如下所示：

```

int main(void)
{
    uint8_t t = 0;
    uint8_t key;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟，168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    lcd_init(); /* 初始化 LCD */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "SLEEP TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    lcd_show_string(30, 110, 200, 16, 16, "KEY0:Enter SLEEP MODE", RED);
    lcd_show_string(30, 130, 200, 16, 16, "KEY_UP:Exit SLEEP MODE", RED);

    while (1)
    {
        key = key_scan(0);
        if (key == KEY0_PRES)
        {
            /* 按下 KEY0 按键，开启 LED1 后进入睡眠模式 */
            LED1(0);
            pwr_enter_sleep();
            LED1(1);
        }

        if (++t == 20)
        {
            t = 0;
            LED0_TOGGLE();
        }

        delay_ms(10);
    }
}

```

从上面的代码中可以看出，在完成相关的初始化操作后，并不断地扫描按键，若扫描到 KEY0 按键被按下，则会点亮 LED1 后进入睡眠模式，此时 ARM Cortex-M4 内核便停止了，从该睡眠模式唤醒需要有任意的中断产生，因此可以由 WKUP 按键产生外部中断来唤醒睡眠。退出睡眠模式后，LED1 将被熄灭。

31.3 下载验证

在完成编译和烧录操作后,可以看到 LED0 闪烁提示系统程序正在运行,此时可以按下 KEY0 按键,可以看到 LED1 亮起,但 LED0 不再闪烁,这是因为系统已经进入睡眠模式了,此时再按下 WKUP 按键,即可从睡眠模式下唤醒,可以看到 LED1 熄灭,LED0 继续闪烁。

第三十二章 停止模式实验

本章介绍 STM32F407 低功耗模式中的停止模式，进入停止模式后，所有的时钟都将被停止以降低功耗，在停止模式下可以被任意中断唤醒。通过本章的学习，读者将学习到低功耗模式中停止模式的使用。

本章分为如下几个小节：

- 32.1 硬件设计
- 32.2 程序设计
- 32.3 下载验证

32.1 硬件设计

32.1.1 例程功能

1. TFTLCD 上显示实验信息
2. LED0 闪烁时，按下 KEY0 按键后，LED1 亮起，LED0 停止闪烁
3. LED0 停止闪烁时，按下 WKUP 按键后，LED1 熄灭，LED0 闪烁

32.1.2 硬件资源

1. LED
 - LED0 - PF9
 - LED1 - PF10
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
 - WKUP - PA0
 - KEY0 - PE4

32.1.3 原理图

本章实验介绍 STM32F407 低功耗模式中的停止模式，不涉及连接原理图。

32.2 程序设计

32.2.1 HAL 库的 PWR 驱动

HAL 库中提供了配置进入停止模式的驱动函数，其配置步骤如下：

①：进入停止模式

在 HAL 库中对应的驱动函数如下：

①：进入停止模式

该函数用于进入停止模式，其函数原型如下所示：

```
void HAL_PWR_EnterSTOPMode(uint32_t Regulator, uint8_t STOPEntry);
```

该函数的形参描述，如下表所示：

形参	描述
Regulator	停止模式下调压器的工作状态 例如：PWR_MAINREGULATOR_ON 和 PWR_LOWPOWER REGULATOR_ON（在 stm32f4xx_hal_pwr.h 文件中有定义）
STOPEntry	进入停止模式的方式 例如：PWR_STOPENTRY_WFI、PWR_STOPENTRY_WFE （在 stm32f4xx_hal_pwr.h 文件中有定义）

表 32.2.1.1 函数 HAL_PWR_EnterSTOPMode()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
无	无

表 32.2.1.2 函数 HAL_PWR_EnterSTOPMode()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 进入停止模式 */
    HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);
}
```

32.2.2 PWR 驱动

本章实验的 PWR 驱动主要负责向应用层提供停止模式唤醒按键的初始化和进入停止模式的函数，同时实现唤醒按键的中断回调函数。本章实验中，PWR 的驱动代码包括 pwr.c 和 pwr.h 两个文件。

因为本章实验中停止模式的唤醒方式与上一章实验中睡眠模式的唤醒方式一致，都是可以由任意的中断唤醒，且均使用了 WKUP 按键的外部中断进行唤醒，因此本章实验中停止模式唤醒按键的相关宏定义、停止模式唤醒按键的初始化及其中断回调函数均与上一章实验一致，因此请读者自行查看上一章中的相关内容。

PWR 驱动中，进入睡眠模式的函数，如下所示：

```
/**
 * @brief 进入停止模式
 * @param 无
 * @retval 无
 */
void pwr_enter_stop(void)
{
    GPIO_InitTypeDef gpio_init_struct = {0};

    __HAL_RCC_PWR_CLK_ENABLE();
    PWR_WKUP_INT_GPIO_CLK_ENABLE();

    HAL_NVIC_SetPriority(PWR_WKUP_INT_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(PWR_WKUP_INT_IRQn);

    /* 配置唤醒引脚 */
    gpio_init_struct.Pin = PWR_WKUP_INT_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_IT_RISING;
    gpio_init_struct.Pull = GPIO_PULLDOWN;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(PWR_WKUP_INT_GPIO_PORT, &gpio_init_struct);

    /* 关闭 SysTick 中断 */
    SysTick->CTRL &= ~SysTick_CTRL_TICKINT_Msk;

    /* 进入停止模式 */
    HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);
}
```

从上面的代码中可以看出，该函数调用了函数 HAL_PWR_EnterSTOPMode()以 WFI 方式进入停止模式，接下来 MCU 便会进入停止模式，等待任意的中断唤醒，因此在执行此函数前，需要先关闭部分中断，以免误唤醒。

32.2.3 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
```

```
{
    uint8_t t = 0;
    uint8_t key;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    lcd_init(); /* 初始化 LCD */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "STOP TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    lcd_show_string(30, 110, 200, 16, 16, "KEY0:Enter STOP MODE", RED);
    lcd_show_string(30, 130, 200, 16, 16, "KEY_UP:Exit STOP MODE", RED);

    while (1)
    {
        key = key_scan(0);
        if (key == KEY0_PRES)
        {
            /* 按下 KEY0 按键, 开启 LED1 后进入停止模式 */
            LED1(0);
            pwr_enter_stop();
            sys_stm32_clock_init(336, 8, 2, 7);
            LED1(1);
        }

        if (++t == 20)
        {
            t = 0;
            LED0_TOGGLE();
        }

        delay_ms(10);
    }
}
```

从上面的代码中可以看出,在完成相关的初始化操作后,便不断地扫描按键,若扫描到 KEY0 按键被按下,则会点亮 LED1 后进入停止模式,此时所有的时钟将会停止,从该停止模式唤醒需要有任意的中断产生,因此可以由 WKUP 按键产生外部中断来唤醒停止模式。退出停止模式后,LED1 将被熄灭。

32.3 下载验证

在完成编译和烧录操作后,可以看到 LED0 闪烁提示系统正在运行,此时可以按下 KEY0 按键,可以看到 LED1 亮起,但 LED0 不再闪烁,这是因为系统已经进入停止模式了,此时再按下 WKUP 按键,即可从停止模式下唤醒,可以看到 LED1 熄灭,LED0 继续闪烁。

第三十三章 待机模式实验

本章介绍 STM32F407 低功耗模式中的待机模式，进入待机模式后，MCU 内部的电压调压器将断开 1.3V 电源域的电，这意味着内核和外设都将停止工作，并且内核寄存器和内存中的数据都将丢失，但这也是功耗最低的模式，待机模式下可被 WKUP 引脚的上升沿唤醒。通过本章的学习，读者将学习到低功耗模式下待机模式的使用。

本章分为如下几个小节：

- 33.1 硬件设计
- 33.2 程序设计
- 33.3 下载验证

33.1 硬件设计

33.1.1 例程功能

1. TFTLCD 上显示实验信息
2. LED0 闪烁时，按下 KEY0 按键后，MCU 进入待机模式
3. MCU 进入待机模式后，按下 WKUP 按键，MCU 重新运行

33.1.2 硬件资源

1. LED
 - LED0 - PF9
 - LED1 - PF10
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
 - WKUP - PA0
 - KEY0 - PE4

33.1.3 原理图

本章实验介绍 STM32F407 低功耗模式中的待机模式，不涉及连接原理图。

33.2 程序设计

33.2.1 HAL 库的 PWR 驱动

本实验配置了 WKUP 按键(连接至 WKUP 引脚)唤醒待机模式，因此具体的配置步骤如下：

①：使能 WKUP 引脚功能

②：进入待机模式

在 HAL 库中对应的驱动函数如下：

①：使能 WKUP 引脚功能

该函数用于使能 WKUP 引脚功能，其函数原型如下所示：

```
void HAL_PWR_EnableWakeUpPin(uint32_t WakeUpPinx);
```

该函数的形参描述，如下表所示：

形参	描述
WakeUpPinx	指定使能的唤醒引脚 例如：PWR_WAKEUP_PIN1 等（在 stm32f4xx_hal_pwr.h 文件中有定义）

表 33.2.1.1 函数 HAL_PWR_EnableWakeUpPin()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
无	无

表 33.2.1.2 函数 HAL_PWR_EnableWakeUpPin()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 使能 WKUP 引脚功能 */
    HAL_PWR_EnableWakeUpPin(PWR_WAKEUP_PIN1);
}
```

②：进入待机模式

该函数用于进入待机模式，其函数原型如下所示：

```
void HAL_PWR_EnterSTANDBYMode(void);
```

该函数的形参描述，如下表所示：

形参	描述
无	无

表 33.2.1.3 函数 HAL_PWR_EnterSTANDBYMode()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
无	无

表 33.2.1.4 函数 HAL_PWR_EnterSTANDBYMode()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 进入待机模式 */
    HAL_PWR_EnterSTANDBYMode();
}
```

33.2.2 PWR 驱动

本章实验的 PWR 驱动主要负责向应用层提供进入待机模式的函数。本章实验中，PWR 的驱动代码包括 pwr.c 和 pwr.h 两个文件。

PWR 驱动中，进入待机模式的函数，如下所示：

```
/**
 * @brief 进入待机模式
 * @param 无
 * @retval 无
 */
void pwr_enter_standby(void)
{
    __HAL_RCC_PWR_CLK_ENABLE();

    /* 配置唤醒引脚 */
    HAL_PWR_EnableWakeUpPin(PWR_WAKEUP_PIN1);
    __HAL_PWR_CLEAR_FLAG(PWR_FLAG_WU);

    /* 进入待机模式 */
    HAL_PWR_EnterSTANDBYMode();
}
```

从上面的代码中可以看出，在进入待机模式前使能了 WKUP 引脚功能，最后调用函数 HAL_PWR_EnterSTANDBYMode()进入待机模式。

33.2.3 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    uint8_t t = 0;
    uint8_t key;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    lcd_init(); /* 初始化 LCD */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "STANDBY TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    lcd_show_string(30, 110, 200, 16, 16, "KEY0:Enter STANDBY MODE", RED);
    lcd_show_string(30, 130, 200, 16, 16, "KEY_UP:Exit STANDBY MODE", RED);

    while (1)
    {
        key = key_scan(0);
        if (key == KEY0_PRES)
        {
            /* 按下 KEY0 按键, 进入待机模式 */
            pwr_enter_standby();
        }

        if (++t == 20)
        {
            t = 0;
            LED0_TOGGLE();
        }

        delay_ms(10);
    }
}
```

从上面的代码中可以看出,在完成相关的初始化操作后,便不断地扫描按键,若扫描到 KEY0 按键被按下,则会进入待机模式,进入待机模式后 LED0 便不再闪烁,说明此时程序已经停止运行,进入了待机模式,此时可以按下 WKUP 按键进行待机模式唤醒,随后便可看到 LED0 恢复闪烁,说明程序再次运行。

33.3 下载验证

在完成编译和烧录操作后,可以看到 LED0 闪烁提示系统正在运行,此时可以按下 KEY0 按键,可以看到 LED0 不再闪烁,这是因为系统已经进入待机模式了,此时再按下 WKUP 按键,即可从待机模式下唤醒,可以看到 LED0 继续闪烁。

第三十四章 DMA 实验

本章介绍 STM32F407 直接存储访问（DMA）的使用，DMA 能够在无 CPU 干预的情况下，实现外设与存储器或存储器与存储器之间数据的高速传输，从而节省 CPU 资源来做其他操作。通过本章的学习，读者将学习到 DMA 的使用。

本章分为如下几个小节：

- 34.1 硬件设计
- 34.2 程序设计
- 34.3 下载验证

34.1 硬件设计

34.1.1 例程功能

1. 按下 KEY0 按键后，串口输出数据
2. LED0 闪烁，提示程序正在运行

34.1.2 硬件资源

1. LED
LED0 - PF9
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
KEY0 - PA15
4. USART1
USART1_TX - PA9
USART1_RX - PA10
5. DMA2
Stream7 Channel4

34.1.3 原理图

本章实验使用的 DMA 为 STM32F407 的片上资源，因此没有对应的连接原理图。

34.2 程序设计

34.2.1 HAL 库的 DMA 驱动

在使用 DMA 之前，需要先根据需求对 DMA 的数据流进行配置，例如本章实验要使用 DMA 进行串口数据的发送，因此应将 DMA 的数据流配置为存储器到外设的模式等等。使用 DMA 的具体步骤如下：

- ①：初始化 DMA
- ②：UART DMA 发送数据

在 HAL 库中对应的驱动函数如下：

①：初始化 DMA

该函数用于初 DMA 始化的各项参数，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_DMA_Init(DMA_HandleTypeDef *hdma);
```

该函数的形参描述，如下表所示：

形参	描述
hdma	指向 DMA 句柄的指针

表 34.2.1.1 函数 HAL_DMA_Init()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 34.2.1.2 函数 HAL_DMA_Init()返回值描述

该函数需要传入 DMA 的句柄指针，该句柄中就包含了 DMA 的初始化配置参数结构体，该结构体的定义如下所示：

```
typedef struct
{
    uint32_t Channel;           /* 通道 */
    uint32_t Direction;        /* 方向 */
    uint32_t PeriphInc;         /* 外设递增 */
    uint32_t MemInc;            /* 存储器递增 */
    uint32_t PeriphDataAlignment; /* 外设数据格式 */
    uint32_t MemDataAlignment; /* 存储器数据格式 */
    uint32_t Mode;              /* 模式 */
    uint32_t Priority;           /* 优先级 */
    uint32_t FIFOMode;          /* FiFo 模式 */
    uint32_t FIFOThreshold;     /* FiFo 阈值 */
    uint32_t MemBurst;          /* 存储区突发传输 */
    uint32_t PeriphBurst;       /* 外设突发传输 */
} DMA_InitTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    DMA_HandleTypeDef dma_handle = {0};

    /* 初始化 DMA */
    dma_handle.Instance = DMA1_Stream0;
    dma_handle.Init.Channel = DMA_CHANNEL_0;
    dma_handle.Init.Direction = DMA_MEMORY_TO_PERIPH;
    dma_handle.Init.PeriphInc = DMA_PINC_DISABLE;
    dma_handle.Init.MemInc = DMA_MINC_ENABLE;
    dma_handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
    dma_handle.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
    dma_handle.Init.Mode = DMA_NORMAL;
    dma_handle.Init.Priority = DMA_PRIORITY_VERY_HIGH;
    dma_handle.Init.FIFOMode = DMA_FIFOMODE_DISABLE;
    dma_handle.Init.FIFOThreshold = DMA_FIFO_THRESHOLD_1QUARTERFULL;
    dma_handle.Init.MemBurst = DMA_MBURST_SINGLE;
    dma_handle.Init.PeriphBurst = DMA_PBURST_SINGLE;
    HAL_DMA_Init(&dma_handle);
}
```

②：UART DMA 发送数据

该函数用于 UART DMA 发送数据，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_UART_Transmit_DMA( UART_HandleTypeDef *huart,
                                           const uint8_t *pData,
                                           uint16_t Size);
```

该函数的形参描述，如下表所示：

形参	描述
huart	指向 UART 句柄的指针
pData	指向待发送数据的指针
Size	待发送数据的大小

表 34.2.1.3 函数 HAL_UART_Transmit_DMA()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 34.2.1.4 函数 HAL_UART_Transmit_DMA()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    const uint8_t buf[] = {"Hello, World!"};

    /* UART DMA 发送数据 */
    HAL_UART_Transmit_DMA(&uart_handle, buf, sizeof(buf));
}
```

34.2.2 DMA 驱动

本章实验的 DMA 驱动主要负责向应用层提供 DMA 的初始化函数。本章实验中，DMA 的驱动代码包括 dma.c 和 dma.h 两个文件。

DMA 驱动中，DMA 的初始化函数，如下所示：

```
/**
 * @brief 初始化 DMA
 * @param 无
 * @retval 无
 */
void dma_init(void)
{
    __HAL_RCC_DMA2_CLK_ENABLE();

    /* 配置 DMA */
    g_dma_handle.Instance = DMA2_Stream7;
    g_dma_handle.Init.Channel = DMA_CHANNEL_4;
    g_dma_handle.Init.Direction = DMA_MEMORY_TO_PERIPH;
    g_dma_handle.Init.PeriphInc = DMA_PINC_DISABLE;
    g_dma_handle.Init.MemInc = DMA_MINC_ENABLE;
    g_dma_handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
    g_dma_handle.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
    g_dma_handle.Init.Mode = DMA_NORMAL;
    g_dma_handle.Init.Priority = DMA_PRIORITY_VERY_HIGH;
    g_dma_handle.Init.FIFOMode = DMA_FIFOMODE_DISABLE;
    g_dma_handle.Init.FIFOThreshold = DMA_FIFO_THRESHOLD_1QUARTERFULL;
    g_dma_handle.Init.MemBurst = DMA_MBURST_SINGLE;
    g_dma_handle.Init.PeriphBurst = DMA_PBURST_SINGLE;
    HAL_DMA_Init(&g_dma_handle);

    /* 与 UART 句柄关联 */
    __HAL_LINKDMA(&g_uart1_handle, hdmatx, g_dma_handle);

    HAL_NVIC_SetPriority(DMA2_Stream7_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(DMA2_Stream7_IRQn);
}
```

从上面的代码中可以看到，DMA 的初始化函数提供了 DMA 数据流通道、外设基地址、存储器基地址、传输数据项数据的配置参数，结合这些传入的参数，通过 DMA 初始化函数 HAL_DMA_Init()初始化了 DMA。

34.2.3 实验应用代码

本章实验的应用代码，如下所示：

```
extern UART_HandleTypeDef g_uart1_handle;

uint8_t temp[] = {"Hello, M144Z-M4 STM32F407!\r\n"};
uint8_t buf[sizeof(temp) * 200];
uint8_t uart_ready = 1;

/**
```

```
* @brief   HAL 库 UART 传输完成回调函数
* @param   无
* @retval   无
*/
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
    /* 标记传输完成, 可以进行下一次传输 */
    uart_ready = 1;
}

int main(void)
{
    uint8_t t = 0;
    uint8_t key;
    uint16_t buf_index;
    uint8_t temp_index;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    lcd_init(); /* 初始化 LCD */
    dma_init(); /* 初始化 DMA */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "DMA TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    lcd_show_string(30, 110, 200, 16, 16, "KEY0:Start", RED);

    /* 准备数据 */
    for (buf_index=0; buf_index<200; buf_index++)
    {
        for (temp_index=0; temp_index<sizeof(temp); temp_index++)
        {
            buf[buf_index*sizeof(temp) + temp_index] = temp[temp_index];
        }
    }

    while (1)
    {
        key = key_scan(0);
        if (key == KEY0_PRES)
        {
            if (uart_ready == 1)
            {
                uart_ready = 0;
                /* DMA 传输串口数据 */
                HAL_UART_Transmit_DMA(&g_uart1_handle, buf, sizeof(buf));
            }

            if (++t == 20)
            {
                t = 0;
                LED0_TOGGLE();
            }

            delay_ms(10);
        }
    }
}
```

从上面的代码中可以看到, 在 DMA 的初始化中, 初始化了 DMA2 数据流 7, 这是因为

USART1 的发送就对应了 DMA2 数据流 7 的通道 4，完成初始化工作后，就不断地扫描按键，若扫描到 KEY0 按键，则调用函数 HAL_UART_Transmit_DMA()通过 DMA 完成 UART 的数据发送。

34.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上显示了本实验的实验信息，此时若按下 KEY0 按键，可以看到串口调试助手不断打印“Hello, M144Z-M4 STM32F407!\r\n”（200 次）。

第三十五章 单通道 ADC 采集实验

本章介绍使用 STM32F407 模数转换器（ADC）进行带通道的电压采集。通过本章的学习，读者将学习到单通道 ADC 的使用。

本章分为如下几个小节：

- 35.1 硬件设计
- 35.2 程序设计
- 35.3 下载验证

35.1 硬件设计

35.1.1 例程功能

1. LCD 上不断刷新显示 PA1 引脚输入电压采样的数字量和模拟量。
2. LED0 闪烁，提示程序正在运行

35.1.2 硬件资源

1. LED
LED0 - PF9
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. ADC2
Channel 1 - PA1

35.1.3 原理图

本章实验使用的 ADC1 为 STM32F407 的片上资源，因此没有对应的连接原理图。

35.2 程序设计

35.2.1 HAL 库的 ADC 驱动

本章实验将使用 ADC1 的通道 1（PA1 引脚）采集外部输入电压的模拟量，并将其转换为数字量，其具体的步骤如下：

- ①：初始化 ADC
- ②：配置 ADC 通道
- ③：开启 ADC
- ④：ADC 轮询转换
- ⑤：获取 ADC 值

在 HAL 库中对应的驱动函数如下：

①：初始化 ADC

该函数用于初始化 ADC，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_ADC_Init(ADC_HandleTypeDef* hadc);
```

该函数的形参描述，如下表所示：

形参	描述
hadc	指向 ADC 句柄的指针

表 35.2.1.1 函数 HAL_ADC_Init()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 35.2.1.2 函数 HAL_ADC_Init()返回值描述

该函数需要传入 ADC 的句柄指针，该句柄中就包含了 ADC 的初始化配置参数结构体，该结构体的定义如下所示：

```
typedef struct
{
    uint32_t ClockPrescaler;          /* 时钟预分频系数 */
    uint32_t Resolution;              /* 精度 */
    uint32_t DataAlign;               /* 数据对齐方式 */
    uint32_t ScanConvMode;            /* 扫描模式 */
    uint32_t EOCSelection;            /* EOC 选择 */
    FunctionalState ContinuousConvMode; /* 连续转换 */
    uint32_t NbrOfConversion;         /* 规则序列 */
    FunctionalState DiscontinuousConvMode; /* 不连续采样模式 */
    uint32_t NbrOfDiscConversion;     /* 不连续采样序列 */
    uint32_t ExternalTrigConv;        /* 外部触发转换 */
    uint32_t ExternalTrigConvEdge;    /* 外部触发转换边沿 */
    FunctionalState DMAContinuousRequests; /* DMA 请求 */
}ADC_InitTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    ADC_HandleTypeDef adc_handle = {0};

    /* 初始化 ADC */
    adc_handle.Instance = ADC1;
    adc_handle.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV2;
    adc_handle.Init.Resolution = ADC_RESOLUTION_12B;
    adc_handle.Init.DataAlign = ADC_DATAALIGN_RIGHT;
    adc_handle.Init.ScanConvMode = DISABLE;
    adc_handle.Init.EOCSelection = ADC_EOC_SEQ_CONV;
    adc_handle.Init.ContinuousConvMode = DISABLE;
    adc_handle.Init.NbrOfConversion = 1;
    adc_handle.Init.DiscontinuousConvMode = DISABLE;
    adc_handle.Init.NbrOfDiscConversion = 1;
    adc_handle.Init.ExternalTrigConv = ADC_SOFTWARE_START;
    adc_handle.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
    adc_handle.Init.DMAContinuousRequests = DISABLE;
    HAL_ADC_Init(&adc_handle);
}
```

②：配置 ADC 通道

该函数用于配置 ADC 通道，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_ADC_ConfigChannel(ADC_HandleTypeDef* hadc,
                                         ADC_ChannelConfTypeDef* sConfig);
```

该函数的形参描述，如下表所示：

形参	描述
hadc	指向 ADC 句柄的指针
sConfig	指向 ADC 通道配置结构体的指针 需自行定义，并根据 ADC 通道配置参数填充结构体中的成员变量

表 35.2.1.3 函数 HAL_ADC_ConfigChannel()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 35.2.1.4 函数 HAL_ADC_ConfigChannel()返回值描述

该函数使用 ADC_ChannelConfTypeDef 类型结构体指针传入了 ADC 通道的配置参数，该结构体的定义如下所示：

```
typedef struct
```

```
{
    uint32_t Channel;      /* 通道 */
    uint32_t Rank;         /* 规则组中的序号 */
    uint32_t SamplingTime; /* 采样时间 */
    uint32_t Offset;      /* 保留 */
}ADC_ChannelConfTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    ADC_ChannelConfTypeDef config = {0};

    /* 配置 ADC 通道 */
    config.Channel = ADC_CHANNEL_0;
    config.Rank = 1;
    config.SamplingTime = ADC_SAMPLETIME_3CYCLES;
    config.Offset = 0;
    HAL_ADC_ConfigChannel(&adc_handle, &config);
}
```

③：开启 ADC

该函数用于开启 ADC，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_ADC_Start(ADC_HandleTypeDef* hadc);
```

该函数的形参描述，如下表所示：

形参	描述
hadc	指向 ADC 句柄的指针

表 35.2.1.5 函数 HAL_ADC_Start()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 35.2.1.6 函数 HAL_ADC_Start()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 开启 ADC */
    HAL_ADC_Start(&adc_handle);
}
```

④：ADC 轮询转换

该函数用于 ADC 轮询转换，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_ADC_PollForConversion(ADC_HandleTypeDef* hadc,
                                              uint32_t Timeout);
```

该函数的形参描述，如下表所示：

形参	描述
hadc	指向 ADC 句柄的指针
Timeout	等待轮询超时时间

表 35.2.1.7 函数 HAL_ADC_PollForConversion()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 35.2.1.8 函数 HAL_ADC_PollForConversion()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
```



```
/* ADC 轮询转换 */
HAL_ADC_PollForConversion(&adc_handle, HAL_MAX_DELAY);
}
```

⑤：获取 ADC 值

该函数用于获取 ADC 值，其函数原型如下所示：

```
uint32_t HAL_ADC_GetValue(ADC_HandleTypeDef* hadc);
```

该函数的形参描述，如下表所示：

形参	描述
hadc	指向 ADC 句柄的指针

表 35.2.1.9 函数 HAL_ADC_GetValue()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
uint32_t 类型数据	ADC 值

表 35.2.1.10 函数 HAL_ADC_GetValue()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    uint32_t value;

    value = HAL_ADC_GetValue(&adc_handle);

    /* Do something. */
}
```

35.2.2 ADC 驱动

本章实验的 ADC 驱动主要负责向应用层提供 ADC 的初始化和获取 ADC 转换结果的函数。本章实验中，ADC 的驱动代码包括 adc.c 和 adc.h 两个文件。

ADC 驱动中，对 ADC、GPIO 的相关宏定义，如下所示：

```
#define ADC_ADCX      ADC2
#define ADC_ADCX_CLK_ENABLE() \
    do { \
        __HAL_RCC_ADC2_CLK_ENABLE(); \
    } while (0)
#define ADC_ADCX_CHY      ADC_CHANNEL_1
#define ADC_ADCX_CHY_GPIO_PORT GPIOA
#define ADC_ADCX_CHY_GPIO_PIN GPIO_PIN_1
#define ADC_ADCX_CHY_GPIO_CLK_ENABLE() \
    do { \
        __HAL_RCC_GPIOA_CLK_ENABLE(); \
    } while (0)
```

ADC 驱动中，ADC 的初始化函数，如下所示：

```
/**
 * @brief  初始化 ADC
 * @param  无
 * @retval 无
 */
void adc_init(void)
{
    /* 配置 ADC */
    g_adc_handle.Instance = ADC_ADCX;
    g_adc_handle.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV4;
    g_adc_handle.Init.Resolution = ADC_RESOLUTION_12B;
    g_adc_handle.Init.DataAlign = ADC_DATAALIGN_RIGHT;
    g_adc_handle.Init.ScanConvMode = DISABLE;
    g_adc_handle.Init.EOCSelection = ADC_EOC_SEQ_CONV;
    g_adc_handle.Init.ContinuousConvMode = DISABLE;
    g_adc_handle.Init.NbrOfConversion = 1;
```

```
g_adc_handle.Init.DiscontinuousConvMode = DISABLE;
g_adc_handle.Init.NbrOfDiscConversion = 1;
g_adc_handle.Init.ExternalTrigConv = ADC_SOFTWARE_START;
g_adc_handle.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
g_adc_handle.Init.DMAContinuousRequests = DISABLE;
HAL_ADC_Init(&g_adc_handle);
}
```

从上面的代码中可以看出，初始化 ADC 就是调用函数 HAL_ADC_Init()来初始化 ADC。
ADC 驱动中，配置 ADC 通道的函数，如下所示：

```
/**
 * @brief 设置ADC通道
 * @param adc_handle: ADC句柄
 * @param channel: ADC通道
 * @param rank: 规则采样的编号
 * @param sampling_time: 采样时间
 * @retval 无
 */
void adc_channel_set(ADC_HandleTypeDef *adc_handle,
                    uint32_t channel,
                    uint32_t rank,
                    uint32_t sampling_time)
{
    ADC_ChannelConfTypeDef adc_channel_conf_struct = {0};

    /* 配置ADC通道 */
    adc_channel_conf_struct.Channel = channel;
    adc_channel_conf_struct.Rank = rank;
    adc_channel_conf_struct.SamplingTime = sampling_time;
    adc_channel_conf_struct.Offset = 0;
    HAL_ADC_ConfigChannel(adc_handle, &adc_channel_conf_struct);
}
```

ADC 驱动中，获取 ADC 转换结果的函数，如下所示：

```
/**
 * @brief 获取ADC结果
 * @param channel: ADC通道
 * @retval ADC结果
 */
uint16_t adc_get_result(uint32_t channel)
{
    uint16_t result;

    adc_channel_set(&g_adc_handle, channel, 1, ADC_SAMPLETIME_480CYCLES);
    HAL_ADC_Start(&g_adc_handle);
    HAL_ADC_PollForConversion(&g_adc_handle, HAL_MAX_DELAY);
    result = HAL_ADC_GetValue(&g_adc_handle);

    return result;
}

/**
 * @brief 均值滤波获取ADC结果
 * @param channel: ADC通道
 * @param times: 均值滤波的原始数据个数
 * @retval ADC结果
 */
uint16_t adc_get_result_average(uint32_t channel, uint8_t times)
{
    uint32_t sum_result = 0;
    uint8_t index;
    uint16_t result;

    for (index=0; index<times; index++)
    {
```

```

        sum_result += adc_get_result(channel);
    }

    result = sum_result / times;

    return result;
}

```

以上两个函数都是用于获取 ADC 转换结果的函数，其中函数 `adc_get_result()` 会配置并开启 ADC 指定通道的规则通道转换，并等待其转换结束后，读取其转换的 1 次结果；而函数 `adc_get_result_averagr()` 则是多次调用啊含糊 `adc_get_result()` 获取多次 ADC 的转换结果，然后进行均值滤波。

35.2.3 实验应用代码

本章实验的应用代码，如下所示：

```

int main(void)
{
    uint16_t adc_result;
    uint16_t voltage;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    lcd_init(); /* 初始化 LCD */
    adc_init(); /* 初始化 ADC */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "ADC TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    lcd_show_string(30, 110, 200, 16, 16, "ADC2_CH1_VAL:0", BLUE);
    lcd_show_string(30, 130, 200, 16, 16, "ADC2_CH1_VOL:0.000V", BLUE);

    while (1)
    {
        /* 均值滤波获取 ADC 结果 */
        adc_result = adc_get_result_average(ADC_ADCX_CHY, 10);
        lcd_show_xnum(134, 110, adc_result, 5, 16, 0, BLUE);

        /* 计算实际电压值 (扩大 1000 倍) */
        voltage = (adc_result * 3300) / 4095;
        lcd_show_xnum(134, 130, voltage / 1000, 1, 16, 0, BLUE);
        lcd_show_xnum(150, 130, voltage % 1000, 3, 16, 0x80, BLUE);

        LED0_TOGGLE();

        delay_ms(100);
    }
}

```

从上面的代码中可以看出，在进行完包括 ADC 的所有初始化工作后，便不断地获取 ADC2 通道 1 进行 10 次转换后经过均值滤波后的结果，并将该原始值显示在 LCD 上，同时还通过该电压的原始值计算出了电压的模拟量，并在 LCD 上进行显示。

35.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上实时刷新显示着 ADC2 通道 1（PA1 引脚）采集到电压的数字量和模拟量，此时可以通过杜邦线给 PA1 引脚接入不同的电压值（注意共地，且输入电压不能超过 3.3V，否则可能损坏开发板），可以看到 LCD 上显示的电压数字量和模拟量也随之改变。

第三十六章 单通道 ADC 采集（DMA 读取）实验

本章介绍使用 STM32F407 的 DMA 进行单通道的 ADC 采集。通过本章的学习，读者将学习到 DMA、ADC 的使用。

本章分为如下几个小节：

- 36.1 硬件设计
- 36.2 程序设计
- 36.3 下载验证

36.1 硬件设计

36.1.1 例程功能

- 1. LCD 上不断刷新显示 PA1 引脚输入电压采样的数字量和模拟量。
- 2. LED0 闪烁，提示程序正在运行

36.1.2 硬件资源

- 1. LED
 - LED0 - PF9
- 2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
- 3. ADC3
 - Channel1 - PA1
- 4. DMA2
 - Stream1 Channel2

36.1.3 原理图

本章实验使用的 ADC3 为 STM32F407 的片上资源，因此没有对应的连接原理图。

36.2 程序设计

36.2.1 HAL 库的 ADC 驱动

本章实验中使用 DAM2 Stream1 Channel2 的外设到存储器模式，将 ADC3 的转换结果保存到指定的存储器中，其具体的步骤如下：

- ①：初始化 ADC
- ②：配置 ADC 通道
- ③：开始 ADC DMA 传输

在 HAL 库中对应的驱动函数如下：

①：初始化 ADC

请见第 35.2.1 小节中初始化 ADC 的相关内容。

②：配置 ADC 通道

请见第 35.2.1 小节中配置 ADC 通道的相关内容。

③：开始 ADC DMA 传输

该函数用于开始 ADC DMA 传输，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_ADC_Start_DMA(ADC_HandleTypeDef* hadc,
                                     uint32_t* pData,
                                     uint32_t Length);
```

该函数的形参描述，如下表所示：

形参	描述
hadc	指向 ADC 句柄的指针

pData	目标存储器的首地址
Length	传输数据的长度

表 36.2.1.1 函数 HAL_ADC_Start_DMA()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 36.2.1.2 函数 HAL_ADC_Start_DMA()形参描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    uint16_t buf;

    /* 开始 ADC DMA 传输 */
    HAL_ADC_Start_DMA(&adc_handle, &buf, sizeof(buf) / sizeof(uint16_t));
}
```

36.2.2 ADC 驱动

本章实验的 ADC 驱动主要负责向应用层提供 ADC 初始化和启动 ADC 的 DMA 采集的函数，同时实现 DMA 的中断回调函数。本章实验中，ADC 的驱动代码包括 adc.c 和 adc.h 两个文件。

ADC 驱动中，对 DMA、GPIO、ADC 的相关宏定义，如下所示：

```
#define ADC_DMA_ADCX ADC3
#define ADC_DMA_ADCX_CLK_ENABLE() \
do { \
    __HAL_RCC_ADC3_CLK_ENABLE(); \
} while (0)
#define ADC_DMA_ADCX_CHY ADC_CHANNEL_1
#define ADC_DMA_ADCX_CHY_GPIO_PORT GPIOA
#define ADC_DMA_ADCX_CHY_GPIO_PIN GPIO_PIN_1
#define ADC_DMA_ADCX_CHY_GPIO_CLK_ENABLE() \
do { \
    __HAL_RCC_GPIOA_CLK_ENABLE(); \
} while (0)
#define ADC_DMA_ADCX_DMASX DMA2_Stream1
#define ADC_DMA_ADCX_DMA_CLK_ENABLE() \
do { \
    __HAL_RCC_DMA2_CLK_ENABLE(); \
} while (0)
#define ADC_DMA_ADCX_DMASX_CHY DMA_CHANNEL_2
#define ADC_DMA_ADCX_DMASX_IRQn DMA2_Stream1_IRQn
#define ADC_DMA_ADCX_DMASX_IRQHandler DMA2_Stream1_IRQHandler
```

ADC 驱动中，ADC 的初始化函数，如下所示：

```
/**
 * @brief 初始化 ADC DMA 读取
 * @param memory_base: 读取目标内存基地址
 * @retval 无
 */
void adc_dma_init(uint32_t memory_base)
{
    /* 配置 ADC */
    g_adc_dma_handle.Instance = ADC_DMA_ADCX;
    g_adc_dma_handle.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV4;
    g_adc_dma_handle.Init.Resolution = ADC_RESOLUTION_12B;
    g_adc_dma_handle.Init.DataAlign = ADC_DATAALIGN_RIGHT;
    g_adc_dma_handle.Init.ScanConvMode = DISABLE;
    g_adc_dma_handle.Init.EOCSelection = ADC_EOC_SEQ_CONV;
    g_adc_dma_handle.Init.ContinuousConvMode = ENABLE;
    g_adc_dma_handle.Init.NbrOfConversion = 1;
```

```

g_adc_dma_handle.Init.DiscontinuousConvMode = DISABLE;
g_adc_dma_handle.Init.NbrOfDiscConversion = 1;
g_adc_dma_handle.Init.ExternalTrigConv = ADC_SOFTWARE_START;
g_adc_dma_handle.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
g_adc_dma_handle.Init.DMAContinuousRequests = ENABLE;
HAL_ADC_Init(&g_adc_dma_handle);

g_adc_dma_memory_base = memory_base;
}

/**
 * @brief   HAL 库 ADC 初始化 MSP 函数
 * @param   hadc: ADC 句柄
 * @retval  无
 */
void HAL_ADC_MspInit(ADC_HandleTypeDef* hadc)
{
    GPIO_InitTypeDef gpio_init_struct = {0};

    if (hadc->Instance == ADC_DMA_ADCX)
    {
        ADC_DMA_ADCX_CLK_ENABLE();
        ADC_DMA_ADCX_CHY_GPIO_CLK_ENABLE();
        ADC_DMA_ADCX_DMA_CLK_ENABLE();

        /* 初始化 ADC 采样引脚 */
        gpio_init_struct.Pin = ADC_DMA_ADCX_CHY_GPIO_PIN;
        gpio_init_struct.Mode = GPIO_MODE_ANALOG;
        gpio_init_struct.Pull = GPIO_NOPULL;
        gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
        HAL_GPIO_Init(ADC_DMA_ADCX_CHY_GPIO_PORT, &gpio_init_struct);

        /* 配置 DMA */
        g_adc_dma_dma_handle.Instance = ADC_DMA_ADCX_DMASX;
        g_adc_dma_dma_handle.Init.Channel = ADC_DMA_ADCX_DMASX_CHY;
        g_adc_dma_dma_handle.Init.Direction = DMA_PERIPH_TO_MEMORY;
        g_adc_dma_dma_handle.Init.PeriphInc = DMA_PINC_DISABLE;
        g_adc_dma_dma_handle.Init.MemInc = DMA_MINC_ENABLE;
        g_adc_dma_dma_handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_HALFWORD;
        g_adc_dma_dma_handle.Init.MemDataAlignment = DMA_MDATAALIGN_HALFWORD;
        g_adc_dma_dma_handle.Init.Mode = DMA_NORMAL;
        g_adc_dma_dma_handle.Init.Priority = DMA_PRIORITY_VERY_HIGH;
        g_adc_dma_dma_handle.Init.FIFOMode = DMA_FIFOMODE_DISABLE;
        g_adc_dma_dma_handle.Init.FIFOThreshold =
            DMA_FIFO_THRESHOLD_1QUARTERFULL;
        g_adc_dma_dma_handle.Init.MemBurst = DMA_MBURST_SINGLE;
        g_adc_dma_dma_handle.Init.PeriphBurst = DMA_PBURST_SINGLE;
        HAL_DMA_Init(&g_adc_dma_dma_handle);

        __HAL_LINKDMA(&g_adc_dma_handle, DMA_Handle, g_adc_dma_dma_handle);

        HAL_NVIC_SetPriority(ADC_DMA_ADCX_DMASX_IRQn, 0, 0);
        HAL_NVIC_EnableIRQ(ADC_DMA_ADCX_DMASX_IRQn);
    }
}

```

从上面的代码中可以看出，该 ADC 初始化函数初始化了 ADC，同时也使能了 ADC 的 DMA 请求，并且在 HAL 库 ADC 初始化 MSP 函数中配置了 DMA。

ADC 驱动中，启动 ADC 的 DMA 采集的函数，如下所示：

```

/**
 * @brief   开启 ADC DMA 读取
 * @param   length: DMA 读取次数
 * @retval  无
 */
void adc_dma_enable(uint32_t length)

```

```
{
    HAL_ADC_Stop_DMA(&g_adc_dma_handle);

    adc_channel_set(&g_adc_dma_handle,
                    ADC_DMA_ADCX_CHY,
                    1,
                    ADC_SAMPLETIME_480CYCLES);

    HAL_ADC_Start_DMA(&g_adc_dma_handle,
                      (uint32_t *)g_adc_dma_memory_base,
                      length);
}
```

从上面的代码中可以看到，开启 ADC 的 DMA 采集就是先配置 ADC 通道，然后调用函数 HAL_ADC_Start_DMA() 开始 ADC DMA 传输。

DAC 驱动代码中，ADC 转换完成回调函数，如下所示：

```
/**
 * @brief   HAL 库 ADC 转换完成回调函数
 * @param   hadc: ADC 句柄
 * @retval  无
 */
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    if (hadc->Instance == ADC_DMA_ADCX)
    {
        g_adc_dma_sta = 1;
    }
}
```

ADC 转换完成回调函数中可以看到，当 ADC 转换完成后，便将相应的标志位置 1，在应用程序中便可以此来判断 ADC 的采集是否完成。

36.2.3 实验应用代码

本章实验的应用代码，如下所示：

```
#define ADC_DMA_BUF_SIZE    50
uint16_t g_adc_dma_buf[ADC_DMA_BUF_SIZE];
extern uint8_t g_adc_dma_sta;

int main(void)
{
    uint16_t adc_result;
    uint16_t voltage;
    uint16_t index;
    uint32_t result_sum;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    lcd_init(); /* 初始化 LCD */
    adc_dma_init((uint32_t)g_adc_dma_buf); /* 初始化 ADC DMA 读取 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "ADC DMA TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    lcd_show_string(30, 110, 200, 16, 16, "ADC3_CH1_VAL:0", BLUE);
    lcd_show_string(30, 130, 200, 16, 16, "ADC3_CH1_VOL:0.000V", BLUE);

    /* 开启 ADC DMA 读取 */
    adc_dma_enable(ADC_DMA_BUF_SIZE);

    while (1)
```

```

{
    if (g_adc_dma_sta == 1)
    {
        g_adc_dma_sta = 0;

        /* 对 DMA 读取的多个 ADC 数据进行均值滤波 */
        for (result_sum = 0, index=0; index<ADC_DMA_BUF_SIZE; index++)
        {
            result_sum += g_adc_dma_buf[index];
        }
        adc_result = result_sum / ADC_DMA_BUF_SIZE;
        lcd_show_xnum(134, 110, adc_result, 5, 16, 0, BLUE);

        /* 计算实际电压值（扩大 1000 倍） */
        voltage = (adc_result * 3300) / 4095;
        lcd_show_xnum(134, 130, voltage / 1000, 1, 16, 0, BLUE);
        lcd_show_xnum(150, 130, voltage % 1000, 3, 16, 0x80, BLUE);

        adc_dma_enable(ADC_DMA_BUF_SIZE);
    }

    LED0_TOGGLE();

    delay_ms(100);
}
}

```

从上面的代码中可以看出，应用代码中定义了一个数组作为 DMA 传输的目的存储器，随后在完成相关初始化后，就开启 ADC 的 DMA 传输，随后便等待 DMA 传输完成，DMA 传输完成后对所有 ADC 转换的结构进行均值滤波后在 LCD 上进行显示，同时显示转换的电压模拟量。

36.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上不断地刷新显示 ADC3 通道 1（PA1 引脚）采集到电压的数字量和模拟量，此时可以通过杜邦线给 PA1 引脚接入不同的电压值（注意共地，且输入电压不能超过 3.3V，否则可能损坏开发板），可以看到 LCD 上显示的电压数字量和模拟量也随之改变。

第三十七章 多通道 ADC 采集（DMA 读取）实验

本章介绍 STM32F407 的 DMA 进行多通道的 ADC 采集。通过本章的学习，读者将学习到 DMA、ADC 的使用。

本章分为如下几个小节：

- 37.1 硬件设计
- 37.2 程序设计
- 37.3 下载验证

37.1 硬件设计

37.1.1 例程功能

- 1. LCD 上不断刷新显示 PA5 和 PA6 引脚输入电压采样的数字量和模拟量。
- 2. LED0 闪烁，提示程序正在运行

37.1.2 硬件资源

- 1. LED
 - LED0 - PF9
- 2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
- 3. ADC1
 - Channel5 - PA5
 - Channel6 - PA6
- 4. DMA2
 - Stream0 Channel0

37.1.3 原理图

本章实验使用的 ADC1 为 STM32F407 的片上资源，因此没有对应的连接原理图。

37.2 程序设计

37.2.1 HAL 库的 ADC 驱动

本章实验与上一章实验中对 ADC 的操作十分类似，不过本章实验是配置、使能和读取 ADC1 的 Channel5 和 Channel6，因此请见第 36.2.1 小节中 HAL 库的 ADC 驱动的相关内容。

37.2.2 ADC 驱动

本章实验的 ADC 驱动主要负责向应用层提供 ADC 的初始化和启动 ADC 的 DMA 采集函数，同时实现了 DMA 的中断回调函数。本章实验中，ADC 的驱动代码包括 adc.c 和 adc.h 两个文件。

ADC 驱动中，对 DMA、GPIO、ADC 的相关宏定义，如下所示：

```
#define ADC_NCH_DMA_ADCX    ADC1
#define ADC_NCH_DMA_ADCX_CLK_ENABLE() \
do { \
    __HAL_RCC_ADC1_CLK_ENABLE(); \
} while (0)
#define ADC_NCH_DMA_ADCX_CH_NUM    2
#define ADC_NCH_DMA_ADCX_CHA      ADC_CHANNEL_5
#define ADC_NCH_DMA_ADCX_CHA_GPIO_PORT    GPIOA
#define ADC_NCH_DMA_ADCX_CHA_GPIO_PIN    GPIO_PIN_5
#define ADC_NCH_DMA_ADCX_CHA_GPIO_CLK_ENABLE() \
do { \
```

```

    __HAL_RCC_GPIOA_CLK_ENABLE();
} while (0)
#define ADC_NCH_DMA_ADCX_CHB ADC_CHANNEL_6
#define ADC_NCH_DMA_ADCX_CHB_GPIO_PORT GPIOA
#define ADC_NCH_DMA_ADCX_CHB_GPIO_PIN GPIO_PIN_6
#define ADC_NCH_DMA_ADCX_CHB_GPIO_CLK_ENABLE() \
do {
    __HAL_RCC_GPIOA_CLK_ENABLE();
} while (0)
#define ADC_NCH_DMA_ADCX_DMASX DMA2_Stream0
#define ADC_NCH_DMA_ADCX_DMA_CLK_ENABLE() \
do {
    __HAL_RCC_DMA2_CLK_ENABLE();
} while (0)
#define ADC_NCH_DMA_ADCX_DMASX_CHY DMA_CHANNEL_0
#define ADC_NCH_DMA_ADCX_DMASX_IRQn DMA2_Stream0_IRQn
#define ADC_NCH_DMA_ADCX_DMASX_IRQHandler DMA2_Stream0_IRQHandler

```

ADC 驱动中，ADC 的初始化函数，如下所示：

```

/**
 * @brief 初始化多通道 ADC DMA 读取
 * @param memory_base: 读取目标内存基地址
 * @retval 无
 */
void adc_nch_dma_init(uint32_t memory_base)
{
    /* 配置 ADC */
    g_adc_nch_dma_handle.Instance = ADC_NCH_DMA_ADCX;
    g_adc_nch_dma_handle.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV4;
    g_adc_nch_dma_handle.Init.Resolution = ADC_RESOLUTION_12B;
    g_adc_nch_dma_handle.Init.DataAlign = ADC_DATAALIGN_RIGHT;
    g_adc_nch_dma_handle.Init.ScanConvMode = ENABLE;
    g_adc_nch_dma_handle.Init.EOCSelection = ADC_EOC_SEQ_CONV;
    g_adc_nch_dma_handle.Init.ContinuousConvMode = ENABLE;
    g_adc_nch_dma_handle.Init.NbrOfConversion = ADC_NCH_DMA_ADCX_CH_NUM;
    g_adc_nch_dma_handle.Init.DiscontinuousConvMode = DISABLE;
    g_adc_nch_dma_handle.Init.NbrOfDiscConversion = 1;
    g_adc_nch_dma_handle.Init.ExternalTrigConv = ADC_SOFTWARE_START;
    g_adc_nch_dma_handle.Init.ExternalTrigConvEdge =
        ADC_EXTERNALTRIGCONVEDGE_NONE;
    g_adc_nch_dma_handle.Init.DMAContinuousRequests = ENABLE;
    HAL_ADC_Init(&g_adc_nch_dma_handle);

    g_adc_nch_dma_memory_base = memory_base;
}

/**
 * @brief HAL 库 ADC 初始化 MSP 函数
 * @param hadc: ADC 句柄
 * @retval 无
 */
void HAL_ADC_MspInit(ADC_HandleTypeDef* hadc)
{
    GPIO_InitTypeDef gpio_init_struct = {0};

    if (hadc->Instance == ADC_NCH_DMA_ADCX)
    {
        ADC_NCH_DMA_ADCX_CLK_ENABLE();
        ADC_NCH_DMA_ADCX_CHA_GPIO_CLK_ENABLE();
        ADC_NCH_DMA_ADCX_CHB_GPIO_CLK_ENABLE();
        ADC_NCH_DMA_ADCX_DMA_CLK_ENABLE();

        /* 初始化 ADC 采样引脚 */
        gpio_init_struct.Pin = ADC_NCH_DMA_ADCX_CHA_GPIO_PIN;
        gpio_init_struct.Mode = GPIO_MODE_ANALOG;
        gpio_init_struct.Pull = GPIO_NOPULL;
    }
}

```

```

gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
HAL_GPIO_Init(ADC_NCH_DMA_ADCX_CHA_GPIO_PORT, &gpio_init_struct);

gpio_init_struct.Pin = ADC_NCH_DMA_ADCX_CHB_GPIO_PIN;
gpio_init_struct.Mode = GPIO_MODE_ANALOG;
gpio_init_struct.Pull = GPIO_NOPULL;
gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
HAL_GPIO_Init(ADC_NCH_DMA_ADCX_CHB_GPIO_PORT, &gpio_init_struct);

/* 配置 DMA */
g_adc_nch_dma_dma_handle.Instance = ADC_NCH_DMA_ADCX_DMASX;
g_adc_nch_dma_dma_handle.Init.Channel = ADC_NCH_DMA_ADCX_DMASX_CHY;
g_adc_nch_dma_dma_handle.Init.Direction = DMA_PERIPH_TO_MEMORY;
g_adc_nch_dma_dma_handle.Init.PeriphInc = DMA_PINC_DISABLE;
g_adc_nch_dma_dma_handle.Init.MemInc = DMA_MINC_ENABLE;
g_adc_nch_dma_dma_handle.Init.PeriphDataAlignment =
DMA_PDATAALIGN_HALFWORD;
g_adc_nch_dma_dma_handle.Init.MemDataAlignment = DMA_MDATAALIGN_HALFWORD;
g_adc_nch_dma_dma_handle.Init.Mode = DMA_NORMAL;
g_adc_nch_dma_dma_handle.Init.Priority = DMA_PRIORITY_VERY_HIGH;
g_adc_nch_dma_dma_handle.Init.FIFOMode = DMA_FIFOMODE_DISABLE;
g_adc_nch_dma_dma_handle.Init.FIFOThreshold =
DMA_FIFO_THRESHOLD_1QUARTERFULL;
g_adc_nch_dma_dma_handle.Init.MemBurst = DMA_MBURST_SINGLE;
g_adc_nch_dma_dma_handle.Init.PeriphBurst = DMA_PBURST_SINGLE;
HAL_DMA_Init(&g_adc_nch_dma_dma_handle);

__HAL_LINKDMA( &g_adc_nch_dma_handle,
DMA_Handle,
g_adc_nch_dma_dma_handle);

HAL_NVIC_SetPriority(ADC_NCH_DMA_ADCX_DMASX_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(ADC_NCH_DMA_ADCX_DMASX_IRQn);
}
}

```

从上面的代码中可以看出，本章实验的 ADC 初始化与上一章实验中的 ADC 初始化步骤基本一致，不过本章实验的 ADC 初始化函数中配置了 ADC1 的 Channel5 和 Channel6。

ADC 驱动中，启动 ADC 的 DMA 采集的函数和 ADC 转换完成回调函数均与上一章实验中的函数一致，请见第 36.2.2 小节中的相关内容。

37.2.3 实验应用代码

本章实验的应用代码，如下所示：

```

#define ADC_NCH_DMA_BUF_SIZE (50 * ADC_NCH_DMA_ADCX_CH_NUM)
uint16_t g_adc_nch_dma_buf[ADC_NCH_DMA_BUF_SIZE];
extern uint8_t g_adc_nch_dma_sta;

int main(void)
{
    uint16_t adc_result[ADC_NCH_DMA_ADCX_CH_NUM];
    uint16_t voltage;
    uint16_t index;
    uint32_t result_sum;
    uint8_t ch_index;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟，168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    lcd_init(); /* 初始化 LCD */
    adc_nch_dma_init((uint32_t)g_adc_nch_dma_buf); /* 初始化多通道 ADC DMA 读取 */
}

```

```

lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
lcd_show_string(30, 70, 200, 16, 16, "ADC NCH DMA TEST", RED);
lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

lcd_show_string(30, 110, 200, 16, 16, "ADC1_CH5_VAL:0", BLUE);
lcd_show_string(30, 130, 200, 16, 16, "ADC1_CH5_VOL:0.000V", BLUE);
lcd_show_string(30, 150, 200, 16, 16, "ADC1_CH6_VAL:0", BLUE);
lcd_show_string(30, 170, 200, 16, 16, "ADC1_CH6_VOL:0.000V", BLUE);

/* 开启多通道 ADC DMA 读取 */
adc_nch_dma_enable(ADC_NCH_DMA_BUF_SIZE);

while (1)
{
    if (g_adc_nch_dma_sta == 1)
    {
        g_adc_nch_dma_sta = 0;

        for (ch_index=0; ch_index<ADC_NCH_DMA_ADCX_CH_NUM; ch_index++)
        {
            /* 对 DMA 读取的多个 ADC 数据进行均值滤波 */
            for (result_sum = 0, index=0;
                index<(ADC_NCH_DMA_BUF_SIZE / ADC_NCH_DMA_ADCX_CH_NUM);
                index++)
            {
                result_sum +=
                    g_adc_nch_dma_buf[(ADC_NCH_DMA_ADCX_CH_NUM * index) +
                                        ch_index];
            }
            adc_result[ch_index] =
                result_sum /
                (ADC_NCH_DMA_BUF_SIZE / ADC_NCH_DMA_ADCX_CH_NUM);
        }

        /* 计算实际电压值（扩大 1000 倍） */
        voltage = (adc_result[0] * 3300) / 4095;
        lcd_show_xnum(134, 110, adc_result[0], 5, 16, 0, BLUE);
        lcd_show_xnum(134, 130, voltage / 1000, 1, 16, 0, BLUE);
        lcd_show_xnum(150, 130, voltage % 1000, 3, 16, 0x80, BLUE);

        /* 计算实际电压值（扩大 1000 倍） */
        lcd_show_xnum(134, 150, adc_result[1], 5, 16, 0, BLUE);
        voltage = (adc_result[1] * 3300) / 4095;
        lcd_show_xnum(134, 170, voltage / 1000, 1, 16, 0, BLUE);
        lcd_show_xnum(150, 170, voltage % 1000, 3, 16, 0x80, BLUE);

        adc_nch_dma_enable(ADC_NCH_DMA_BUF_SIZE);
    }

    LED0_TOGGLE();

    delay_ms(100);
}

```

本章实验的应用代码与上一章实验中的引用代码基本一致，只不过本章实验要处理的是 2 个 ADC 通道的数据，因此 DMA 传输的目的存储器容量的定义变大了，随后便可在 DMA 传输完成后，采用与上一章实验一样的处理方式处理 ADC 采集并转换后的数据，最后将每个通道采集到电压的数字量和模拟量显示置 LCD 上。

37.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上不断地刷新显示 ADC1 的 Channel5 和 Channel6（PA5 引脚和 PA6 引脚）采集到电压的数字量和模拟量，此时可以通过杜邦线给 PA5 和 PA6 中

的任意一个或多个引脚接入不同的电压值（注意共地，且输入电压不能超过 3.3V，否则可能损坏开发板），可以看到 LCD 上显示的电压数字量和模拟量也随之改变。

第三十八章 内部温度传感器实验

本章介绍使用 ADC 采集 STM32F407 内部温度传感器输出的电压值，并根据该电压值计算出 STM32F407 芯片的大致温度。通过本章的学习，读者将学习到 ADC、内部温度传感器的使用。

本章分为如下几个小节：

- 38.1 硬件设计
- 38.2 程序设计
- 38.3 下载验证

38.1 硬件设计

38.1.1 例程功能

- 1. LCD 上不断刷新显示温度
- 2. LED0 闪烁，提示程序正在运行

38.1.2 硬件资源

- 1. LED
 - LED0 - PF9
- 2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
- 3. ADC1
 - Channel16

38.1.3 原理图

本章实验使用的内部温度传感器为 STM32F407 的片上资源，因此没有对应的连接原理图。

38.2 程序设计

38.2.1 HAL 库的 ADC 驱动

本章实验与第三十五章“单通道 ADC 采集实验”十分相似，第三十五章实验中使用 ADC2 的 Channel1 采集 PA1 引脚上的电压，而本章实验是使用 ADC1 的 Channel16 采集 STM32F407 片内温度传感器的输出电压，因此大部分的内容都是相似的，以及操作 ADC 的方式都一样，因此请见第 35.2.1 小节中 HAL 库的 ADC 驱动的相关内容。

38.2.2 ADC 驱动

本章实验的 ADC 驱动主要负责向应用层提供 ADC 的初始化以及获取内部温度传感器温度的函数。本章实验中，ADC 的驱动代码包括 adc.c 和 adc.h 两个文件。

因为本章实验的 ADC 驱动与第三十五章“单通道 ADC 采集实验”的 ADC 驱动代码十分相似，因此请参考第 35.2.1 小节 ADC 驱动的相关内容，本小节仅讲解获取内部温度传感器温度的函数，如下所示：

```
/**
 * @brief  获取内部温度传感器结果
 * @param  无
 * @retval 内部温度传感器结果（扩大 100 倍）
 */
int16_t adc_get_temperature(void)
{
    uint16_t result;
```

```
double voltage;
double temperature;
int16_t temperature_x100;

result = adc_get_result_average(ADC_CHANNEL_TEMPSENSOR, 10);
voltage = ((double)result * 3.3) / 4095;
temperature = (voltage - 0.76) * 400 + 25;
temperature_x100 = (int16_t)(temperature * 100);

return temperature_x100;
}
```

从上面的代码中可以看到，在获取温度传感器的输出电压后，需要将电压值转换为实际的温度值，转换公式涉及内部温度传感器的物理特性，由芯片的设计厂家给出。

38.2.3 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    int16_t temperature;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    uart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    lcd_init(); /* 初始化 LCD */
    adc_temperature_init(); /* 初始化 ADC 采集内部温度传感器 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "Temperature TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    lcd_show_string(30, 110, 200, 16, 16, "TEMPERATE: 00.00C", BLUE);

    while (1)
    {
        /* 获取内部温度传感器结果 */
        temperature = adc_get_temperature();

        /* 显示温度 */
        if (temperature < 0)
        {
            temperature = -temperature;
            lcd_show_string(30 + 10 * 8, 110, 16, 16, 16, "-", BLUE);
        }
        else
        {
            lcd_show_string(30 + 10 * 8, 110, 16, 16, 16, " ", BLUE);
        }
        lcd_show_xnum(30 + 11 * 8, 110, temperature / 100, 2, 16, 0, BLUE);
        lcd_show_xnum(30 + 14 * 8, 110, temperature % 100, 2, 16, 0x80, BLUE);

        LED0_TOGGLE();

        delay_ms(100);
    }
}
```

本章实验的应用代码很简单，在完成包括 ADC 之内的初始化后，便重复通过函数 `adc_get_temperature()` 获取内部温度传感器采集的温度值，并在 LCD 上实时显示。

38.3 下载验证

在完成编译和烧录操作后，便可看到 LCD 上不断地刷新显示 STM32F407 内部温度传感器采集到的温度值。

第三十九章 DAC 输出实验

本章介绍使用 STM32F407 的 DAC 输出指定的电压值。通过本章的学习。读者将学习到 DAC 的使用。

本章分为如下几个小节：

- 39.1 硬件设计
- 39.2 程序设计
- 39.3 下载验证

39.1 硬件设计

39.1.1 例程功能

1. LCD 上不断刷新显示 PA4 引脚输出电压的数字量、模拟量和 PA1 引脚输入电压的模拟量
2. 按下 WKUP 按键可以加大 PA4 引脚输出的电压，按下 KEY0 按键可以减小 PA4 引脚输出的电压
3. LED0 闪烁，提示程序正在运行

39.1.2 硬件资源

1. LED
 - LED0 - PF9
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
 - WKUP - PA0
 - KEY0 - PE4
4. ADC2
 - Channel1 - PA1
5. DAC
 - Channel1 - PA4

39.1.3 原理图

本章实验使用的 DAC 为 STM32F407 的片上资源，因此没有对应的连接原理图。

39.2 程序设计

39.2.1 HAL 库的 DAC 驱动

本章实验主要使用 DAC 的 Channel1（PA4 引脚）输出指定的电压值（0V~3.3V），其具体的步骤如下：

- ①：初始化 DAC
- ②：配置 DAC 通道
- ③：开启 DAC 转换
- ④：设置 DAC 输出值

在 HAL 库中对应的驱动函数如下：

①：初始化 DAC

该函数用于初始化 DAC 的相关参数，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_DAC_Init(DAC_HandleTypeDef *hdac);
```

该函数的形参描述，如下表所示：

形参	描述
----	----

hdac	指向 DAC 句柄的指针
------	--------------

表 39.2.1.1 函数 HAL_DAC_Init()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 39.2.1.2 函数 HAL_DAC_Init()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"
```

```
void example_fun(void)
```

```
{
```

```
    DAC_HandleTypeDef dac_handle = {0};
```

```
    /* 初始化 DAC */
```

```
    dac_handle.Instance = DAC;
```

```
    HAL_DAC_Init(&dac_handle);
```

```
}
```

②：配置 DAC 通道

该函数用于配置 DAC 通道，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_DAC_ConfigChannel(    DAC_HandleTypeDef *hdac,
                                           DAC_ChannelConfTypeDef *sConfig,
                                           uint32_t Channel);
```

该函数的形参描述，如下表所示：

形参	描述
hdac	指向 DAC 句柄的指针
sConfig	指向 DAC 通道配置结构体的指针 需自行定义，并根据 DAC 通道配置参数填充结构体中的成员变量
Channel	DAC 通道 例如：DAC_CHANNEL_1 和 DAC_CHANNEL_2（在 stm32f4xx_hal_dac.h 文件中有定义）

表 39.2.1.3 函数 HAL_DAC_ConfigChannel()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 39.2.1.4 函数 HAL_DAC_ConfigChannel()返回值描述

该函数使用 DAC_ChannelConfTypeDef 类型结构体指针传入了 DAC 通道的配置参数，该结构体的定义如下所示：

```
typedef struct
```

```
{
```

```
    uint32_t DAC_Trigger;        /* 外部触发源 */
```

```
    uint32_t DAC_OutputBuffer;  /* 输出缓冲 */
```

```
} DAC_ChannelConfTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"
```

```
void example_fun(void)
```

```
{
```

```
    DAC_ChannelConfTypeDef config = {0};
```

```
    /* 配置 DAC 通道 */
```

```
    config.DAC_Trigger = DAC_TRIGGER_SOFTWARE;
```

```
    config.DAC_OutputBuffer = DAC_OUTPUTBUFFER_DISABLE;
```

```
    HAL_DAC_ConfigChannel(&dac_handle, &config);
```

```
}
```

③：开启 DAC 转换

该函数用于开启 DAC 转换，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_DAC_Start(DAC_HandleTypeDef *hdac, uint32_t Channel);
```

该函数的形参描述，如下表所示：

形参	描述
hdac	指向 DAC 句柄的指针
Channel	DAC 通道 例如：DAC_CHANNEL_1 和 DAC_CHANNEL_2（在 stm32f4xx_hal_dac.h 文件中有定义）

表 39.2.1.5 函数 HAL_DAC_Start()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 39.2.1.6 函数 HAL_DAC_Start()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 开启 DAC 转换 */
    HAL_DAC_Start(&dac_handle, DAC_CHANNEL_1);
}
```

④：设置 DAC 输出值

该函数用于设置 DAC 输出值，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_DAC_SetValue(DAC_HandleTypeDef *hdac,
    uint32_t Channel,
    uint32_t Alignment,
    uint32_t Data);
```

该函数的形参描述，如下表所示：

形参	描述
hdac	指向 DAC 句柄的指针
Channel	DAC 通道 例如：DAC_CHANNEL_1 和 DAC_CHANNEL_2（在 stm32f4xx_hal_dac.h 文件中有定义）
Alignment	数据对齐方式 例如：DAC_ALIGN_8B_R、DAC_ALIGN_12B_L 等（在 stm32f4xx_hal_dac.h 文件中有定义）
Data	输出值

表 39.2.1.7 函数 HAL_DAC_SetValue()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 39.2.1.8 函数 HAL_DAC_SetValue()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 设置 DAC 输出值 */
    HAL_DAC_SetValue(&dac_handle, DAC_CHANNEL_1, DAC_ALIGN_12B_R, 1000);
}
```

39.2.2 DAC 驱动

本章实验的 DAC 驱动主要负责向应用层提供 DAC 的初始化和配置其输出指定电压的函数。

本章实验中，DAC 的驱动代码包括 dac.c 和 dac.h 两个文件。

DAC 驱动中，DAC 的初始化函数，如下所示：

```
/**
 * @brief 初始化 DAC
 * @param 无
 * @retval 无
 */
void dac_init(void)
{
    /* 配置 DAC */
    g_dac_handle.Instance = DAC_DACX;
    HAL_DAC_Init(&g_dac_handle);
    HAL_DAC_Start(&g_dac_handle, DAC_DACX_CHY);
}

/**
 * @brief HAL 库 DAC 初始化 MSP 函数
 * @param hdac: DAC 句柄
 * @retval 无
 */
void HAL_DAC_MspInit(DAC_HandleTypeDef *hdac)
{
    GPIO_InitTypeDef gpio_init_struct = {0};
    DAC_ChannelConfTypeDef dac_channel_conf_struct = {0};

    if (hdac->Instance == DAC_DACX)
    {
        DAC_DACX_CLK_ENABLE();
        DAC_DACX_CHY_GPIO_CLK_ENABLE();

        /* 配置 DAC 输出引脚 */
        gpio_init_struct.Pin = DAC_DACX_CHY_GPIO_PIN;
        gpio_init_struct.Mode = GPIO_MODE_ANALOG;
        gpio_init_struct.Pull = GPIO_NOPULL;
        gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
        HAL_GPIO_Init(DAC_DACX_CHY_GPIO_PORT, &gpio_init_struct);

        /* 配置 DAC 通道 */
        dac_channel_conf_struct.DAC_Trigger = DAC_TRIGGER_NONE;
        dac_channel_conf_struct.DAC_OutputBuffer = DAC_OUTPUTBUFFER_DISABLE;
        HAL_DAC_ConfigChannel(&g_dac_handle,
                              &dac_channel_conf_struct,
                              DAC_DACX_CHY);
    }
}
```

从上面的代码中可以看出，DAC 的初始化函数会初始化 DAC、配置 DAC 通道并开启 DAC 转换。

DAC 驱动中，配置 DAC 通道输出指定电压的函数，如下所示：

```
/**
 * @brief 设置 DAC 输出电压
 * @param voltage: DAC 输出电压（扩大 1000 倍）
 * @retval 无
 */
void dac_set_voltage(uint16_t voltage)
{
    uint16_t value;

    value = (voltage * 4095) / 3300;
    value &= 0xFFF;
    HAL_DAC_SetValue(&g_dac_handle, DAC_DACX_CHY, DAC_ALIGN_12B_R, value);
}
```

该函数将输入的电压模拟量转换为 DAC 输出的数字量后，将该值写入指定 DAC 通道的数

据保持寄存器。

39.2.3 实验应用代码

本章实验的应用代码，如下所示：

```
extern DAC_HandleTypeDef g_dac_handle;

int main(void)
{
    uint8_t t = 0;
    uint8_t key;
    uint16_t dac_value;
    uint16_t dac_voltage = 100;
    uint16_t adc_value;
    uint16_t adc_voltage;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟，168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    lcd_init(); /* 初始化 LCD */
    adc_init(); /* 初始化 ADC */
    dac_init(); /* 初始化 DAC */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "DAC TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    lcd_show_string(30, 110, 200, 16, 16, "WK_UP:+ KEY0:-", RED);

    lcd_show_string(30, 130, 200, 16, 16, "DAC_CH1 VAL:", BLUE);
    lcd_show_string(30, 150, 200, 16, 16, "DAC_CH1_VOL:0.000V", BLUE);
    lcd_show_string(30, 170, 200, 16, 16, "ADC2_CH1_VOL:0.000V", BLUE);

    /* 设置 DAC 输出电压 */
    dac_set_voltage(dac_voltage);

    while (1)
    {
        key = key_scan(0);
        if (key == WKUP_PRES)
        {
            /* 加大 DAC 输出 */
            if (dac_voltage < 3300)
            {
                dac_voltage += 100;
                dac_set_voltage(dac_voltage);
            }
        }
        else if (key == KEY0_PRES)
        {
            /* 减小 DAC 输出 */
            if (dac_voltage > 100)
            {
                dac_voltage -= 100;
                dac_set_voltage(dac_voltage);
            }
        }

        if (++t == 20)
        {
            t = 0;
        }
    }
}
```

```

/* 显示 DAC 输出数字量 */
dac_value = HAL_DAC_GetValue(&g_dac_handle, DAC_DACX_CHY);
lcd_show_xnum(126, 130, dac_value, 4, 16, 0, BLUE);

/* 显示 DAC 输出模拟量 */
lcd_show_xnum(126, 150, dac_voltage / 1000, 1, 16, 0, BLUE);
lcd_show_xnum(142, 150, dac_voltage % 1000, 3, 16, 0x80, BLUE);

/* 显示 ADC 采集模拟量 */
adc_value = adc_get_result_average(ADC_ADCX_CHY, 10);
adc_voltage = (adc_value * 3300) / 4095;
lcd_show_xnum(134, 170, adc_voltage / 1000, 1, 16, 0, BLUE);
lcd_show_xnum(150, 170, adc_voltage % 1000, 3, 16, 0x80, BLUE);

    LED0_TOGGLE();
}

    delay_ms(10);
}
}

```

可以看到，应用代码中不仅初始化了 DAC 的 Channel1（PA4 引脚）同时还使能了 ADC2 的 Channel1（PA1 引脚）用于采集 DAC Channel1 的输出电压。在完成初始化后，便不断地扫描按键并将 DAC 输出电压的模拟量和数字量以及 ADC 采集到电压的模拟量在 LCD 上进行显示，此时若扫描到 KEY0 按键，则减少 DAC 输出电压的数字量，若扫描到 WKUP 按键，则增加 DAC 输出电压的数字量。

39.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上实时刷新显示着 DAC 输出电压的模拟量和数字量以及 ADC 采集到电压的模拟量，此时可以将 PA4 引脚（DAC Channel1 输出引脚）和 PA1 引脚（ADC Channel1 采集引脚）通过杜邦线相连，在按下 KEY0 或 WKUP 按键来调整 DAC 通道 1 的输出电压，可以看到 LCD 上显示的 DAC 输出电压的模拟量和数字量以及 ADC 采集到电压的模拟量也随之变化，并且 ADC 采集到电压的模拟量也十分接近 DAC 输出电压的模拟量。

第四十章 DAC 输出三角波实验

本章将使用软件的方式控制 STM32F407 的 DAC 输出指定幅值、频率和个数的三角波。通过本章的学习，读者将学习到 DAC 的使用。

本章分为如下几个小节：

- 40.1 硬件设计
- 40.2 程序设计
- 40.3 下载验证

40.1 硬件设计

40.1.1 例程功能

1. 按下 WKUP 按键，PA4 引脚输出 100 个三角波 1（幅值 3.3V，频率 100Hz，采样点 2000）
2. 按下 KEY0 按键，PA4 引脚输出 100 个三角波 2（幅值 3.3V，频率 100Hz，采样点 20）
3. LED0 闪烁，提示程序正在运行

40.1.2 硬件资源

1. LED
LED0 - PF9
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
WKUP - PA0
KEY0 - PE4
4. DAC
Channel1 - PA4

40.1.3 原理图

本章实验使用的 DAC 为 STM32F407 的片上资源，因此没有对应的连接原理图。

40.2 程序设计

40.2.1 HAL 库的 DAC 驱动

本章实验与上一章实验十分相似，上一章实验使用按键控制 DAC Channel1 输出“离散”的电压，而本章实验的是使用软件算法控制 DAC Channel1 输出的电压，使之输出一个三角波，因此大部分内容都是相似的，以及操作 DAC 的方式都是一样的，因此请见第 39.2.1 小节中 HAL 库的 DAC 驱动的相关内容。

40.2.2 DAC 驱动

本章实验的 DAC 驱动主要负责向应用层提供 DAC 的初始化以及控制 DAC 输出指定幅值、频率和个数的三角波的函数。本章实验中，DAC 的驱动代码包括 dac.c 和 dac.h 两个文件。

本章实验 DAC 驱动中对 DAC 的初始化与上一章实验中对 DAC 的初始化方式一致，请见第 39.2.2 小节中 DAC 初始化的相关内容提供。本小节仅介绍通过软件控制 DAC 输出三角波的函数，如下所示：

```
/**
 * @brief    设置 DAC 输出三角波
 * @param    max_value: 三角波波峰电压数字量
 * @param    interval: 每个采样点的时间间隔，单位：微妙
```

```

* @param samples: 一个三角波周期采样点的个数
* @param number: 输出三角波的个数
* @retval 无
*/
void dac_triangular_wave( uint16_t max_value,
                          uint16_t interval,
                          uint16_t samples,
                          uint16_t number)
{
    uint16_t incval;
    uint16_t curval;
    uint16_t sample_index;
    uint16_t wave_index;

    /* 确保采样点个数为偶数 */
    samples = ((samples + 1) >> 1) << 1;
    /* 计算每个采样点的步进值 */
    incval = max_value / (samples >> 1);
    if (incval == 0)
    {
        return;
    }

    for (wave_index=0; wave_index<number; wave_index++)
    {
        curval = 0;
        HAL_DAC_SetValue(&g_dac_handle, DAC_DACX_CHY, DAC_ALIGN_12B_R, curval);

        /* 输出上升沿 */
        for (sample_index=0; sample_index<(samples >> 1); sample_index++)
        {
            curval += incval;
            HAL_DAC_SetValue( &g_dac_handle,
                              DAC_DACX_CHY,
                              DAC_ALIGN_12B_R,
                              curval);

            delay_us(interval);
        }

        /* 输出下降沿 */
        for (sample_index=0; sample_index<(samples >> 1); sample_index++)
        {
            curval -= incval;
            HAL_DAC_SetValue( &g_dac_handle,
                              DAC_DACX_CHY,
                              DAC_ALIGN_12B_R,
                              curval);

            delay_us(interval);
        }
    }
}

```

可以看到，该函数就是每间隔一段时间就修改一次 DAC 的输出电压，以控制 DAC 输出指定的三角波，对于该函数的实现，读者无需深究，仅需会使用该函数即可。

40.2.3 实验应用代码

本章实验的应用代码，如下所示：

```

int main(void)
{
    uint8_t t = 0;
    uint8_t key;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
}

```



```

delay_init(168);          /* 初始化延时 */
usart_init(115200);        /* 初始化串口 */
led_init();                /* 初始化 LED */
key_init();                /* 初始化按键 */
lcd_init();                /* 初始化 LCD */
dac_init();                /* 初始化 DAC */

lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
lcd_show_string(30, 70, 200, 16, 16, "DAC Triangular Wave TEST", RED);
lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

lcd_show_string(30, 110, 200, 16, 16, "WK_UP:Wave1 KEY0:Wave2", RED);

while (1)
{
    key = key_scan(0);
    if (key == WKUP_PRES)
    {
        /* DAC 输出三角波 1, 幅值 3.3V, 频率 100Hz, 采样点 2000 */
        lcd_show_string(30, 130, 200, 16, 16, "DAC Wave1", BLUE);
        dac_triangular_wave(4095, 5, 2000, 100);
        lcd_show_string(30, 130, 200, 16, 16, "DAC None", BLUE);
    }
    else if (key == KEY0_PRES)
    {
        /* DAC 输出三角波 2, 幅值 3.3V, 频率 100Hz, 采样点 20 */
        lcd_show_string(30, 130, 200, 16, 16, "DAC Wave2", BLUE);
        dac_triangular_wave(4095, 500, 20, 100);
        lcd_show_string(30, 130, 200, 16, 16, "DAC None", BLUE);
    }

    if (++t == 20)
    {
        t = 0;
        LED0_TOGGLE();
    }

    delay_ms(10);
}
}

```

应用代码中会初始化 DAC Channel1 (PA4 引脚), 完成初始化后, 便不断地扫描按键, 并根据扫描到的按键, 控制 DAC Channel1 输出指定幅值、频率和个数三角波。

40.3 下载验证

在完成编译和烧录操作后, 便可分别按下 KEY0 按键和 WKUP 按键控制 DAC Channel1 输出不同类型的三角波, DAC Channel1 输出的三角波可通过示波器观察 PA4 引脚 (DAC Channel1 输出引脚) 看到。

第四十一章 DAC 输出正弦波实验

本章将使用 TIM7 的更新事件用于 TRGO 触发 DAC Channel1 输出 DMA1 Stream5 Channel7 从使用软件算法生成的正弦波数据，以输出正弦波。通过本章的学习，读者将学习到 DAC、TIM、DMA 的使用。

本章分为如下几个小节：

- 41.1 硬件设计
- 41.2 程序设计
- 41.3 下载验证

41.1 硬件设计

41.1.1 例程功能

1. 按下 WKUP 按键，PA4 引脚输出正弦波 1（幅值 3.3V，频率 1KHz，采样点 100）
2. 按下 KEY0 按键，PA4 引脚输出正弦波 2（幅值 3.3V，频率 10KHz，采样点 10）
3. LED0 闪烁，提示程序正在运行

41.1.2 硬件资源

1. LED
LED0 - PF9
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
WKUP - PA0
KEY0 - PE4
4. DAC
Channel1 - PA4
5. DMA1
Stream5 Channel7
6. TIM7

41.1.3 原理图

本章实验使用的 DAC 为 STM32F407 的片上资源，因此没有对应的连接原理图。

41.2 程序设计

41.2.1 HAL 库的 TIM 驱动

本章实验中将使用 TIM7 的更新事件产生 TRGO 事件，该 TRGO 事件将用于触发 DAC，其具体的步骤如下：

- ①：初始化 TIM
- ②：配置 TIM 主模式

在 HAL 库中对应的驱动函数如下：

①：初始化 TIM

请见第 16.2.1 小节中初始化 TIM 的相关章节。

②：配置 TIM 主模式

该函数用于配置 TIM 的主模式，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_TIMEx_MasterConfigSynchronization(
    TIM_HandleTypeDef *htim,
    TIM_MasterConfigTypeDef *sMasterConfig);
```

该函数的形参描述，如下表所示：

形参	描述
htim	指向 TIM 句柄的指针
sMasterConfig	指向 TIM 主模式配置结构体的指针 需自行定义，并根据 TIM 主模式配置参数填充结构体中的成员变量

表 41.2.1.1 函数 HAL_TIMEx_MasterConfigSynchronization()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 41.2.1.2 函数 HAL_TIMEx_MasterConfigSynchronization()返回值描述

该函数使用 TIM_MasterConfigTypeDef 类型结构体指针传入了 TIM 主模式的配置参数，该结构体的定义如下所示：

```
typedef struct
{
    uint32_t MasterOutputTrigger; /* 触发输出选择 */
    uint32_t MasterSlaveMode; /* 主从模式 */
} TIM_MasterConfigTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    TIM_MasterConfigTypeDef config = {0};

    /* 配置 TIM 主模式 */
    config.MasterOutputTrigger = TIM_TRGO_RESET;
    config.MasterSlaveMode = TIM_MASTERSLAVEMODE_ENABLE;
    HAL_TIMEx_MasterConfigSynchronization(&tim_handle, &config);
}
```

41.2.2 HAL 库的 DAC 驱动

本章实验与第四十章一样，使用 DAC 通道 1（PA4 引脚）输出电压，不同之处在于，本章使用 DMA 自动将 DAC 通道 1 待输出的数据写入 DAC 的数据保持寄存器，以输出正弦波。其中对 DAC 的操作请见第四十章中 HAL 库的 DAC 驱动的相关内容，本小节仅介绍 DAC 使用 DMA 的相关步骤，其具体的步骤如下：

①：开启 DAC DMA 转换

在 HAL 库中对应的驱动函数如下：

①：开启 DAC DMA 转换

该函数用于开启 DAC DMA 转换，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_DAC_Start_DMA(DAC_HandleTypeDef *hdac,
    uint32_t Channel,
    uint32_t *pData,
    uint32_t Length,
    uint32_t Alignment);
```

该函数的形参描述，如下表所示：

形参	描述
hdac	指向 DAC 句柄的指针
Channel	DAC 通道 例如：DAC_CHANNEL_1 和 DAC_CHANNEL_2（在 stm32f4xx_hal_dac.h 文件中有定义）
pData	数据首地址
Length	数据长度
Alignment	数据对齐方式

	例如：DAC_ALIGN_8B_R、DAC_ALIGN_12B_L 等（在 stm32f4xx_hal_dac.h 文件中有定义）
--	---

表 41.2.2.1 函数()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 41.2.2.2 函数()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    uint16_t buf;

    /* 开启 DAC DMA 传输 */
    HAL_DAC_Start_DMA(&dac_handle,
        DAC_CHANNEL_1,
        buf,
        sizeof(buf) / sizeof(buf),
        DAC_ALIGN_12B_R);
}
```

41.2.3 DAC 驱动

本章实验的 DAC 驱动主要负责向应用层提供 DAC 的初始化和使能 DAC 输出指定幅值、频率的正弦波。本章实验中，DAC 的驱动代码包括 dac.c 和 dac.h 两个文件。

DAC 驱动中，DAC 的初始化函数，如下所示：

```
/**
 * @brief 初始化 DAC DMA 波形输出
 * @param 无
 * @retval 无
 */
void dac_dma_wave_init(void)
{
    /* 配置 DAC */
    g_dac_dma_handle.Instance = DAC_DMA_DACX;
    HAL_DAC_Init(&g_dac_dma_handle);
}

/**
 * @brief HAL 库 DAC 初始化 MSP 函数
 * @param hdac: DAC 句柄
 * @retval 无
 */
void HAL_DAC_MspInit(DAC_HandleTypeDef *hdac)
{
    GPIO_InitTypeDef gpio_init_struct = {0};
    DAC_ChannelConfTypeDef dac_channel_conf_struct = {0};
    TIM_MasterConfigTypeDef tim_master_config_struct = {0};

    if (hdac->Instance == DAC_DMA_DACX)
    {
        DAC_DMA_DACX_CLK_ENABLE();
        DAC_DMA_DACX_CHY_GPIO_CLK_ENABLE();
        DAC_DMA_DACX_DMA_CLK_ENABLE();
        DAC_DMA_DACX_TIMX_CLK_ENABLE();

        /* 配置 DAC 输出引脚 */
        gpio_init_struct.Pin = DAC_DMA_DACX_CHY_GPIO_PIN;
        gpio_init_struct.Mode = GPIO_MODE_ANALOG;
        gpio_init_struct.Pull = GPIO_NOPULL;
```

```

gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
HAL_GPIO_Init(DAC_DMA_DACX_CHY_GPIO_PORT, &gpio_init_struct);

/* 配置 DAC 通道 */
dac_channel_conf_struct.DAC_Trigger = DAC_TRIGGER_T7_TRGO;
dac_channel_conf_struct.DAC_OutputBuffer = DAC_OUTPUTBUFFER_DISABLE;
HAL_DAC_ConfigChannel(hdac, &dac_channel_conf_struct, DAC_DMA_DACX_CHY);

/* 配置 DMA */
g_dac_dma_dma_handle.Instance = DAC_DMA_DACX_DMASX;
g_dac_dma_dma_handle.Init.Channel = DAC_DMA_DACX_DMASX_CHY;
g_dac_dma_dma_handle.Init.Direction = DMA_MEMORY_TO_PERIPH;
g_dac_dma_dma_handle.Init.PeriphInc = DMA_PINC_DISABLE;
g_dac_dma_dma_handle.Init.MemInc = DMA_MINC_ENABLE;
g_dac_dma_dma_handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_HALFWORD;
g_dac_dma_dma_handle.Init.MemDataAlignment = DMA_MDATAALIGN_HALFWORD;
g_dac_dma_dma_handle.Init.Mode = DMA_CIRCULAR;
g_dac_dma_dma_handle.Init.Priority = DMA_PRIORITY_VERY_HIGH;
g_dac_dma_dma_handle.Init.FIFOMode = DMA_FIFOMODE_DISABLE;
g_dac_dma_dma_handle.Init.FIFOThreshold =
    DMA_FIFO_THRESHOLD_1QUARTERFULL;
g_dac_dma_dma_handle.Init.MemBurst = DMA_MBURST_SINGLE;
g_dac_dma_dma_handle.Init.PeriphBurst = DMA_PBURST_SINGLE;
HAL_DMA_Init(&g_dac_dma_dma_handle);

__HAL_LINKDMA(hdac, DMA_Handle1, g_dac_dma_dma_handle);

HAL_NVIC_SetPriority(DAC_DMA_DACX_DMASX_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(DAC_DMA_DACX_DMASX_IRQn);

/* 配置 TIM */
g_dac_dma_tim_handle.Instance = DAC_DMA_DACX_TIMX;
HAL_TIM_Base_Init(&g_dac_dma_tim_handle);

tim_master_config_struct.MasterOutputTrigger = TIM_TRGO_UPDATE;
HAL_TIMEx_MasterConfigSynchronization(&g_dac_dma_tim_handle,
    &tim_master_config_struct);
}
}

```

可以看到本章实验 DAC 驱动中的 DAC 初始化函数与第三十九章“DAC 输出实验”中对 DAC 的初始化基本一致，不过本章配置 DAC Channel1 的触发源为 TIM7 的 TRGO 事件，同时还配置了 DMA。

DAC 驱动中使能 DAC 输出指定幅值、频率的正弦波的函数，如下所示：

```

/**
 * @brief 使能 DAC DMA 波形输出
 * @param ndtr: DMA 数据流传输一次数据项的数目
 * @param arr: DAC 触发定时器的自动重装载值
 * @param psc: DAC 触发定时器的预分频器数值
 * @retval 无
 */
void dac_dma_wave_enable(uint16_t ndtr, uint16_t arr, uint16_t psc)
{
    /* 配置 TIM */
    g_dac_dma_tim_handle.Init.Prescaler = psc;
    g_dac_dma_tim_handle.Init.Period = arr;
    HAL_TIM_Base_Init(&g_dac_dma_tim_handle);
    HAL_TIM_Base_Start(&g_dac_dma_tim_handle);

    /* 重新开启 DAC DMA 传输 */
    HAL_DAC_Stop_DMA(&g_dac_dma_handle, DAC_DMA_DACX_CHY);
    HAL_DAC_Start_DMA(&g_dac_dma_handle,
        DAC_DMA_DACX_CHY,
        (uint32_t *)g_dac_sin_buf,
        ndtr,

```

```
DAC_ALIGN_12B_R);
```

该函数配置了 TIM7 的溢出频率,因此 TIM7 的溢出频率就决定了 DAC 输出正弦波的频率,同时也开启了 DAC DMA 传输。

41.2.4 实验应用代码

本章实验的应用代码,如下所示:

```
/* 正弦波数据缓冲区 */
uint16_t g_dac_sin_buf[4096];

/**
 * @brief 生成正弦数据
 * @param maxval : 正弦的振幅
 * @param samples: 正弦一个周期的采样点个数
 * @retval 无
 */
static void dac_creat_sin_buf(uint16_t maxval, uint16_t samples)
{
    uint16_t i;
    double w;
    uint16_t outdata;

    w = (2 * 3.1415926) / samples; /*  $\omega=2\pi/T$  */

    for (i=0; i<samples; i++)
    {
        outdata = maxval * sin(w * i + 0) + maxval; /*  $y=A\sin(\omega x+\phi)+b$  */
        if (outdata > 4095) /* 限制上限 */
        {
            outdata = 4095;
        }
        g_dac_sin_buf[i] = outdata;
    }
}

int main(void)
{
    uint8_t t = 0;
    uint8_t key;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    lcd_init(); /* 初始化 LCD */
    dac_dma_wave_init(); /* 初始化 DAC DMA 波形输出 */
    dac_creat_sin_buf(2048 - 1, 100); /* 生成正弦数据 */
    dac_dma_wave_enable(100, 10 - 1, 84 - 1); /* 使能 DAC DMA 波形输出 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "DAC DMA Sine Wave TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    lcd_show_string(30, 110, 200, 16, 16, "WK_UP:Wave1 KEY0:Wave2", RED);

    while (1)
    {
        key = key_scan(0);
        if (key == WKUP_PRES)
        {

```

```

        /* 生成正弦数据 */
        dac_creat_sin_buf(2048 - 1, 100);
        /* 使能 DAC DMA 波形输出 */
        dac_dma_wave_enable(100, 10 - 1, 84 - 1);
    }
    else if (key == KEY0_PRES)
    {
        /* 生成正弦数据 */
        dac_creat_sin_buf(2048 - 1, 10);
        /* 使能 DAC DMA 波形输出 */
        dac_dma_wave_enable(10, 10 - 1, 84 - 1);
    }

    if (++t == 20)
    {
        t = 0;
        LED0_TOGGLE();
    }

    delay_ms(10);
}
}

```

可以看到，在实验应用代码中，定义了函数 `dac_creat_sin_buf()`，该函数用于生成正弦波数据，并保存至数组 `g_dac_sin_buf` 中。在完成 DAC 初始化后，便生成了一组正弦波数据，并调用函数 `dac_dma_wave_enable()` 使能 DAC Channel1 输出指定的正弦波，随后便通过扫描到的不同按键值，输出不同频率的正弦波。

41.3 下载验证

在完成编译和烧录操作后，便可分别按下 KEY0 按键和 WKUP 按键控制 DAC Channel1 输出不同类型的正弦波，DAC Channel1 输出的正弦波可通过示波器观察 PA4 引脚（DAC Channel1 输出引脚）看到。

第四十二章 IIC 实验

本章将介绍使用 STM32F407 驱动板载的 EEPROM 进行读写操作。通过本章的学习，读者将学习到使用 GPIO 模拟 IIC 时序以及 EEPROM 的驱动。

本章分为如下几个小节：

- 42.1 硬件设计
- 42.2 程序设计
- 42.3 下载验证

42.1 硬件设计

42.1.1 例程功能

1. 按下 WKUP 按键可往 24C02 写入 “STM32 IIC TEST”
2. 按下 KEY0 按键可从 24C02 读取到 “STM32 IIC TEST”，并显示至 LCD
3. LED0 闪烁，提示程序正在运行

42.1.2 硬件资源

1. LED
LED0 - PF9
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
WKUP - PA0
KEY0 - PE4
4. 24C02
IIC_SCL - PB8
IIC_SDA - PB9

42.1.3 原理图

本章实验使用了板载的 24C02 芯片，该芯片是一个 EEPROM，MCU 是通过两个 GPIO 与该 EEPROM 进行连接与通信的，该 EEPROM 与 MCU 的连接原理图，如下图所示：

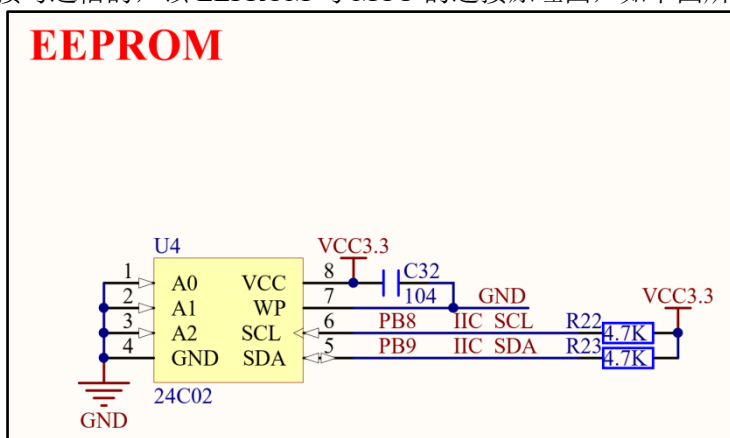


图 42.1.3.1 EEPROM 与 MCU 的连接原理图

42.2 程序设计

42.2.1 HAL 库的 GPIO 驱动

本章实验通过控制与 EEPROM 连接的 GPIO 模拟 IIC 时序，实现 EEPROM 的读写操作，对于 GPIO，主要涉及 GPIO 的配置和读写操作，需要一下几个步骤：

- ①：配置 GPIO 引脚为通用输出模式、开漏输出和禁止上拉/下拉
 - ②：在与 EEPROM 通信时，根据需求控制指定 GPIO 引脚输出指定电平
 - ③：在与 EEPROM 通信时，根据需求读取指定 GPIO 引脚上的输入电平
- 在 HAL 库中对应的驱动函数如下：

①：配置 GPIO 引脚

请见第 10.2.1 小节中配置 GPIO 引脚的相关内容。

②：设置 GPIO 引脚输出电平

请见第 10.2.1 小节中设置 GPIO 引脚输出电平的相关内容。

③：读取 GPIO 引脚输入电平

请见第 11.2.1 小节中读取 GPIO 引脚输入电平的相关内容。

42.2.2 IIC 驱动

本章实验使用的 IIC 的软件模拟的 IIC，即控制 GPIO 模拟 IIC 的时序与外部器件进行通信。IIC 驱动主要负责向 EEPROM 驱动提供 IIC 操作的各种函数，例如：IIC 起始信号、IIC 停止信号等。本章实验中，IIC 的驱动代码包括 myiic.c 和 myiic.h 两个文件。

IIC 驱动中，对 GPIO 相关的宏定义，如下所示：

```
#define IIC_SCL_GPIO_PORT    GPIOB
#define IIC_SCL_GPIO_PIN    GPIO_PIN_8
#define IIC_SCL_GPIO_CLK_ENABLE() \
do { \
    __HAL_RCC_GPIOB_CLK_ENABLE(); \
} while (0)
#define IIC_SDA_GPIO_PORT    GPIOB
#define IIC_SDA_GPIO_PIN    GPIO_PIN_9
#define IIC_SDA_GPIO_CLK_ENABLE() \
do { \
    __HAL_RCC_GPIOB_CLK_ENABLE(); \
} while (0)
#define IIC_SCL(x) \
do { (x) ? \
    HAL_GPIO_WritePin(IIC_SCL_GPIO_PORT, IIC_SCL_GPIO_PIN, GPIO_PIN_SET); \
    HAL_GPIO_WritePin(IIC_SCL_GPIO_PORT, IIC_SCL_GPIO_PIN, GPIO_PIN_RESET); \
} while (0)
#define IIC_SDA(x) \
do { (x) ? \
    HAL_GPIO_WritePin(IIC_SDA_GPIO_PORT, IIC_SDA_GPIO_PIN, GPIO_PIN_SET); \
    HAL_GPIO_WritePin(IIC_SDA_GPIO_PORT, IIC_SDA_GPIO_PIN, GPIO_PIN_RESET); \
} while (0)
#define IIC_SDA_READ ( \
    (HAL_GPIO_ReadPin(IIC_SDA_GPIO_PORT, IIC_SDA_GPIO_PIN) == GPIO_PIN_RESET) ? \
    0 : \
    1)
```

IIC 驱动中，IIC 的初始化函数，如下所示：

```
/**
 * @brief    f 初始化 IIC
 * @param    无
 * @retval    无
 */
void iic_init(void)
{
    GPIO_InitTypeDef gpio_init_struct = {0};
```

```

/* 时钟使能 */
IIC_SCL_GPIO_CLK_ENABLE();
IIC_SDA_GPIO_CLK_ENABLE();

/* 配置 SCL 引脚 */
gpio_init_struct.Pin = IIC_SCL_GPIO_PIN;
gpio_init_struct.Mode = GPIO_MODE_OUTPUT_OD;
gpio_init_struct.Pull = GPIO_NOPULL;
gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
HAL_GPIO_Init(IIC_SCL_GPIO_PORT, &gpio_init_struct);

/* 配置 SDA 引脚 */
gpio_init_struct.Pin = IIC_SDA_GPIO_PIN;
gpio_init_struct.Mode = GPIO_MODE_OUTPUT_OD;
gpio_init_struct.Pull = GPIO_NOPULL;
gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
HAL_GPIO_Init(IIC_SDA_GPIO_PORT, &gpio_init_struct);

iic_stop();
}

```

可以看到，IIC 的初始化实际上就是配置 IIC 时钟与数据信号对应的 GPIO 引脚。

IIC 驱动中对 IIC 的各种操作，例如产生 IIC 起始信号、产生 IIC 停止信号等，请读者结合 IIC 的时序规定查看本实验的配套实验源码。

42.2.3 EEPROM 驱动

本章实验的 EEPROM 驱动主要负责向应用层提供 EEPROM 的初始化和读写数据等操作函数。本章实验中，EEPROM 的驱动代码包括 24cxx.c 和 24cxx.h 两个文件。

EEPROM 驱动中，EEPROM 的初始化函数，如下所示：

```

/**
 * @brief  初始化 AT24CXX
 * @param  无
 * @retval 无
 */
void at24cxx_init(void)
{
    iic_init();
}

```

可以看到，该函数实际上就是初始化了与 EEPROM 通讯的 IIC。

EEPROM 驱动中的其他对 EEPROM 的操作函数，例如：EEPROM 的读写函数，请读者结合 24C02 EEPROM 芯片的数据手册查看本实验的配套实验源码。

42.2.4 实验应用代码

本章实验的应用代码，如下所示：

```

static const uint8_t g_text_buf[] = {"STM32 IIC TEST"};
#define TEXT_SIZE sizeof(g_text_buf)

int main(void)
{
    uint8_t t = 0;
    uint8_t key;
    uint8_t data[TEXT_SIZE];

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    lcd_init(); /* 初始化 LCD */
}

```

```

at24cxx_init();                               /* 初始化 AT24CXX */

lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
lcd_show_string(30, 70, 200, 16, 16, "IIC TEST", RED);
lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

lcd_show_string(30, 110, 200, 16, 16, "WK_UP:Write KEY0:Read", RED);

/* 检测 AT24CXX 是否正常 */
while (at24cxx_check() != 0)
{
    lcd_show_string(30, 130, 200, 16, 16, "24C02 Check Failed!", RED);
    delay_ms(500);
    lcd_show_string(30, 130, 200, 16, 16, "Please Check!      ", RED);
    delay_ms(500);
    LED0_TOGGLE();
}
lcd_show_string(30, 130, 200, 16, 16, "24C02 Ready!      ", RED);

while (1)
{
    key = key_scan();
    if (key == WKUP_PRES)
    {
        /* 往 AT24CXX 写入数据 */
        lcd_fill(0, 150, 239, 319, WHITE);
        lcd_show_string(30, 150, 200, 16, 16, "Start Write 24C02....", BLUE);
        at24cxx_write(0, (uint8_t *)g_text_buf, TEXT_SIZE);
        lcd_show_string(30, 150, 200, 16, 16, "24C02 Write Finished!", BLUE);
    }
    else if (key == KEY0_PRES)
    {
        /* 从 AT24CXX 读取数据 */
        lcd_show_string(30, 150, 200, 16, 16, "Start Read 24C02.... ", BLUE);
        at24cxx_read(0, data, TEXT_SIZE);
        lcd_show_string(30, 150, 200, 16, 16, "The Data Readed Is: ", BLUE);
        lcd_show_string(30, 170, 200, 16, 16, (char *)data, BLUE);
    }

    if (++t == 20)
    {
        t = 0;
        LED0_TOGGLE();
    }

    delay_ms(10);
}
}

```

从本章实验的应用代码中可以看到，在初始化完 EEPROM 后，会检测与 EEPROM 的连接是否正常，若与 EEPROM 的连接正常，则会不断地等待按键输入，若检测到 WKUP 按键被按下，则会往 EEPROM 的指定地址中写入指定的数据，若检测到 KEY_0 按键被按下，则会从 EEPROM 的指定地址中读取数据，并在 LCD 上进行显示。

42.3 下载验证

在完成编译和烧录操作后，若 MCU 与 EEPROM 的连接无误，则可以在 LCD 上看到“24C02 Ready!”的提示信息，此时可以按下 WKUP 按键往 EEPROM 的指定地址写入指定数据，然后再按下 KEY_0 按键从 EEPROM 的指定地址将写入的数据读回来在 LCD 上进行显示，此时便可以看到在 LCD 上显示了“STM32 IIC TEST”的提示信息，该提示信息就是从 EEPROM 中读回的数据。

第四十三章 SPI 实验

本章将介绍使用 STM32F407 驱动板载的 NOR Flash 进行读写操作。通过本章的学习，读者将学习到使用 SPI 驱动 NOR Flash 的使用。

本章分为如下几个小节：

- 43.1 硬件设计
- 43.2 程序设计
- 43.3 下载验证

43.1 硬件设计

43.1.1 例程功能

1. 按下 WKUP 按键可往 NOR Flash 写入 “STM32 SPI TEST”
2. 按下 KEY0 按键可从 NOR Flash 读取到 “STM32 SPI TEST”，并显示至 LCD
3. LED0 闪烁，提示程序正在运行

43.1.2 硬件资源

1. LED
 - LED0 - PF9
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
 - WKUP - PA0
 - KEY0 - PE4
5. NOR Flash
 - SPI1_SCK - PB3
 - SPI1_MISO - PB4
 - SPI1_MOSI - PB5
 - SPI1_CS - PB14

43.1.3 原理图

本章实验使用了板载的 25Q128 芯片，该芯片是一个 NOR Flash，MCU 是通过 SPI 与该 NOR Flash 进行连接与通信的，该 NOR Flash 与 MCU 的连接原理图，如下如图所示：

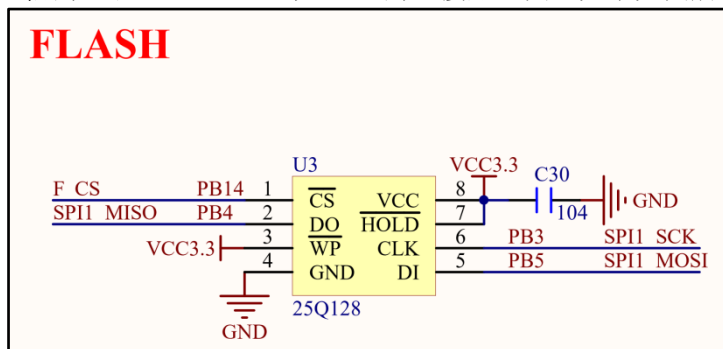


图 43.1.3.1 NOR Flash 与 MCU 的连接原理图

43.2 程序设计

43.2.1 HAL 库的 SPI 驱动

本章实验通过 SPI1 驱动 NOR Flash，因此需要对 SPI1 进行相应的配置，并使用 SPI1 与 NOR

Flash 进行通信，其具体的步骤如下所示：

- ①：初始化 SPI
- ②：SPI 发送并接收数据

在 HAL 库中对应的驱动函数如下：

①：初始化 SPI

该函数用于初始化 SPI，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_SPI_Init(SPI_HandleTypeDef *hspi);
```

该函数的形参描述，如下表所示：

形参	描述
hspi	指向 SPI 句柄的指针

表 43.2.1.1 函数 HAL_SPI_Init()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 43.2.1.2 函数 HAL_SPI_Init()返回值描述

该函数需要传入 SPI 的句柄指针，该句柄中就包含了 SPI 的初始化配置参数结构体，该结构体的定义如下所示：

```
typedef struct
{
    uint32_t Mode; /* 模式 */
    uint32_t Direction; /* 方向 */
    uint32_t DataSize; /* 数据位宽 */
    uint32_t CLKPolarity; /* 时钟极性 */
    uint32_t CLKPhase; /* 时钟相位 */
    uint32_t NSS; /* 片选 */
    uint32_t BaudRatePrescaler; /* 波特率分频系数 */
    uint32_t FirstBit; /* 高位 */
    uint32_t TIMode; /* 帧格式 */
    uint32_t CRCCalculation; /* CRC 使能 */
    uint32_t CRCPolynomial; /* CRC 多项式 */
} SPI_InitTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    SPI_HandleTypeDef spi_handle = {0};

    /* 初始化 SPI */
    spi_handle.Instance = SPI1;
    spi_handle.Init.Mode = SPI_MODE_MASTER;
    spi_handle.Init.Direction = SPI_DIRECTION_2LINES;
    spi_handle.Init.DataSize = SPI_DATASIZE_8BIT;
    spi_handle.Init.CLKPolarity = SPI_POLARITY_HIGH;
    spi_handle.Init.CLKPhase = SPI_PHASE_2EDGE;
    spi_handle.Init.NSS = SPI_NSS_SOFT;
    spi_handle.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_256;
    spi_handle.Init.FirstBit = SPI_FIRSTBIT_MSB;
    spi_handle.Init.TIMode = SPI_TIMODE_DISABLE;
    spi_handle.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
    spi_handle.Init.CRCPolynomial = 1;
    HAL_SPI_Init(&spi_handle);
}
```

②：SPI 发送并接收数据

该函数用于 SPI 发送并接收数据，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_SPI_TransmitReceive(SPI_HandleTypeDef *hspi,
                                           uint8_t *pTxData,
                                           uint8_t *pRxData,
```

```
uint16_t Size,
uint32_t Timeout);
```

该函数的形参描述，如下表所示：

形参	描述
hspi	指向 SPI 句柄的指针
pTxData	待发送数据首地址
pRxData	存放接收数据的首地址
Size	数据量
Timeout	超时等待时间

表 43.2.1.3 函数 HAL_SPI_TransmitReceive()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 43.2.1.4 函数 HAL_SPI_TransmitReceive()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    uint8_t txbuf;
    uint8_t rxbuf;

    /* SPI 发送并接收数据 */
    HAL_SPI_TransmitReceive(&spi_handle, &txbuf, &rxbuf, 1, HAL_MAX_DELAY);
}
```

43.2.2 SPI 驱动

本章实验的 SPI 驱动主要负责向 NOR Flash 驱动提供 SPI 的各种操作函数，例如：SPI 初始化、SPI 读写等。本章实验中，SPI 的驱动代码包括 spi.c 和 spi.h 两个文件。

SPI 驱动中，对 SPI、GPIO 相关的宏定义，如下所示：

```
#define SPI1_SPI SPI1
#define SPI1_SPI_CLK_ENABLE() \
    do { \
        __HAL_RCC_SPI1_CLK_ENABLE(); \
    } while (0)
#define SPI1_SCK_GPIO_PORT GPIOB
#define SPI1_SCK_GPIO_PIN GPIO_PIN_3
#define SPI1_SCK_GPIO_AF GPIO_AF5_SPI1
#define SPI1_SCK_GPIO_CLK_ENABLE() \
    do { \
        __HAL_RCC_GPIOB_CLK_ENABLE(); \
    } while (0)
#define SPI1_MISO_GPIO_PORT GPIOB
#define SPI1_MISO_GPIO_PIN GPIO_PIN_4
#define SPI1_MISO_GPIO_AF GPIO_AF5_SPI1
#define SPI1_MISO_GPIO_CLK_ENABLE() \
    do { \
        __HAL_RCC_GPIOB_CLK_ENABLE(); \
    } while (0)
#define SPI1_MOSI_GPIO_PORT GPIOB
#define SPI1_MOSI_GPIO_PIN GPIO_PIN_5
#define SPI1_MOSI_GPIO_AF GPIO_AF5_SPI1
#define SPI1_MOSI_GPIO_CLK_ENABLE() \
    do { \
        __HAL_RCC_GPIOB_CLK_ENABLE(); \
    } while (0)
```

SPI 驱动中，SPI 的初始化函数，如下所示：

```
/**
 * @brief 初始化 SPI1
```

```

* @param  无
* @retval 无
*/
void spil_init(void)
{
    /* 配置 SPI1 */
    g_spil_handle.Instance = SPI1_SPI;
    g_spil_handle.Init.Mode = SPI_MODE_MASTER;
    g_spil_handle.Init.Direction = SPI_DIRECTION_2LINES;
    g_spil_handle.Init.DataSize = SPI_DATASIZE_8BIT;
    g_spil_handle.Init.CLKPolarity = SPI_POLARITY_HIGH;
    g_spil_handle.Init.CLKPhase = SPI_PHASE_2EDGE;
    g_spil_handle.Init.NSS = SPI_NSS_SOFT;
    g_spil_handle.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_256;
    g_spil_handle.Init.FirstBit = SPI_FIRSTBIT_MSB;
    g_spil_handle.Init.TIMode = SPI_TIMODE_DISABLE;
    g_spil_handle.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
    g_spil_handle.Init.CRCPolynomial = 1;
    HAL_SPI_Init(&g_spil_handle);
}

/**
 * @brief  HAL 库 SPI1 初始化 MSP 函数
 * @param  无
 * @retval 无
 */
void HAL_SPI_MspInit(SPI_HandleTypeDef *hspi)
{
    GPIO_InitTypeDef gpio_init_struct = {0};

    if (hspi->Instance == SPI2_SPI)
    {
        /* 时钟使能 */
        SPI1_SPI_CLK_ENABLE();
        SPI1_SCK_GPIO_CLK_ENABLE();
        SPI1_MISO_GPIO_CLK_ENABLE();
        SPI1_MOSI_GPIO_CLK_ENABLE();

        /* 配置 SCK 引脚 */
        gpio_init_struct.Pin = SPI1_SCK_GPIO_PIN;
        gpio_init_struct.Mode = GPIO_MODE_AF_PP;
        gpio_init_struct.Pull = GPIO_PULLUP;
        gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
        gpio_init_struct.Alternate = SPI1_SCK_GPIO_AF;
        HAL_GPIO_Init(SPI1_SCK_GPIO_PORT, &gpio_init_struct);

        /* 配置 MISO 引脚 */
        gpio_init_struct.Pin = SPI1_MISO_GPIO_PIN;
        gpio_init_struct.Mode = GPIO_MODE_AF_PP;
        gpio_init_struct.Pull = GPIO_PULLUP;
        gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
        gpio_init_struct.Alternate = SPI1_MISO_GPIO_AF;
        HAL_GPIO_Init(SPI1_MISO_GPIO_PORT, &gpio_init_struct);

        /* 配置 MOSI 引脚 */
        gpio_init_struct.Pin = SPI1_MOSI_GPIO_PIN;
        gpio_init_struct.Mode = GPIO_MODE_AF_PP;
        gpio_init_struct.Pull = GPIO_PULLUP;
        gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
        gpio_init_struct.Alternate = SPI1_MOSI_GPIO_AF;
        HAL_GPIO_Init(SPI1_MOSI_GPIO_PORT, &gpio_init_struct);
    }
}

```

可以看到，该函数会初始化 SPI1。

SPI 驱动中，使用 SPI 传输 1 字节数据的函数，如下所示：

```
/**
 * @brief   SPI1 读写一字节数据
 * @param   txdata: 待写入的一字节数据
 * @retval   读取到的一字节数据
 */
uint8_t spi1_read_write_byte(uint8_t txdata)
{
    uint8_t rxdata;

    if (HAL_SPI_TransmitReceive(&g_spi1_handle, &txdata, &rxdata, 1, 1000) !=
        HAL_OK)
    {
        return 0;
    }

    return rxdata;
}
```

该函数使用函数 HAL_SPI_TransmitReceive() 发送并接收一个字节数据。

43.2.3 NOR Flash 驱动

本章实验的 NOR Flash 驱动主要负责向应用层提供 NOR Flash 的初始化和读写操作等函数。本章实验中，NOR Flash 的驱动代码包括 norflash.c 和 norflash.h 两个文件。

NOR Flash 驱动中，对 GPIO 的相关宏定义，如下所示：

```
#define NORFLASH_CS_GPIO_PORT  GPIOB
#define NORFLASH_CS_GPIO_PIN  GPIO_PIN_14
#define NORFLASH_CS_GPIO_CLK_ENABLE() \
    do { \
        __HAL_RCC_GPIOB_CLK_ENABLE(); \
    } while (0)
#define NORFLASH_CS(x) do { (x) ? \
    HAL_GPIO_WritePin( NORFLASH_CS_GPIO_PORT, \
        NORFLASH_CS_GPIO_PIN, \
        GPIO_PIN_SET); \
    HAL_GPIO_WritePin( NORFLASH_CS_GPIO_PORT, \
        NORFLASH_CS_GPIO_PIN, \
        GPIO_PIN_RESET); \
    } while (0)
```

NOR Flash 驱动中，初始化 NOR Flash 的函数，如下所示：

```
/**
 * @brief   初始化 NOR Flash
 * @param   无
 * @retval   无
 */
void norflash_init(void)
{
    GPIO_InitTypeDef gpio_init_struct = {0};
    uint8_t temp;

    /* 时钟使能 */
    NORFLASH_CS_GPIO_CLK_ENABLE();

    /* 配置 CS 引脚 */
    gpio_init_struct.Pin = NORFLASH_CS_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;
    gpio_init_struct.Pull = GPIO_PULLUP;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(NORFLASH_CS_GPIO_PORT, &gpio_init_struct);
    NORFLASH_CS(1);

    /* 配置 SPI 接口 */
    spi1_init();
}
```



```

spi1_set_speed(SPI_BAUDRATEPRESCALER_4);

g_norflash_type = norflash_read_id(); /* 读取 NOR Flash 芯片 ID */
if (g_norflash_type == W25Q256) /* W25Q256 需使能 4 字节地址模式 */
{
    temp = norflash_read_sr(3); /* 读状态寄存器 3，判断地址模式 */
    if ((temp & 0x01) == 0) /* 如果不是 4 字节地址模式，则需进行相应配置 */
    {
        norflash_write_enable(); /* NOR Flash 写使能 */
        temp |= (1 << 1); /* ADP=1，配置上电 4 字节地址模式 */
        norflash_write_sr(3, temp); /* 写状态寄存器 3 */

        NORFLASH_CS(0); /* 使能 NOR Flash 片选 */
        spi2_read_write_byte(NORFLASH_Enable4ByteAddr); /* 使能 4 字节地址模式 */
        NORFLASH_CS(1); /* 失能 NOR Flash 片选 */
    }
}
}

```

可以看到，在 NOR Flash 的初始化函数中，先初始化了控制 NOR Flash 片选的 GPIO 引脚，然后再是初始化与 NOR Flash 通讯的 SPI 并配置其通讯波特率，最后还会根据不同容量的 NOR Flash 做相应的配置操作。

NOR Flash 驱动中其他对 NOR Flash 的操作函数，例如，NOR Flash 的读写函数、擦除函数等，请读者结合 25Q128 NOR Flash 芯片的数据手册查看本实验的配套实验源码。

43.2.4 实验应用代码

本章实验的应用代码，如下所示：

```

static const uint8_t g_text_buf[] = {"STM32 SPI TEST"};
#define TEXT_SIZE sizeof(g_text_buf)

int main(void)
{
    uint8_t t = 0;
    uint8_t key;
    uint8_t data[TEXT_SIZE];
    uint16_t id;
    uint32_t flashsize;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟，168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    lcd_init(); /* 初始化 LCD */
    norflash_init(); /* 初始化 NOR Flash */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "SPI TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    lcd_show_string(30, 110, 200, 16, 16, "WK_UP:Write KEY0:Read", RED);

    id = norflash_read_id();
    while ((id == 0) || (id == 0xFFFF))
    {
        lcd_show_string(30, 130, 200, 16, 16, "NOR Flash Check Failed!", RED);
        delay_ms(500);
        lcd_show_string(30, 130, 200, 16, 16, "Please Check! ", RED);
        delay_ms(500);
        LED0_TOGGLE();
    }
}

```

```

lcd_show_string(30, 130, 200, 16, 16, "NOR Flash Ready!", RED);
flashsize = 16 * 1024 * 1024;

while (1)
{
    key = key_scan(0);
    if (key == WKUP_PRES)
    {
        /* 往 NOR Flash 写入数据 */
        lcd_fill(0, 150, 239, 319, WHITE);
        lcd_show_string(30, 150, 200, 16, 16, "Start Write Flash...", BLUE);
        norflash_write((uint8_t *)g_text_buf,
                        flashsize - TEXT_SIZE,
                        TEXT_SIZE);
        lcd_show_string(30, 150, 200, 16, 16, "Flash Write Finished!", BLUE);
    }
    else if (key == KEY0_PRES)
    {
        /* 从 NOR Flash 读取数据 */
        lcd_show_string(30, 150, 200, 16, 16, "Start Read Flash.... ", BLUE);
        norflash_read(data, flashsize - TEXT_SIZE, TEXT_SIZE);
        lcd_show_string(30, 150, 200, 16, 16, "The Data Readed Is: ", BLUE);
        lcd_show_string(30, 170, 200, 16, 16, (char *)data, BLUE);
    }

    if (++t == 20)
    {
        t = 0;
        LED0_TOGGLE();
    }

    delay_ms(10);
}
}

```

从本章实验的应用代码中可以看到，在初始化完 NOR Flash 后，会检测与 NOR Flash 的连接是否正常，若与 NOR Flash 的连接正常，则会不断地等待按键输入，若检测到 WKUP 按键被按下，则会往 NOR Flash 的指定地址中写入指定的数据，若检测 KEY0 按键被按下，则会从 NOR Flash 的指定地址中读取数据，并在 LCD 上进行显示。

43.3 下载验证

在完成编译和烧录操作后，若 MCU 与 NOR Flash 的连接无误，则可以在 LCD 上看到“NOR Flash Ready!”的提示信息，此时可以按下 WKUP 按键往 NOR Flash 的指定地址写入指定的数据，然后再按下 KEY_0 按键从 NOR Flash 的指定地址将写入的数据读回来在 LCD 上进行显示，此时便可以看到 LCD 上显示“STM32 SPI TEST”的提示信息，该提示信息就是从 NOR Flash 中读回的数据。

第四十四章 触摸屏实验

本章将介绍使用 STM32F407 驱动 TFTLCD 模块上的触摸屏，实现一个类似画板的应用。通过本章的学习，读者将学习到使用 GPIO 模拟 IIC 和 SPI 时序以及触摸屏的驱动。

本章分为如下几个小节：

- 44.1 硬件设计
- 44.2 程序设计
- 44.3 下载验证

44.1 硬件设计

44.1.1 例程功能

1. LCD 上实时显示触摸屏被触摸的触摸轨迹，并可通过触摸右上角的 RST 来清空轨迹
2. 按下 KEY0 按键可进行电阻触摸屏的触摸校准
3. LED0 闪烁，提示程序正在运行

44.1.2 硬件资源

1. LED
LED0 - PF9
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
WKUP - PA0
4. 24C02
IIC_SCL - PB8
IIC_SDA - PB9

44.1.3 原理图

本章实验使用了正点原子的 TFTLCD 模块（兼容正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块），该模块需通过 LCD 转接板与板载的 TFTLCD 接口进行连接，该接口与板载 MCU 的连接原理图，如图 26.1.3.1 所示。

如图 26.1.3.1 所示，TFTLCD 模块上的触摸屏使用 T_MISO、T_MOSI、T_PEN、T_CS、T_SCK 信号与 MCU 进行通讯，正点原子的 TFTLCD 模块采用了两种触摸屏，分别为：电阻式触摸屏和电容式触摸屏，其中电阻式触摸屏使用 SPI 协议与 MCU 进行通信，而电容式触摸屏则是使用 IIC 协议与 MCU 进行通讯，因此 TFT 模块上的触摸屏引脚对于不同的触摸屏有不同的引脚定义，请读者自行查看 TFTLCD 模块的用户手册查看具体的引脚定义。

44.2 程序设计

44.2.1 触摸屏驱动

本章实验的触摸屏驱动主要负责向应用层提供触摸屏的初始化和获取触摸屏触摸坐标等函数。本章实验中，触摸屏的驱动代码包括 touch.c、touch.h、ctiic.c、ctiic.h、ft5206.c、ft5206.h、gt9xxx.c、gt9xxx.h 八个文件。

触摸屏驱动中，触摸屏的初始化函数，如下所示：

```
/**
 * @brief    初始化触摸屏
 * @param    无
 * @retval   触摸屏是否校准
 *          0: 否
```

```

*           1: 是
*/
uint8_t tp_init(void)
{
    tp_dev.touchtype = 0; /* 默认设置（电阻屏、竖屏） */
    tp_dev.touchtype |= lcddev.dir & 0x01; /* 根据 LCD 判断是横屏还是竖屏 */

    if (lcddev.id == 0x7796)
    {
        if (gt9xxx_init() == 0) /* 初始化 GT 系列触摸屏成功,即当前 3.5 寸屏为电容触摸屏 */
        {
            tp_dev.scan = gt9xxx_scan; /* 扫描函数指向 GT9147 触摸屏扫描 */
            tp_dev.touchtype |= 0x80; /* 电容屏 */
            return 0;
        }
    }

    if (lcddev.id == 0x5510 ||
        lcddev.id == 0x9806 ||
        lcddev.id == 0x4342 ||
        lcddev.id == 0x4384 ||
        lcddev.id == 0x1018) /* 电容触摸屏,4.3 寸/10.1 寸屏 */
    {
        gt9xxx_init();
        tp_dev.scan = gt9xxx_scan; /* 扫描函数指向 GT9147 触摸屏扫描 */
        tp_dev.touchtype |= 0x80; /* 电容屏 */
        return 0;
    }
    else if (lcddev.id == 0x1963 ||
             lcddev.id == 0x7084 ||
             lcddev.id == 0x7016) /* SSD1963 7 寸屏或者 7 寸 800*480/1024*600 RGB 屏 */
    {
        if (!ft5206_init())
        {
            tp_dev.scan = ft5206_scan; /* 扫描函数指向 FT5206 触摸屏扫描 */
        }
        else
        {
            gt9xxx_init();
            tp_dev.scan = gt9xxx_scan; /* 扫描函数指向 GT9147 触摸屏扫描 */
        }
        tp_dev.touchtype |= 0x80; /* 电容屏 */
        return 0;
    }
    /* 电阻屏 */
    else
    {
        /* 电阻屏的初始化操作,代码省略 */
    }

    return 1;
}

```

从上面的代码中可以看出，触摸屏的初始化会读取 TFTLCD 模块 LCD 的 ID 号，来判断触摸屏的型号，因此在使用本触摸屏驱动初始化触摸屏前，需要先进行 LCD 的初始化。触摸屏的初始化函数会依据 LCD 的 ID 对不同型号的触摸屏进行初始化。

对于触摸屏驱动中的其他内容，请读者自行结合实际使用的 TFTLCD 的用户手册产看本章配套实验例程的源码。

44.2.2 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
```

```
{
    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    lcd_init(); /* 初始化 LCD */
    tp_dev.init(); /* 初始化触摸屏 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "TOUCH TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    /* 电阻屏显示触摸校准提示 */
    if ((tp_dev.touchtype & 0x80) == 0)
    {
        lcd_show_string(30, 110, 200, 16, 16, "Press KEY0 to Adjust", RED);
    }
    delay_ms(1500);
    load_draw_dialog();

    if (tp_dev.touchtype & 0x80)
    {
        /* 电容屏测试 */
        ctp_test();
    }
    else
    {
        /* 电阻屏测试 */
        rtp_test();
    }
}
```

在进行完触摸屏的初始化后，便根据触摸屏的不同类型调用了不同的测试函数，这是因为电容屏是支持多点触控的，而电阻屏并不支持。

电容屏测试测试函数，如下所示：

```
/**
 * @brief 电容触摸屏测试
 * @param 无
 * @retval 无
 */
static void ctp_test(void)
{
    uint8_t t = 0;
    uint8_t i = 0;
    uint16_t lastpos[10][2];
    uint8_t maxp = 5;

    if (lcddev.id == 0x1018)
    {
        maxp = 10;
    }

    while (1)
    {
        tp_dev.scan(0);
        for (t=0; t<maxp; t++)
        {
            if ((tp_dev.sta) & (1 << t))
            {
                /* 坐标在屏幕范围内 */
                if ( (tp_dev.x[t] < lcddev.width) &&
                    (tp_dev.y[t] < lcddev.height) )
```

```

        {
            if (lastpos[t][0] == 0xFFFF)
            {
                lastpos[t][0] = tp_dev.x[t];
                lastpos[t][1] = tp_dev.y[t];
            }

            lcd_draw_bline( lastpos[t][0], /* 画线 */
                           lastpos[t][1],
                           tp_dev.x[t],
                           tp_dev.y[t],
                           2,
                           POINT_COLOR_TBL[t]);
            lastpos[t][0] = tp_dev.x[t];
            lastpos[t][1] = tp_dev.y[t];

            if (tp_dev.x[t] > (lcddev.width - 24) && tp_dev.y[t] < 20)
            {
                load_draw_dialog();          /* 清除 */
            }
        }
    }
    else
    {
        lastpos[t][0] = 0xFFFF;
    }
}

delay_ms(5);
i++;
if ((i % 20) == 0)
{
    LED0_TOGGLE();
}
}
}

```

因为电容屏触摸屏支持多点触控，因此电容屏测试函数在扫描到每一个触摸点坐标后，便将每一个触摸点对应 LCD 屏幕上的坐标在 LCD 屏幕上进行绘制，并且每个触摸点使用不同的颜色进行绘制。

电阻屏测试函数，如下所示：

```

/**
 * @brief   电阻触摸屏测试
 * @param   无
 * @retval  无
 */
void rtp_test(void)
{
    uint8_t key;
    uint8_t i = 0;

    while (1)
    {
        key = key_scan(0);
        tp_dev.scan(0);

        if (tp_dev.sta & TP_PRES_DOWN)          /* 触摸屏被按下 */
        {
            if ( (tp_dev.x[0] < lcddev.width) &&
                  (tp_dev.y[0] < lcddev.height))
            {
                if ( (tp_dev.x[0] > (lcddev.width - 24)) &&
                      (tp_dev.y[0] < 16))
                {
                    load_draw_dialog();          /* 清除 */
                }
            }
        }
    }
}

```

```

        }
        else
        {
            tp_draw_big_point(tp_dev.x[0], tp_dev.y[0], RED); /* 画点 */
        }
    }
}
else
{
    delay_ms(10); /* 没有按键按下时候 */
}

if (key == KEY0_PRES) /* KEY0 按下，则执行校准程序 */
{
    tp_adjust(); /* 屏幕校准 */
    tp_save_adjust_data();
    load_draw_dialog();
}

i++;
if ((i % 20) == 0)
{
    LED0_TOGGLE();
}
}
}

```

电阻触摸屏就相对简单，因为电阻触摸屏仅支持单点触控，因此仅需将触摸的 1 个触摸点对应 LCD 屏幕上坐标的点进行绘制即可，同时因为电阻触摸屏是需要校准的，因此当检测到 KEY0 按键被按下时，便会进行电阻触摸屏校准。

44.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上显示了本实验的实验信息，随后便进入“白板”界面，此时便可在 LCD 上通过触摸屏绘制出任意的图案，若是电容屏，还支持多点触控，若是电阻屏出现触摸点与 LCD 上显示的绘制点坐标不吻合，可以按下 KEY0 按键进行电阻触摸屏的校准。

第四十五章 FLASH 模拟 EEPROM 实验

本章将介绍使用 STM32F407 的片上 Flash 模拟 EEPROM，并对齐进行读写操作。通过本章的学习，读者将学习到 Flash 的使用。

本章分为如下几个小节：

- 45.1 硬件设计
- 45.2 程序设计
- 45.3 下载验证

45.1 硬件设计

45.1.1 例程功能

1. 按下 WKUP 按键可往 Flash 写入“STM32 FLASH TEST”
2. 按下 KEY0 按键可从 Flash 读取到“STM32 FLASH TEST”，并显示至 LCD
3. LED0 闪烁，提示程序正在运行

45.1.2 硬件资源

1. LED
LED0 - PF9
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
WKUP - PA0
KEY0 - PE4

45.1.3 原理图

本章实验使用的 Flash 为 STM32F407 的片上资源，因此没有对应的连接原理图。

45.2 程序设计

45.2.1 HAL 库的 Flash 驱动

STM32F407 的片上 Flash 是可以直接读取的，但 Flash 无法直接写入，写入 Flash 前，需要先对其进行擦除操作，其具体的操作步骤如下：

- ①：解锁访问 Flash 控制寄存器
- ②：关闭 Flash 数据 Cache
- ③：擦除 Flash
- ④：编程 Flash
- ⑤：开启 Flash 数据 Cache
- ⑥：上锁访问 Flash 控制寄存器

在 HAL 库中对应的驱动函数如下：

①：解锁访问 Flash 控制寄存器

该函数用于解锁访问 Flash 控制寄存器，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_FLASH_Unlock(void);
```

该函数的形参描述，如下表所示：

形参	描述
无	无

表 45.2.1.1 函数 HAL_FLASH_Unlock()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 45.2.1.2 函数 HAL_FLASH_Unlock()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 解锁访问 Flash 控制寄存器 */
    HAL_FLASH_Unlock();
}
```

②：关闭 Flash 数据 Cache

该函数实际上是一个宏定义，用于关闭 Flash 数据 Cache，该宏的定义如下所示：

```
#define __HAL_FLASH_DATA_CACHE_DISABLE() (FLASH->ACR &= (~FLASH_ACR_DCEN))
```

该宏的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 关闭 Flash 数据 Cache */
    __HAL_FLASH_DATA_CACHE_DISABLE();
}
```

③：擦除 Flash

该函数用于擦除 Flash，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_FLASHEx_Erase( FLASH_EraseInitTypeDef *pEraseInit,
                                     uint32_t *SectorError);
```

该函数的形参描述，如下表所示：

形参	描述
pEraseInit	指向 Flash 擦除信息结构体的指针 需自行定义，并根据 Flash 的擦除信息填充结构体中的成员变量
SectorError	擦除失败的扇区

表 45.2.1.3 函数 HAL_FLASHEx_Erase()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 45.2.1.4 函数 HAL_FLASHEx_Erase()返回值描述

该函数使用 FLASH_EraseInitTypeDef 类型结构体指针传入了 Flash 的擦除信息，该结构体的定义如下所示：

```
typedef struct
{
    uint32_t TypeErase;      /* 擦除类型 */
    uint32_t Banks;          /* 块编号 */
    uint32_t Sector;         /* 起始扇区 */
    uint32_t NbSectors;      /* 扇区数量 */
    uint32_t VoltageRange;   /* 电压范围 */
} FLASH_EraseInitTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    FLASH_EraseInitTypeDef erase = {0};
    uint32_t sector

    /* 擦除 Flash */
    HAL_FLASHEx_Erase(&erase, &sector);
}
```

}

④：编程 Flash

该函数用于编程 Flash，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_FLASH_Program(   uint32_t TypeProgram,
                                         uint32_t Address,
                                         uint64_t Data);
```

该函数的形参描述，如下表所示：

形参	描述
TypeProgram	编程类型 例如：FLASH_TYPEPROGRAM_BYTE、FLASH_TYPEPROGRAM_HALFWORD 等（在 stm32f4xx_hal_flash.h 文件中有定义）
Address	起始地址
Data	数据

表 45.2.1.5 函数 HAL_FLASH_Program()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 45.2.1.6 函数 HAL_FLASH_Program()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    uint8_t buf;

    /* 编程 Flash */
    HAL_FLASH_Program(FLASH_TYPEPROGRAM_BYTE, 0, &buf);
}
```

⑤：开启 Flash 数据 Cache

该函数实际上是一个宏定义，用于开启 Flash 数据 Cache，该宏的定义如下所示：

```
#define __HAL_FLASH_DATA_CACHE_ENABLE() (FLASH->ACR |= FLASH_ACR_DCEN)
```

该宏的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 开启 Flash 数据 Cache */
    __HAL_FLASH_DATA_CACHE_ENABLE();
}
```

⑥：上锁访问 Flash 控制寄存器

该函数用于上锁访问 Flash 控制寄存器，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_FLASH_Lock(void);
```

该函数的形参描述，如下表所示：

形参	描述
无	无

表 45.2.1.7 函数 HAL_FLASH_Lock()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 45.2.1.8 函数 HAL_FLASH_Lock()返回值描述

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
```

```
{
    /* 上锁访问 Flash 控制寄存器 */
    HAL_FLASH_Lock();
}
```

45.2.2 Flash 驱动

本章实验的 Flash 驱动主要负责向应用层提供 Flash 的读写操作函数。本章实验中，Flash 的驱动代码包括 stmflash.c 和 stmflash.h 两个文件。

Flash 驱动中，读取 Flash 数据的函数，如下所示：

```
/**
 * @brief 从指定地址读取一字的数据
 * @param addr: 指定读取数据的地址
 * @retval 读取到的一字数据
 */
uint32_t stmflash_read_word(uint32_t addr)
{
    return (*(volatile uint32_t *)addr);
}

/**
 * @brief 从指定地址读取指定字的数据
 * @param addr: 指定读取数据的地址
 * @param buf: 存储读取数据的起始地址
 * @param length: 指定读取数据的长度，单位：字
 * @retval 无
 */
void stmflash_read(uint32_t addr, uint32_t *buf, uint32_t length)
{
    uint32_t index;

    for (index=0; index<length; index++)
    {
        buf[index] = stmflash_read_word(addr);
        addr += sizeof(uint32_t);
    }
}
```

STM32F407 片上 Flash 的读取十分简单，仅需读取对应地址的数据即可。

Flash 驱动中，往 Flash 写入数据的函数，如下所示：

```
/**
 * @brief 往指定地址不检查地写入指定字的数据
 * @param addr: 指定写入数据的地址
 * @param buf: 存储写入数据的起始地址
 * @param length: 指定写入数据的长度，单位：字
 * @retval 无
 */
void stmflash_write_noccheck(uint32_t addr, uint32_t *buf, uint16_t length)
{
    uint16_t index;

    HAL_FLASH_Unlock();
    __HAL_FLASH_DATA_CACHE_DISABLE();
    for (index=0; index<length; index++)
    {
        HAL_FLASH_Program( FLASH_TYPEPROGRAM_WORD,
                           addr + (index << 2),
                           buf[index]);
    }
    __HAL_FLASH_DATA_CACHE_ENABLE();
    HAL_FLASH_Lock();
}

/**
```

```

* @brief 往指定地址写入指定字的数据
* @param addr: 指定写入数据的地址
* @param buf: 存储写入数据的起始地址
* @param length: 指定写入数据的长度, 单位: 字
* @retval 无
*/
void stmflash_write(uint32_t addr, uint32_t *buf, uint16_t length)
{
    uint32_t addrx;
    uint32_t endaddr;
    FLASH_EraseInitTypeDef flash_erase_init_struct = {0};
    uint32_t sectorerr;
    HAL_StatusTypeDef status = HAL_OK;

    /* 检查写入地址范围的合法性 */
    if ( (!IS_FLASH_ADDRESS(addr)) ||
          (!IS_FLASH_ADDRESS(addr + (length * sizeof(uint32_t)) - 1)) ||
          ((addr & 3) != 0) )
    {
        return;
    }

    addrx = addr;
    endaddr = addr + (length << 2);
    if (addrx <= FLASH_END)
    {
        while (addrx < endaddr)
        {
            /* 判断是否需要擦除 */
            if (stmflash_read_word(addrx) != 0xFFFFFFFF)
            {
                /* 擦除扇区 */
                flash_erase_init_struct.TypeErase = FLASH_TYPEERASE_SECTORS;
                flash_erase_init_struct.Banks = FLASH_BANK_1;
                flash_erase_init_struct.Sector =
                    stmflash_get_flash_sector(addrx);
                flash_erase_init_struct.NbSectors = 1;
                flash_erase_init_struct.VoltageRange = FLASH_VOLTAGE_RANGE_3;
                HAL_FLASH_Unlock();
                status = HAL_FLASHEx_Erase(&flash_erase_init_struct, &sectorerr);
                HAL_FLASH_Lock();
                if (status != HAL_OK)
                {
                    break;
                }
            }
            else
            {
                addrx += sizeof(uint32_t);
            }
        }

        if (status == HAL_OK)
        {
            while (addr < endaddr)
            {
                /* 写入数据 */
                stmflash_write_noccheck(addr, buf, 1);
                addr += sizeof(uint32_t);
                buf++;
            }
        }
    }
}

```

在写 Flash 前需要先判断待写入的比特位是否为 1, 若不为 1 则需要先进行擦除操作, 否则

将写入失败，保证待写入位置的比特位全部为 0 后，方可调用函数 HAL_FLASH_Program()对 Flash 进行编程。

45.2.3 实验应用代码

本章实验的应用代码，如下所示：

```
static const uint8_t g_text_buf[] = {"STM32 FLASH TEST"};
#define TEXT_SIZE (((sizeof(g_text_buf) >> 2) << 2) + 4)

int main(void)
{
    uint8_t t = 0;
    uint8_t key;
    uint8_t data[TEXT_SIZE];
    uint8_t wdata[TEXT_SIZE] = {0};
    uint8_t index;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    lcd_init(); /* 初始化 LCD */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "FLASH TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    lcd_show_string(30, 110, 200, 16, 16, "WK_UP:Write KEY0:Read", RED);

    for (index=0; index<sizeof(g_text_buf); index++)
    {
        wdata[index] = g_text_buf[index];
    }

    while (1)
    {
        key = key_scan(0);
        if (key == WKUP_PRES)
        {
            /* 往 Flash 写入数据 */
            lcd_fill(0, 130, 239, 319, WHITE);
            lcd_show_string(30, 130, 200, 16, 16, "Start Write Flash...", BLUE);
            stmflash_write(FLASH_END + 1 - TEXT_SIZE,
                           (uint32_t *)wdata,
                           TEXT_SIZE / sizeof(uint32_t));
            lcd_show_string(30, 130, 200, 16, 16, "Flash Write Finished!", BLUE);
        }
        else if (key == KEY0_PRES)
        {
            /* 从 Flash 读取数据 */
            lcd_show_string(30, 130, 200, 16, 16, "Start Read Flash.... ", BLUE);
            stmflash_read(FLASH_END + 1 - TEXT_SIZE,
                           (uint32_t *)data,
                           TEXT_SIZE / sizeof(uint32_t));
            lcd_show_string(30, 130, 200, 16, 16, "The Data Readed Is: ", BLUE);
            lcd_show_string(30, 150, 200, 16, 16, (char *)data, BLUE);
        }

        if (++t == 20)
        {
            t = 0;
            LED0_TOGGLE();
        }
    }
}
```

```
    }  
  
    delay_ms(10);  
}  
}
```

从本章实验的应用代码中可以看到,在完成相关的初始化工作后,便会不断地等待按键输入,若检测到 WKUP 按键被按下,则会往 Flash 的指定地址中写入指定的数据,若检测到 KEY_0 按键被按下,则会从 Flash 的指定地址中读取数据,并在 LCD 上进行显示。

45.3 下载验证

在完成编译和烧录操作后,可以看到 LCD 上显示了本实验相关的信息,此时便可按下 WKUP 按键往 Flash 的指定地址写入指定数据,然后再按下 KEY_0 按键从 Flash 的指定地址将写入的数据读回来在 LCD 上进行显示,此时便可以看到在 LCD 上显示了“STM32 FLASH TEST”的提示信息,该提示信息就是从 Flash 中读回的数据。

第四十六章 摄像头实验

本章将介绍使用 STM32F407 驱动 OV2640 摄像头，从而获取摄像头输出的图像数据，并将其显示在 LCD 上，也可通过串口发送至 PC 上位机软件。通过本章的学习，读者将学习到数字摄像头接口（DCMI）的使用。

本章分为如下几个小节：

- 46.1 硬件设计
- 46.2 程序设计
- 46.3 下载验证

46.1 硬件设计

46.1.1 例程功能

1. 初始化摄像头后，可通过按下 WKUP 按键或 KEY0 按键进入 RGB565 或 JPEG 测试模式
2. RGB565 测试模式下，LCD 显示 DCMI 捕获的摄像头画面，可通过按键修改画面对比度或缩放
3. JPEG 测试模式下，USART2 输出 DCMI 捕获的 JPEG 数据，可通过按键对画面对比度或尺寸进行设置
4. 串口 1 输出 DCMI 接收摄像头数据帧率
5. LED1 闪烁，提示 DCMI 接收到一帧数据

46.1.2 硬件资源

1. LED
 - LED0 - PF9
 - LED1 - PF10
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
 - WKUP - PA0
 - KEY0 - PE4
4. USART1
 - USART1_TX - PA9
 - USART1_RX - PA10
5. USART2
 - USART2_TX - PA2
6. TIM6
7. ATK-MC2640 摄像头模块
 - OV_D0~D7 - PC6/PC7/PC8/PC9/PC11/PB6/PE5/PE6
 - OV_SCL - PD6
 - OV_SDA - PD7
 - OV_VSYNC - PB7
 - OV_HREF - PA4
 - OV_PCLK - PA6
 - OV_PWDN - PG9
 - OV_RESET - PG15
 - OV_XCLK - PA8

46.1.3 原理图

本章实验使用了一个正点原子 OV2640 摄像头模块（ATK-MC2640），该模块需通过摄像头模块延长线与板载的 CAMERA 接口进行连接，该接口也可与 OLED 模块进行连接，该接口与板

载 MCU 的连接原理图，如下图所示：

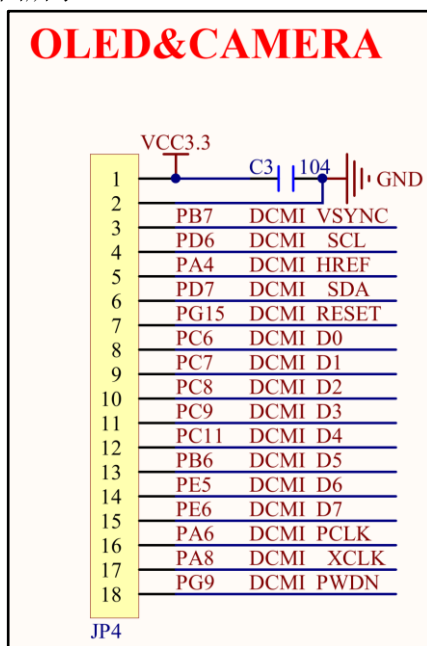


图 46.1.3.1 摄像头模块与 MCU 的连接原理图

46.2 程序设计

46.2.1 HAL 库的 DCMI 驱动

本章使用通过 DCMI 驱动摄像头模块，并获取摄像头模块返回的图像数据，分为 RGB565 模式和 JPEG 模式，其中 RGB565 模式直接将摄像头返回的数据流通过 DMA 传输至 LCD，JPEG 模式下将整帧的 JPEG 图像数据通过 USART2 传输至 PC 上位机软件，因为需要获取 JPEG 整帧的数据，因此 JPEG 模式下需要使用到 DCMI 的捕获完成中断，其具体的步骤如下：

①：初始化 DCMI

⑤：使能 DCMI 捕获

在 HAL 库中对应的驱动函数如下：

①：初始化 DCMI

该函数用于初始化 DCMI 的各项参数，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_DCMI_Init(DCMI_HandleTypeDef *hdcmi);
```

该函数的形参描述，如下表所示：

形参	描述
hdcmi	指向 DCMI 句柄的指针

表 46.2.1.1 函数 HAL_DCMI_Init()形参描述

该函数的返回值描述，如下表所示：

返回值	描述
HAL_StatusTypeDef	HAL 状态

表 46.2.1.2 函数 HAL_DCMI_Init()返回值描述

该函数需要传入 DCMI 的句柄指针，该句柄中就包含了 DCMI 的初始化配置参数结构体，该结构体的定义如下所示：

```
typedef struct
{
    uint32_t SynchroMode;          /* 同步模式 */
    uint32_t PCKPolarity;          /* PCLK 极性 */
    uint32_t VSPolarity;           /* VSYNC 极性 */
    uint32_t HSPolarity;           /* HSYNC 极性 */
    uint32_t CaptureRate;          /* 捕获率 */
}
```



```
uint32_t ExtendedDataMode; /* 数据位宽 */
DCMI_CodesInitTypeDef SyncroCode; /* 帧分隔符 */
uint32_t JPEGMode; /* JPEG 模式 */
}DCMI_InitTypeDef;
```

该函数的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    DCMI_HandleTypeDef dcmi_handle = {0};

    /* 初始化 DCMI */
    dcmi_handle.Instance = DCMI;
    dcmi_handle.Init.SynchroMode = DCMI_SYNCHRO_HARDWARE;
    dcmi_handle.Init.PCKPolarity = DCMI_PCKPOLARITY_RISING;
    dcmi_handle.Init.VSPolarity = DCMI_VSPOLARITY_LOW;
    dcmi_handle.Init.HSPolarity = DCMI_HSPOLARITY_LOW;
    dcmi_handle.Init.CaptureRate = DCMI_CR_ALL_FRAME;
    dcmi_handle.Init.ExtendedDataMode = DCMI_EXTEND_DATA_8B;
    dcmi_handle.Init.JPEGMode = DCMI_JPEG_DISABLE;
    HAL_DCMI_Init(&dcmi_handle);
}
```

②：使能 DCMI 捕获

该函数实际上是一个宏定义，用于使能 DCMI 捕获，该宏的定义如下所示：

```
#define __HAL_DMA_ENABLE(__HANDLE__) ((__HANDLE__)->Instance->CR |= DMA_SxCR_EN)
```

该宏的使用示例，如下所示：

```
#include "stm32f4xx_hal.h"

void example_fun(void)
{
    /* 使能 DCMI 捕获 */
    __HAL_DMA_ENABLE(&dcmi_handle);
}
```

46.2.2 DCMI 驱动代码

本章实验的 DCMI 驱动主要负责向应用层提供 DCMI 的初始化和启动 DCMI 传输等函数。本章实验中，DCMI 的驱动代码包括 dcmi.c 和 dcmi.h 两个文件。

由于 DCMI 使用了大量的 GPIO 引脚，因此对于 GPIO 的相关定义，请读者自行查看 dcmi.c 这个文件。

DCMI 驱动中，DCMI 的初始化函数，如下所示：

```
/**
 * @brief 初始化 DCMI
 * @param 无
 * @retval 无
 */
void dcmi_init(void)
{
    /* 配置 DCMI */
    g_dcmi_handle.Instance = DCMI;
    g_dcmi_handle.Init.SynchroMode = DCMI_SYNCHRO_HARDWARE;
    g_dcmi_handle.Init.PCKPolarity = DCMI_PCKPOLARITY_RISING;
    g_dcmi_handle.Init.VSPolarity = DCMI_VSPOLARITY_LOW;
    g_dcmi_handle.Init.HSPolarity = DCMI_HSPOLARITY_LOW;
    g_dcmi_handle.Init.CaptureRate = DCMI_CR_ALL_FRAME;
    g_dcmi_handle.Init.ExtendedDataMode = DCMI_EXTEND_DATA_8B;
    g_dcmi_handle.Init.JPEGMode = DCMI_JPEG_DISABLE;
    HAL_DCMI_Init(&g_dcmi_handle);

    DCMI->IER = 0x0;

    __HAL_DCMI_ENABLE_IT(&g_dcmi_handle, DCMI_IT_FRAME); /* 使能帧中断 */
}
```

```

    __HAL_DCMI_ENABLE(&g_dcmi_handle);
}

/**
 * @brief   HAL 库 DCMI 初始化 MSP 函数
 * @param   hdcmi: DCMI 句柄
 * @retval  无
 */
void HAL_DCMI_MspInit(DCMI_HandleTypeDef* hdcmi)
{
    GPIO_InitTypeDef gpio_init_struct = {0};

    if (hdcmi->Instance == DCMI)
    {
        __HAL_RCC_DCMI_CLK_ENABLE();
        __HAL_RCC_GPIOA_CLK_ENABLE();
        __HAL_RCC_GPIOB_CLK_ENABLE();
        __HAL_RCC_GPIOC_CLK_ENABLE();
        __HAL_RCC_GPIOE_CLK_ENABLE();

        gpio_init_struct.Pin = GPIO_PIN_4 |
                                GPIO_PIN_6;
        gpio_init_struct.Mode = GPIO_MODE_AF_PP;
        gpio_init_struct.Pull = GPIO_PULLUP;
        gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
        gpio_init_struct.Alternate = GPIO_AF13_DCMI;
        HAL_GPIO_Init(GPIOA, &gpio_init_struct);

        gpio_init_struct.Pin = GPIO_PIN_6 |
                                GPIO_PIN_7;
        HAL_GPIO_Init(GPIOB, &gpio_init_struct);

        gpio_init_struct.Pin = GPIO_PIN_6 |
                                GPIO_PIN_7 |
                                GPIO_PIN_8 |
                                GPIO_PIN_9 |
                                GPIO_PIN_11;
        HAL_GPIO_Init(GPIOC, &gpio_init_struct);

        gpio_init_struct.Pin = GPIO_PIN_5 |
                                GPIO_PIN_6;
        HAL_GPIO_Init(GPIOE, &gpio_init_struct);

        HAL_NVIC_SetPriority(DCMI_IRQn, 0, 0);
        HAL_NVIC_EnableIRQ(DCMI_IRQn);
    }
}

```

在 DCMI 的初始化函数中可以看到，配置并使能了 DCMI，还为 JPEG 模式使能了 DCMI 的帧中断。

DCMI 驱动中，DCMI 的帧事件回调函数，如下所示：

```

/**
 * @brief   DCMI 中断回调服务函数
 * @param   hdcmi: DCMI 句柄
 * @retval  无
 */
void HAL_DCMI_FrameEventCallback(DCMI_HandleTypeDef *hdcmi)
{
    /* 清除帧中断 */
    __HAL_DCMI_CLEAR_FLAG(&g_dcmi_handle, DCMI_FLAG_FRAMERI);
    jpeg_data_process(); /* jpeg 数据处理 */
    LED1_TOGGLE(); /* LED1 闪烁 */
    g_ov_frame++;
    /* 重新使能帧中断,因为 HAL_DCMI_IRQHandler() 函数会关闭帧中断 */
    __HAL_DCMI_ENABLE_IT(&g_dcmi_handle, DCMI_IT_FRAME);
}

```

}

可以看到，该函数主要就是调用了 JPEG 数据的处理函数，以及统计帧率。

摄像头返回的数据是使用 DMA 进行传输的，DCMI 驱动中，DMA 的初始化函数，如下所

示：

```
/**
 * @brief    初始化 DCMI DMA
 * @param    mem0addr: 存储器 0 地址
 * @param    mem1addr: 存储器 1 地址
 * @param    memsize: 存储器大小
 * @param    memblen: 存储器位宽
 * @param    meminc: 存储器增长模式
 * @retval    无
 */
void dcmi_dma_init(uint32_t mem0addr,
                  uint32_t mem1addr,
                  uint16_t memsize,
                  uint32_t memblen,
                  uint32_t meminc)
{
    /* 配置 DMA */
    __HAL_RCC_DMA2_CLK_ENABLE();
    g_dcmi_dma_handle.Instance = DMA2_Stream1;
    g_dcmi_dma_handle.Init.Channel = DMA_CHANNEL_1;
    g_dcmi_dma_handle.Init.Direction = DMA_PERIPH_TO_MEMORY;
    g_dcmi_dma_handle.Init.PeriphInc = DMA_PINC_DISABLE;
    g_dcmi_dma_handle.Init.MemInc = meminc;
    g_dcmi_dma_handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD;
    g_dcmi_dma_handle.Init.MemDataAlignment = memblen;
    g_dcmi_dma_handle.Init.Mode = DMA_CIRCULAR;
    g_dcmi_dma_handle.Init.Priority = DMA_PRIORITY_VERY_HIGH;
    g_dcmi_dma_handle.Init.FIFOMode = DMA_FIFOMODE_ENABLE;
    g_dcmi_dma_handle.Init.FIFOThreshold = DMA_FIFO_THRESHOLD_HALFFULL;
    g_dcmi_dma_handle.Init.MemBurst = DMA_MBURST_SINGLE;
    g_dcmi_dma_handle.Init.PeriphBurst = DMA_PBURST_SINGLE;
    HAL_DMA_DeInit(&g_dcmi_dma_handle);
    HAL_DMA_Init(&g_dcmi_dma_handle);

    __HAL_LINKDMA(&g_dcmi_handle, DMA_Handle, g_dcmi_dma_handle);

    if (mem1addr == 0)
    {
        HAL_DMA_Start(&g_dcmi_dma_handle,
                     (uint32_t)&DCMI->DR,
                     mem0addr, memsize);
    }
    else
    {
        HAL_DMAEx_MultiBufferStart(&g_dcmi_dma_handle,
                                   (uint32_t)&DCMI->DR,
                                   mem0addr,
                                   mem1addr,
                                   memsize);
        __HAL_DMA_ENABLE_IT(&g_dcmi_dma_handle, DMA_IT_TC);
        HAL_NVIC_SetPriority(DMA2_Stream1_IRQn, 0, 0);
        HAL_NVIC_EnableIRQ(DMA2_Stream1_IRQn);
    }
}
```

可以看到，该函数配置 DMA 将 DCMI 的数据传输至存储器，且可以配置单缓冲和双缓冲。

DCMI 驱动中，启动 DCMI 传输的函数，如下所示：

```
/**
 * @brief    启动 DCMI 传输
 * @param    无
 * @retval    无
```

```

*/
void dcmi_start(void)
{
    lcd_set_cursor(0, 0);           /* 设置坐标到原点 */
    lcd_write_ram_prepare();        /* 开始写入 GRAM */
    __HAL_DMA_ENABLE(&g_dcmi_dma_handle); /* 使能 DMA */
    DCMI->CR |= DCMI_CR_CAPTURE;    /* DCMI 捕获使能 */
}

```

因为在 RGB565 模式下，DMA 将直接将 DCMI 的数据传输至 LCD 上进行显示，因此在 DCMI 的启动传输函数中对 LCD 进行了相应的操作，并使能了 DMA 和 DCMI。

DCMI 的驱动就介绍这么多，更多的请读者查看本章实验配套实验例程的源码。

46.2.3 SCCB 驱动

本章实验的 SCCB 驱动主要负责向 OV2640 驱动提供配置 OV2640 摄像头的各种函数，SCCB 协议与 IIC 协议十分相似，也可兼容 IIC 协议，因此请读者结合 SCCB 和 IIC 协议的相关文档查看本章实验配套实验源码中 SCCB 的相关驱动文件。本章实验中，SCCB 的驱动代码包括 sccb.c 和 sccb.h 两个文件。

46.2.4 OV2640 驱动

本章实验的 OV2640 驱动主要负责向应用层提供 OV2640 的初始化和各种配置函数，请读者结合正点原子 OV2640 模块(ATK-MC2640)的用户手册查看本章实验配套例程源码中的 OV2640 驱动代码。本章实验中，OV2640 的驱动代码包括 ov2640.c 和 ov2640.h 两个文件。

46.2.5 实验应用代码

本章实验的应用代码，如下所示：

```

int main(void)
{
    uint8_t t = 0;
    uint8_t key;

    HAL_Init();           /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟，168MHz */
    delay_init(168);      /* 初始化延时 */
    usart_init(115200);   /* 初始化串口 */
    led_init();           /* 初始化 LED */
    key_init();           /* 初始化按键 */
    lcd_init();           /* 初始化 LCD */
    btim_timx_int_init(10000 - 1, 8400 - 1); /* 初始化基本定时器 */
    usart2_init(921600);  /* 初始化 USART2 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "OV2640 TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    /* 初始化 OV2640 */
    while (ov2640_init() != 0)
    {
        lcd_show_string(30, 130, 200, 16, 16, "OV2640 ERROR ", RED);
        delay_ms(200);
        lcd_show_string(30, 130, 200, 16, 16, "Please Check! ", RED);
        delay_ms(500);
        LED0_TOGGLE();
    }
    lcd_show_string(30, 130, 200, 16, 16, "OV2640 OK ", RED);
    ov2640_flash_intctrl();

    while (1)

```

```

{
    t++;
    key = key_scan(0);

    if (key == KEY0_PRES)
    {
        /* RGB565 模式 */
        g_ov_mode = 0;
        break;
    }
    else if (key == WKUP_PRES)
    {
        /* JPEG 模式 */
        g_ov_mode = 1;
        break;
    }

    if (t == 100)
    {
        lcd_show_string(30, 150, 230, 16, 16,
                        "KEY0:RGB565 KEY_UP:JPEG", RED);
    }
    else if (t == 200)
    {
        t = 0;
        lcd_fill(30, 150, 230, 150 + 16, WHITE);
        LED0_TOGGLE();
    }

    delay_ms(5);
}

if (g_ov_mode == 0)
{
    /* RGB565 模式测试 */
    rgb565_test();
}
else
{
    /* JPEG 模式测试 */
    jpeg_test();
}
}

```

从上面的代码中可以看出，在初始化完 OV2640 摄像头后，并不断地扫描按键，若检测到 KEY0 按键被按下，则进入 RGB565 模式的测试，即将 DCMI 获取到的摄像头图像数据，直接在 LCD 上进行显示，若检测到 WKUP 按键被按下，则进入 JPEG 模式的测试，即将 DCMI 获取到的摄像头数据，通过 USART2 传输至 PC 上位机。

46.3 下载验证

在完成编译和烧录操作后，先将开发板断电，随后将摄像头模块通过摄像头延长线与开发板进行连接，同时连接好 LCD 模式和 USART2 与 PC 的连接，最后再给开发板供电（在插拔开发板上的接插件和模块时，要求必须断电操作，否则容易烧毁硬件）。程序运行后，可以看到 LCD 上提示按下 KEY0 按键进入 RGB565 模式的测试，按下 WKUP 按键进入 JPEG 模式的测试。

若此时按下 KEY0 按键，若开发板连接了 LCD 模块，便能够在 LCD 显示屏上看到摄像头采集到的实时画面，并且按下 KEY0 按键可以改变画面的对比度，按下 WKUP 按键可以改变画面的尺寸。

若此时按下 WKUP 按键，若开发板的 USART2 通过 USB 转串口模式与 PC 进行连接，并配置好 PC 端的上位机（正点原子 视频传输上位机，读者可前往正点原子资料下载中心下载该上位机软件），便可以在上位机软件中看到摄像头采集到的实时画面，并且按下 KEY0 按键可以改变画面的对比度，按下 WKUP 按键可以改变画面的尺寸。

在进入 RGB565 模式或 JPEG 模式测试后，可以通过串口调试助手查看到摄像头的实时帧率。

第四十七章 外部 SRAM 实验

本章将介绍使用 STM32F407 驱动 SRAM 进行 SRAM 的数据读写。通过本章的学习，读者将学习到 FSMC 的使用。

本章分为如下几个小节：

- 47.1 硬件设计
- 47.2 程序设计
- 47.3 下载验证

47.1 硬件设计

47.1.1 例程功能

1. 按下 WKUP 按键可对外部 SRAM 进行数据测试，并在 LCD 上显示测试结果
2. 按下 KEY0 按键可对外部 SRAM 进行容量测试，并在 LCD 上显示测试结果
3. J LED0 闪烁，提示程序正在运行

47.1.2 硬件资源

1. LED
 - LED0 - PF9
2. 按键
 - WKUP - PA0
 - KEY0 - PE4
3. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
4. 外部 SRAM

47.1.3 原理图

本章实验使用了一个板载的 SRAM 芯片，该 SRAM 芯片通过 MCU 的 SMC 接口与 MCU 进行连接，该 SRAM 与 MCU 的连接原理图，如下图所示：

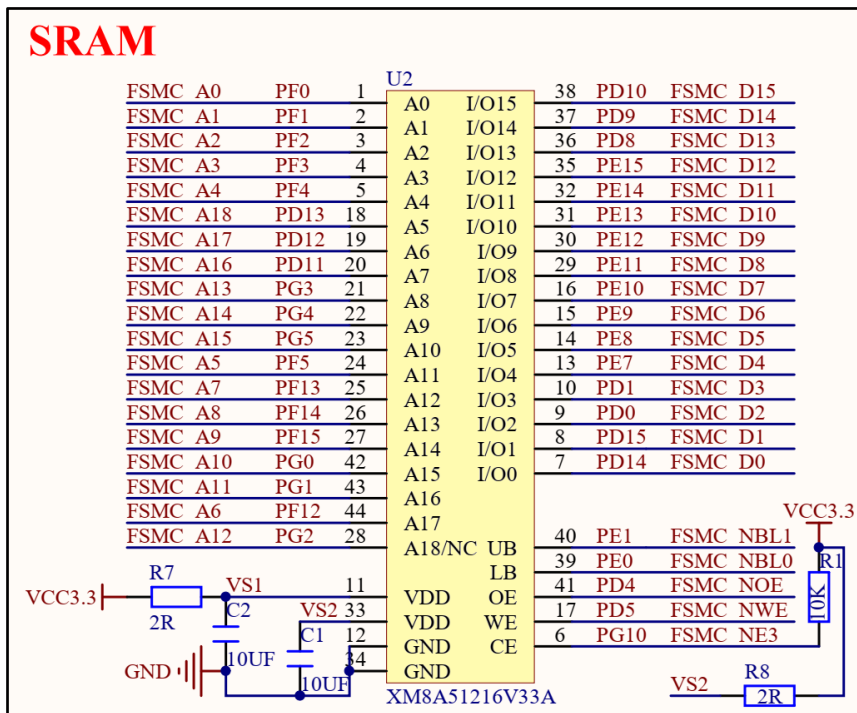


图 47.1.3.1 摄像头模块与 MCU 的连接原理图

47.2 程序设计

47.2.1 HAL 库的 FSMC 驱动

本章实验通过 FSMC 驱动 SRAM 芯片，通过 FSMC 可以将外部 SRAM 芯片的数据访问映射成一段内存空间，通过访问这段内存空间，即可访问 SRAM 芯片中的数据，因此需要对 FSMC 做相应的配置，FSMC 的配置方式，请读者查看第 26.2.1 小节中 HAL 库的 FSMC 驱动中相关的内容。

47.2.2 SRAM 驱动代码

本章实验的 SRAM 驱动主要负责向应用层提供 SRAM 的初始化函数，因为 SRAM 在初始化后，SRAM 将被映射为一段内存空间，对 SRAM 的访问操作就是访问这段内存空间。本章实验中，SRAM 的驱动代码包括 sram.c 和 sram.h 两个文件。

由于 SRAM 需要使用大量的 GPIO 引脚，因此对于 GPIO 的相关定义，请读者自行查看 sram.c 和 sram.h 这两个文件。

SRAM 驱动中，SRAM 的初始化函数，如下所示：

```
/**
 * @brief  初始化外部 SRAM
 * @param  无
 * @retval 无
 */
void sram_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;
    FSMC_NORSRAM_TimingTypeDef fsmc_timing_struct = {0};

    /* 使能时钟 */
    __HAL_RCC_FSMC_CLK_ENABLE();
    SRAM_WR_GPIO_CLK_ENABLE();
    SRAM_RD_GPIO_CLK_ENABLE();
    SRAM_CS_GPIO_CLK_ENABLE();
    __HAL_RCC_GPIOC_CLK_ENABLE();
}
```



```
__HAL_RCC_GPIOE_CLK_ENABLE();
__HAL_RCC_GPIOF_CLK_ENABLE();
__HAL_RCC_GPIOG_CLK_ENABLE();

/* 配置 FSMC_NWE 引脚 */
gpio_init_struct.Pin = SRAM_WR_GPIO_PIN;
gpio_init_struct.Mode = GPIO_MODE_AF_PP;
gpio_init_struct.Pull = GPIO_PULLUP;
gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
gpio_init_struct.Alternate = SRAM_WR_GPIO_AF;
HAL_GPIO_Init(SRAM_WR_GPIO_PORT, &gpio_init_struct);

/* 配置 FSMC_NOE 引脚 */
gpio_init_struct.Pin = SRAM_RD_GPIO_PIN;
HAL_GPIO_Init(SRAM_RD_GPIO_PORT, &gpio_init_struct);

/* 配置 FSMC_NEx 引脚 */
gpio_init_struct.Pin = SRAM_CS_GPIO_PIN;
HAL_GPIO_Init(SRAM_CS_GPIO_PORT, &gpio_init_struct);

/* 配置 FSMC_NBLn 引脚 */
gpio_init_struct.Pin = GPIO_PIN_0 |
GPIO_PIN_1;
HAL_GPIO_Init(GPIOE, &gpio_init_struct);

/* 配置 FSMC_Dn 引脚 */
gpio_init_struct.Pin = GPIO_PIN_14 |
GPIO_PIN_15 |
GPIO_PIN_0 |
GPIO_PIN_1 |
GPIO_PIN_8 |
GPIO_PIN_9 |
GPIO_PIN_10;
HAL_GPIO_Init(GPIOD, &gpio_init_struct);
gpio_init_struct.Pin = GPIO_PIN_7 |
GPIO_PIN_8 |
GPIO_PIN_9 |
GPIO_PIN_10 |
GPIO_PIN_11 |
GPIO_PIN_12 |
GPIO_PIN_13 |
GPIO_PIN_14 |
GPIO_PIN_15;
HAL_GPIO_Init(GPIOE, &gpio_init_struct);

/* 配置 FSMC_An 引脚 */
gpio_init_struct.Pin = GPIO_PIN_0 |
GPIO_PIN_1 |
GPIO_PIN_2 |
GPIO_PIN_3 |
GPIO_PIN_4 |
GPIO_PIN_5 |
GPIO_PIN_12 |
GPIO_PIN_13 |
GPIO_PIN_14 |
GPIO_PIN_15;
HAL_GPIO_Init(GPIOF, &gpio_init_struct);
gpio_init_struct.Pin = GPIO_PIN_0 |
GPIO_PIN_1 |
GPIO_PIN_2 |
GPIO_PIN_3 |
GPIO_PIN_4 |
GPIO_PIN_5;
HAL_GPIO_Init(GPIOG, &gpio_init_struct);
gpio_init_struct.Pin = GPIO_PIN_11 |
GPIO_PIN_12 |
```

```

        GPIO_PIN_13;
    HAL_GPIO_Init(GPIOD, &gpio_init_struct);

    /* 配置 FSMC 时序 */
    fsmc_timing_struct.AddressSetupTime = 0x02;
    fsmc_timing_struct.AddressHoldTime = 0x00;
    fsmc_timing_struct.DataSetupTime = 0x08;
    fsmc_timing_struct.AccessMode = FSMC_ACCESS_MODE_A;

    /* 配置 FSMC */
    g_sram_handle.Instance = FSMC_NORSRAM_DEVICE;
    g_sram_handle.Extended = FSMC_NORSRAM_EXTENDED_DEVICE;
    g_sram_handle.Init.NSBank = (SRAM_FSMC_NEX == 1) ? FSMC_NORSRAM_BANK1 :
    (SRAM_FSMC_NEX == 2) ? FSMC_NORSRAM_BANK2 :
    (SRAM_FSMC_NEX == 3) ? FSMC_NORSRAM_BANK3 :
    FSMC_NORSRAM_BANK4;
    g_sram_handle.Init.DataAddressMux = FSMC_DATA_ADDRESS_MUX_DISABLE;
    g_sram_handle.Init.MemoryType = FSMC_MEMORY_TYPE_SRAM;
    g_sram_handle.Init.MemoryDataWidth = FSMC_NORSRAM_MEM_BUS_WIDTH_16;
    g_sram_handle.Init.BurstAccessMode = FSMC_BURST_ACCESS_MODE_DISABLE;
    g_sram_handle.Init.WaitSignalPolarity = FSMC_WAIT_SIGNAL_POLARITY_LOW;
    g_sram_handle.Init.WrapMode = FSMC_WRAP_MODE_DISABLE;
    g_sram_handle.Init.WaitSignalActive = FSMC_WAIT_TIMING_BEFORE_WS;
    g_sram_handle.Init.WriteOperation = FSMC_WRITE_OPERATION_ENABLE;
    g_sram_handle.Init.WaitSignal = FSMC_WAIT_SIGNAL_DISABLE;
    g_sram_handle.Init.ExtendedMode = FSMC_EXTENDED_MODE_DISABLE;
    g_sram_handle.Init.AsynchronousWait = FSMC_ASYNCHRONOUS_WAIT_DISABLE;
    g_sram_handle.Init.WriteBurst = FSMC_WRITE_BURST_DISABLE;
    g_sram_handle.Init.PageSize = FSMC_PAGE_SIZE_NONE;

    HAL_SRAM_Init(&g_sram_handle, &fsmc_timing_struct, NULL);
}

```

从原理图中可以看到，SRAM 芯片的 CE 引脚连接到了 PG10 引脚（SMC_NE3 信号），因此在进行 SRAM 初始化后，SRAM 映射的内存基地址为 0x68000000，访问 SRAM 中存储的数据仅需访问 0x68000000 加上数据偏移后的地址即可。

47.2.5 实验应用代码

```

/* 定义测试数组
 * 起始地址为 SRAM_BASE_ADDR
 */
uint32_t g_test_buffer[250000] __attribute__((at(SRAM_BASE_ADDR)));

int main(void)
{
    uint32_t ts;
    uint8_t t = 0;
    uint8_t key;

    NVIC_ConfigPriorityGroup(NVIC_PRIORITY_GROUP_3); /* 设置中断优先级分组为组 3 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置系统时钟 */
    delay_init(168); /* 初始化延时功能 */
    usart_init(115200); /* 初始化串口 */
    usmart_dev_init(84); /* 初始化 USMART */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    lcd_init(); /* 初始化 LCD */
    sram_init(); /* 初始化外部 SRAM */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "SRAM TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY0:Test SRAM", RED);
}

```

```

lcd_show_string(30, 130, 200, 16, 16, "KEY_UP:Test Data", RED);

for (ts=0; ts<250000; ts++)                /* 创建测试数据 */
{
    g_test_buffer[ts] = ts;
}

while (1)
{
    t++;
    key = key_scan(0);

    if (key == KEY0_PRES)
    {
        /* 测试外部 SRAM 容量 */
        smc_sram_test(30, 150);
    }
    else if (key == WKUP_PRES)
    {
        /* 显示测试数据 */
        for (ts=0; ts<250000; ts++)
        {
            lcd_show_xnum(30, 170, g_test_buffer[ts], 6, 16, 0, BLUE);
        }
    }

    if (t == 20)
    {
        LED0_TOGGLE();
        t = 0;
    }

    delay_ms(10);
}
}

```

可以看到，应用代码中定义了一个起始地址为 `SRAM_BASE_ADDR` 的数组，`SRAM_BASE_ADDR` 是在 `sram.h` 文件中的一个宏定义，该宏定义用于表示 SRAM 进行映射后的内存起始地址，因此访问数组 `g_test_buffer` 就能访问 SRAM 中的数据。

在完成 SRAM 初始化后，便往 SRAM 中填充数据，随后便不断地检测按键输入，若检测到 `KEY_UP` 按键被按下，则将 SRAM 中的数据逐一地读出，然后在 LCD 上进行显示，这实际是测试了 SRAM 的读操作，若检测到 `KEY0` 按键被按下，则调用函数 `smc_sram_test()` 对 SRAM 进行读写测试，该函数如下所示：

```

/**
 * @brief    测试外部 SRAM 容量
 * @note     最大支持 1MB 容量的 SRAM
 * @param    x: LCD 上显示提示信息的起始 X 坐标
 * @param    y: LCD 上显示提示信息的起始 Y 坐标
 * @retval    无
 */
static void smc_sram_test(uint16_t x, uint16_t y)
{
    uint32_t i;
    uint8_t temp;
    uint8_t sval;

    lcd_show_string(x, y, 239, y + 16, 16, "Ex Memory Test: 0KB", BLUE);

    /* 每间隔 4KB 写入一个数据，总共写入 256 个数据，刚好为 1MB */
    for (temp=0, i=0; i<(1 * 1024 * 1024); i+=4096)
    {
        sram_write(&temp, i, 1);
        temp++;
    }
}

```

```

/* 读出写入的数据进行校验 */
for (i=0; i<(1 * 1024 * 1024); i+=4096)
{
    sram_read(&temp, i, 1);

    if (i == 0)
    {
        sval = temp;
    }
    else if (temp <= sval)
    {
        break;
    }

    /* 显示内存容量 */
    lcd_show_xnum( x + 15 * 8,
                    y,
                    (uint16_t)(temp - sval + 1) * 4,
                    4,
                    16,
                    0,
                    BLUE);
}

```

可以看到，函数 `smc_sram_test()` 就是往 SRAM 中写入数据，然后再读出校验，一次来进行 SRAM 的读写测试。

47.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上显示了本实验的实验信息，此时按下 `KEY_0` 按键对 SRAM 进行读写测试，便可以看到 LCD 上提示了测试外部 SRAM 的容量结果，如果一切正常，将提示 “Ex Memory Test:1024KB”，其中 1024KB 也就是开发板板载 SRAM 的实际容量（1M 字节），接着按下 `KEY_UP` 按键对 SRAM 进行读测试，可以看到 LCD 上不断地刷新显示一串数字，这便是从 SRAM 中读出的在 SRAM 初始化后被写入 SRAM 的 250000 个数据（0~249999）。

第三篇 提高篇

前面的章节介绍了开发 STM32 的工具和方法以及 STM32 的一些基础外设，本章将进入提高篇的学习，本篇的大部分内容都和 STM32 硬件关系不大，主要是介绍各种软件在 STM32 上的移植和使用，例如：内存管理、文件系统、图片解码库、中文字库、手写识别库、T9 拼音输入法、USB 库、OS 等。

本篇依然采用一章一示例的方式进行 STM32 各种高级应用的介绍，但将更多地介绍各种软件库的使用方法，而不会去分析各种软件库的实现方式，相信通过本篇的学习，读者能够使用 STM32 完成更复杂的项目开发。

第四十八章 内存管理实验

本章将介绍正点原子提供的内存管理库的使用,通过使用内存管理库可以提高内存的使用效率,在大型的应用开发中,是必不可少的工具。通过本章的学习,读者将学习到正点原子内存管理库的使用。

本章分为如下几个小节:

- 48.1 硬件设计
- 48.2 程序设计
- 48.3 下载验证

48.1 硬件设计

48.1.1 例程功能

1. 按下 WKUP 按键可从内部 SRAM、CCM 和外部 SRAM 申请内存,并在 LCD 上显示相关信息
2. 按下 KEY0 按键可释放最近一次从内部 SRAM、CCM 和外部 SRAM 申请到的内存
3. LED0 闪烁,提示程序正在运行

48.1.2 硬件资源

1. LED
LED0 - PF9
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
WKUP - PA0
KEY0 - PE4
4. 外部 SRAM

48.1.3 原理图

本章实验使用的内存管理库为软件库,因此没有对应的连接原理图。

48.2 程序设计

48.2.1 内存管理库的使用

正点原子提供的内存管理库包含两个文件,分别为: malloc.c 和 malloc.h,本章实验配套的实验例程中已经提供了这两个文件,并且已经针对正点原子 M144Z-M4 最小系统板 STM32F407 版进行了移植适配,用户在使用时,仅需将这两个文件添加到自己的工程即可,如下图所示:

```
./Middlewares/MALLOC/  
|-- malloc.c  
'-- malloc.h
```

图 48.2.1.1 正点原子内存管理库文件

内存管理库中提供了内存管理初始化、申请内存和释放内存等函数,使用非常方便。

在进行内存申请和释放之前,需要使用内存初始化函数对内存进行初始化,内存管理初始化的使用示例,如下所示:

```
#include " ./MALLOC/malloc.h"  
  
void example_fun(void)  
{  
    /* 初始化内部 SRAM 内存池 */  
}
```

```
my_mem_init(SRAMIN);
}
```

初始化内存后，便可以在需要内存的时候申请内存，申请内存函数的使用示例，如下所示：

```
#include "MALLOC/malloc.h"

void example_fun(void)
{
    uint8_t ptr;

    /* 初始化内部 SRAM 内存池 */
    my_mem_init(SRAMIN);

    /* 申请 1KB 内存 */
    ptr = (uint8_t *)mymalloc(SRAMIN, 1024);

    /* Do something. */
}
```

在内存使用完毕后，需要及时释放内存，否则可能产生内存泄漏，释放内存函数的使用示例，如下所示：

```
#include "MALLOC/malloc.h"

void example_fun(void)
{
    uint8_t ptr;

    /* 初始化内部 SRAM 内存池 */
    my_mem_init(SRAMIN);

    /* 申请 1KB 内存 */
    ptr = (uint8_t *)mymalloc(SRAMIN, 1024);

    /* Do something. */

    /* 释放无用内存 */
    myfree(SRAMIN, ptr);
}
```

48.2.2 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    uint8_t t = 0;
    uint8_t key;
    uint8_t *p_sramin = NULL;
    uint8_t *p_sramccm = NULL;
    uint8_t *p_sramex = NULL;
    uint32_t tp_sramin = 0;
    uint32_t tp_sramccm = 0;
    uint32_t tp_sramex = 0;
    uint8_t paddr[32];
    uint16_t memused;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟，168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    lcd_init(); /* 初始化 LCD */
    sram_init(); /* 初始化外部 SRAM */
    my_mem_init(SRAMIN); /* 初始化内部 SRAM 内存池 */
    my_mem_init(SRAMCCM); /* 初始化 CCM 内存池 */
}
```

```
my_mem_init(SRAMEX);          /* 初始化外部 SRAM 内存池 */

lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
lcd_show_string(30, 70, 200, 16, 16, "MALLOC TEST", RED);
lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

lcd_show_string(30, 110, 200, 16, 16, "KEY0:Malloc & WR & Show", RED);
lcd_show_string(30, 130, 200, 16, 16, "KEY_UP:Free", RED);

lcd_show_string(30, 162, 200, 16, 16, "SRAMIN USED:", BLUE);
lcd_show_string(30, 178, 200, 16, 16, "SRAMCCM USED:", BLUE);
lcd_show_string(30, 194, 200, 16, 16, "SRAMEX USED:", BLUE);

while (1)
{
    key = key_scan(0);
    if (key == WKUP_PRES)
    {
        /* 申请内存 */
        p_sramin = mymalloc(SRAMIN, 2048);
        p_sramccm = mymalloc(SRAMCCM, 2048);
        p_sramex = mymalloc(SRAMEX, 2048);

        /* 判断内存申请是否成功 */
        if ((p_sramin != NULL) && (p_sramccm != NULL) && (p_sramex != NULL))
        {
            /* 使用申请到的内存 */
            sprintf((char *)p_sramin,
                    "SRAMIN: Malloc Test%03d", t + SRAMIN);
            lcd_show_string(30, 258, 239, 16, 16, (char *)p_sramin, BLUE);
            sprintf((char *)p_sramccm,
                    "SRAMCCM: Malloc Test%03d", t + SRAMCCM);
            lcd_show_string(30, 274, 239, 16, 16, (char *)p_sramccm, BLUE);
            sprintf((char *)p_sramex,
                    "SRAMEX: Malloc Test%03d", t + SRAMEX);
            lcd_show_string(30, 290, 239, 16, 16, (char *)p_sramex, BLUE);
        }
        else
        {
            myfree(SRAMIN, p_sramin);
            myfree(SRAMCCM, p_sramccm);
            myfree(SRAMEX, p_sramex);
            p_sramin = NULL;
            p_sramccm = NULL;
            p_sramex = NULL;
        }
    }
    else if (key == KEY0_PRES)
    {
        /* 释放内存 */
        myfree(SRAMIN, p_sramin);
        myfree(SRAMCCM, p_sramccm);
        myfree(SRAMEX, p_sramex);
        p_sramin = NULL;
        p_sramccm = NULL;
        p_sramex = NULL;
    }

    /* 显示申请到内存的首地址 */
    if ((tp_sramin != (uint32_t)p_sramin) ||
        (tp_sramccm != (uint32_t)p_sramccm) ||
        (tp_sramex != (uint32_t)p_sramex))
    {
        tp_sramin = (uint32_t)p_sramin;
        tp_sramccm = (uint32_t)p_sramccm;
        tp_sramex = (uint32_t)p_sramex;
    }
}
```



```

        sprintf((char *)paddr,
                "SRAMIN: Addr: 0x%08X", (uint32_t)p_sramin);
        lcd_show_string(30, 110, 239, 16, 16, (char *)paddr, BLUE);
        sprintf((char *)paddr,
                "SRAMCCN: Addr: 0x%08X", (uint32_t)p_sramccm);
        lcd_show_string(30, 226, 239, 16, 16, (char *)paddr, BLUE);
        sprintf((char *)paddr,
                "SRAMEX: Addr: 0x%08X", (uint32_t)p_sramex);
        lcd_show_string(30, 242, 239, 16, 16, (char *)paddr, BLUE);
    }
    else if ((p_sramin == NULL) || (p_sramccm == NULL) || (p_sramex == NULL))
    {
        lcd_fill(30, 210, 239, 319, WHITE);
    }

    if (++t == 20)
    {
        t = 0;

        /* 显示内部 SRAM 使用率 */
        memused = my_mem_perused(SRAMIN);
        sprintf((char *)paddr, "%d.%01d%%", memused / 10, memused % 10);
        lcd_show_string(30 + 96, 162, 200, 16, 16, (char *)paddr, BLUE);

        /* 显示 CCM 使用率 */
        memused = my_mem_perused(SRAMCCM);
        sprintf((char *)paddr, "%d.%01d%%", memused / 10, memused % 10);
        lcd_show_string(30 + 104, 178, 200, 16, 16, (char *)paddr, BLUE);

        /* 显示外部 SRAM 使用率 */
        memused = my_mem_perused(SRAMEX);
        sprintf((char *)paddr, "%d.%01d%%", memused / 10, memused % 10);
        lcd_show_string(30 + 96, 194, 200, 16, 16, (char *)paddr, BLUE);

        LED0_TOGGLE();
    }

    delay_ms(10);
}
}

```

可以看到,本实验的应用代码使用到了三个内存池,分别为内部 SRAM、CCM 和外部 SRAM,在完成内存池初始化后,便在 LCD 上实时刷新显示三个内存池的使用量,以及检测按键输入,若检测到 WKUP 按键被按下,则从三个内存池中申请三块内存,并写入测试数据,然后将申请到的三块内存的起始地址即内存中写入的测试数据在 LCD 上进行显示,若检测到 KEY0 按键被按下,则释放最近一次申请的三块内存回对应的内存池中。

48.3 下载验证

在完成编译和烧录操作后,可以看到 LCD 上实时地显示了三个内存池的使用情况,此时按下 WKUP 按键申请内存,可以看到三个内存池的使用量增加,并且 LCD 上显示了申请到的三个内存的起始地址和内存中写入的测试数据,接着按下 KEY0 按键释放内存,可以看到, LCD 上显示的内存信息消失,并且因为内存已经被释放回内存池,因此内存池的使用量较少。

第四十九章 SD 卡实验

本章将介绍使用 STM32F407 驱动 SD 卡进行 SD 卡的识别、读写等操作。通过本章的学习，读者将学习到 SD 卡的使用。

本章分为如下几个小节：

- 49.1 硬件设计
- 49.2 程序设计
- 49.3 下载验证

49.1 硬件设计

49.1.1 例程功能

- 1. LCD 上显示 SD 卡信息
- 2. 按下 WKUP 按键可读取 SD 卡第 0 块的数据并通过串口显示
- 3. LED0 闪烁，提示程序正在运行

49.1.2 硬件资源

- 1. LED
 - LED0 - PF9
- 2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
- 3. 按键
 - WKUP - PA0
- 4. USART1
 - USART1_TX - PA9
 - USART1_RX - PA10
- 5. SD
 - SDIO_D0 - PC8
 - SDIO_D1 - PC9
 - SDIO_D2 - PC10
 - SDIO_D3 - PC11
 - SDIO_SCK - PC12
 - SDIO_CMD - PD2

49.1.3 原理图

本章实验使用 SDIO 接口与 SD 卡进行连接，开发板板载了一个 Micro SD 卡座，用于连接 SD 卡，SD 卡与 MCU 的连接原理图，如下图所示：

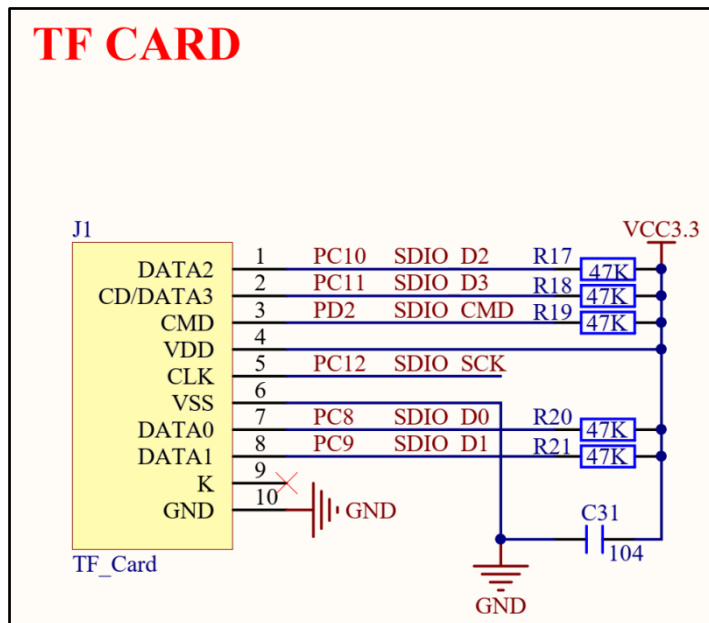


图 49.1.3.1 SD 卡与 MCU 的连接原理图

49.2 程序设计

49.2.1 SD 卡驱动

本章实验的 SD 卡驱动包含两个文件，分别为：sdio_sdcard.c 和 sdio_sdcard.h，SD 卡的驱动涉及 SD 的通信协议，本章实验配套实验例程的 SD 卡驱动已经根据 SD 协议实现了 SD 卡的初始化和读写等操作，对具体实现过程感兴趣的读者，可结合 SD 的协议查看本章实验配套实验例程的 SD 卡驱动。

49.2.2 实验应用代码

本章实验的应用代码，如下所示：

```
/**
 * @brief 显示 SD 卡信息
 * @param 无
 * @retval 无
 */
static void show_sd_info(void)
{
    HAL_SD_CardCIDTypeDef sd_card_cid = {0};

    HAL_SD_GetCardCID(&g_sd_handle, &sd_card_cid);

    printf("Card Type: %s\r\n",
        (g_sd_card_info.CardType == CARD_SDSC) ?
        ((g_sd_card_info.CardVersion == CARD_V1_X) ? ("SDSC V1") :
        ((g_sd_card_info.CardVersion == CARD_V1_X) ? ("SDSC V2") :
        (""))) :
        ((g_sd_card_info.CardType == CARD_SDHC_SDXC) ? ("SDHC") :
        ((g_sd_card_info.CardType == CARD_SECURED) ? ("SECURE") :
        (""))));

    printf("Card ManufacturerID:%d\r\n", sd_card_cid.ManufacturerID);
    printf("Card RCA:%d\r\n", g_sd_card_info.RelCardAdd);
    printf("LogBlockNbr:%d \r\n", g_sd_card_info.LogBlockNbr);
    printf("LogBlockSize:%d \r\n", g_sd_card_info.LogBlockSize);
    printf("Card Capacity:%d MB\r\n",
        (uint32_t)((uint64_t)g_sd_card_info.LogBlockNbr *
        g_sd_card_info.LogBlockSize) >> 20));
```

```
printf("Card BlockSize:%d\r\n\r\n", g_sd_card_info.BlockSize);

lcd_show_string(30, 146, 200, 16, 16, "SD Card Size:    MB", BLUE);
lcd_show_num(30 + 13 * 8, 146,
             (uint32_t)((uint64_t)g_sd_card_info.LogBlockNbr *
                       g_sd_card_info.LogBlockSize) >> 20), 5, 16, BLUE);
}

static void sd_read_test(void)
{
    uint8_t *buf;
    uint16_t index;

    buf = (uint8_t *)mymalloc(SRAMIN, g_sd_card_info.BlockSize);
    if (buf == NULL)
    {
        return;
    }

    /* 读取并打印 SD 卡第 0 个块的数据 */
    if (sd_read_disk(buf, 0, 1) == 0)
    {
        lcd_show_string(30, 170, 200, 16, 16, "USART1 Sending Data...", BLUE);
        printf("Block 0 Data:\r\n");
        for (index=0; index<g_sd_card_info.BlockSize; index++)
        {
            printf("%02X ", buf[index]);
        }
        printf("\r\nData End\r\n");
        lcd_show_string(30, 170, 200, 16, 16, "USART1 Send Data Over!", BLUE);
    }
    else
    {
        printf("SD read Failure!\r\n");
        lcd_show_string(30, 170, 200, 16, 16, "SD read Failure!    ", BLUE);
    }

    myfree(SRAMIN, buf);
}

int main(void)
{
    uint8_t t = 0;
    uint8_t key;

    HAL_Init();                /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168);           /* 初始化延时 */
    usart_init(115200);         /* 初始化串口 */
    led_init();                 /* 初始化 LED */
    key_init();                 /* 初始化按键 */
    lcd_init();                 /* 初始化 LCD */
    my_mem_init(SRAMIN);        /* 初始化内部 SRAM 内存池 */
    my_mem_init(SRAMCCM);       /* 初始化 CCM 内存池 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "SD TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    lcd_show_string(30, 110, 200, 16, 16, "WKUP: Read Block 0", RED);

    while (sd_init() != 0)
    {
        lcd_show_string(30, 130, 200, 16, 16, "SD Card Error!", RED);
        delay_ms(500);
    }
}
```

```

        lcd_show_string(30, 130, 200, 16, 16, "Please Check! ", RED);
        delay_ms(500);
        LED0_TOGGLE();
    }

    lcd_show_string(30, 130, 200, 16, 16, "SD Card OK    ", BLUE);

    /* 显示 SD 卡信息 */
    show_sd_info();

    while (1)
    {
        key = key_scan(0);
        if (key == WKUP_PRES)
        {
            /* 进行 SD 卡读测试 */
            sd_read_test();
        }

        if (++t == 20)
        {
            t = 0;
            LED0_TOGGLE();
        }

        delay_ms(10);
    }
}

```

可以看到，本实验的应用代码中，通过初始化 SD 卡判断与 SD 卡的连接是否有误，SD 卡初始化成功后便通过函数 `show_sd_info()` 打印 SD 卡的卡类型、容量等信息，同时也在 LCD 上显示了 SD 的容量信息，随后便不断地检测按键输入，若检测到 WKUP 按键被按下，则读取 SD 卡扇区 0 的 512 字节数据并通过串口输出至串口调试助手。

49.3 下载验证

在完成编译和烧录操作后，将准备好的 SD 卡插入开发板板载的 SD 卡卡座（请确保 SD 卡中没有有用的数据，或已做好备份），接着便能在 LCD 上看到 SD 卡的容量，以及串口调试助手显示了 SD 卡的卡类型、容量等信息，接着可以按下 WKUP 按键读取 SD 卡扇区 0 的 512 字节数据，可以通过串口调试助手查看读出的 512 字节数据。

第五十章 FATFS 实验

上一章实验中已经成功驱动 SD 卡，并可对 SD 卡进行读写操作，但读写 SD 卡时都是直接读出或写入二进制数据，这样使用起来显得十分不方便，因此本章将介绍 FATFS，FATFS 是一个通用的 FAT 文件系统模块，能够帮助实现文件系统，方便对 SD 卡、NOR Flash 或其他存储介质中数据的管理。通过本章的学习，读者将学习到 FATFS 的基本使用。

本章分为如下几个小节：

- 50.1 硬件设计
- 50.2 程序设计
- 50.3 下载验证

50.1 硬件设计

50.1.1 例程功能

- 1. LCD 上显示 SD 卡容量和剩余容量
- 2. 可通过 USMART 对 SD 卡和 NOR Flash 的文件系统进行操作
- 3. LED0 闪烁，提示程序正在运行

50.1.2 硬件资源

- 1. LED
 - LED0 - PF9
- 2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
- 3. 按键
 - WKUP - PA0
- 4. USART1
 - USART1_TX - PA9
 - USART1_RX - PA10
- 5. NOR Flash
 - SPI1_SCK - PB3
 - SPI1_MISO - PB4
 - SPI1_MOSI - PB5
 - SPI1_CS - PB14
- 6. SD
 - SDIO_D0 - PC8
 - SDIO_D1 - PC9
 - SDIO_D2 - PC10
 - SDIO_D3 - PC11
 - SDIO_SCK - PC12
 - SDIO_CMD - PD2

50.1.3 原理图

本章实验使用的 FATFS 为软件库，因此没有对应的连接原理图。

50.2 程序设计

50.2.1 FATFS 的使用

FATFS 涉及多个文件，在本章实验的配套实验例程中，如下图所示：

```
./Middlewares/FATFS/source/
|-- 00history.txt
|-- 00readme.txt
|-- diskio.c
|-- diskio.h
|-- ff.c
|-- ff.h
|-- ffconf.h
|-- ffsystem.c
|-- ffunicode.c
```

图 50.2.1.1 实验例程中的 FATFS 源文件

对于不同的硬件，使用 FATFS 仅需修改 diskio.c 和 ffconf.h 文件即可，并且本章实验配套的实验例程中的 FATFS 文件已经针对正点原子 M144Z-M4 最小系统板 STM32F407 版进行了移植适配，用户在使用时，仅需直接将这些文件添加到自己的工程，并根据实际需求适当修改 diskio.c 和 ffconf.h 文件即可。

为了方便读者使用 FATFS，本章实验配套的实验例程另外提供了四个文件，如下图所示：

```
./Middlewares/FATFS/exfuns/
|-- exfuns.c
|-- exfuns.h
|-- fattester.c
|-- fattester.h
```

图 50.2.1.2 实验 FATFS 扩展文件

这几个文件中提供了许多便捷使用和测试 FATFS 的函数，例如：获取文件类型、获取磁盘剩余容量和复制文件夹等。

50.2.2 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    FRESULT res;
    uint32_t total;
    uint32_t free;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    lcd_init(); /* 初始化 LCD */
    my_mem_init(SRAMIN); /* 初始化内部 SRAM 内存池 */
    my_mem_init(SRAMCCM); /* 初始化 CCM 内存池 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "FATFS TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "Use USMART for test", RED);

    while (sd_init() != 0)
    {
        lcd_show_string(30, 130, 200, 16, 16, "SD Card Error!", RED);
        delay_ms(500);
        lcd_show_string(30, 130, 200, 16, 16, "Please Check! ", RED);
        delay_ms(500);
        LED0_TOGGLE();
    }

    exfuns_init();

    /* 挂载 SD 卡 */
```

```
f_mount(fs[0], "0:", 1);

/* 挂载 NOR Flash, 如果 NOR Flash 没有文件系统则需要进行格式化 */
res = f_mount(fs[1], "1:", 1);
if (res == FR_NO_FILESYSTEM)
{
    lcd_show_string(30, 130, 200, 16, 16, "Flash Disk Formatting...", RED);
    res = f_mkfs("1:", NULL, NULL, FF_MAX_SS);
    if (res == FR_OK)
    {
        f_setlabel((const TCHAR *)"1:ALIEN TEK");
        lcd_show_string(30, 130, 200, 16, 16,
            "Flash Disk Format Finish", RED);
    }
    else
    {
        lcd_show_string(30, 130, 200, 16, 16,
            "Flash Disk Format Error ", RED);
    }
    delay_ms(1000);
}
lcd_fill(30, 130, 239, 145, WHITE);

/* 获取 SD 卡容量 */
while (exfuns_get_free("0", &total, &free) != 0)
{
    lcd_show_string(30, 130, 200, 16, 16, "SD Card FatFs Error!", RED);
    delay_ms(200);
    lcd_fill(30, 130, 240, 150 + 16, WHITE);
    delay_ms(200);
    LED0_TOGGLE();
}

lcd_show_string(30, 130, 200, 16, 16, "FATFS OK!", BLUE);
lcd_show_string(30, 150, 200, 16, 16, "SD Total Size:    MB", BLUE);
lcd_show_string(30, 170, 200, 16, 16, "SD Free Size:    MB", BLUE);
lcd_show_num(142, 150, total >> 10, 5, 16, BLUE);
lcd_show_num(134, 170, free >> 10, 5, 16, BLUE);

while (1)
{
    delay_ms(200);
    LED0_TOGGLE();
}
```

可以看到，本实验的应用代码中，使用函数 `exfuns_init()` 函数为使用 and 测试 FATFS 完成一些必要的初始化后，便挂载了 SD 卡和 NOR Flash，并且在 NOR Flash 上没有文件系统的情况下对其进行格式化，随后借助函数 `exfuns_get_free()` 获取 SD 卡的总容量和剩余容量，并且将其在 LCD 上进行显示。

本实验还使用到了 USMART 调试组件，方便使用串口调试助手测试 FATFS，`usmart_config.c` 文件中添加的函数，如下所示：

```
struct _m_usmart_nametab usmart_nametab[] =
{
    #if USMART_USE_WRFUNS == 1 /* 如果使能了读写操作 */
    (void *)read_addr, "uint32_t read_addr(uint32_t addr)",
    (void *)write_addr, "void write_addr(uint32_t addr,uint32_t val)",
    #endif
    (void *)delay_ms, "void delay_ms(uint16_t nms)",
    (void *)delay_us, "void delay_us(uint32_t nus)",

    (void *)mf_init, "uint8_t mf_init(void)",
    (void *)mf_free, "void mf_free(void)",
    (void *)mf_mount, "uint8_t mf_mount(uint8_t *path, uint8_t mt)",
    (void *)mf_open, "uint8_t mf_open(uint8_t *path, uint8_t mode)",
}
```



```
(void *)mf_close, "uint8_t mf_close(void)",
(void *)mf_read, "uint8_t mf_read(uint16_t len)",
(void *)mf_write, "uint8_t mf_write(uint8_t *pdata, uint16_t len)",
(void *)mf_opendir, "uint8_t mf_opendir(uint8_t *path)",
(void *)mf_closedir, "uint8_t mf_closedir(void)",
(void *)mf_readdir, "uint8_t mf_readdir(void)",
(void *)mf_scan_files, "uint8_t mf_scan_files(uint8_t *path)",
(void *)mf_showfree, "uint32_t mf_showfree(uint8_t *path)",
(void *)mf_lseek, "uint8_t mf_lseek(uint32_t offset)",
(void *)mf_tell, "uint32_t mf_tell(void)",
(void *)mf_size, "uint32_t mf_size(void)",
(void *)mf_mkdir, "uint8_t mf_mkdir(uint8_t *path)",
(void *)mf_fmks,
    "uint8_t mf_fmks(uint8_t *path, uint8_t opt, uint16_t au)",
(void *)mf_unlink, "uint8_t mf_unlink(uint8_t *path)",
(void *)mf_rename, "uint8_t mf_rename(uint8_t *oldname, uint8_t *newname)",
(void *)mf_getlabel, "void mf_getlabel(uint8_t *path)",
(void *)mf_setlabel, "void mf_setlabel(uint8_t *path)",
(void *)mf_gets, "void mf_gets(uint16_t size)",
(void *)mf_putc, "uint8_t mf_putc(uint8_t c)",
(void *)mf_puts, "uint8_t mf_puts(uint8_t *str)",
};
```

这么一来便可以通过串口调试助手借助 USMART 调试组件对 FATFS 进行测试。

50.3 下载验证

在完成编译和烧录操作后，将准备好的 SD 卡插入开发板板载的 SD 卡卡座（请确保 SD 卡中没有有用的数据，或已做好备份），接着若 SD 卡和 NOR Flash 都初始化并挂载和格式化成功，便能在 LCD 上看到 SD 卡的总容量和剩余容量。

接下来便可以通过串口调试助手借助 USMART 对 FATFS 进行测试，例如：“mf_scan_files(“0:”)”和 “mf_scan_files(“1:”)” 可分别查看 SD 卡和 NOR Flash 中文件系统根目录下的目录结构等。

第五十章 汉字显示实验

本章将介绍正点原子提供的字库管理库的使用，通过使用字库能够在 LCD 或其他显示设备上显示中文字符，对应中文应用的开发，是一个很有用的工具。通过本章的学习，读者将学习到正点原子字库管理库的使用。

本章分为如下几个小节：

- 51.1 硬件设计
- 51.2 程序设计
- 51.3 下载验证

51.1 硬件设计

51.1.1 例程功能

1. 自动检查并在无字库时自动更新字库
2. LCD 上不断刷新显示汉字
3. LED0 闪烁，提示程序正在运行

51.1.2 硬件资源

1. LED
 - LED0 - PF9
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
 - WKUP - PA0
4. NOR Flash
 - SPI1_SCK - PB3
 - SPI1_MISO - PB4
 - SPI1_MOSI - PB5
 - SPI1_CS - PB14
5. SD
 - SDIO_D0 - PC8
 - SDIO_D1 - PC9
 - SDIO_D2 - PC10
 - SDIO_D3 - PC11
 - SDIO_SCK - PC12
 - SDIO_CMD - PD2

51.1.3 原理图

本章实验使用的字库管理库为软件库，因此没有对应的连接原理图。

51.2 程序设计

51.2.1 字库管理库的使用

正点原子提供的字库管理库包含了四个文件，分别为：fonts.c、fonts.h、text.c 和 text.h，本章实验配套实验例程中已经提供了这四个文件，并且已经针对正点原子 M144Z-M4 最小系统板 STM32F407 版软硬件进行了移植适配，用户在使用时，仅需将这四个文件添加到自己的工程中即可，如下图所示：

```
./Middlewares/TEXT/
|-- fonts.c
|-- fonts.h
|-- text.c
`-- text.h
```

图 51.2.1.1 正点原子字库管理库文件

字库管理库中 fonts.c 和 fonts.h 两个文件提供了字库更新和初始化的函数，test.c 和 test.h 文件中提供了在 LCD 上显示中文字符的函数。

字库管理库在显示中文字符至 LCD 上时会使用 NOR Flash 中的中文字库，因此需要确保 NOR Flash 中的中文字库无误，若 NOR Flash 中没有中文字库的数据，那么在进行字库初始化时就会提示失败，这时就需要使用字库管理库中提供的字库更新函数更新 NOR Flash 中的中文字库数据，更新字库是读取 SD 卡中的字库文件将其写入 NOR Flash，因此需确保 SD 卡中有对应的中文字库文件。

本章实验所需的中文字库文件可在 A 盘→5，SD 卡根目录文件→SYSTEM→FONT 中找到，建议将 A 盘→5，SD 卡根目录文件中的所有文件按照该目录的目录结构复制到 SD 卡中，方便后续实验的使用。

51.2.2 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    uint8_t t = 0;
    uint8_t key;
    uint8_t res;
    uint32_t fontcnt;
    uint8_t i;
    uint8_t j;
    uint8_t fontx[2];

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    lcd_init(); /* 初始化 LCD */
    my_mem_init(SRAMIN); /* 初始化内部 SRAM 内存池 */
    my_mem_init(SRAMCCM); /* 初始化 CCM 内存池 */
    exfuncs_init(); /* 为 exfuncs 申请内存 */
    f_mount(fs[0], "0:", 1); /* 挂载 SD 卡 */
    f_mount(fs[1], "1:", 1); /* 挂载 NOR Flash */

    /* 检查字库 */
    while (fonts_init() != 0)
    {
UPD:
        lcd_clear(WHITE);
        lcd_show_string(30, 30, 200, 16, 16, "STM32", RED);

        /* 初始化 SD 卡 */
        while (sd_init() != 0)
        {
            lcd_show_string(30, 30, 200, 16, 16, "SD Card Error!", RED);
            delay_ms(500);
            lcd_show_string(30, 30, 200, 16, 16, "Please Check! ", RED);
            delay_ms(500);
            LED0_TOGGLE();
        }
    }
}
```

```
lcd_show_string(30, 50, 200, 16, 16, "SD Card OK", RED);
lcd_show_string(30, 70, 200, 16, 16, "Font Updating...", RED);

/* 更新字库 */
res = fonts_update_font(30, 90, 16, (uint8_t *) "0:", RED);
while (res != 0)
{
    lcd_show_string(30, 90, 200, 16, 16, "Font Update Failed!", RED);
    delay_ms(200);
    lcd_show_string(30, 90, 200, 16, 16, "Please Check! ", RED);
    delay_ms(200);
}

lcd_show_string(30, 90, 200, 16, 16, "Font Update Success! ", RED);
delay_ms(1500);
lcd_clear(WHITE);
}

text_show_string(30, 30, 200, 16, "正点原子 STM32 开发板", 16, 0, RED);
text_show_string(30, 50, 200, 16, "GBK 字库测试程序", 16, 0, RED);
text_show_string(30, 70, 200, 16, "ATOM@ALIENTEK", 16, 0, RED);
text_show_string(30, 90, 200, 16, "WKUP: 更新字库", 16, 0, RED);

text_show_string(30, 110, 200, 16, "内码高字节:", 16, 0, BLUE);
text_show_string(30, 130, 200, 16, "内码低字节:", 16, 0, BLUE);
text_show_string(30, 150, 200, 16, "汉字计数器:", 16, 0, BLUE);

text_show_string(30, 180, 200, 24, "对应汉字为:", 24, 0, BLUE);
text_show_string(30, 204, 200, 16, "对应汉字(16*16)为:", 16, 0, BLUE);
text_show_string(30, 220, 200, 12, "对应汉字(12*12)为:", 12, 0, BLUE);

while (1)
{
    fontcnt = 0;

    /* GBK 内码高字节范围为 0x81~0xFE */
    for (i=0x81; i<0xFF; i++)
    {
        fontx[0] = i;
        lcd_show_num(118, 110, i, 3, 16, BLUE);

        /* GBK 内码低字节范围为 0x40~0x7E、0x80~0xFE */
        for (j=0x40; j<0xFE; j++)
        {
            if (j == 0x7F)
            {
                continue;
            }

            fontcnt++;
            lcd_show_num(118, 130, j, 3, 16, BLUE);
            lcd_show_num(118, 150, fontcnt, 5, 16, BLUE);
            fontx[1] = j;
            text_show_font(30 + 132, 180, fontx, 24, 0, BLUE);
            text_show_font(30 + 144, 204, fontx, 16, 0, BLUE);
            text_show_font(30 + 108, 220, fontx, 12, 0, BLUE);

            t = 200;
            while ((t--) != 0)
            {
                delay_ms(1);

                key = key_scan(0);
                if (key == WKUP_PRES)
```

```

        {
            /* 更新字库 */
            goto UPD;
        }
    }

    LED0_TOGGLE();
}
}
}
}
}

```

从上面的代码中可以看出，本实验的应用代码中，在完成文件系统等一系列初始化后，便调用函数 `fonts_init()` 初始化字库管理库，若初始化失败则说明 NOR Flash 中没有对应的字库数据，因此需要进行更新字库，更新字库时，会从 SD 卡中读入字库数据并将其写入 NOR Flash 中，因此需要在 SD 中提前准备好字库文件。

当 NOR Flash 有了字库数据后，便在 LCD 上使用中文显示本实验的实验信息，并不断的显示中文字符，同时检测按键输入，若检测到 WKUP 按键被按下，则强制进行字库更新。

51.3 下载验证

在完成编译和烧录操作后，将根目录存放了 A 盘→5，SD 卡根目录文件中文件的 SD 卡插入开发板板载的 SD 卡卡座后，若 NOR Flash 中没有字库数据，则能够看到 LCD 显示了正在更新字库的提示，更新完成后，便可以在 LCD 上看到中文显示的实验信息和不断刷新显示的中文字符，此时可以按下 WKUP 按键强制更新字库。

第五十二章 图片显示实验

本章将介绍使用 STM32F407 软件解码 BMP、JPG 和 GIF 等格式的图片，并在 LCD 进行显示。通过本章的学习，读者将学习到图片解码库的使用。

52.1 硬件设计

52.2 程序设计

52.3 下载验证

52.1 硬件设计

52.1.1 例程功能

1. LCD 上不断刷新显示图片
2. 按下 WKUP 按键可切换至上一个图片
3. 按下 KEY0 按键可切换至下一个图片
4. LED0 闪烁，提示程序正在运行

52.1.2 硬件资源

1. LED
LED0 - PF9
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
WKUP - PA0
4. NOR Flash
SPI1_SCK - PB3
SPI1_MISO - PB4
SPI1_MOSI - PB5
SPI1_CS - PB14
5. SD
SDIO_D0 - PC8
SDIO_D1 - PC9
SDIO_D2 - PC10
SDIO_D3 - PC11
SDIO_SCK - PC12
SDIO_CMD - PD2

52.1.3 原理图

本章实验使用的图片解码库为软件库，因此没有对应的连接原理图。

52.2 程序设计

52.2.1 图片解码库的使用

正点原子提供的图片解码库包含了九个文件，分别为：piclib.c、piclib.h、bmp.c、bmp.h、gif.c、gif.h、tjpgd.c、tjpgd.h、tjpgdcnf.h，这几个文件的描述，如下表所示：

文件	描述
piclib.c piclib.h	正点原子图片解码库文件，主要提供图片解码库和画图的初始化和根据文件类型自动解码并画图的函数等
bmp.c bmp.h	正点原子 BMP 图片编解码库，主要用于解码 BMP 图片文件

gif.c gif.h	正点原子 GIF 图片解码库，主要用于解码 GIF 图片文件
tjpgd.c tjpgd.h tjpgdcnf.h	TjpgDec（JPEG 图片解码库）的源文件和配置文件，为了方便使用，添加了用于正点原子图片解码库的函数

表 52.2.1.1 图片解码库各文件描述

以上图片解码库的九个文件，在本章实验配套的实验例程中都已提供，并且针对正点原子 M144Z-M4 最小系统板 STM32F407 版软硬件进行了移植适配，用户在使用时，仅需将这九个文件添加到自己的工程即可，如下所示：

```
./Middlewares/PICTURE/
|-- bmp.c
|-- bmp.h
|-- gif.c
|-- gif.h
|-- piclib.c
|-- piclib.h
|-- tjpgd.c
|-- tjpgd.h
`-- tjpgdcnf.h
```

图 52.2.1.1 正点原子图片解码库文件

52.2.2 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    uint8_t t = 0;
    uint8_t key;
    uint8_t res;
    DIR picdir;
    uint16_t totpicnum;
    FILINFO *picfileinfo;
    char *pname;
    uint32_t *picoffsettbl;
    uint16_t curindex;
    uint16_t temp;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    lcd_init(); /* 初始化 LCD */
    my_mem_init(SRAMIN); /* 初始化内部 SRAM 内存池 */
    my_mem_init(SRAMCCM); /* 初始化 CCM 内存池 */
    exfuncs_init(); /* 为 exfuncs 申请内存 */
    f_mount(fs[0], "0:", 1); /* 挂载 SD 卡 */
    f_mount(fs[1], "1:", 1); /* 挂载 NOR Flash */

    /* 检查字库 */
    while (fonts_init() != 0)
    {
        lcd_show_string(30, 30, 200, 16, 16, "STM32", RED);

        /* 初始化 SD 卡 */
        while (sd_init() != 0)
        {
            lcd_show_string(30, 30, 200, 16, 16, "SD Card Error!", RED);
        }
    }
}
```

```

        delay_ms(500);
        lcd_show_string(30, 30, 200, 16, 16, "Please Check! ", RED);
        delay_ms(500);
        LED0_TOGGLE();
    }

    lcd_show_string(30, 50, 200, 16, 16, "SD Card OK", RED);
    lcd_show_string(30, 70, 200, 16, 16, "Font Updating...", RED);

    /* 更新字库 */
    res = fonts_update_font(30, 90, 16, (uint8_t *) "0:", RED);
    while (res != 0)
    {
        lcd_show_string(30, 90, 200, 16, 16, "Font Update Failed!", RED);
        delay_ms(200);
        lcd_show_string(30, 90, 200, 16, 16, "Please Check! ", RED);
        delay_ms(200);
    }

    lcd_show_string(30, 90, 200, 16, 16, "Font Update Success! ", RED);
    delay_ms(1500);
    lcd_clear(WHITE);
}

text_show_string(30, 30, 200, 16, "正点原子 STM32 开发板", 16, 0, RED);
text_show_string(30, 50, 200, 16, "图片显示实验", 16, 0, RED);
text_show_string(30, 70, 200, 16, "ATOM@ALIENTEK", 16, 0, RED);
text_show_string(30, 90, 200, 16, "WKUP: PREV", 16, 0, RED);
text_show_string(30, 110, 200, 16, "KEY0: NEXT", 16, 0, RED);

/* 打开图片文件夹 */
while (f_opendir(&picdir, "0:/PICTURE") != FR_OK)
{
    text_show_string(30, 130, 200, 16, "PICTURE 文件夹错误!", 16, 0, RED);
    delay_ms(200);
    lcd_fill(30, 130, 200, 16, WHITE);
    delay_ms(200);
}

/* 获取有效图片文件数量 */
totpicnum = pic_get_tnum("0:/PICTURE");
while (totpicnum == 0)
{
    text_show_string(30, 130, 200, 16, "没有图片文件!", 16, 0, RED);
    delay_ms(200);
    lcd_fill(30, 130, 200, 16, WHITE);
    delay_ms(200);
}

/* 申请内存 */
picfileinfo = (FILINFO *)mymalloc(SRAMIN, sizeof(FILINFO));
pname = (char *)mymalloc(SRAMIN, FF_MAX_LFN * 2 + 1);
picoffsettbl = (uint32_t *)mymalloc(SRAMIN, 4 * totpicnum);
while ((picfileinfo == NULL) || (pname == NULL) || (picoffsettbl == NULL))
{
    text_show_string(30, 130, 200, 16, "内存分配失败!", 16, 0, RED);
    delay_ms(200);
    lcd_fill(30, 130, 200, 16, WHITE);
    delay_ms(200);
}

/* 打开目录并记录图片索引 */
res = (uint8_t)f_opendir(&picdir, "0:/PICTURE");
if (res == 0)
{

```



```

    curindex = 0;
    while (1)
    {
        temp = picdir.dptr;
        res = (uint8_t)f_readdir(&picdir, picfileinfo);
        if ((res != 0) || (picfileinfo->fname[0] == 0))
        {
            break;
        }

        res = exfuns_file_type(picfileinfo->fname);
        if ((res & 0xF0) == 0x50)
        {
            picoffsettbl[curindex] = temp;
            curindex++;
        }
    }
}

/* 开始显示图片 */
text_show_string(30, 130, 200, 16, "开始显示...", 16, 0, RED);
delay_ms(1500);
piclib_init();
curindex = 0;
res = (uint8_t)f_opendir(&picdir, (const TCHAR *)"0:/PICTURE");
while (res == 0)
{
    /* 获取下一个图片文件信息 */
    dir_sdi(&picdir, picoffsettbl[curindex]);
    res = (uint8_t)f_readdir(&picdir, picfileinfo);
    if ((res != 0) || (picfileinfo->fname[0] == 0))
    {
        break;
    }

    /* 根据图片路径显示图片 */
    strcpy((char *)pname, "0:/PICTURE/");
    strcat((char *)pname, (const char *)picfileinfo->fname);
    lcd_clear(BLACK);
    piclib_ai_load_picfile(pname, 0, 0, lcddev.width, lcddev.height, 1);
    text_show_string(2, 2, lcddev.width, 16, (char *)pname, 16, 1, RED);

    while (1)
    {
        key = key_scan(0);
        if (key == KEY0_PRES)
        {
            /* 切换上一张 */
            if (curindex != 0)
            {
                curindex--;
            }
            else
            {
                curindex = totpicnum - 1;
            }
            break;
        }
        else if (key == WKUP_PRES)
        {
            /* 切换下一张 */
            curindex++;
            if (curindex >= totpicnum)
            {
                curindex = 0;
            }
        }
    }
}

```

```

        break;
    }

    if (++t == 20)
    {
        t = 0;
        LED0_TOGGLE();
    }

    delay_ms(10);
}

/* 释放内存 */
myfree(SRAMIN, picfileinfo);
myfree(SRAMIN, pname);
myfree(SRAMIN, picoffsettbl);

while (1);
}

```

从上面的代码中可以看出，本实验的应用代码中，在完成文件系统、SD 卡等一些列必要的初始化后，便会统计 SD 卡中 PICTURE 文件夹下图片的数量和路径，然后根据 WKUP 和 KEY0 按键，调用函数 `piclib_ai_load_picfile()` 切换显示 SD 卡 PICTURE 文件夹中的图片文件至 LCD 上。

本章实验所需的图片文件可以在 A 盘→5，SD 卡根目录文件→PICTURE 中找到，建议将 A 盘→5，SD 卡根目录文件中的所有文件按照该目录的目录结构复制进 SD 卡，方便后续实验的使用。

52.3 下载验证

在完成编译和烧录操作后，将根目录存放了 A 盘→5，SD 卡根目录文件中文件的 SD 卡插入开发板板载的 SD 卡卡座后，便能看到 LCD 上显示了 SD 卡 PICTURE 文件夹中的图片，并且按下 WKUP 按键或 KEY0 按键可以切换 LCD 显示 SD 卡 PICTURE 文件夹中的上一张或下一张图片。

第五十三章 照相机实验

本章将介绍使用 STM32F407 模拟照相机将摄像头采集到的图像数据以 BMP 或 JPEG 的文件格式保存至 SD 中，是前面章节中 SD 卡驱动、摄像头驱动、FATFS、字库管理库、图片编解码库等的综合应用。通过本章的学习，读者将学习到编码 BMP 文件以及保存摄像头采集到图像数据的保存。

本章分为如下几个小节：

53.1 硬件设计

53.2 程序设计

53.3 下载验证

53.1 硬件设计

53.1.1 例程功能

1. 按下 WKUP 按键对摄像头输出的数据进行 BMP 编码并保存值 SD 卡中
2. LED1 闪烁，提示 DCMI 接收到一帧数据
3. LED0 闪烁，提示程序正在运行

53.1.2 硬件资源

1. LED
 - LED0 - PF9
 - LED1 - PF10
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
 - WKUP - PA0
4. ATK-MC2640 摄像头模块
 - OV_D0~D7 - PC6/PC7/PC8/PC9/PC11/PB6/PE5/PE6
 - OV_SCL - PD6
 - OV_SDA - PD7
 - OV_VSYNC - PB7
 - OV_HREF - PA4
 - OV_PCLK - PA6
 - OV_PWDN - PG9
 - OV_RESET - PG15
 - OV_XCLK - PA8
5. SD
 - SDIO_D0 - PC8
 - SDIO_D1 - PC9
 - SDIO_D2 - PC10
 - SDIO_D3 - PC11
 - SDIO_SCK - PC12
 - SDIO_CMD - PD2

53.1.3 原理图

本章实验涉及的 SD 卡、摄像头、LCD 等的连接原理图，请读者自行查看前面对应章节中的连接原理图。

53.2 程序设计

53.2.1 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    uint8_t t = 0;
    uint8_t key;
    uint8_t res;
    uint8_t sd_ok = 1;
    uint8_t *pname;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    lcd_init(); /* 初始化 LCD */
    piclib_init(); /* 初始化画图 */
    my_mem_init(SRAMIN); /* 初始化内部 SRAM 内存池 */
    my_mem_init(SRAMCCM); /* 初始化 CCM 内存池 */
    exfuncs_init(); /* 为 exfuncs 申请内存 */
    f_mount(fs[0], "0:", 1); /* 挂载 SD 卡 */
    f_mount(fs[1], "1:", 1); /* 挂载 NOR Flash */

    /* 检查字库 */
    while (fonts_init() != 0)
    {
        lcd_show_string(30, 90, 200, 16, 16, "Font error!", RED);
        delay_ms(200);
        lcd_fill(20, 90, 200 + 20, 90 + 16, WHITE);
        delay_ms(200);
    }

    /* 初始化 OV2640 */
    while (ov2640_init() != 0)
    {
        text_show_string(30, 170, 240, 16, "OV2640 错误!", 16, 0, RED);
        delay_ms(200);
        lcd_fill(30, 150, 239, 206, WHITE);
        delay_ms(200);
    }

    sw_sdcard_mode();

    /* 初始化 SD 卡 */
    while (sd_init() != 0)
    {
        lcd_show_string(30, 50, 200, 16, 16, "SD Card Failed!", RED);
        delay_ms(200);
        lcd_fill(30, 50, 200 + 30, 50 + 16, WHITE);
        delay_ms(200);
    }

    text_show_string(30, 50, 200, 16, "正点原子 STM32 开发板", 16, 0, RED);
    text_show_string(30, 70, 200, 16, "照相机实验", 16, 0, RED);
    text_show_string(30, 90, 200, 16, "WKUP:拍照 (bmp 格式)", 16, 0, RED);

    /* 创建 PHOTO 文件夹 */
    res = (uint8_t)f_mkdir("0:/PHOTO");
}
```

```

if ((res != (uint8_t)FR_EXIST) && (res != 0))
{
    text_show_string(30, 150, 240, 16, "SD 卡错误!", 16, 0, RED);
    text_show_string(30, 150, 240, 16, "拍照功能将不可用!", 16, 0, RED);
    sd_ok = 0;
}

/* 为带路径的文件名申请内存 */
pname = (uint8_t *)mymalloc(SRAMIN, 30);
while (pname == NULL)
{
    text_show_string(30, 150, 240, 16, "内存分配失败!", 16, 0, RED);
    delay_ms(200);
    lcd_fill(30, 150, 240, 146, WHITE);
    delay_ms(200);
}

sw_ov2640_mode();
ov2640_rgb565_mode();
dcmi_init();
dcmi_dma_init( (uint32_t)&(LCD->LCD_RAM),
               0,
               1,
               DMA_MDATAALIGN_HALFWORD,
               DMA_MINC_DISABLE);
ov2640_outsize_set(lcddev.width, lcddev.height);
dcmi_start();
ov2640_flash_intctrl();

/* 配置 OV2640 为 RGB565 模式 */
/* 初始化 DCMI */
/* 配置 DCMI DMA */

/* 全屏缩放 */
/* 启动 DCMI 传输 */
/* 内部控制闪光灯 */

while (1)
{
    t++;
    key = key_scan(0);

    if (key != 0)/* 有按键按下 */
    {
        dcmi_stop();/* 先禁止 DCMI 传输 */
        if (sd_ok == 1)/* SD 卡正常 */
        {
            sw_sdcard_mode();/* 切换为 SD 卡模式 */
            switch (key)
            {
                case WKUP_PRES:
                {
                    /* 得到 BMP 格式文件名 */
                    camera_new_pathname(pname, 0);
                    /* 编码并保存 BMP 文件 */
                    res = bmp_encode(pname,
                                     0,
                                     0,
                                     lcddev.width,
                                     lcddev.height,
                                     0);

                    break;
                }
                default:
                {
                    break;
                }
            }
        }
        sw_ov2640_mode();
        if (res != 0)
        {
            text_show_string(30, 130, 240, 16,

```

```

        "写入文件错误!", 16, 0, RED);
    }
    else
    {
        text_show_string(10, 130, 240, 16, "拍照成功!", 16, 0, RED);
        text_show_string(10, 150, 240, 16, "保存为:", 16, 0, RED);
        text_show_string(10 + 56, 150, 240, 16,
                        (char*)pname, 16, 0, RED);
    }

    delay_ms(1000);
}
else/* SD 卡不可用 */
{
    text_show_string(30, 130, 240, 16, "SD 卡错误!", 16, 0, RED);
    text_show_string(30, 150, 240, 16, "拍照功能不可用!", 16, 0, RED);
}
delay_ms(2000);
dcmi_start();/* 恢复 DCMI 传输 */
}

if (t == 20)
{
    LED0_TOGGLE();
    t = 0;
}

delay_ms(10);
}
}

```

可以看到，本章实验实际上是前面章节中 SD 卡驱动、摄像头驱动、FATFS、字库管理库、图片编解码库等的综合应用，并没有涉及新的内容。要注意的是，对于正点原子 M144Z-M4 最小系统板 STM32F407 版，其 SD 卡和摄像头是无法同时工作的，因为它们使用到了部分共同的 GPIO 引脚，因此在本章实验中，需要对 GPIO 引脚的复用模式进行切换，以正常使用 SD 卡或摄像头，用于切换 GPIO 引脚复用模式的函数，如下所示：

```

/**
 * @brief    切换为 OV2640 模式
 * @note     切换 PC8/PC9/PC11 为 DCMI 复用功能（AF13）
 * @param    无
 * @retva    1 无
 */
static void sw_ov2640_mode(void)
{
    GPIO_InitTypeDef gpio_init_struct = {0};

    OV2640_PWDN(0);
    gpio_init_struct.Pin = GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_11;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;
    gpio_init_struct.Pull = GPIO_PULLUP;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
    gpio_init_struct.Alternate = GPIO_AF13_DCMI;
    HAL_GPIO_Init(GPIOC, &gpio_init_struct);
}

/**
 * @brief    切换为 SD 卡模式
 * @note     切换 PC8/PC9/PC11 为 SDIO 复用功能（AF12）
 * @param    无
 * @retval    无
 */
static void sw_sdcard_mode(void)
{
    GPIO_InitTypeDef gpio_init_struct = {0};

```

```
OV2640_PWDN(1);
gpio_init_struct.Pin = GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_11;
gpio_init_struct.Mode = GPIO_MODE_AF_PP;
gpio_init_struct.Pull = GPIO_PULLUP;
gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
gpio_init_struct.Alternate = GPIO_AF12_SDIO;
HAL_GPIO_Init(GPIOC, &gpio_init_struct);
}
```

从应用代码的初始化函数中可以看到，在初始化过程中，配置了 DCMI 和 DMA 将摄像头返回的图像数据传输至 LCD 上进行显示，初始化完成后，便检测按键输入，若无按键输入，则一直在 LCD 上显示摄像头采集到的图像数据，若检测到 WKUP 按键被按下，则调用函数 `bmp_encode()` 读取 LCD 上整个屏幕显示的数据并编码保存到 SD 卡上。

53.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上实时显示这摄像头采集到的图像数据，此时若按下 WKUP 按键，可以看到 LCD 上显示了拍照成功和 BMP 图片保存路径的提示信息。拍照完成后，可通过 PC 或图片显示实验例程（需修改获取图片的文件夹路径为“0:PHOTO”）查看所拍摄的 BMP 格式的照片。

第五十四章 视频播放器实验

本章将介绍使用 STM32F407 实现一个简单的视频播放器应用。通过本章的学习，读者将学习到使用 libjpeg 库软解 JPEG 实现播放 AVI 视频的使用。

本章分为如下几个小节：

- 54.1 硬件设计
- 54.2 程序设计
- 54.3 下载验证

54.1 硬件设计

54.1.1 例程功能

1. LCD 上自动播放 AVI 格式视频
2. 按下 WKUP 按键可切换至上一个视频
3. 按下 KEY0 按键可切换至下一个视频
4. LED0 闪烁，提示程序正在运行

54.1.2 硬件资源

1. LED
 - LED0 - PF9
 - LED1 - PF10
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
 - WKUP - PA0
 - KEY0 - PE4
4. NOR Flash
 - SPI1_SCK - PB3
 - SPI1_MISO - PB4
 - SPI1_MOSI - PB5
 - SPI1_CS - PB14
5. SD
 - SDIO_D0 - PC8
 - SDIO_D1 - PC9
 - SDIO_D2 - PC10
 - SDIO_D3 - PC11
 - SDIO_SCK - PC12
 - SDIO_CMD - PD2

54.1.3 原理图

本章实验主要涉及 libjpeg 软件库的使用和 AVI 文件的解析，因此没有对应的连接原理图。

54.2 程序设计

54.2.1 libjpeg 库的使用

本章实验要实现一个简单的视频播放器，以播放 AVI 格式的视频，AVI 格式的视频数据可以采用 MJPG 进行编码，因此在播放时，需要对 MJPG 数据流进行解码，这样便可以从 MJPG 数据流中获得视频的逐帧画面，只需要在 LCD 上连续显示这些画面，就能够实现视频播放。

在第五十二章“图片显示实验”中使用 TjpgDec 解码 JPEG 图片，虽然 TjpgDec 占用资源少，

但解码速度慢，若用于解码 MJPG 数据流，会导致视频播放不够流畅，因此本章实验使用解码速度更快的 libjpeg 库解码 MJPG 数据流。

关于 libjpeg 库的移植和使用，请读者自行查看 libjpeg 源码中的介绍文件，可以重点看 readme.txt、filelist.txt、install.txt 和 libjpeg.txt 等文件，可以参考本章实验配套实验例程进行移植和使用。

为了方便地使用 libjpeg 实现本章实验的功能，正点原子提供了 mjpeg.c 和 mjpeg.h 这两个文件，这两个文件提供了初始化、解码等三个函数，大大简化了 libjpeg 的使用流程，函数原型，如下所示：

```
uint8_t mjpegdec_init(uint16_t offx,uint16_t offy);
void mjpegdec_free(void);
uint8_t mjpegdec_decode(uint8_t* buf,uint32_t bsize);
```

这三个函数的使用也非常简单，光看函数名也能猜出各个函数的作用了，因此具体的使用方法，请读者自行查看本章实验的配套实验例程。

54.2.2 AVI 文件解析

有关 AVI 文件格式的介绍和解析方式，请感兴趣的读者自行查阅相关的资料。对于 AVI 文件的解析，正点原子提供了 avi.c 和 avi.h 两个文件，请读者结合本章实验配套的实验例程进行查看。

54.2.3 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    /* 必要初始化，代码省略 */

    while (1)
    {
        video_play();
    }
}
```

可以看到，本章实验在完成一些必要的软硬件初始化后，便调用函数 video_play()进行视频播放，该函数如下所示：

```
/**
 * @brief 播放视频
 * @param 无
 * @retval 无
 */
void video_play(void)
{
    uint8_t res;
    DIR vdir;
    FILINFO *vfileinfo;
    uint8_t *pname;
    uint16_t totavinum;
    uint16_t curindex;
    uint32_t *voffsettbl;
    uint8_t key;

    /* 检查 VIDEO 文件夹是否存在，代码省略 */

    /* 检查是否有视频文件，代码省略 */

    /* 申请内存，代码省略 */

    /* 记录视频文件的索引，代码省略 */

    curindex = 0;
```

```

res = (uint8_t)f_opendir(&vdir, "0:/VIDEO");/* 打开目录 */
while (res == 0)
{
    /* 索引到待播放的视频文件 */
    dir_sdi(&vdir, voffsettbl[curindex]);
    res = (uint8_t)f_readdir(&vdir, vfileinfo);
    if ((res != 0) || (vfileinfo->fname[0] == 0))
    {
        break;
    }

    /* 生成视频文件路径信息 */
    strcpy((char *)pname, "0:/VIDEO/");
    strcat((char *)pname, (const char *)vfileinfo->fname);

    /* 显示视频基本信息 */
    lcd_clear(WHITE);
    video_bmsg_show((uint8_t *)vfileinfo->fname, curindex + 1, totavinum);

    /* 播放视频 */
    key = video_play_mjpeg(pname);

    /* 根据键值，切换播放上（下）一个视频文件 */
}

/* 释放内存，代码省略 */
}

```

从上面的代码中可以看出，主要就是调用函数 `video_play_mjpeg()` 来播放 AVI 视频文件，该函数如下所示：

```

/**
 * @brief   播放 MJPEG 视频
 * @param   pname: 视频文件名
 * @retval  按键键值
 *          KEY0_PRES: 上一个视频
 *          WKUP_PRES: 下一个视频
 *          其他值: 错误代码
 */
static uint8_t video_play_mjpeg(uint8_t *pname)
{
    uint8_t *framebuf;
    uint8_t *pbuf;
    uint8_t res = 0;
    uint16_t offset;
    uint32_t nr;
    uint8_t key;
    FIL *favi;
    uint8_t *psaibuf;

    /* 申请内存，代码省略 */

    while (res == 0)
    {
        /* 打开文件 */
        res = (uint8_t)f_open(favi, (const TCHAR *)pname, FA_READ);
        if (res == 0)
        {
            pbuf = framebuf;
            /* 读取文件 */
            res = (uint8_t)f_read(favi, pbuf, AVI_VIDEO_BUF_SIZE, &nr);
            if (res != 0)
            {
                printf("fread error:%d\r\n", res);
                break;
            }
        }
    }
}

```

```

    }

    /* AVI 初始化解析，获取 AVI 视频的帧间隔时间、总帧数等信息 */
    res = avi_init(pbuf, AVI_VIDEO_BUF_SIZE);
    if (res != 0)
    {
        printf("avi error:%d\r\n", res);
        break;
    }

    /* 显示 AVI 初始化解析得到的部分信息 */
    video_info_show(&g_avix);
    /* 初始化 TIM7 以 AVI 视频的帧间隔时间为周期产生中断，
     * 以逐帧显示 AVI 视频画面
     */
    btmr_tmr7_int_init(g_avix.SecPerFrame / 100 - 1, 8400 - 1);
    /* 获取第一个 MJPG 数据流位置 */
    offset = avi_srarch_id(pbuf, AVI_VIDEO_BUF_SIZE, "movi");
    avi_get_streaminfo(pbuf + offset + 4);
    f_lseek(favi, offset + 12);
    /* 初始化 libjpeg 解码 MJPG 数据流 */
    res = mjpegdec_init((lcddev.width - g_avix.Width) / 2,
                        10 + (lcddev.height - 110 - g_avix.Height) / 2);

    while (1)
    {
        /* 视频流 */
        if (g_avix.StreamID == AVI_VIDS_FLAG)
        {
            pbuf = framebuf;
            /* 读取当前视频流的整帧数据和下一帧数据流的 ID 信息 */
            f_read(favi, pbuf, g_avix.StreamSize + 8, &nr);
            /* 解码当前视频流数据，并在 LCD 上进行显示 */
            res = mjpegdec_decode(pbuf, g_avix.StreamSize);
            if (res != 0)
            {
                printf("decode error!\r\n");
            }

            /* 等待 TIM7 中断，以播放下一帧 */
            while (frameup == 0);
            frameup = 0;
            frame++;
        }
        /* 其他数据流，如音频流等 */
        else
        {
            /* 显示当前播放时间 */
            video_time_show(favi, &g_avix);
            /* 读取当前数据流的整帧数据和下一帧数据流的 ID 信息 */
            f_read(favi, psaibuf, g_avix.StreamSize + 8, &nr);
            pbuf = psaibuf;
        }

        /* 按键按下，提前退出 */
        key = key_scan(0);
        if (key != 0)
        {
            res = key;
            break;
        }

        /* 解析下一帧数据流的 ID 信息，以判断是否为视频流 */
        if (avi_get_streaminfo(pbuf + g_avix.StreamSize) != 0)
    }

```

```

        {
            printf("g_frame error\r\n");
            res = WKUP_PRES;
            break;
        }
    }

    /* 关闭当前视频播放，代码省略 */
}

/* 释放内存，代码省略 */

return res;
}

```

从上面的代码中可以看出，主要就是逐帧的读取 AVI 文件的数据流和下一帧数据流的 ID 信息，如果当前数据流为视频流则调用函数 `mjpegdec_decode()` 对齐进行解码并在 LCD 上进行显示，若不是视频流则直接跳过，不进行处理，并且在每一帧数据流处理完毕后，都会检测一次按键，以进行上（下）一个视频切换播放的操作。

54.3 下载验证

在完成编译和烧录操作后，将根目录存放了 A 盘→5，SD 卡根目录文件中文件的 SD 卡插入开发板板载的 SD 卡卡座后，便能看到 LCD 上显示了 SD 卡 VIDEO 文件夹中的视频信息，并且在 LCD 上自动播放了该视频文件，此时，若按下 WKUP 按键或 KEY0 按键可以切换 LCD 显示播放 SD 卡 VIDEO 文件夹中的上一个或下一个视频。

第五十五章 FPU 测试（Julia 分形）实验

本章介绍使用 STM32F407 的 FPU 加速浮点运算。通过本章的学习，读者将学习到如何开启 STM32F407 的 FPU。

本章分为如下几个小节：

- 55.1 硬件设计
- 55.2 程序设计
- 55.3 下载验证

55.1 硬件设计

55.1.1 例程功能

1. 自动计算 Julia 图形并在 LCD 进行显示，同时显示单次计算耗时
2. 按下 WKUP 按键可增加缩放因子
3. 按下 KEY0 按键可切换自动和手动增加缩放因子
4. LED0 闪烁，提示程序正在运行

55.1.2 硬件资源

1. LED
 - LED0 - PF9
 - LED1 - PF10
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
 - WKUP - PA0
 - KEY0 - PE4
4. FPU
5. TIM6
6. USART1
 - USART1_TX - PA9
 - USART1_RX - PA10

55.1.3 原理图

本章实验使用的 FPU 为 STM32F407 的片上资源，因此没有对应的连接原理图。

55.2 程序设计

55.2.1 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    uint8_t key;
    uint8_t zoom = 0;
    uint8_t autorun = 0;
    float time;
    char buf[50];

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
}
```

```

led_init(); /* 初始化 LED */
key_init(); /* 初始化按键 */
lcd_init(); /* 初始化 LCD */
btim_timx_int_init(0xFFFF, 8400 - 1); /* 初始化基本定时器 */

lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
lcd_show_string(30, 70, 200, 16, 16, "FPU TEST", RED);
lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

lcd_show_string(30, 110, 200, 16, 16, "KEY UP:+", RED);
lcd_show_string(30, 130, 200, 16, 16, "KEY0:Auto/Manul", RED);

delay_ms(500);

/* 初始化颜色表 */
julia_clut_init(g_color_map);

while (1)
{
    key = key_scan(0);
    if (key == WKUP_PRES)
    {
        if (++zoom > (sizeof(zoom_ratio) / sizeof(uint16_t) - 1))
        {
            zoom = 0;
        }
    }
    else if (key == KEY0_PRES)
    {
        autorun = !autorun;
    }

    if (autorun != 0)
    {
        LED1(0);
        if (++zoom > (sizeof(zoom_ratio) / sizeof(uint16_t) - 1))
        {
            zoom = 0;
        }
    }
    else
    {
        LED1(1);
    }

    /* 绘图及计时准备工作 */
    lcd_set_window(0, 0, lcddev.width, lcddev.height);
    lcd_write_ram_prepare();
    __HAL_TIM_SET_COUNTER(&g_timx_handle, 0);
    g_timeout = 0;

    /* 绘图及计算耗时 */
    julia_generate_fpu(lcddev.width, lcddev.height,
                      lcddev.width / 2, lcddev.height / 2,
                      zoom_ratio[zoom]);
    time = __HAL_TIM_GET_COUNTER(&g_timx_handle) +
            (uint32_t)g_timeout * 0x10000;
    sprintf(buf, "%s: zoom:%d runtime:%0.1fms\r\n",
            FPU_MODE, zoom_ratio[zoom], time / 10);
    lcd_show_string(5, lcddev.height - 5 - 12, lcddev.width - 5, 12,
                    12, buf, RED);
    printf("%s", buf);
    LED0_TOGGLE();
}
}

```

从上面的代码中可以看出,本实验就是使用函数 `julia_genetate_fpu()` 实时产生 Julia 分形图形,并在 LCD 上进行显示,同时通过 TIM6 计算函数 `julia_generate_fpu()` 的执行时间,并将执行时间在 LCD 上进行显示。

本章实验要对比开启和关闭 FPU 的实验效果,在 Options for Target 窗口的 Target 选项卡中可以开启或关闭使用 FPU,如下图所示:

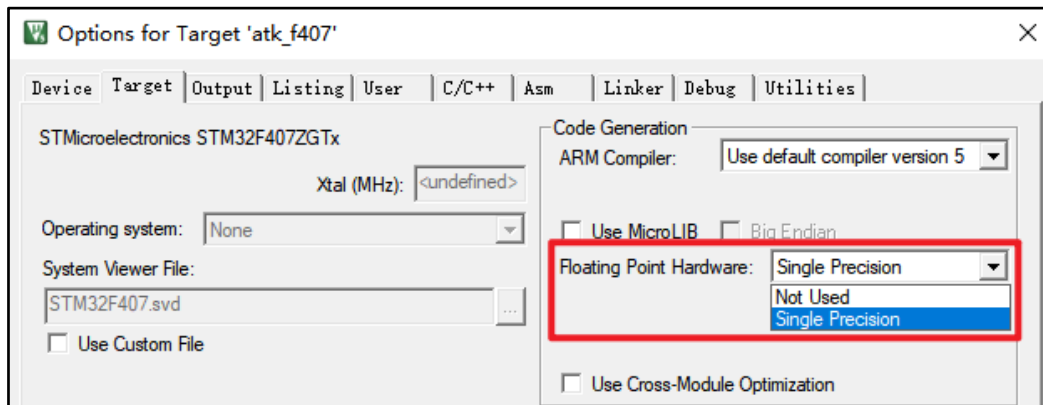


图 55.2.1.1 FPU 开关

上图中的 Floating Point Hardware 选择 “Not Used” 就表示不使用硬件 FPU,选择 “Single Precision” 就表示使用硬件 FPU。

55.3 下载验证

在 FPU 开启和关闭的情况下,分别完成编译和烧录操作后,可以看到, LCD 自动刷新显示了 Julia 分形图形,并且可以看到在不开启 FPU 的情况下生成并显示第一张 Julia 分形图像,大约耗时 3000+毫秒,而在开启 FPU 后生成并显示第一张 Julia 分形图像的耗时缩短为 300+毫秒,两者相差近 10 倍,由此可见使用 FPU 的优势。

通过可按下 WKUP 按键重新生成并显示不同缩放因子的 Julia 分形图像,也可按下 KEY0 按键使程序不断自动切换缩放因子,并重新生成和显示 Julia 分形图形。

第五十六章 DSP BasicMath 实验

本章介绍使用 DSP 库进行基本的数学运算。通过本章的学习，读者将学习到如何使用 DSP 库进行数学运算。

本章分为如下几个小节：

56.1 硬件设计

56.2 程序设计

56.3 下载验证

56.1 硬件设计

56.1.1 例程功能

1. 分别在使用和不使用 DSP 库的情况下进行正弦余弦计算，并分别在 LCD 上显示计算耗时
2. LED0 闪烁，提示程序正在运行

56.1.2 硬件资源

1. LED
LED0 - PF9
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块

56.1.3 原理图

本章实验使用的 DSP 库为软件库，因此没有对应的连接原理图。

56.2 程序设计

56.2.1 DPS 库的使用

本章实验使用 STM32CubeF4 软件包中提供的 DSP 库，将其添加到工程文件夹后，如下图所示：

```
./Drivers/CMSIS/  
|-- DSP  
|   |-- Include  
|   |   |-- arm_common_tables.h  
|   |   |-- arm_const_structs.h  
|   |   |-- arm_math.h  
|-- Device  
|-- Include  
|-- Lib  
|   |-- ARM  
|   |-- arm_cortexM4lf_math.lib
```

图 56.2.1.1 DSP 库

DSP 中就提供了大量用于数学运算的函数，例如：正弦、余弦的计算等。使用 DSP 库能加快各种算法的实现，并且 DSP 中的函数是专门为 DSP 指令集做了相应的优化的，因此能以极高的效率在开启 FPU 且具有 DSP 指令集的 MCU 执行。

DSP 库的使用方法也很简单，仅需将 DSP 库文件添加到工程中，并包含相应的头文件，就能调用 DSP 库中的函数了，具体请见本章实验的配套实验例程。

56.2.2 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
```



```

{
    float time;
    char buf[50];

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    lcd_init(); /* 初始化 LCD */
    btim_timx_int_init(0xFFFF, 8400 - 1); /* 初始化基本定时器 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "DSP BasicMath TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    lcd_show_string(30, 110, 200, 16, 16, "No DSP runtime:", RED);
    lcd_show_string(30, 130, 200, 16, 16, "Use DSP runtime:", RED);

    while (1)
    {
        /* 不使用 DSP 库 */
        __HAL_TIM_SET_COUNTER(&g_timx_handle, 0);
        g_timeout = 0;
        if (sin_cos_test(PI / 6, 200000, 0) == 0)
        {
            time = __HAL_TIM_GET_COUNTER(&g_timx_handle) +
                (uint32_t)g_timeout * 0x10000;
            sprintf(buf, "%0.1fms\r\n", time / 10);
            lcd_show_string(150, 110, 200, 16, 16, buf, BLUE);
        }
        else
        {
            lcd_show_string(150, 110, 200, 16, 16, "Error", BLUE);
        }

        /* 使用 DSP 库 */
        __HAL_TIM_SET_COUNTER(&g_timx_handle, 0);
        g_timeout = 0;
        if (sin_cos_test(PI / 6, 200000, 1) == 0)
        {
            time = __HAL_TIM_GET_COUNTER(&g_timx_handle) +
                (uint32_t)g_timeout * 0x10000;
            sprintf(buf, "%0.1fms\r\n", time / 10);
            lcd_show_string(158, 130, 200, 16, 16, buf, BLUE);
        }
        else
        {
            lcd_show_string(158, 130, 200, 16, 16, "Error", BLUE);
        }

        LED0_TOGGLE();
    }
}

```

从上面的代码中可以看出，分别调用了两次函数 `sin_cos_test()` 在使用 DSP 库和不使用 DSP 的情况下分别进行 200000 次进行正弦和余弦的计算测试，并使用定时器统计两者的计算耗时，最终在 LCD 上显示两个的计算耗时情况。

函数 `sin_cos_test()` 如下所示：

```

/* 定义误差值 */
#define DELTA 0.0001f

/**
 * @brief 正弦余弦测试

```

```

* @param   angle: 起始角度
* @param   times: 运算次数
* @param   mode: 是否使用 DSP 库
* @arg     0: 不使用 DSP 库
* @arg     1: 使用 DSP 库
* @retval  计算结果
* @arg     0: 计算无误
* @arg     1: 计算错误
*/
uint8_t sin_cos_test(float angle, uint32_t times, uint8_t mode)
{
    float sinx;
    float cosx;
    float result;
    uint32_t i = 0;

    if (mode == 0)
    {
        /* 不使用 DSP 库计算正弦余弦 */
        for (i=0; i<times; i++)
        {
            cosx = cosf(angle);
            sinx = sinf(angle);
            result = sinx * sinx + cosx * cosx;
            result = fabsf(result - 1.0f);

            if (result > DELTA)
            {
                return 0xFF;
            }

            angle += 0.001f;
        }
    }
    else
    {
        /* 使用 DSP 库计算正弦余弦 */
        for (i=0; i<times; i++)
        {
            cosx = arm_cos_f32(angle);
            sinx = arm_sin_f32(angle);
            result = sinx * sinx + cosx * cosx;
            result = fabsf(result - 1.0f);

            if (result > DELTA)
            {
                return 0xFF;
            }

            angle += 0.001f;
        }
    }

    return 0;
}

```

从上面的代码中可以看出，使用 DSP 库计算正弦、余弦时，使用了 DSP 库中的函数 `arm_cos_f32()` 和函数 `arm_sin_f32()`，不使用 DSP 库正弦、余弦时，使用了 C 标准库中的函数 `cosf()` 和函数 `sinf()`。

56.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上分别显示了不使用 DSP 库和使用 DSP 库的运算耗时时间，从两者的耗时时间中，能看出使用 DSP 库进行数学运算有明显的速度优势。

第五十七章 DSP FFT 实验

本章将使用 DSP 进行快速傅里叶变换（FFT）的运算测试。通过本章的学习，读者将学习到 DSP 库中 FFT 运算的简单应用。

本章分为如下几个章节：

- 57.1 硬件设计
- 57.2 程序设计
- 57.3 下载验证

57.1 硬件设计

57.1.1 例程功能

1. 按下 WKUP 按键可进行 FFT 计算，并在 LCD 上显示计算耗时，同时通过串口输出计算结果
2. LED0 闪烁，提示程序正在运行

57.1.2 硬件资源

1. LED
LED0 - PF9
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
WKUP - PA0
4. USART1
USART1_TX - PA9
USART1_RX - PA10

57.1.3 原理图

本章使用的 DSP 库为软件库，因此没有对应的连接原理图。

57.2 程序设计

57.2.1 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    uint8_t t = 0;
    uint8_t key;
    float time;
    char buf[50];
    arm_cfft_radix4_instance_f32 s;
    uint16_t i;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟, 168MHz */
    delay_init(168); /* 初始化延时 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    lcd_init(); /* 初始化 LCD */
    btim_timx_int_init(0xFFFF, 8400 - 1); /* 初始化基本定时器 */
}
```

```

lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
lcd_show_string(30, 70, 200, 16, 16, "DSP FFT TEST", RED);
lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

lcd_show_string(30, 110, 200, 16, 16, "WKUP:Run FFT", RED);
lcd_show_string(30, 130, 200, 16, 16, "FFT runtime:", RED);

/* FFT 计算初始化 */
arm_cfft_radix4_init_f32(&s, FFT_LENGTH, 0, 1);

while (1)
{
    key = key_scan(0);
    if (key == WKUP_PRES)
    {
        /* 生成输入数据 */
        for (i=0; i<FFT_LENGTH; i++)
        {
            fft_inputbuf[2 * i] = 100 +
                10 * arm_sin_f32(2 * PI * i / FFT_LENGTH) +
                30 * arm_sin_f32(2 * PI * i * 4 / FFT_LENGTH) +
                50 * arm_cos_f32(2 * PI * i * 8 / FFT_LENGTH);
            fft_inputbuf[2 * i + 1] = 0;
        }

        /* FFT 计算并计算耗时 */
        __HAL_TIM_SET_COUNTER(&g_timx_handle, 0);
        g_timeout = 0;
        arm_cfft_radix4_f32(&s, fft_inputbuf);
        time = __HAL_TIM_GET_COUNTER(&g_timx_handle) +
            (uint32_t)g_timeout * 0x10000;
        sprintf(buf, "%0.1fms\r\n", time / 10);
        lcd_show_string(126, 130, 200, 16, 16, buf, BLUE);

        /* 对 FFT 计算结果取模并通过串口输出 */
        arm_cmplx_mag_f32(fft_inputbuf, fft_outputbuf, FFT_LENGTH);
        printf("\r\n%d point FFT runtime:%0.1fms\r\n",
            FFT_LENGTH, time / 10);
        printf("FFT Result:\r\n");
        for (i=0; i<FFT_LENGTH; i++)
        {
            printf("fft_outputbuf[%d]:%f\r\n", i, fft_outputbuf[i]);
        }
    }

    if (++t == 20)
    {
        t = 0;
        LED0_TOGGLE();
    }

    delay_ms(10);
}
}

```

从上面的代码中可以看出，在检测到 WKUP 按键被按下后，便会调用 DSP 库中的函数 arm_cfft_radix4_f32() 进行 FFT 运算测试，同时使用定时器统计运算耗时，最后调用函数 arm_cmplx_mag_f32() 对 FFT 的运算结果取模，然后在串口调试助手上显示最终的结果。在使用了 DSP 库后，进行 FFT 运算变得十分简便，这非常有利于缩短应用开发周期。

57.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上显示了本实验的相关信息，此时若按下 WKUP 按键进行 FFT 运算测试，可以看到，在 FFT 运算结束后，LCD 上显示了本次 FFT 运算的耗时

时长，同时串口调试助手上也显示了本次 FFT 运算结果取模后的结果值。

第五十八章 手写识别实验

本章将介绍正点原子提供的手写识别库的使用,通过使用手写识别库可以实现一些简单的数字字母手写识别的应用。通过本章的学习,读者将学习到正点原子手写识别库的使用。

本章分为如下几个小节:

- 58.1 硬件设计
- 58.2 程序设计
- 58.3 下载验证

58.1 硬件设计

58.1.1 例程功能

1. LCD 上显示手写触摸轨迹,并自动识别手写输入
2. 按下 WKUP 按键可进行电容触摸屏触摸校准
3. 按下 KEY0 按键可切换手写识别模式
4. LED0 闪烁,提示程序正在运行

58.1.2 硬件资源

1. LED
LED0 - PF9
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. USART1
USART1_TX - PA9
USART1_RX - PA10
4. NOR Flash
SPI1_SCK - PB3
SPI1_MISO - PB4
SPI1_MOSI - PB5
SPI1_CS - PB14
5. SD
SDIO_D0 - PC8
SDIO_D1 - PC9
SDIO_D2 - PC10
SDIO_D3 - PC11
SDIO_SCK - PC12
SDIO_CMD - PD2
6. 24C02
IIC_SCL - PB8
IIC_SDA - PB9

58.1.3 原理图

本章实验使用的手写识别库为软件库,因此没有对应的连接原理图。

58.2 程序设计

58.2.1 手写识别库的使用

正点原子提供的手写识别库包含四个文件,分别为: ATKNCR_M_Vx.x.lib、ATKNCR_N_Vx.x.lib、atk_ocr.x 和 atk_ocr.h,本章实验配套的实验例程中已经提供了这四个文件,并且针对正点原子 M144Z-M4 最小系统板 STM32F407 版进行了移植适配,在使用时,仅需将

atk_ncr.c 和 atk_ncr.h 这两个文件添加到自己的工程中，并根据是否使用内存管理动态管理手写识别库运行时所需的内存，添加 ATKNCR_M_Vx.x.lib（使用内存管理）或 ATKNCR_N_Vx.x.lib（不使用内存管理）即可，如下图所示：

```
./Middlewares/ATKNCR/
|-- ATKNCR_M_V2.0.lib
|-- ATKNCR_N_V2.0.lib
|-- atk_ncr.c
`-- atk_ncr.h
```

图 58.2.1.1 正点原子手写识别库文件

手写识别库中提供了手写识别初始化、识别和停止等函数，使用非常方便。

在进行手写识别前，需要使用手写识别初始化函数对其进行初始化，手写识别初始化的使用示例，如下所示：

```
#include "../ATKNCR/atk_ncr.h"

void example_fun(void)
{
    /* 初始化手写识别 */
    alientek_ncr_init();
}
```

手写识别初始化后，便可开始进行手写识别，手写识别函数的使用示例，如下所示：

```
#include "../ATKNCR/atk_ncr.h"

void example_fun(void)
{
    /* 定义点阵数据缓存 */
    atk_ncr_point input_buf[200];
    int input_cnt;
    char output_buf[2];

    /* 初始化手写识别 */
    alientek_ncr_init();

    /* 从触摸屏或其他方式获取输入的点阵数据 */
    input_cnt = get_data(input_buf);

    /* 进行手写识别 */
    alientek_ncr(input_buf, cnt, 1, 1, output_buf);

    /* 处理识别结果 */
    /* printf("Detect result: %s\r\n", output_buf); */
}
```

在进行手写识别后，若不再需要进行手写识别，可以调用结束手写识别函数，结束手写识别，该函数的使用示例，如下所示：

```
#include "../ATKNCR/atk_ncr.h"

void example_fun(void)
{
    /* 定义点阵数据缓存 */
    atk_ncr_point input_buf[200];
    int input_cnt;
    char output_buf[2];

    /* 初始化手写识别 */
    alientek_ncr_init();

    /* 从触摸屏或其他方式获取输入的点阵数据 */
    input_cnt = get_data(input_buf);

    /* 进行手写识别 */
    alientek_ncr(input_buf, cnt, 1, 1, output_buf);
}
```

```

/* 处理识别结果 */
/* printf("Detect result: %s\r\n", output_buf); */

/* 停止手写识别 */
alientek_ncr_stop();
}

```

58.2.2 实验应用代码

本章实验的应用代码，如下所示：

```

int main(void)
{
    /* 仅保留关键代码，其余代码省略 */
    /* 初始化手写识别 */
    alientek_ncr_init();

RESTART:
    while (1)
    {
        key = key_scan(0);

        switch (key)
        {
            /* 切换识别模式 */
            case KEY0_PRES:
            {
                mode++;
                if (mode > 4)
                {
                    mode = 1;
                }
                break;
            }
            /* 进行触摸校准 */
            case WKUP_PRES:
            {
                if ((tp_dev.touchtype & 0x80) == 0)
                {
                    tp_adjust();
                    goto RESTART;
                }
                break;
            }
        }

        tp_dev.scan(0);
        /* 有触摸按下，记录触摸点阵数据 */
        if (tp_dev.sta & TP_PRES_DOWN)
        {
            ncr_input_buf[pcnt].x = tp_dev.x[0];
            ncr_input_buf[pcnt].y = tp_dev.y[0];
        }
        /* 触摸松开，进行手写识别 */
        else
        {
            alientek_ncr(ncr_input_buf, pcnt, 6, mode, sbuf);
            printf("识别结果:%s\r\n", sbuf);
        }
    }
}

```

由于本实验的应用代码过于冗长，因此上面代码仅保留了部分关键代码，完整代码请读者自行查看本实验的配套实验例程。从上面的代码中可以看出，本实验实现的应用中，通过触摸屏获取点阵数据，将点阵数据传入手写识别函数后，获取手写识别结果，然后将手写识别结果通过串

口等方式输出，并且可通过 KEY0 按键修改手写识别的模式，也可通过 WKUP 按键随时进行电阻屏的触摸校准。

58.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上显示了本实验的相关实验信息，此时便可以在 LCD 上提示的“手写区”进行手写操作，完后手写操作后，可以看到 LCD 上提示了手写识别的结果，此时也可以按下 KEY0 按键修改手写识别的模式，也可以按下 WKUP 按键进行电阻屏的触摸校准。

第五十九章 T9 拼音输入法实验

本章将介绍正点原子提供的 T9 拼音输入法库的使用，用其实现一个简单的 T9 拼音输入法应用。通过本章的学习，读者将学习到正点原子 T9 拼音输入法库的使用。

本章分为如下几个小节：

59.1 硬件设计

59.2 程序设计

59.3 下载验证

59.1 硬件设计

59.1.1 例程功能

1. 输入拼音后，LCD 和串口显示匹配结果
2. 按下 WKUP 按键可清除输入
3. 按下 KEY0 按键可换页匹配结果
4. LED0 闪烁，提示程序正在运行

59.1.2 硬件资源

1. LED
LED0 - PF9
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. USART1
USART1_TX - PA9
USART1_RX - PA10
4. NOR Flash
SPI1_SCK - PB3
SPI1_MISO - PB4
SPI1_MOSI - PB5
SPI1_CS - PB14
5. SD
SDIO_D0 - PC8
SDIO_D1 - PC9
SDIO_D2 - PC10
SDIO_D3 - PC11
SDIO_SCK - PC12
SDIO_CMD - PD2
6. 24C02
IIC_SCL - PB8
IIC_SDA - PB9

59.1.3 原理图

本章实验使用的 T9 拼音输入法库为软件库，因此没有对应的连接原理图。

59.2 程序设计

59.2.1 T9 拼音输入法库的使用

正点原子提供的 T9 拼音输入法库包含三个文件，分别为：pyinput.c、pyinput.h 和 pymb.h，本章实验配套的实验例程中已经提供了这三个文件，并且针对正点原子 M144Z-M4 最小系统板 STM32F407 版进行了移植适配，在使用时，仅需将这三个文件添加到自己的工程即可，如下图

所示：

```
./Middlewares/T9INPUT/
|-- pyinput.c
|-- pyinput.h
`-- pymb.h
```

图 59.2.1.1 正点原子 T9 拼音输入法库文件

T9 拼音输入法库提供了匹配码表的函数，使用非常方便。

匹配码表函数的使用示例，如下所示：

```
#include "../T9INPUT/pyinput.h"

void example_fun(void)
{
    uint8_t inputstr[5];
    uint8_t res;
    uint8_t index;

    /* 拼音输入 */
    // inputstr[0] = 9; // wxyz
    // inputstr[1] = 4; // ghi
    // inputstr[2] = 3; // def
    // inputstr[3] = 6; // mno
    // inputstr[4] = 4; // ghi

    /* 匹配码表 */
    res = t9.getpymb(inputstr);
    /* 有匹配结果 */
    if (res != 0)
    {
        /* 展示所有匹配结果 */
        for (index=0; index<(res&0x7F); index++)
        {
            // printf("拼音:  %s\r\n", t9.pymb[index]->py);
            // printf("结果:  %s\r\n", t9.pymb[index]->pymb);
        }
    }
    /* 无匹配结果 */
    else
    {
        /* Do something */
    }
}
```

59.2.2 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    /* 仅保留关键代码，其余代码省略 */

    RESTART:
    while (1)
    {
        /* 获取触摸点对应的键值 */
        pykey = py_get_keynum(30, 195);
        /* 有有效键值 */
        if (pykey != 0)
        {
            /* 退格键 */
            if (pykey == 1)
            {
                if (inputlen != 0)
                {

```

```

        inputlen--;
    }
    inputstr[inputlen] = '\0';
}
/* 其他有效键值 */
else
{
    inputstr[inputlen] = pykey + '0';
    if (inputlen < 7)
    {
        inputlen++;
    }
}

/* 有输入 */
if (inputstr[0] != '\0')
{
    /* 匹配码表 */
    pykey = t9.getpymb(inputstr);
    /* 有匹配结果 */
    if (pykey != 0)
    {
        result_num = pykey & 0x7F;
        cur_index = 1;
    }
    /* 无匹配结果 */
    else
    {
        inputlen--;
        inputstr[inputlen] = '\0';
    }
}
/* 无输入 */
else
{
    cur_index = 0;
    result_num = 0;
}

/* 显示输入和匹配结果 */
lcd_fill(30 + 40, 105, 30 + 40 + 48, 110 + 16, WHITE);
lcd_show_num(30 + 144, 105, result_num, 1, 16, BLUE);
text_show_string(30 + 40, 105, 200, 16, (char *)inputstr, 16, 0, BLUE);
py_show_result(cur_index);
}

/* 有匹配结果 */
if (result_num != 0)
{
    key = key_scan(0);
    switch (key)
    {
        /* 匹配翻页 */
        case KEY0_PRES:
        {
            if (cur_index < result_num)
            {
                cur_index++;
            }
            else
            {
                cur_index = 1;
            }
            py_show_result(cur_index);
            break;
        }
    }
}

```

由于本实验的应用代码过于冗长，因此上面代码仅保留了部分关键代码，完成代码请读者自行查看本实验的配套实验例程。从上面的代码中可以看出，本实验实验的应用中，通过触摸屏获取拼音的键值，然后调用函数 `t9.getpymb()` 进行码表匹配，有匹配结果时，则在 LCD 上显示匹配结果，若有多个匹配结果，则可通过 KEY0 按键进行翻页，通过也可通过 WKUP 按键清除输入。

在完成编译和烧录操作后，可以看到 LCD 上显示了本实验的相关实验信息，通过也显示了 T9 拼音的输入键盘，此时便可通过键盘输入拼音，随即便可在 LCD 上看到输入拼音对应的汉字匹配结果，若有多个匹配结果，例如输入“64”，则会匹配到拼音“mi”和“ni”，此时按下 KEY0 按键，可对多个匹配结果进行翻页，若按下 WKUP 按键则会清除所有输入。

第六十章 串口 IAP 实验

本章将介绍在 STM32F407 上使用串口进行 IAP, 以实现简单的 IAP 功能。通过本章的学习, 读者将学习到 IAP 的使用。

本章分为如下几个小节:

- 60.1 硬件设计
- 60.2 程序设计
- 60.3 下载验证

60.1 硬件设计

60.1.1 例程功能

1. 串口接收 APP 固件数据
2. 按下 WKUP 按键可将串口接收到的 APP 固件数据写入 Flash
3. 按下 KEY0 按键可执行写入 Flash 的 APP
4. LED0 闪烁, 提示程序正在运行

60.1.2 硬件资源

1. LED
LED0 - PF9
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. 按键
WKUP - PA0
KEY0 - PE4
4. USART1
USART1_TX - PA9
USART1_RX - PA10

60.1.3 原理图

本章实验使用的 IAP 为软件算法, 因此没有对应的连接原理图。

60.2 程序设计

60.2.1 IAP 的实现

本章实验提供了用于 IAP 的驱动代码, 如下图所示:

```
./IAP Bootloader V1.0/User/IAP/  
|-- iap.c  
'-- iap.h
```

图 60.2.1.1 IAP 驱动代码

IAP 的实现最主要分为两个步骤, 分别为加载 APP 固件至 Flash 和跳转到 APP 中运行, 为此本实验实现了以上两个函数, 如下所示:

```
void iap_write_appbin(uint32_t appxaddr, uint8_t *appbuf, uint32_t applen);  
void iap_load_app(uint32_t appxaddr);
```

以上两个函数就分别实现了加载 APP 固件至 Flash 和跳转到 APP 运行的功能, 这两个函数的使用示例, 如下所示:

```
#include "../IAP/iap.h"  
  
#define FLASH_APP_ADDR (0x08010000)  
#define APP_MAX_SIZE (120*1024)
```

```
static uint8_t appbin[APP_MAX_SIZE];

void example_fun(void)
{
    uint32_t appsize;

    /* 通过串口等方式获取 APP 的二进制数据 */
    appsize = get_app_bin(appbin);

    /* 将 APP 的二进制数据写入 Flash 的指定地址中 */
    iap_write_appbin(FLASH_APP_ADDR, appbin, appsize);

    /* 跳转到 APP 在 Flash 中的起始地址运行 */
    iap_load_app(FLASH_APP_ADDR);
}
```

60.2.2 APP 工程修改

APP 程序要能够在指定的 Flash 起始地址运行，需要编译器在编译 APP 的时候知道 APP 将被保存在 Flash 的那个位置，对于 MDK 软件，可以在 Options for Target 窗口中设置，如下图所示：

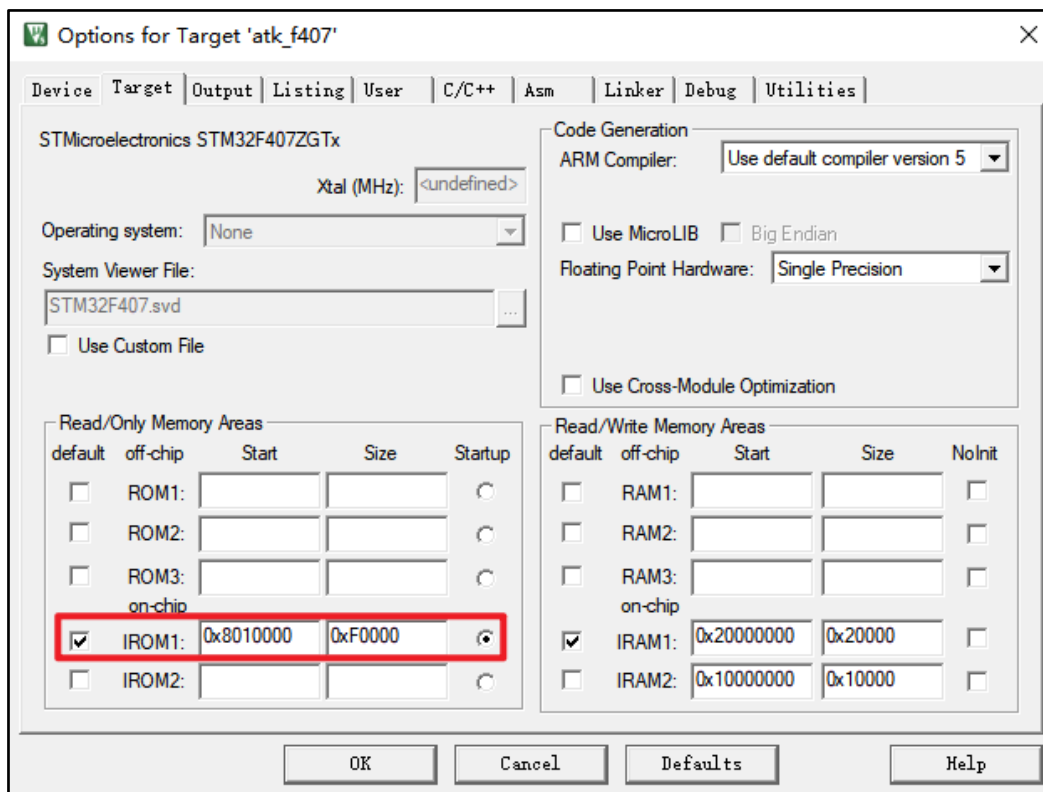


图 60.2.2.1 配置 MDK 中的 Flash 空间

上图中将 APP 程序保存在 Flash 中的空间配置为 0x8010000~(0x8010000+0xF0000)，通过这样的配置便可以使 APP 被保存在指定 Flash 位置中执行。读者可以自行配置“Start”和“Size”参数，但要注意不能超过 Flash 的范围，并且也不要覆盖 IAP 程序（Bootloader）。

上一小节中将 APP 保存到 Flash 中的操作操作的是二进制数据，因此 APP 也应当被编译为二进制文件，然后 MDK 软件中并没有直接编译出二进制文件的选项，因此需要配置 MDK 在编译完成后执行指定的命令，如下图所示：

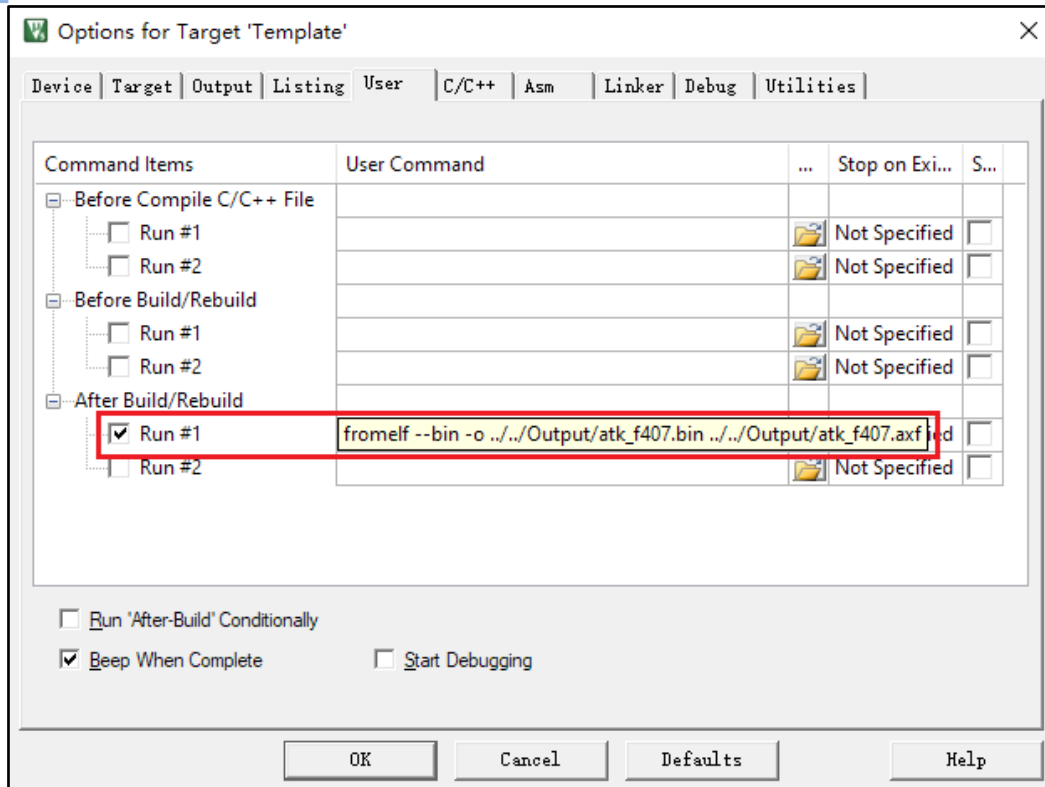


图 60.2.2.2 配置 MDK 生成二进制文件

完成以上配置后，便可在 APP 程序编译完成后在工程的 Output 目录下得到正确的二进制文件。

60.2.3 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    /* 仅保留关键代码，其余代码省略 */

    /* 初始化串口 */
    usart_init(115200);

    while (1)
    {
        /* 串口已接收到数据 */
        if (g_usart_rx_cnt != 0)
        {
            /* 新的一次循环串口还没有接收到新数据，
             * 则断定串口接收完毕
             */
            if (lastcount == g_usart_rx_cnt)
            {
                /* 记录 APP 固件大小 */
                applenth = g_usart_rx_cnt;
                lastcount = 0;
                g_usart_rx_cnt = 0;
            }
            else
            {
                lastcount = g_usart_rx_cnt;
            }
        }
    }
}
```



```
key = key_scan(0);
switch (key)
{
    case WKUP_PRES:
    {
        /* 将串口接收到的 APP 二进制数据写入 Flash 中的指定位置 */
        iap_write_appbin(FLASH_APP1_ADDR, g_usart_rx_buf, applenth);
        break;
    }
    case KEY0_PRES:
    {
        /* 跳转到 Flash 的指定位置执行 APP 程序 */
        iap_load_app(FLASH_APP1_ADDR);
        break;
    }
}

delay_ms(100);
}
```

从上面的代码中可以看出，程序是通过串口接收 APP 的二进制数据的，因此串口驱动中的串口接收缓存因该设置的足够大，以能够容纳 APP 的二进制数据，本实验中串口接收缓存的配置如下所示：

```
/* 定义最大接收 120KB 数据 */
#define USART_REC_LEN (120 * 1024)
```

在判断 APP 二进制数据接收完毕后，便可通过 WKUP 按键调用函数 iap_write_appbin() 将 APP 的二进制数据写入 Flash 中的 FLASH_APP1_ADDR 位置中，FLASH_APP1_ADDR 是一个宏，该宏的定义如下所示：

```
#define FLASH_APP1_ADDR 0x08010000
```

需要注意的是，APP 写入的位置应当与上一小节中配置 MDK 软件的指定 Flash 位置相同。

将 APP 的二进制数据写入到 Flash 中后，此时运行的依旧是 IAP (Bootloader) 程序，此时可以按下 WKUP 按键来跳转到 Flash 中 APP 的位置执行 APP 程序。

60.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上显示了本实验的相关实验信息，此时程序正在等待串口接收 APP 的二进制数据，此时便可通过串口调试助手将实现编译生成的 APP 二进制文件发送给 MCU，待发送完毕后按下 WKUP 按键，将 APP 的二进制数据写入到 Flash 中，若写入成功，LCD 上将会有相应的提示，此时便可按下 KEY0 按键将程序跳转到 APP 的保存位置中运行，若指定的 Flash 位置保存了正确的 APP 固件，那么便可看到 MCU 正在运行 APP 程序，而非 IAP (Bootloader) 程序。

第六十一章 USB 读卡器（Slave）实验

本章将介绍使用 STM32F407 作为 USB 从设备，模拟出 USB 读卡器，使得 PC 与 STM32F407 通过 USB 连接后，能够访问与 STM32F407 连接的 SD 中的数据。通过本章的学习，读者将学习到 STM32F407 作为 USB 从设备模拟出 USB 读卡器的使用。

本章分为如下几个小节：

- 61.1 硬件设计
- 61.2 程序设计
- 61.3 下载验证

61.1 硬件设计

61.1.1 例程功能

1. LCD 上显示 SD 卡和 NOR Flash 容量
2. 当通过 USB 接口连接至 USB Host 时，LCD 上显示 USB 连接状态和读写状态，同时 USB Host 设备可对 SD 卡和 NOR Flash 中的文件进行操作
3. LED0 闪烁，提示程序正在运行

61.1.2 硬件资源

1. LED
 - LED0 - PF9
 - LED1 - PE6
2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
3. NOR Flash
 - SPI1_SCK - PB3
 - SPI1_MISO - PB4
 - SPI1_MOSI - PB5
 - SPI1_CS - PB14
4. SD
 - SDIO_D0 - PC8
 - SDIO_D1 - PC9
 - SDIO_D2 - PC10
 - SDIO_D3 - PC11
 - SDIO_SCK - PC12
 - SDIO_CMD - PD2
5. USB
 - USB_DM - PA11
 - USB_DP - PA12

61.1.3 原理图

本章实验使用 USB 接口与 PC 进行连接，开发板板载了一个 USB 接口，用于连接其他 USB 设备，USB 接口与 MCU 的连接原理图，如下图所示：

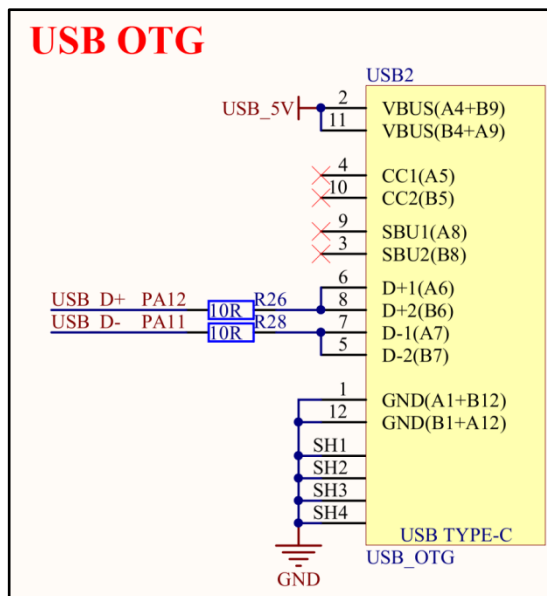


图 61.1.3.1 USB 接口与 MCU 的连接原理图

61.2 程序设计

61.2.1 ST 的 USB 设备驱动库

ST 针对 STM32F4 的 USB 设备驱动库为 STM32_USB_Device_Library，该 USB 设备驱动库由 ST 提供对 STM32CubeF4 软件包一起提供，如下图所示：

```
STM32Cube_FW_F4_V1.27.1/Middlewares/ST/
|-- STM32_Audio
|-- STM32_USB_Device_Library
|-- STM32_USB_Host_Library
|-- STemWin
'-- TouchGFX
```

图 61.2.1.1 STM32_USB_Device_library

从上图中也可以看出，STM32CubeF4 软件包提供了 USB 主机和设备的驱动库，本书主要介绍 USB 设备的应用，因此主要关注 STM32_USB_Device_Library。

本书不展开讲解 USB 的相关原理，若读者想更加深入地了解 USB，请自行查阅相关资料。

61.2.2 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    /* 仅保留关键代码，其余代码省略 */

    /* 初始化 USB */
    USB_D_Init(&USB_D_Device, &MSC_Desc, 0);
    /* 添加类 */
    USB_D_RegisterClass(&USB_D_Device, USB_D_MSC_CLASS);
    /* 为 MSC 类添加回调函数 */
    USB_D_MSC_RegisterStorage(&USB_D_Device, &USB_D_DISK_fops);
    /* 开启 USB */
    USB_D_Start(&USB_D_Device);

    while (1)
    {
        t++;
    }
}
```

```

/* USB 状态改变 */
if (usb_sta != g_usb_state_reg)
{
    lcd_fill(30, 190, 240, 190 + 16, WHITE);

    /* 正在写入 */
    if (g_usb_state_reg & 0x01)
    {
        LED1(0);
        lcd_show_string(30, 190, 200, 16, 16, "USB Writing...", RED);
    }

    /* 正在读取 */
    if (g_usb_state_reg & 0x02)
    {
        LED1(0);
        lcd_show_string(30, 190, 200, 16, 16, "USB Reading...", RED);
    }

    /* 写入错误 */
    if (g_usb_state_reg & 0x04)
    {
        lcd_show_string(30, 210, 200, 16, 16, "USB Write Err ", RED);
    }
    else
    {
        lcd_fill(30, 210, 240, 230 + 16, WHITE);
    }

    /* 读取错误 */
    if (g_usb_state_reg & 0x08)
    {
        lcd_show_string(30, 230, 200, 16, 16, "USB Read Err ", RED);
    }
    else
    {
        lcd_fill(30, 230, 240, 250 + 16, WHITE);
    }

    usb_sta = g_usb_state_reg;
}

/* USB 连接状态改变 */
if (device_sta != g_device_state)
{
    /* 连接建立 */
    if (g_device_state == 1)
    {
        lcd_show_string(30, 170, 200, 16, 16, "USB Connected ", RED);
    }
    /* 连接断开 */
    else
    {
        lcd_show_string(30, 170, 200, 16, 16, "USB DisConnected ", RED);
    }

    device_sta = g_device_state;
}
}

```

从上面的代码中可以看出，在调用完 USB 设备驱动库中的相关初始化函数后并不断地获取 USB 的读写状态和连接状态，并在 LCD 上进行显示。

在调用完 USB 相关的初始化函数后，USB 设备驱动库便会自动模拟出一个 USB 读卡器，

当然也需要实现配置好 USB 读卡器模拟出的设备信息，以及读写、初始化 SD 卡、NOR Flash 等的操作，这些操作全部在 `usbd_storage.c` 文件中完成了，请读者自行参考该文件。

61.3 下载验证

在完成编译和烧录操作后，将 SD 卡正确插入开发板板载的 SD 卡卡座，可以看到 LCD 上显示了 SD 卡的容量信息以及 USB 的连接状态，若开发板还未通过 USB 接口与 PC 进行连接，那么 LCD 上会有 USB 连接断开的提示，此时可以将开发板通过 USB 接口与 PC 进行连接，稍等一会后，可以看到 LCD 上显示了 USB 已连接的提示，并且 PC 上也多出了两个磁盘，磁盘中的文件就是 SD 卡和 NOR Flash 中的文件，并且在 PC 上也可以直接对 SD 卡和 NOR Flash 中的文件进行读写操作。

第六十二章 USB 虚拟串口（Slave）实验

本章将介绍使用 STM32F407 作为 USB 从设备，虚拟出串口与 PC 进行通信。通过本章的学习，读者将学习到 STM32F407 作为 USB 从设备虚拟出串口的使用。

本章分为如下几个小节：

- 62.1 硬件设计
- 62.2 程序设计
- 62.3 下载验证

62.1 硬件设计

62.1.1 例程功能

- 1. 回显虚拟串口接收到的数据
- 2. 每间隔一定时间，虚拟串口发送一段提示信息
- 3. LED0 闪烁，提示程序正在运行

62.1.2 硬件资源

- 1. LED
 - LED0 - PF9
- 2. 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块
- 3. USB
 - USB_DM - PA11
 - USB_DP - PA12

62.1.3 原理图

请见第 60.1.3 小节中 USB OTG 接口与 MCU 的连接原理图的相关内容。

62.2 程序设计

62.2.1 ST 的 USB 设备驱动库

请见第 60.2.1 小节中 ST 的 USB 设备驱动库的相关内容。

62.2.2 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    /* 仅保留关键代码，其余代码省略 */

    /* 初始化 USB */
    USBD_Init(&USBD_Device, &VCP_Desc, 0);
    /* 添加类 */
    USBD_RegisterClass(&USBD_Device, USBD_CDC_CLASS);
    /* 为 CDC 类添加回调函数 */
    USBD_CDC_RegisterInterface(&USBD_Device, &USBD_CDC_fops);
    /* 开启 USB */
    USBD_Start(&USBD_Device);

    while(1)
    {
        /* 连接状态改变 */
    }
}
```

```

if (device_sta != g_device_state)
{
    if (device_sta == 1)
    {
        lcd_show_string(30, 110, 200, 16, 16, "USB Connected ", RED);
        LED1(0);
    }
    else
    {
        lcd_show_string(30, 110, 200, 16, 16, "USB Disconnected ", RED);
        LED1(1);
    }
    device_sta = g_device_state;
}

/* 接收到数据 */
if (g_usb_usart_rx_sta & 0x8000)
{
    len = g_usb_usart_rx_sta & 0x3FFF;
    usb_printf("\r\n 您发送的消息为: \r\n");
    cdc_vcp_data_tx(g_usb_usart_rx_buffer, len);
    usb_printf("\r\n\r\n");
    g_usb_usart_rx_sta = 0;
}
else
{
    if ((times % 5000) == 0)
    {
        usb_printf("\r\n 正点原子 \
                    M144Z-M4 最小系统板 STM32F407 版 \
                    USB 虚拟串口实验\r\n");
        usb_printf("正点原子@ALIENTEK\r\n\r\n\r\n");
    }
    if ((times % 200) == 0)
    {
        usb_printf("请输入数据，以回车键结束\r\n");
    }
    if ((times % 30) == 0)
    {
        LED0_TOGGLE();
    }
    times++;
}

delay_ms(10);
}
}

```

从上面的代码中可以看出，在调用完 USB 设备驱动库中相关的初始化函数后便等待 USB 虚拟串口接收数据，若接收到数据便通过函数 `usb_printf()` 将数据发送出去，`usb_printf()` 的实现如下所示：

```

/**
 * @brief 通过 USB 格式化输出函数
 * @note 通过 USB VCP 实现 printf 输出
 * 确保一次发送数据长度不超 USB_USART_REC_LEN 字节
 * @param 格式化输出
 * @retval 无
 */
void usb_printf(char *fmt, ...)
{
    uint16_t i;
    va_list ap;
    va_start(ap, fmt);
    vsprintf((char *)g_usb_usart_printf_buffer, fmt, ap);
    va_end(ap);
}

```

```
i = strlen((const char *)g_usb_usart_printf_buffer); /* 此次发送数据的长度 */  
cdc_vcp_data_tx(g_usb_usart_printf_buffer, i); /* 发送数据 */  
}
```

62.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上显示了本实验的相关实验信息，此时可以将开发板通过 USB 接口与 PC 进行连接，待 PC 自动完成驱动安装后，便可看到 PC 上多出了一个端口设备，此时可以通过串口调试助手与该端口进行通信，可以看到无论串口调试助手发送任何数据，都会在串口调试助手的接受区看到发送出去的数据，这是因为 STM32F407 将虚拟串口接收到的数据通过虚拟串口发送回去，实现了“回显”的效果。

第六十三章 FreeRTOS 移植实验

前面章节中的实验都是在裸机环境下实现，本章将介绍 FreeRTOS 的简单使用，对 RTOS 感兴趣的读者，可以参考正点原子推出的 FreeRTOS 或 μ C/OS-III 的全套教程资料，包含教学视频、开发指南并配套有例程源码。通过本章的学习，读者将学习到 FreeRTOS 的简单应用。

本章分为如下几个小节：

- 63.1 硬件设计
- 63.2 程序设计
- 63.3 下载验证

63.1 硬件设计

63.1.1 例程功能

1. LED0 由 LED0 任务控制 80 毫秒亮，920 毫秒灭
2. LED1 由 LED1 任务控制 300 毫秒亮，300 毫秒灭

63.1.2 硬件资源

1. LED
 - LED0 - PF9
 - LED1 - PF10

63.1.3 原理图

本章实验使用的 FreeRTOS 为软件库，因此没有对应的连接原理图。

63.2 程序设计

63.2.1 FreeRTOS 的移植

FreeRTOS 的源码可参考正点原子推出的《FreeRTOS 开发指南》或相关的配套教学视频至 FreeRTOS 的官网下载，也可在 A 盘→6，软件资料→FreeRTOS 学习资料中找到。

FreeRTOS 的移植请读者参考正点原子推出的《FreeRTOS 开发指南》或相关的配套教学视频，在上述资料中，对 FreeRTOS 的移植和使用都做了非常详细地介绍，是入门 FreeRTOS 的一个非常好的选择。

63.2.2 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    HAL_Init();                /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟，168MHz */
    delay_init(168);           /* 初始化延时 */
    usart_init(115200);        /* 初始化串口 */
    led_init();                /* 初始化 LED */

    xTaskCreate((TaskFunction_t)start_task, /* 任务函数 */
               (const char*) "start_task", /* 任务名称 */
               (uint16_t) START_STK_SIZE, /* 任务堆栈大小 */
               (void*) NULL, /* 传入给任务函数的参数 */
               (UBaseType_t) START_TASK_PRIO, /* 任务优先级 */
               (TaskHandle_t*) &StartTask_Handler); /* 任务句柄 */
}
```

```
vTaskStartScheduler();
}
```

从上面的代码中可以看出，在完成必要的初始化后，便创建了“start_task”任务，随后便开始了任务调度，“start_task”任务的入口函数为函数 start_task()，如下所示：

```
/**
 * @brief    start_task
 * @param    pvParameters: 传入参数(未用到)
 * @retval    无
 */
void start_task(void *pvParameters)
{
    taskENTER_CRITICAL();                /* 进入临界区 */

    /* 创建 LED0 任务 */
    xTaskCreate( (TaskFunction_t) led0_task,
                (const char*) "led0_task",
                (uint16_t) LED0_STK_SIZE,
                (void*) NULL,
                (UBaseType_t) LED0_PRIO,
                (TaskHandle_t*) &LED0Task_Handler);

    /* 创建 LED1 任务 */
    xTaskCreate( (TaskFunction_t) led1_task,
                (const char*) "led1_task",
                (uint16_t) LED1_STK_SIZE,
                (void*) NULL,
                (UBaseType_t) LED1_PRIO,
                (TaskHandle_t*) &LED1Task_Handler);

    vTaskDelete(StartTask_Handler);      /* 删除开始任务 */
    taskEXIT_CRITICAL();                  /* 退出临界区 */
}
```

从上面的代码中可以看出，“start_task”任务主要用于创建“led0_task”和“led1_task”任务，这两个任务的入口函数，如下所示：

```
/**
 * @brief    LED0 任务
 * @param    pvParameters: 传入参数(未用到)
 * @retval    无
 */
void led0_task(void *pvParameters)
{
    while(1)
    {
        LED0(0);
        vTaskDelay(80);
        LED0(1);
        vTaskDelay(920);
    }
}

/**
 * @brief    LED1 任务
 * @param    pvParameters: 传入参数(未用到)
 * @retval    无
 */
void led1_task(void *pvParameters)
{
    while(1)
    {
        LED1(0);
        vTaskDelay(300);
        LED1(1);
        vTaskDelay(300);
    }
}
```

```
}  
}
```

可以看到，两个任务分别控制开发板板载的两个 LED 闪烁，但其闪烁的频率和亮灭时间都不相同，因此因该能看到开发板板载的两个 LED 以不同的频率和不同的亮灭时间进行闪烁。

63.3 下载验证

在完成编译和烧录操作后，可以看到开发板板载的两个 LED 以不用的频率和不同的亮灭时间间闪烁着，其中 LED0 在一个闪烁周期内的亮灭时间约分别为 80 毫秒和 920 毫秒，LED1 在一个闪烁周期内的亮灭时间约分别为 300 毫秒和 300 毫秒。